Dr.BGR
Publications

SIMPLE

PYTHON

MADE FOR

BEGINNERS

"Every expert was once a beginner
— start your journey here."

BY
SAHAYA ANTO NISHANTHI

SIMPLE

PYTHON

MADE FOR

BEGINNERS

BY
Mrs. M. SAHAYA ANTO NISHANTHI, MCA

Dr.BGR
Publications

Verso Page

Publishing House	Dr. BGR Publications
	Tuticorin - 05
	9003494749
	drbgrpublications@gmail.com
	https://drbgrpublications.in/books/
	https://www.instagram.com/drbgrpublications/
Title	Simple Python Made for Beginners
e-ISBN	978-81-990337-7-1
Print ISBN	978-81-990337-9-5
Language	English
Country of Publication	India
Product Composition	Single-Component Retail Product
First published	2025
Author	M. Sahaya Anto Nishanthi
e-Book MRP	INR 50
Paperback MRP	INR 390
Copyright	© M. Sahaya Anto Nishanthi
Edited and typeset by	Dr. BGR Publications
Cover design by	M. Sahaya Anto Nishanthi

Disclaimer

The authors are solely responsible for the contents of the book. The publishers or editors do not take any responsibility for the same in any manner. Errors, if any, are purely unintentional and readers are requested to communicate such errors to the editors or publishers to avoid discrepancies in future.

Introductory Note with Gratitude:

Holding a Master's degree in Computer Applications (MCA) with a focus on Computer Science. I've spent my teaching career helping students break through the fear of programming. ***Whether in front of a blackboard or behind a keyboard, my goal has remained the same—to make programming more accessible and empowering for students.***

This book of notes provides a structured journey into Python programming, focusing on the essential principles and problem-solving strategies necessary for success in computer science and software development. Each chapter breaks down complex concepts into digestible explanations, followed by practical exercises and projects that reinforce learning. Whether you're a college student, a hobbyist, or someone exploring Python for career development, this book offers a thorough understanding of how Python can be used in everything from data analysis and web development to machine learning and automation.

I believe that with the right approach, anyone can learn to code—and I've written this book to be that guiding support.

This journey would not have been possible without the encouragement and contributions of many. With much appreciation, I extend my sincere thanks to professor **Mrs. R. Subasri, M.Sc., M.Phil., B.Ed** for her careful review and kind encouragement, which added both quality and confidence to this project.

I owe deep thanks to My companion in life **Mr. F. Michael Manojan**, whose patience, support, understanding, and unwavering belief in me made this work possible.

Special thanks to **Dr. BGR Publications** for believing in this project and providing the platform to share it with a wider audience.

— Mrs. M. Sahaya Anto Nishanthi, MCA

Witnessed Moments: Reader Imprints

"Simple Python Made for Beginners" is a well-structured and thoughtfully written book that serves as a perfect starting point for anyone stepping into the world of programming with python. Mrs. M. Sahaya Anto Nishanthi has done a commendable job in breaking down complex programming concepts into clear , digestible lessons that even a non-programmer can grasp with ease.

The book begins with the very basics-explaining variables, data types, and control structures- and gradually progresses to more advanced topics like functions, file handling, and simple data structures. What stands out is the clarity of explanation, step-by-step examples, and practice exercises.

J. Saravanakumar
Director, Futuro Focus, chennai

Witnessed Moments: Reader Imprints

"Simple Python Made for Beginners" is an excellent entry-level textbook that thoughtfully bridges the gap between foundational computer science principles and practical Python programming. Designed specifically for 11th and 12th standard students as well as BSc computer science undergraduates, the book covers essential topics such as computer fundamentals, Python's core design, data types, control structures, functions, and object-oriented programming in a clear and structured manner.

As a technical project manager working in Workday integrations, I appreciate how the book balances theory with hands-on practice, making it highly suitable for learners who are new to programming. The sections on dictionaries, sets, and database programming are particularly valuable for students who wish to build real-world applications early in their learning journey.

Overall, this book is a well-organized and accessible resource that lays a strong foundation for further studies in computer science and software development. It's a commendable guide for both classroom use and self-study.

Maria Pravin Vaz,
Project Manager – Workday Technology,
7th Floor, Block B, Futura Tech Park, 334,
Rajiv Gandhi Salai (OMR) Sholinganallur, Kancheepuram,
Chennai – 600 119, Tamil Nadu

Witnessed Moments: Reader Imprints

The book **"Simple Python Made for Beginners"** emphasizes developing good programming habits and algorithmic thinking. It's a good choice for those who want to build a strong foundation in computer science principles.

It also known for its hands-on approach, guiding beginners to focuses on practical applications of Python, teaching users how to automate tasks like file manipulation, web scraping, and working with Excel. It's a great choice for those who want to see immediate real-world applications of Python.

Selvendran Rajendran
Associate Consultant
HCL Technologies Ltd

Why Python Is Hot

Python is hot primarily because it has all the right stuff for the kind of software development that's really driving the whole software development world these days. Machine learning, robotics, artificial intelligence, and data science are the leading technologies today and for the foreseeable future. Python is popular mainly because it already has lots of capabilities in those areas, while many older languages lag behind in these technologies

Python, The Best Bet?

Python is generally more beginner-friendly, with simpler syntax and a wide variety of use cases like automation, data analysis, and web development. Python is a powerful, high-level programming language that has captured the hearts of developers around the world due to its elegant syntax and broad range of applications.

Several popular companies make use of Python as well, including:

- **Google:** Uses Python for various services, web applications, and internal tools.
- **Facebook:** Employs Python for infrastructure management, binary distribution, and more.
- **Instagram:** Built primarily on Django, a popular Python web framework.
- **Netflix:** Relies on Python for its backend services and data analysis.
- **Spotify:** Uses Python for data analysis and backend services.
- **Dropbox:** Developed its desktop client and many backend services using Python.
- **Quora:** The platform's backend and server code are written in Python.
- **Pinterest:** Leverages Python for backend services and machine learning tasks.
- **NASA:** Employs Python for data analysis, simulations, and system management.
- **Reddit:** Originally built on Lisp, later rewritten using Python for its simplicity and flexibility.

Python's widespread adoption makes it a valuable skill to learn for anyone looking to break into the world of programming or enhance their existing skill set.

TABLE OF CONTENTS

BOOK 1: COMPUTER FUNDAMENTALS

1.1 What does computer science focus on?

Computer science is the study of problems, problem-solving, and the solutions that come out of the problem-solving process. Given a problem, a computer scientist's goal is to develop an algorithm, a step-by-step list of instructions for solving any instance of the problem that might arise. Algorithms are finite processes that if followed will solve the problem. Algorithms are solutions.

Key quote :- Computer Science is fundamentally about computational

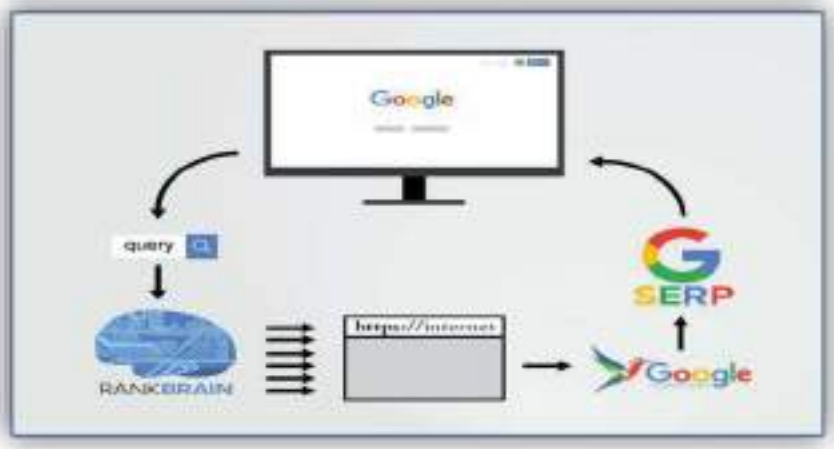
An alternative definition for computer science, then, is to say that computer science is the study of problems that are and that are not computable, the study of the existence and the nonexistence of algorithms.

What is a Computer?

A computer is an electronic machine that runs a set of programs, known as software, to perform computations. A computer is designed to work on information , often taken as input from the user. The software in the computer then performs certain processing on this data and produces a result called input.

1.2. THE ESSENCE OF COMPUTATIONAL PROBLEM SOLVING:

Have you ever wonder how Google's search engine can find results in milliseconds? The answer lies in the power of computational algorithm. *"Computational problem solving is not just about coding; it's about thinking logically and creatively".*



Computational problem solving' is the iterative process of developing computational solutions to problems. Computational solutions are expressed as logical sequences of steps (i.e. algorithms), where each step is precisely defined so that it can be expressed in a form that can be executed by a computer. The Core Principles of Computational Problem-Solving lies in breaking down complex challenges into logical steps, designing efficient algorithms ,and leveraging the power of programming like python is to transform abstract ideas into executable solutions to real-world problems.

In order to solve a problem computationally, two things are needed:

- A representation that captures all the relevant aspects of the problem.
- An algorithm that solves the problem by use of the representation.

"From the authors" viewpoint ,the future of computational problem-solving will be driven by innovations in artificial intelligence and machine learning.

1. **Guido Van Rossum(Creator of Python):**
   "python is an experiment in how much freedom programmers need. Too much freedom and nobody can read another's code; too little and expressiveness is endangered."
   Vital Aspect: Python balances simplicity and power ,making problem-solving both efficient and collaborative.
2. **Allen B. Downey(Author of Think Python):**
   "The goal of programming is not just to tell a computer what to do , but to express solutions clearly and concisely."
   Vital Aspect: Python's readability helps focus on solving problems rather than struggling with syntax.

1.2.1 Intellectual Decomposition of computational problem-solving:

To tackle any challenge effectively , we need to first dissect it. A decomposition or a breakdown is the process that allows us to make sense of the parts ,so we can focus on solving the whole.
Solving problems computationally involves four fundamental steps:

i. Understanding the Problem - Clearly define what needs to be solved.
ii. Devising a Plan(Algorithm design) - Break the problem into logical steps.
iii. Implementing the solution in python - Convert the plan into executable code.
iv. Optimizing for Efficiency - Improve the speed and performance of the solution

1.2.2 Key Aspects of Computational problem-solving:

Some of the key aspect for determine the computational problem solving are mentioned below

1. Problem Understanding:

- ***Clarifying the Problem***: Before diving into solutions, it's crucial to thoroughly understand the problem – what's being asked, what the inputs and outputs are and any constraints or limitations.
- ***Decomposition:*** Breaking the problem into smaller , more manageable sub-problems to simplify the overall process.

2. Algorithm Design:

- ***Step-by-step Approach:*** Designing a clear and systematic method (algorithm)to solve the problem. Algorithm are the foundation of computational problem-solving as they provide an ordered sequence of steps that lead to a solution.
- ***Efficiency:*** Ensuring the algorithm is not only correct but also efficient in terms of time and space.

3. Simplicity and Elegance:

- ***Clean Code:*** Writing simple, clean ,and readable code is crucial for long-term maintenance. Clear, minimalistic solutions are often effective than overly complicated ones.
- ***Avoiding Overengineering:*** A simple solution that solves the problem effectively is often better than a complex one that adds unnecessary features.

4. Scalability:

- ***Handling Larger Data:*** Solutions need to be scalable ,meaning they should work effectively even as the size of the input or data grows.
- ***Big Data Consideration:*** When dealing with large datasets, algorithms must be designed to handle high volumes of data without compromising performance.

5. Artificial Intelligence and Machine Learning:

- ***AI Integration:*** Many modern computational problems involve AI or machine learning, requiring techniques like pattern recognition, supervised and unsupervised learning, and deep learning.

- ***Data-Driven problem solving:*** Solving problems based on data patterns or models built from large datasets, especially in areas like natural language processing or image recognition.

6. Constraints Satisfaction and Optimization:

- ***Solving Problems within Given Limits:*** Many Computational problems involve constraints , requiring constraints satisfaction techniques(e.g., scheduling problems, resource allocation).
- ***Linear and Non-Linear Optimization:*** Finding the best solution among many possible ones using mathematical optimization techniques.

7. Interactive and Real-Time Computing:

- ***Handling Real-Time Constraints:*** Some applications, such as video games and robotics, require solving problems within strict time limits.
- ***Event-Driven Programming:*** Designing systems that respond dynamically to user input or real-time events.

1.3 Limits of Computational problem solving:

Once an algorithm is developed, a critical question arises: Can it find a solution within a reasonable time? If not, its *practical use* is limited. Any algorithm that correctly solves a given problem must solve the problem in a reasonable amount of time, otherwise it is of limited practical use.

While computers are powerful, they do have limitations. Some limits of computation are outlined below, highlighting the challenges computers face in problem-solving.

1.Computational Feasibility:

- Algorithms must solve problems within a reasonable timeframe.
- Time complexity scales runtime with input size.
- Some problems exceed current computational capabilities.

2.Growth in Problem Complexity:

- As input size increases, solutions grow exponentially, hindering exhaustive searches.
- Factorial growth in configurations results in an impractically large number of solutions.

3.**Need for Efficient Algorithms**:

- Efficient algorithms employ strategies such as educated guessing, dynamic programming, greedy algorithms, and utilize AI and machine learning to effectively solve complex problems compared to traditional methods

4.**Hardware & Physical Limits:**

- Computers have limits in processing power, memory, and energy.
- Quantum computing offers potential solutions but has its own limitations.

5. Practical and Ethical Considerations:

- Bias in AI and Machine Learning – Algorithms can inherit bias from data, affecting fairness.
- Cryptography Limits – Some encryption methods rely on computational hardness but may be broken in the future.
- Human Interpretation – Some problems require human intuition, emotions, or ethical decision-making beyond computation.

1.4 Computer Algorithms:

An algorithm solves a problem by utilizing its representation. It consists of a finite number of clearly described unambiguous "doable" steps that can be systematically followed to produce a desired result for given input within a finite amount of time.

1.4.1 What does the term 'algorithm' mean?

An algorithm is a procedure used for solving a problem or performing a computation. Algorithms act as an exact list of instructions that conduct specified actions step by step in either hardware- or software-based routines.

Machine learning is a good example of an algorithm, as it uses multiple algorithms to predict outcomes without being explicitly programmed to do so.

1.4.2 How do algorithms work?

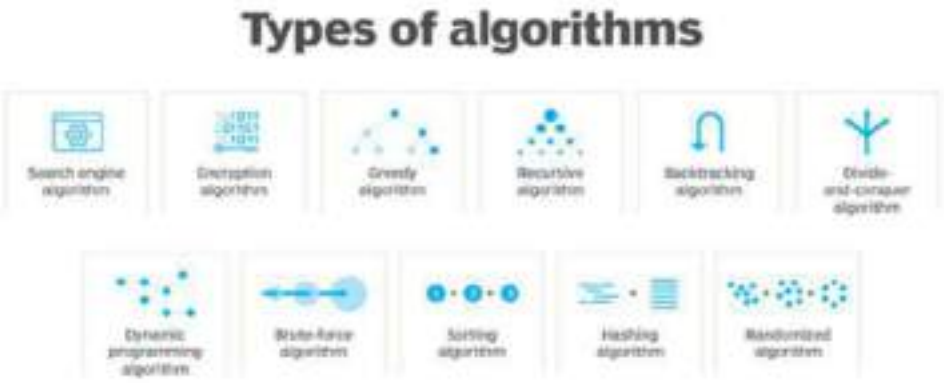
Algorithms work by following a set of instructions or rules to complete a task or solve a problem. They can be expressed as natural languages, programming languages, pseudocode, flowcharts and control tables. Natural language expressions are rare, as they are more ambiguous. Programming languages are normally used for expressing algorithms executed by a computer.

For example, a search algorithm takes a search query as input and runs it through a set of instructions for searching through a database for relevant items to the query.

1.4.3 What are the various categories of algorithms?

There are several types of algorithms, all designed to accomplish different tasks:

1. **Search engine algorithm:**
   - ✓ This algorithm takes search strings of keywords and operators as input, searches its associated database for relevant webpages and returns results.
2. **Encryption algorithm:**
   - ✓ This computing algorithm transforms data according to specified actions to protect it.
3. **Greedy algorithm.:**
   - ✓ This algorithm solves optimization problems by finding the locally optimal solution, hoping it is the optimal solution at the global level.
4. **Recursive algorithm :**
   - ✓ This algorithm calls itself repeatedly until it solves a problem.
   - ✓ Recursive algorithms call themselves with a smaller value every time a recursive function is invoked.
5. **Backtracking algorithm:**
   - ✓ This algorithm finds a solution to given problem in incremental approaches and solves it one piece at a time.
6. **Divide-and-conquer algorithm:**
   - ✓ This common algorithm is divide into two parts. One part divides a problem into smaller subproblems.
   - ✓ The second solved these problems and then combines them to produce a solution.
7. **Dynamic Programming algorithm:**
   - ✓ This algorithm solves problems by dividing them into subproblems.
8. **Brute-force algorithm:**
   - ✓ This algorithm iterates all possible solutions to a problem blindly, searching for one or more solutions to a function.
9. **Sorting algorithm:**
   - ✓ Sorting algorithms are used to rearrange data structures based on a comparison operator, Which is used to decide a new order for data.
10. **Hashing algorithm:**
    - ✓ This algorithm takes and converts it into a uniform message with a hashing.
11. **Random algorithm:**
    - ✓ This algorithm reduces running times and time-based complexities.



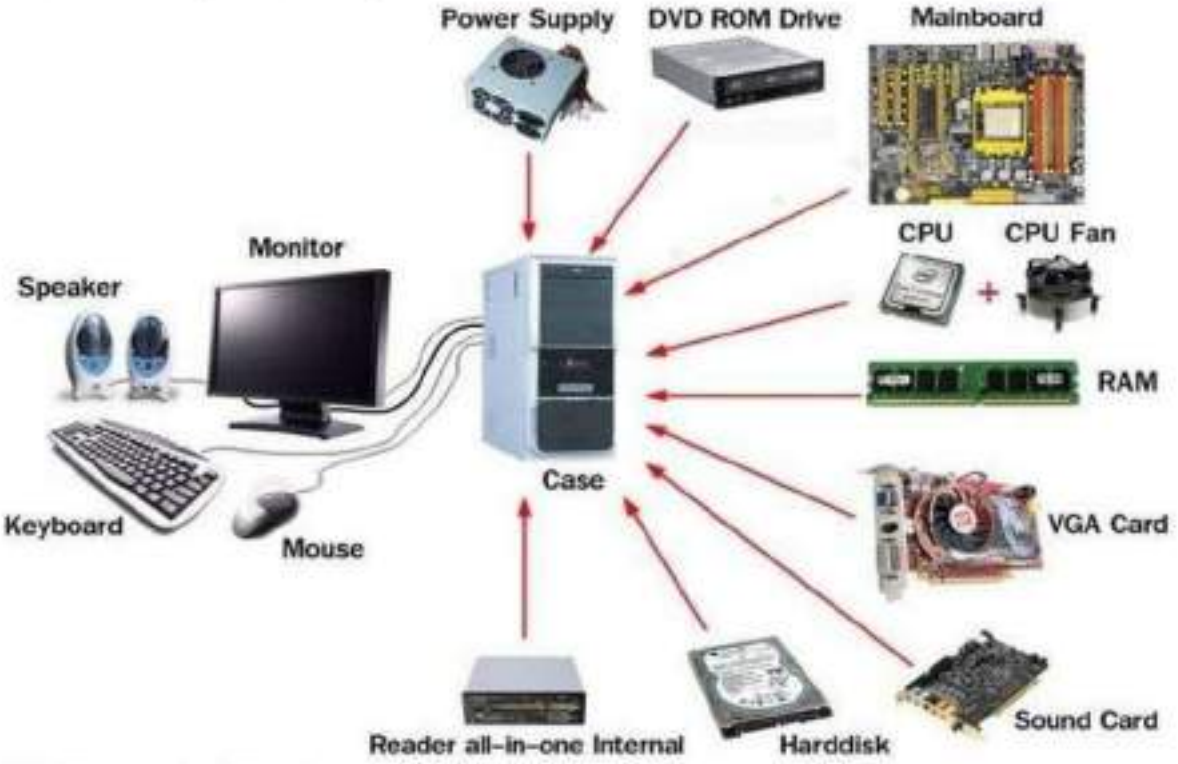
1.4.4 Algorithm and computer: An Ideal Combination

Computers run algorithms to perform calculations and tasks and an efficient algorithms improve computer speed. Without algorithms, computers cannot function. Because computers can execute instructions very quickly and reliably without error, algorithms and computers are a perfect match.

1.5 Computer Hardware and Computer Software:

Every computer is composed of two basic components: hardware and software.

1.5.1 What is Computer Hardware? Computer hardware is a collective term used to describe any of the physical components. Computer hardware includes the physical parts of a computer, such as the central processing unit (CPU), random access memory (RAM), motherboard, computer data storage, graphics card, sound card, and computer case. It includes external devices such as a monitor, mouse, keyboard, and speakers.



1.5.1.1. Computer Hardware Basics:

Computer consists of two main components: memory and CPU (Central Processing Unit). The rest of the computer can be viewed as input/output devices for the two main components. (i.e)

a. External Hardware(peripherals): means any hardware device that is located outside the computer.

- **Input device-**a piece of hardware device which is used to enter information to a computer for processing.
- **Examples:** keyboard, mouse, trackpad (or touchpad), touchscreen, joystick, microphone, light pen, webcam ,etc.
- **Output device -** a piece of hardware device that receives information from a computer.
- **Examples:** monitor, printer, scanner, speaker, display screen (tablet, smartphone …), projector, head phone, etc

b. Internal Hardware components - any piece of hardware device that is located inside the computer.

- **Examples:** CPU, hard disk drive, ROM, RAM, etc.

The motherboard is the main component of a computer. It is a board with integrated circuitry that connects the other parts of the computer including the CPU, the RAM, the disk drives (CD, DVD, hard disk, or any others) as well as any peripherals connected via the ports or the expansion slots. The integrated circuit (IC) chips in a computer typically contain billions of tiny metal–oxide–semiconductor field-effect transistors.

1.5.1.2. How does hardware improve computing efficiency?

All information within a computer system is represented using only two digits, 0 and 1, called binary representation.

This binary representation is based on the use of electronic switches called transistors that can be either "on" or "off" (similar to a light switch). Computer hardware is built using integrated circuits (chips), which consist of millions or billions of transistors.

1.5.1.3. Binary Number system: The Language of a computer

A computer is an electronic device. It does not understand the language understand by humans. It operates on voltage signals-***On/Off, High/Low*** . Hence ,it can be established that a computer understands only 2 types of values. The language of a computer is known as the ***Binary Language.***

The word ***"Bi"*** means 2, denoting the 2 values understand by a computing device. Binary is a base-2 number system, which only uses two digits (0 & 1). It is a system used at the heart of every digital computer, allowing them to encode information, perform arithmetic operations and execute logical control processes.

Gottfried Wilhelm Leibniz (1646–1716)

A 17th-century thinker devised a way to express all numbers using only two digits. Developed the binary system (0s and 1s), which became the foundation of modern computing

To understand binary values imagine each digit (or 'bit') of the binary notation as representing an increasing power of 2 - with the rightmost digit representing 2^0, the next representing 2^1, then 2^2 and so on.

Bit:

The smallest unit of information in a computer is known as a ***bit,*** short for binary digit. A bit have 2 values—either 1 or 0.

For each bit, the 1 or 0 signifies whether the value of the increasing power of two summates towards the number's total.

Figure 1. Increasing powers of two represented with its decimal result.

2^n	2^3	2^2	2^1	2^0
Decimal	**8**	**4**	**2**	**1**

Byte: A byte consists of 8 bits. It is a fundamental unit of data storage in computers and typically represents a single character, such as a letter or number, in text.

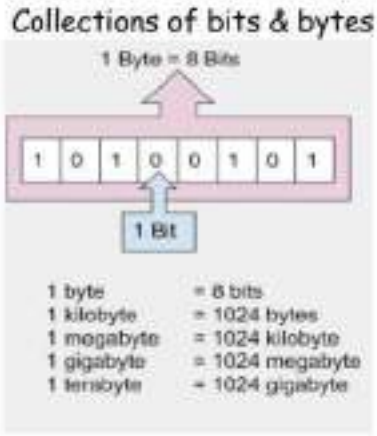
Half a byte, i.e., a group of 4 bits is called ***nibble.***

Group of Bits: Bytes are groups of bits that are operated on as single units within computer systems.

Base (Radix): While computers use binary (base-2) for internal data representation, numbers can be represented in any base. Common bases of number system has include four types.

They are

- **Binary(Base-2)**
- **Decimal (Base-10)**
- **Hexadecimal (Base-16)**
- **Octal (Base-8)**



Comparison of Number Systems:

Number System	**Base**	**Digits Used**	**Example**
Binary	**2**	**0,1**	**1010_2**
Decimal	**10**	**0-9**	**10_{10}**
Octal	**8**	**0-7**	**12_8**
Hexadecimal	**16**	**0-9, A-F**	**A_{16}**

1.5.2 Computer Software:

The first computer programs ever written were for a mechanical computer designed by **Charles Babbage** in the mid-1800s. The person who wrote these programs was a woman, Ada Lovelace who was a talented mathematician. Thus, she is referred to as "the first computer programmer."

Ada Lovelace

What is Software?

Hardware is the physical parts of the computer and software is the programs that run on a computer.

Computer software is a set of program instructions, including related data and documentation, that can be executed by computer.

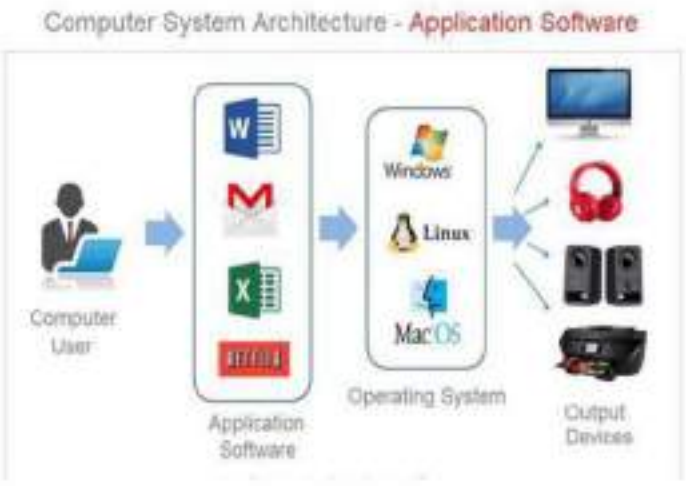
The Main types of software is – **systems software** and **application software**.

1.5.2.1 Application software:

The application software is a computer program designed to offer a specific functionality to the user. These software are developed by the software companies to provide a specific solution to the user.

All the application software comes with built-in features, such as a GUI or CLI interface, that aid in interaction with the operating system, hardware units, and external components to execute operations.

Application software , often simply referred to as "apps" or "applications" is a category of computer software designed to perform specific tasks or functions foe end-users.



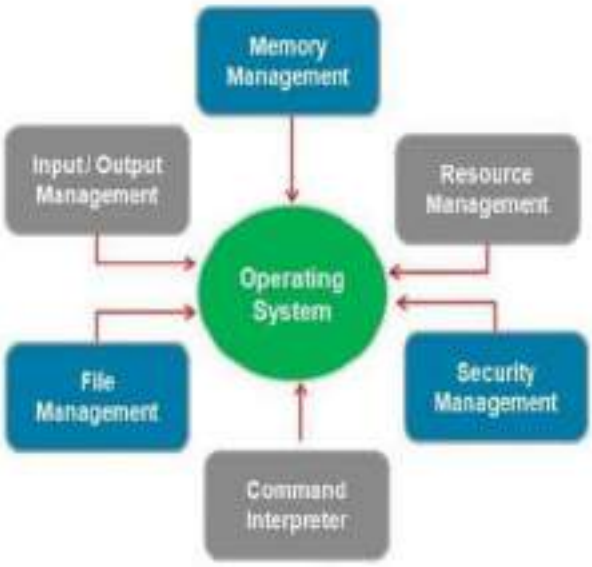
1.5.2.2 System software:

System Software refers to a set of programs and applications that manage and control the operations of a computer system., which means Software that provides the operating and management instructions for the underlying hardware and other components.

The system software comprises the operating system, utility programmes, and device drivers, and other tools that enable the computer to perform various tasks.

1. Operating System (OS): (Windows, Linux, macOS, etc.)

An operating system (OS) is a type of system software that manages a computer's hardware and software resources. It provides common services for computer programs. An OS acts as a link between the software and hardware.The main functions of operating systems are as follow:



2.Device Drivers:

Device drivers are a class of system software that minimizes the need for system troubleshooting. In computer the software that enables the communication between hardware and OS. To operate the hardware components, the operating system comes with a variety of device drivers.

3. Firmware:

In software the firmware is a pre-installed low-level software that controls a device's basic functions. These are the operational programs installed on computer motherboards that assist the operating system in distinguishing between Flash, ROM, EPROM, and memory chips.

There are mainly two main types of firmware chips:

- BIOS (Basic Input/Output System) chip.
- UEFI (Unified Extended Firmware Interface) chips.

4. Programming Language Translator:

Programming language translators are programs that translate code written in one programming language into another programming language.

5. Utility software: tools for system maintenance and optimization. System Software and application software interact through utility software.

A third-party product called utility software is created to lessen maintenance problems and find computer system defects. It is included with your computer's operating system.

1.6. The process of Computational problem solving:

Computational problem-solving is the process of using logical steps and algorithms to find solutions to problems with the help of computers. There are four steps to define the computational process. They are

1. Analyse the Problem:

- To clearly understand and define the problem.
- This includes identifying the inputs, outputs, and any constraints.

2. Algorithm Design:

- To develop a step-by-step algorithm to solve the problem.
- Determine what type of data is needed.
- Write an algorithm using pseudocode and/or a flowchart.

3. Implementation:

- At this stage translate the algorithm into code.
- This includes choosing the appropriate programming language to writing the code.
- Implement the algorithms in programming language.

4.Testing and Debugging:

- Test the program on a selected set of problem instances to ensure accuracy.

1.7. Exercise:

1.What is a computer?

2.Define: 1) Program 2) Software 3)Hardware
4)ALU 5)CPU 6)Data

3. Differentiate between the software, data, hardware.
4. List the components of computer hardware.
5. Write about the OS functions.
6. What is software ? Write a note on Application software.
7. Write the difference between Bit and Grouped bit.
8. Write a note about process of computational problem solving.
9. Define Firmware.
10. What is radix? Mention its types.
11. Define the benefits of computer hardware.
12. What does the term 'algorithm' mean?

****

BOOK 2:
PYHTON
BY
DESIGN

2.1 On Python: What is Python?

Python is a general-purpose programming language that is easy to learn and use. It is used in a wide variety of applications, including web development, data science, machine learning, and more.

2.1.1 History of Python

Guido van Rossum

The programming language Python was conceived in the late 1980s, and its implementation was started in December 1989 by Guido van Rossum at CWI in the Netherlands.

First released in 1991 by Guido van Rossum, Python was designed with the philosophy that code should be easy to read and understand, allowing developers to focus on problem-solving rather than syntax complexities.

Python's syntax is often described as "executable pseudocode," meaning it reads almost like plain English, making it an excellent choice for both beginners and experts.

2.1.2 Features of Python

Below are mentioned some of notable features of python:

1. ***Easy to use and learn:*** Python has a simple syntax that is easy to learn.
2. ***Multi-Paradigm*:** Python supports multiple programming paradigms.
3. ***Open Source*:** Python is an open-source language. It can be freely modified, used, and distributed.
4. ***Large Standard Library:*** Python comes with a comprehensive standard library.
5. ***Portable:*** Python code can be run on different operating systems and platforms, making it a highly portable language.
6. ***Interpreted:*** Python is an interpreted language executed line by line by an interpreter. This feature makes it easier to debug and test code.

Python Logo

7. ***Cross-Platform*:** Python can run on various operating systems like Windows, Linux, and macOS, making it a highly portable language.
8. ***Object-Oriented Programming Language*:** Python is a powerful object-oriented programming language that supports OOPs concepts like inheritance, polymorphism, and encapsulation.
9. ***Dynamically Typed***: Python is a dynamically typed language. The developers do not have to worry about type errors and can focus on the code's functionality.
10. ***GUI Programming Support*:** Python provides support for Graphical User Interface (GUI) programming.

2.1.3. Diverse Applications of Python

- Web and Internet Development (**Django** and **Flask**)
- Desktop GUI Application (**Tkinter, OpenGL , QT**)
- Software Development (**SCons, Buildbot and Apache Gump**)
- AI and Machine Learning (**scikit-learn or TensorFlow**)
- Database access (**PyMySQL, PyMongo**)
- Network Programming (**Simple Client-Server Chat**)
- Games and 3D Graphics (**PyGame and PyOpenGL**)

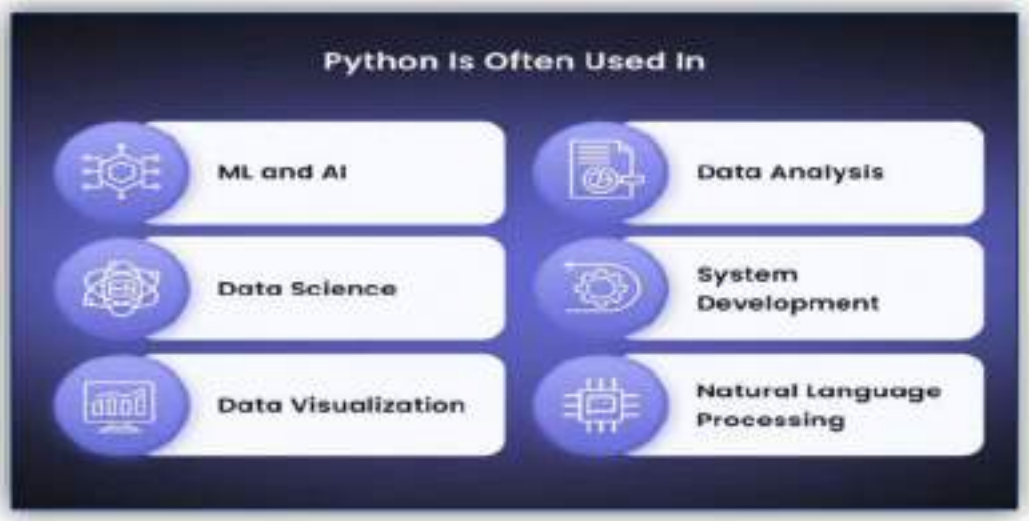
2.2 Python vs Other programming languages:

Python enables programs to be written compactly and readably. Programs written in Python are typically much shorter than equivalent C, C++, or Java programs, for several reasons:

- the high-level data types allow you to express complex operations in a single statement;
- statement grouping is done by indentation instead of beginning and ending brackets;
- no variable or argument declarations are necessary.
- the program in python are easily readable and understandable.

Python is a widely used programming language known for its simplicity, readability, and flexibility. Unlike many traditional programming languages, Python allows developers to write clean and concise code, making it a preferred choice for beginners and professionals alike.

Other languages like Java, C++, and JavaScript excel in their own domains, Python's ability to handle multiple use cases with minimal syntax makes it a top choice for developers worldwide.



2.3 Imperative programming technique:

Python IDLE (Integrated Development and Learning Environment) is a lightweight, user-friendly environment for writing, testing, and debugging Python code. It is pre-installed with Python, making it the default IDE (Integrated Development Environment) for Python programmers. IDLE has two main window types, the *Shell window* and the *Editor window*. It is possible to have multiple editor windows simultaneously.

2.3.1. Interactive Python Shell (REPL):

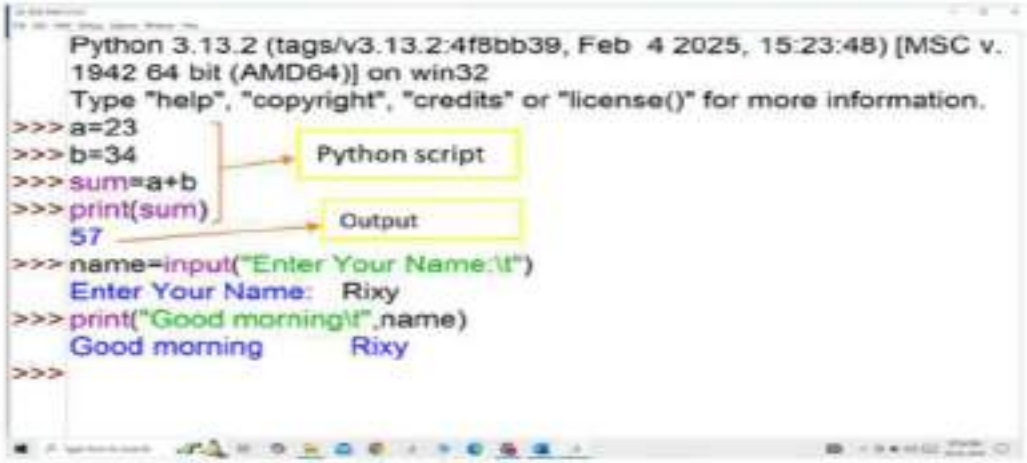
IDLE opens with an **interactive shell (>>>),** allowing users to execute Python commands line by line and see immediate results by pressing enter button.

- This is similar to running Python in the command line but with added features like syntax highlighting and auto-indentation.
- ***Interactive Python shell*** is more suitable for one-time tasks or exploration of data.
- ***Interactive Python shell*** is where you to type your code into the python interpreter directly. This is useful for trying out small snippets of code, or for testing things out as you're writing them.

How Interactive shell works ?

Step 1: Open the Python Interpreter

- Command prompt will present then type your python script after the >>> prompt line by line .
- Press *Enter* to execute the script.
- The interactive shell allows testing small code snippets quickly.
- Python immediately runs the command and displays the output.



2.3.2. Script Editor:

The Python Script Editor is a tool that allows you to write, edit, save, and run Python programs as files. Unlike the interactive mode, where you type and execute one line at a time, the script editor lets you write complete programs, save them, and run them whenever needed. This is useful for creating reusable programs instead of entering commands one by one in the interactive shell.

- IDLE includes a built-in text editor for writing and saving Python programs (.py files).
- It supports multiple tabs, allowing users to work on several scripts at once.
- Once the scripts is created, it can be executed again and again without retyping. The scripts are editable.

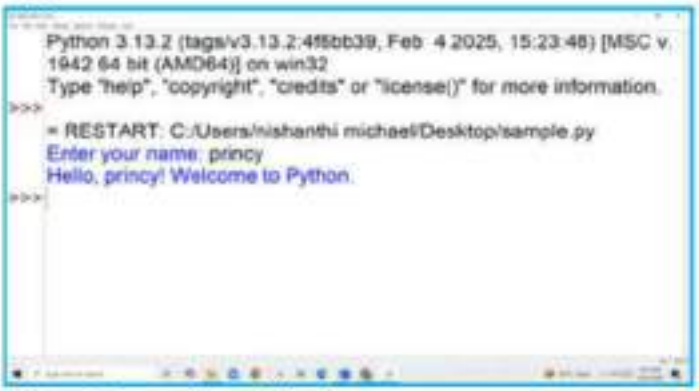
How to run python scripts in Python scripts editor?

Step 1: Create a New File – Click on "File" → "New File" (or press Ctrl + N).

Step 2: Type your **Python script** in the new window.

Step 3: Save the File – Click on **"File" → "Save As"**, then choose a location and give the file a name (with **.py** extension).

Step 4: Run the File – from menu bar Click **"Run" → "Run Module"** (or press **F5**). The result will be shown in python shell.



2.3.3. Setting Up python Tool:

Setting up a Python tool involves installing the Python programming language, ensuring compatibility with the necessary version, and configuring the environment to support the tool's functionality. This typically includes setting up a virtual environment to isolate dependencies, installing required libraries or frameworks using a package manager like pip, and verifying the installation with test scripts or commands.

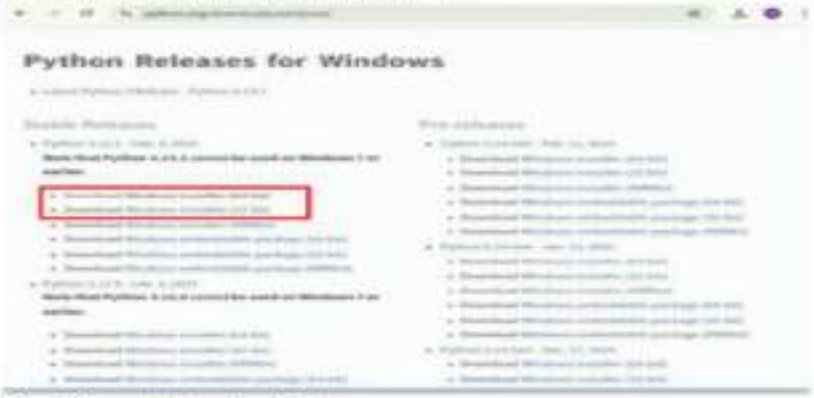
In Windows:

Step 1: Select the python version

- Open your browser and navigate to the official website of **https://www.python.org/downloads/windows/** and select the python version

Step 2: Download Python Executable Installer

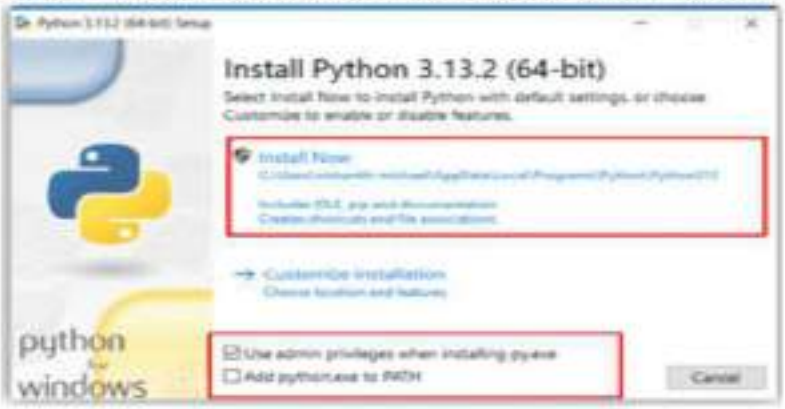
- Scroll to the bottom and select either Windows installer (64-bit) or Windows installer (32-bit).
- Locate the desired Python version.



Step 3: Run Executable Installer

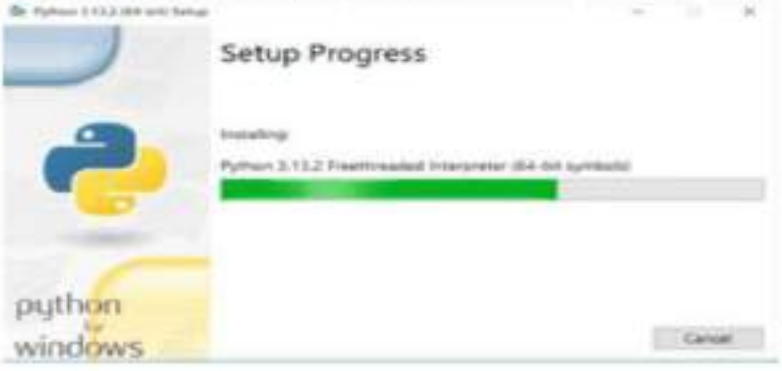
1. Once you've chosen and downloaded an installer, run it by double-clicking on the file. An installation wizard like the one below will appear on your screen:
2. The Installation window shows two checkboxes:

- *Admin privileges* The parameter controls whether to install python for the current or all system users. This option allows you to change the installation folder for python.
- *Add Python to Path* The second option places the executable in the PATH variable after installation. You can also add to the PATH environment variable manually later.



- For the most straightforward installation, we recommend ticking both checkboxes.

3. Select the Install Now option for the recommended installation. Once you have selected the option the installation process will start into your device .



- The default installation installs Python to *C:\Users\[user]\AppData\Local\Programs\Python\Python[version]* for the current user. It includes IDLE (the default Python editor), the PIP package manager, and additional documentation. The installer also creates necessary shortcuts and file associations.

Step 4: Verify python was installed on windows

- The first way to verify that Python was installed successfully is through the command line. Open the command prompt and run the following command.

```
Python --version
```

- The output shows the installed Python version.

Step 5: Verify PIP Was Installed

1. To verify whether PIP was installed, enter the following command in the command prompt:
2. If it was installed successfully, you should see the PIP version number, the executable path,

```
Pip --version
```

and the Python version.

3. To verify the pip library package module (like numpy, matplotlib, flask, pandas, etc.), enter the following command in the command prompt:

```
pip list
```

4. If all the library modules are installed successfully means, you can see the modules as listed

5. If library modules are not installed means, you can type the following command line to install them into pip library.

6. After successful installation, check the packages using pip list.

```
Pip install numpy
Pip install pandas
Pip install flask
Pip install matplotlib
Pip install seaborn
```

Or

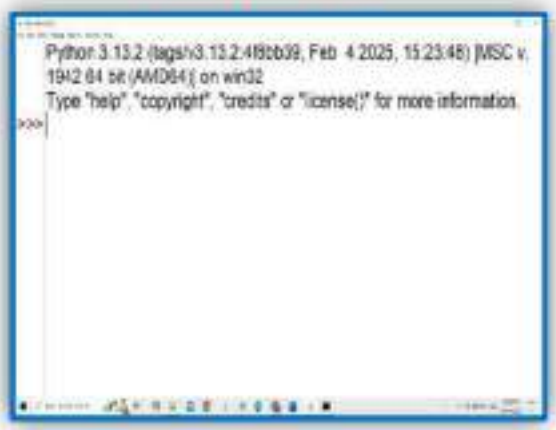
All at once
pip install numpy matplotlib seaborn pandas flask

2.3.4. Python IDLE:

The second way is to use the GUI to verify the Python installation.

- Navigate to the directory where Python was installed on the system .
- Double-click *python.exe* (the Python interpreter) or IDLE.

Python's built-in IDE



2.3.4.1. What is IDLE ?

Python IDLE stands for Integrated Development and Learning Environment. It is a bundle of software tool for program development.

2.3.4.2. The primary features of python IDLE:

The Python Standard Library is a collection of modules, each providing specific functionality beyond what is included in the core part of Python.

1. **Interactive Shell:** IDLE provides an interactive Python shell that allows users to execute Python commands and see the results immediately.
2. **File Editor:** A built-in file editor for writing and editing Python scripts with features like syntax highlighting and auto-completion.
3. **Debugger:** A basic debugger with support for setting breakpoints and stepping through code.
4. **Multi-window Text Editor:** Supports multiple windows for editing code, allowing you to work on multiple scripts simultaneously.
5. **Integrated Documentation:** Quick access to Python documentation within the environment.

6. **Configuration Options:** Customize the appearance and behaviour of the editor to suit your preferences

2.4. Python Character set:

Python program is a sequence of characters. When these characters are submitted to the python interpreter, they are interpreted in various contexts as characters, name/identifiers, constant and statements. Python uses the character set to declare identifiers as given below:

- **Letters:** Both lowercase(a, b, c, d,….,etc)and uppercase(A,B,C,D,….,etc).
- **Digits:** 0 – 9
- **Special symbols:** _ , (,),[,], {,},+, -, *, /, %, &,$,<,>,=,dot(.),colon(☹),semi-colon(;), single quote(‘), double quote (“),back slash (\)
- **White space :** (‘\t\n\x0b\x0c\r’), Space, tab(→),New line (↵)

2.5. What is python Identifiers?

Identifiers in Python is a names used to identify a variable, function, class, module, or other objects. An identifier can only contain letters, digits, and underscores, and cannot start with a digit. Python is case sensitive and hence uppercase and lowercase letters are considered distinct.

The following rules are for naming an identifier in python.

- Identifiers can be a combination of letters in lowercase(a to z) or uppercase (A to Z) or digit(0 to 9) or an underscore(_). For example Price and price is different.
- Reserved keywords cannot be used as an identifier.
- Identifiers cannot begin with a digit. For example 2times,3apple etc are invalid identifiers.
- Special symbol like @,!,#,$,%,^,&,* etc. cannot be used in an identifier.

For example

- sum@, $total, &avg are invalid identifiers.
- (a/b),(a+b) are invalid identifiers because they were use the special symbols /,+ .

Some examples for valid identifiers are total, Total, max_avg, count_2,

2.6. Keywords:

Python keywords are reserved words used for specific purposes that have specific meanings and functions, and these keywords cannot be used for anything other than their specific use. These keywords are prevent the user or the programmer from using it as an identifier in a program.

To retrieve the keywords in python environment the following script can be given at prompt…

Example:

```
>>> import keywords
>>>print(keywords.kwlist)
```

Note: There are different count of keyword in python environment. They are

- 33 keywords in python 3.3
- 35 keywords in python 3.11
- 36 keywords in python 3.13

Output:

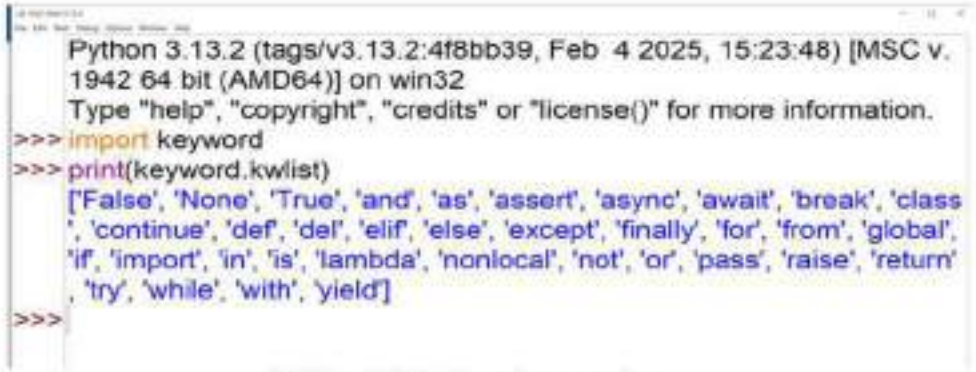


Table 2.6 Python keywords

False	**await**	**else**	**import**	**pass**
None	**break**	**except**	**in**	**raise**
True	**class**	**finally**	**is**	**return**
and	**continue**	**for**	**lambda**	**try**
as	**def**	**from**	**nonlocal**	**while**
assert	**del**	**global**	**not**	**with**
async	**elif**	**if**	**or**	**yield**

2.7. Python variables :

In programming, **variables are fundamental building blocks** that allow us to store, manipulate, and reuse data in our code. They are essential for creating dynamic and flexible programs.

The process of creating variables with data is referred to as "assignment." A variable assignment usually consists of a name, followed by an equal sign (=), and the value assigned by the programmer. An immutable value is a value that cannot be changed.

- The process of creating a variable to store a value is called Declaration and
- The process of setting its value for the first time is called Initialization.

The following guidelines for variable names:

• A variable name must start with a letter or the underscore character

• A variable name cannot start with a number

• A variable name can only contain alpha-numeric characters and underscores A-z, 0-9, and _

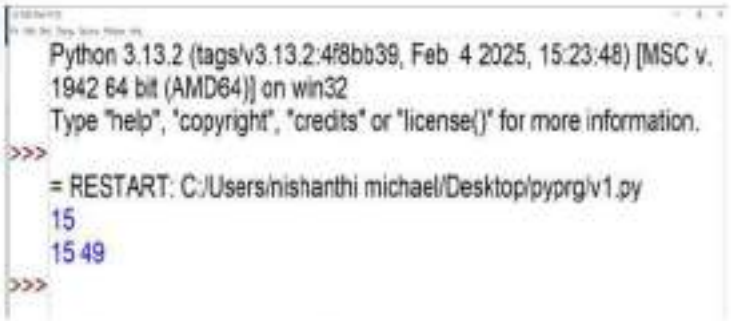
• Variable names are case-sensitive (age, Age, and AGE are three different variables)

For example , the code

```
>>> num_1 = 4
>>> num_2 = 6
>>> mul = num_1 * num_2
>>> add = mul + 45
>>> print(mul)
>>> print(mul,end=' ')
>>> print(add)
```

In the example code, there are four variables. Two of them are assigned values directly, and the third is assigned the value of the first two variable multiplied together, and the fourth variable is assigned the value of third variable .

Output:



Practice Quiz:

1. Create two variables and assign the values and print the value of difference, division.

2.8. Python comment lines :

Comments are for programmers for better understanding of a program. These are the statements that are ignored by the python interpreter and not executed at all. In python ,we use the hash **(#)** symbol to start writing a comment. Comments in Python can be added using either a single line or multiple lines.

For example:

#1

>>>print("Hello") **# Display Hello**

Output:

Hello

#2

```
>>>print("Hi")    '''This is an example of multiline
                    comment from python'''
>>>print("Welcome to coding")
```

After ignoring the comments, the output will be:

Hi

Welcome to coding

Key: The hash(#) symbol is for single line comment and by using triple quotes(''' ''') or(""" """) the comments spanning across multiple lines in python code.

2.9. Indentation: A Key Part of Python Syntax

Python Indentation is the whitespace (space or tab) before a block of code. It is the cornerstone of any Python syntax and is not just a convention but a requirement. Indentation is significant in Python as it ensures code readability.

Unlike C, C++, and other languages, where curly braces { } represent a block of code, Python uses indentation level (number of leading whitespaces) to show a black with the same group of statements. We have to increase the indentation in the if statement so that the code will be grouped together. Let's look at an example of indentation in Python for if statement.

The code is

```
a=23
b=12
if a==b:
  print("equal")
else:
  print("not equal")
```

Output is

not equal

2.9.1. Indentation Errors:

Let's look at some examples of the indentation error in Python.

Example 1 : Python shell snippet to show the Indentation Error

```
>>> x = 10
  File "<stdin>", line 1
  x = 10
  ^
IndentationError: unexpected indent
>>>
```

Example 2: Python script mode code to show the indentation error.

```
if x > 0:
   print("Positive number")
else:
print("Non-positive number")
```

Output:

We can't have an indentation in the else part print function. That's why the Python interpreter throws IndentationError.

2.10.The Role of input() and print() in Python Programming:

In Python, program output is displayed using the print( ) function, while user input is obtained with the input( ) function. Python treats all input as strings by default, requiring explicit conversion for other data types.

2.10.1.The print( ) Function:

We use the widely used print( ) statement to display some data on the screen.The function used to print output on a screen is the print statement where you can pass zero or more expressions seperated by commas. The print function converts the expression you pass into a string and writes the result to standard output.

Syntax for print( ):

```
>>>print("string to be displayed as output")
>>>print("String to be displayed as output by variable:", variable)
>>>print(variable)
>>>print("string1",variable,"string2",variable,"string",variable,....)
```

Example 1:

 # 1. Printing the simple text.

```
>>> print("welcome to the Adventure")
```

welcome to the Adventure

 # 2. printing the variable value.

```
>>> a=34
>>> print("The value of a is :",a)
```

The value of a is :3

3.Printing the multiple string and variable

```
>>>name="sweety"
>>>m1=67
>>>m2=87
>>>res="pass"
>>>print("Name:",name,"Mark_1:",m1,"Mark_2:",m2,"Result:",res)
```

Name:sweety Mark_1:67 Mark_2:87 Result:pass

Example 2: Python print() with end Parameter

The end parameter in the print() function specifies what to print at the end of the output. By default, it is a newline character (\n), but you can set it to a different string.

 # 1. Print with end parameter as space

```
Print("login to check the progress,",end=' ')
Print("login  confirmed")
```

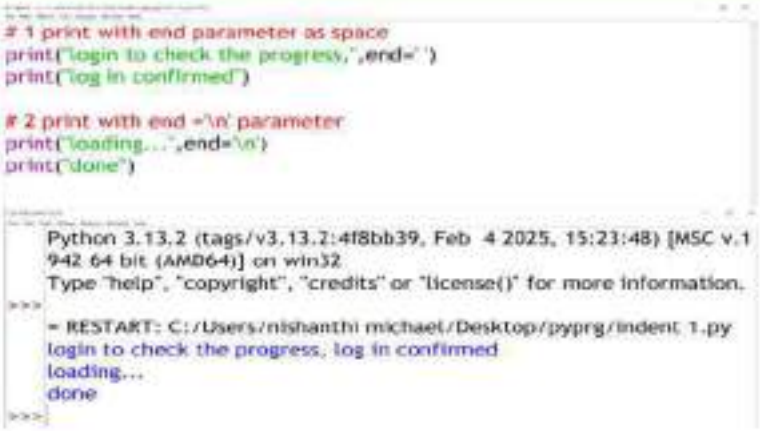
 # 2. Print with end='\n' parameter

```
Print("loading…",end='\n')
Print("done")
```

Output :

login to check the progress, login confirmed
loading…
done

Note: *Print ( )is flexible and can print strings, numbers,variables,etc.*
By default, it seperates items with a space (sep=' ') and ends with a newline (end='\n')



In the above example, the end=' ' parameter replaces the newline with a space, so the two print statements output on the same line

Example 3: Python print() with sep parameter

The sep parameter defines the separator between multiple items passed to the print() function. By default, it is a space.

 # 1. Print with a custom seperator'/'

```
Print("janu","Meena","Gugan","Ritu",sep='/')
```

Output:

janu/Meena/Gugan/Ritu

 # 2. Print with a custom seperator '+'

```
a=10
b=34
c=12
print(a,b,c,sep='+')
```

output:

10+34+1

 # 3. Print with a custom seperator % in list

```
sub=['tamil','english','maths','science','social']
sep='%'
res=sep.join(sub)
print(res)
```

Output:

tamil%english%maths%science%social

Note: join( ) is the most efficient way to join elements of a list with a seperator.

As shown above, the third example utilizes the join() function.Sep.join(sub)joins all elements of the list sub using the seperator sep.

Example 4: python Print( ) Concatenated Strings

We can also join two strings together inside the print() statement by using (+) operator.

#1. Using + to concatenate string in python in single command line.

```
print('Hey siri,' + '\t play a song')
```

Output:

Heysiri, play a song

This line prints the string **"Hey siri,"** followed by a **tab space** (due to \t) and then **"play a song"**, all on the same line.

#2. Concatenate the string variables by using + symbol:

```
first_name='Michael'
second_name='manojan'
print(first_name+second_name)
```

Output:

Michaelmanojan

This code defines two string variables: first_name and second_name, and then prints their **concatenation** (joining them with no space in between).

2.10.2 The Input( ) Function:

While programming, we might want to take the input from the user. In Python, we can use the default built-in input() function.

Syntax:

variable=input('prompt')

Here, prompt is the string we want to display while taking input from the user. This is optional.

Note: raw_input('prompt') is another built-in input function. This was a Python 2-only feature and is no longer available in Python 3.

Example 1: To get the input from the user

```
num=input("Enter your mark:")
print("Your mark is:", num)
```

Output:

Enter your mark:67

Your mark is:67

Example 2: To find the datatype of user input

 # 1. display the user input type:

```
a=input("enter the name:")
print("data type of a:",type(a))
```

Output:

enter the name: sabitha

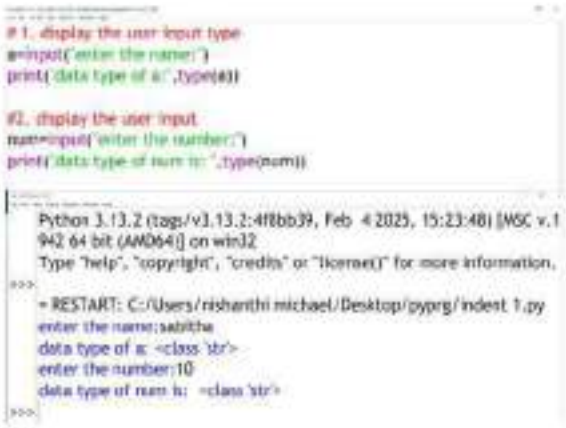
data type of a: <class 'str'>

#2. display the user input:

```
num=input("enter the number:")
print("data type of num is: ",type(num))
```

Output:

```
enter the number:10
data type of num is:  <class 'str'>
```



In the above second example, the input looks like a number, but Python treats it as a string. Even if a number is typed by the user, Python read it as a string. That's why the **type()** function shows it as str. To change it into a number, use **int()**.

 # 3. Taking integer inputs from users:

```
x=int(input("enter the value for x :"))
print("data type is :",type(x))
```

Output:

```
Enter the value of  x:56
    <class 'int'>
```

This program 3 takes user input using input(), converts it to an integer using int(), and stores it in the variable x. Then, it prints the **data type** of x using the type() function.

 # 4. Taking float inputs from users:

```
side=float(input("enter the side of square:"))
res=side*side
print(type(side))
print(res)
```

Output:

```
enter the side of square:12
    <class 'float'>
    144.0
```

Note: float() works like int(), but it's meant for decimal or fractional numbers.

Python's ***split()*** method allows users to enter multiple values in a single input line by breaking the string into a list of individual elements, using whitespace as the default separator. These elements can then be converted into specific data types like integers or floats for further processing.

Example 3: Taking multiple inputs

 # 1. *Taking multiple string inputs*

```
x= input("Enter three fruits name: ").split()
print(f"Fruits: {x}")
```

Output:

```
Enter three fruits name: Apple orange banana
Fruits: ['Apple', 'orange', 'banana']
```

The program prompts the user to enter three fruit names in a single line, separated by spaces. It uses the split() method to divide the input string into a list of individual fruit names, which are stored in the variable x. This allows the program to treat each fruit name as a separate element in the list. Finally, it prints the entire list using an f-string, displaying the fruits entered by the user in list format.

2.11. Expression:

A combination of operands and operators is called an **expression**. An expression in python is any valid combination of operators, literals and variables.

The expression type in python are

- Arithmetic Expression
- String Expression
- Relational Expression
- Logical Expression
- Compound Expression etc.

EXPRESSION: An expression in python is any valid combination of operators and atoms. An expression is composed of one or more operation.

2.11.1. Arithmetic Expression:

In arithmetic expression, we perform the mathematical calculation using arithmetic operators like +, -, *, /, etc. Arithmetic expression involve numbers(integers,floating point,complex numbers)

Examples

#1. Basic arithmetic expression

```
2+5**4,-8*6/5
```

 # 2. Program to calculate the sum of two numbers with variables.

```
num1=34
num2=56
sum=num1+num2
print("The value of sum is :",sum)
```

Arithmetic expression

Output:

```
The value of sum is : 90
```

2.11.2. String Expression:

Python also provides two string operators + and *, when combined with string operands and integers, from string expressions.

Examples:

 # 1. String expression for concatenation

```
>>> "Hello"+"Python"
```

Output:

```
'HelloPython'
```

#2. String expression for replication with variable

```
a="hello"
b="joe \t"
print((a+b)*2)
```

Output:

```
hellojoe he
```

2.11.3. Relational Expression:

An expression having literals and/or variables of any valid type and relational operators is a **relational expression.** The overall expression results in either True or False (boolean result). We have **four** types of relational operators in Python (***i.e.***>,<,>=,<=).

Example:

```
a=23
b=12
c=15
d=15
res_1=(a*b)==(c*b)
res_2=(a+d)>=(b+c)
print("type:",type(res_1))
print("The first result is:",res_1)
print("The second result is:",res_2)
```

Relational Expression

Output:

```
type: <class 'bool'>
The first result is: False
The second result is: True
```

2.11.4. Logical Expression:

An expression having literals and/or variables of any valid type and logical operators is a logical expression. We have three types of logical expressions in Python, they are

Operator	Syntax	Working
and	*x* and y	The expression return True if both x and y are true, else it returns False.
or	*x* or yy	The expression return True if at least one of x or y is True.
not	not x	The expression returns True if the condition of x is False.

Example:

```
a = 10
b = 5
c = 20
print((a > b) and (c > a))
print((a < b) or (c == 20))
print(not (a == c))
```

Output:

```
True
True
True
```

2.11.5. Compound Expression:

A compound expression is an expression that includes multiple operators and sub-expressions, which Python evaluates from left to right(**Parentheses** are evaluated first).

For example

 # 1. Execute arithmetic steps and comparisons to determine the answer.

```
>>>print ((3+4) * 4>13)
>>> print ("The value is:",(6+24) / 8)
```

Output:

```
True
The value is: 3.75
```

 # 2. Use the arithmetic, logic and comparison steps to find the answer.

```
a = 10
b = 5
c = 20
print((a+c)*3<15 and b!=(a/b))
```

Output:

```
False
```

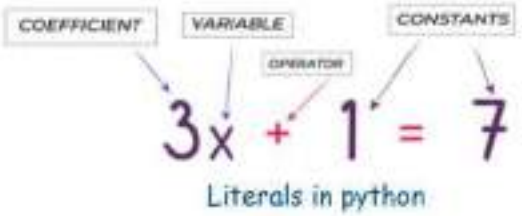
Calculation Process:

- ((a+c)*3<15 and b!=(a/b))
- (30*3<15 and 5!=2)
- 90<15 and TRUE
- FALSE and TRUE
- FALSE

2.12. Literals:

Python literals or constants are the notation for representing a fixed value in source code. They provide a way to store numbers, text, or other essential information that does not change during program execution.

Note: Literals is a raw data given to variable or constant.



Literals in python

Python provides following literals …

- Numeric literals
- String literals
- Boolean literals
- Collection literals
- Special literals

2.12.1.Numerical Literals:

Python numeric literals are immutable and contain digits.Numeric literals can belong to three different numerical types ***Integer,Float and Complex.***

i. Integer Literals:

Integer literals include positive and negative numbers, including zero. However, there shouldn't be any decimal or fractional parts.

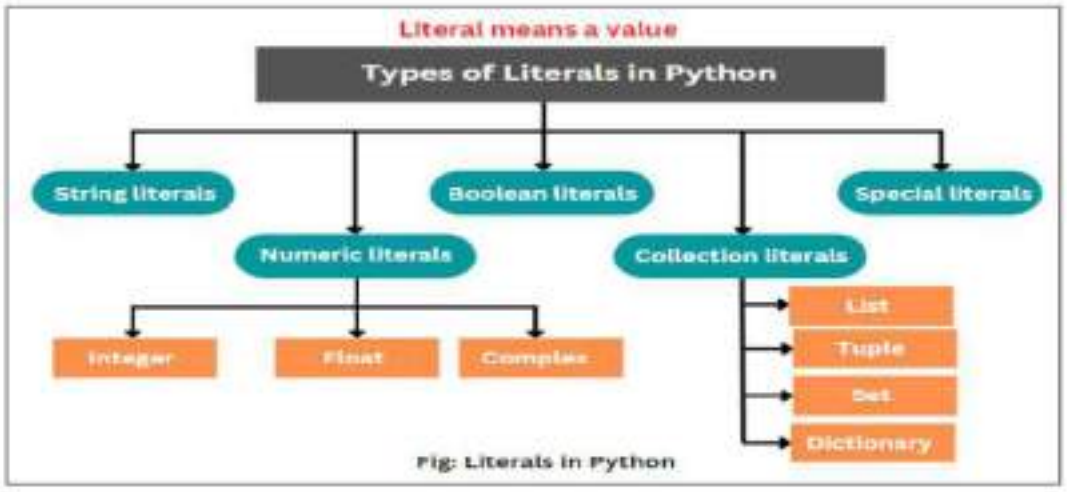


Fig: Literals in Python

There are different types of integer literals:

- Decimal literal (base 10) - starts with regular number
- Boolean literal (base 2)- starts with 0b or 0B
- Octal literal (base 8) - starts with 0o or 0O
- Hexadecimal litreal (base 16) – starts with 0x or 0X

Example: Python program to display the Numerical Literals.

```
p=45          #decimal integer
q=0o62        #octal integer
r=0b101110    # binary integer
z=0x53        #hexadecimal integer
print("\t decimal integer is:",p)
print("\t octal integer is:",q)
print("\t binary integer is:",r)
print("\t hexadecimal integer is:",z)
```

Output:

```
decimal integer is: 45
octal integer is: 50
binary integer is: 46
hexadecimal integer is: 83
```

ii. Float literal:

A floating-point literal is a real number that contains a decimal point (.) or exponent sign or both, differentiating them from integer literals. We can express them in either standard or scientific notation.

Valid floating literals are: 12.4,1.5e2, -10.5, -1.3E2

The letter E or e separates the mantissa and exponential portion of a number.

- ✓ Mantissa is the digit before E in an exponential literal. While computing, this part represents significant digits of a floating-point number.
- ✓ Exponent is the digit after E in an exponential literal. It represents where to place the decimal point.

Example:

```
x=10.25
y=1.6e2
print("Float literals: ",x,"\t",y)
```

Output:

```
Float literals:  10.25      160.0
```

iii. Complex literal:

A complex number comprises two floating-point values, one each for the real and imaginary.

A complex literal is represented by (a + bj).

Here, a is a real part and the b with j is the imaginary or complex part. The letter j represents the square root of -1. In mathematics, we use the letter i.

Example:

```
a = 5 + 10j
b = 2j
print(a)
print(b)
print(a,"imaginary part of a is =",a.imag,"real part of a is =",a.real)
```

Output:

```
(5+10j)
2j
(5+10j) imaginary part of a is = 10.0 real part of a is = 5.0
```

2.12.2. String literal:

A string literal in Python is a consecutive sequence of characters used to store and represent messages, heading, and other text-based information. We can easily create a string literal in Python by enclosing the text or group of characters in single (' '), double (" "), or triple (''' ''' or """ """) quotes. We use triple quotes to write multi-line strings or represent them in the required way.

Example:

```
# 1.Single quote message
```

```
msg='Hi ,how are you'
print(msg)
# 2. double quote message
msg_1="Hi,i'm python"
print(msg_1)
#3. multi line messages can create in two ways
# 1.
msg_2="I\
am a\
robo\
boy"
print(msg_2)
# 2.
msg_3="""This is
a multiline
string
literal example"""
print(msg_3)
```

Output:

```
Hi ,how are you
Hi,i'm python
Iam aroboboy
This is
a multiline
string
```

In strings, you can include non graphic characters theough escape sequences. It is used in representing certain white spaces.
Escape sequences are given below:

Escape sequence	What it does [Non-graphic character]	Escape sequence	What it does [Non-graphic character]
\\	Backslash (\)	\r	Carriage Return (CR)
\'	Single quote (')	\t	Horizontal Tab (TAB)
\"	Double quote (")	\uxxxx	Character with 16-bit hex value xxxx (Unicode only)
\a	ASCII Bell (BEL)	\Uxxxxxxxx	Character with 32-bit hex value xxxxxxxx (Unicode only)
\b	ASCII Backspace (BS)	\v	ASCII Vertical Tab (VT)
\f	ASCII Formfeed (FF)	\ooo	Character with octal value ooo
\n	New line character	\xhh	Character with hex value hh
\N {name}	Character named name in the Unicode database (Unicode only)		

2.12.3. Special literals:

Python has one special literal known as None, which specifies the field that isn't created. Python special literals are used to define a null variable. When we print a variable without assigning it a value, it returns None as output. None is often used as a **default value** for variables, function arguments, or return types when no actual value is yet assigned or returned.

Example:

```
a = None
print(a)
```

Output:

```
None
```

This program assigns the special Python constant None to the variable a. The None keyword represents the absence of a value or a null value in Python. When print(a) is called, it outputs None, indicating that a currently holds no usable data or value.

2.12.4. Boolean Literals:

A boolean literal is a logical value that can have any of the two values: True or False. True represents the value 1 whereas False represents the value 0.

Example:

```
a = (1 == True)
b = (1 == False)
c = True + 4
d = False + 7
x=c
z=d
if x is c:
  print(x)
else:
  print(d)
print("a is", a)
print("b is", b)
print("c:", c)
print("d:", d)
```

Output:

```
5
a is True
b is False
c: 5
d: 7
```

2.12.5. Collection literals:

Collection literals in Python are used when working with more than one value, allowing to store collections of data in the source code. There are four types of literal collections:

i. List literals: A list is a collection of mutable items of different data types declared using square brackets [], and values are separated using commas.

Example:

```
name="srinivasan"
address=['no:4','2nd street','wcc road','chennai']
print("person detail is:",name,address)
```

Output:

```
person detail is: srinivasan ['no:4', '2nd street', 'wcc road', 'chennai']
```

ii.Tuple literals: A python tuple is a collection of immutable items, so we can't change or modify them once created. Tuples using paraenthesis( ) and separate elements by commas.

Example:

```
num=(2,5,6,7)
x=('h','e','l','l','o')
print(x)
if 6 in num:
    print(num)
else:
    print("false")
```

Output:

```
('h', 'e', 'l', 'l', 'o')
(2, 5, 6, 7)
```

iii.Dictionary literals: A Python dictionary literal stores data in ***key-value format***. It is enclosed by curly braces {}, and each pair is separated using commas. A dictionary is mutable and allows us to store different types of data.

Example 1: Python program to display a dictionary literals keys with values.

```
my_dict = {'b': 'bear', 'd': 'dear', 'f': 'fear'}
print(my_dict)
```

Output:

```
{'b': 'bear', 'd': 'dear', 'f': 'fear'}
```

iv. Set literals: A Python set is a collection of unordered data sets that we can't changeor modify. It is enclosed within curly braces {}, and every element is separated by a comma.

Example 2: Python program to display the set literals.

```
fruits = {'apple', 'cherry', 'melon', 'orange'}
print(fruits)
```

Output:

```
{'orange', 'cherry', 'apple', 'melon'}
```

2.13. Python Operators:

Operators are the constructs which can manipulate the value of operands.

For example:

a=b+c

In the above expression a,b and c are called the operands and +, = are called the operators. Those operators that work with only one operand are called *unary operator*. Those which work with two operands are called *binary operators*.

There are different type of operators. They are

- **Arithmetic operators**
- **Comparision (relational) operators**
- **Assignment operators**
- **Logical operators**
- **Bitwise operators**
- **Membership operators**
- **Identity operators**

2.13.1. Arithmetic operators:

Aritmetic operators is a mathematical operator that takes two operands and performs a calculation on them. They used for performing basic arithmetic operations.

The arithmetic operators are

Operators	Examples
+(Addtion)	**>>>a+b**
- (Subtraction)	**>>>a-b**
*** (Multiplication)**	**>>>a*b**
/ (Division)	**>>>a / b**
% (Modulus)	**>>>a%b**
**** (Exponent)**	**>>>a**b**
// (Floor Division)	**>>>a//b (Integer Devision)**

Example:

```
a,b,c=10,12,5
print("The values of a,b,c is:",a,b,c)
print("the sum=",(a+b))
print("The difference=",(b-c))
print("The Quotient=",(a/c))
print("The Reminder=",(b%c))
print("The Exponent=",(b**c))
print("The Floor Division=",(b//c))
```

Output:

```
The values of a,b,c is: 10 12 5
the sum= 22
The difference= 7
The Quotient= 2.0
The Reminder= 2
The Exponent= 248832
The Floor Division= 2
```

For practise:

1. write a program to print the sum and integer devision of 56 and 34 by using variables:

Solution:

```
x=56
y=34
print("The Sum is: ",x+y)
print("The Integer Devision is:",x//y)
```

Output:

```
The Sum is: 90
The Integer Devision is: 1
```

2.Write a program to print the area of rectangle and square.

Solution:

```
#Rectangle
length=int(input("Enter the length of rectangle:"))
breadth=int(input("Enter the breadth of rectangle:"))
area=length*breadth
print("Area of the rectangle is:",area)
#Square
side=int(input("enter the side of square :"))
s_area=side*side
print("Area of square=",s_area)
```

output:

```
Enter the length of rectangle:12
Enter the breadth of rectangle:14
Area of the rectangle is: 168
enter the side of square :16
Area of square= 256
```

Practice Quize:

1.Type the following code and see the result

```
a,b=12,5
add=a+b
sub=a-b
mul=a*b
divi=a/b
power=a**b
print("Addition=",add,"\nsubtraction=",sub,"\nmultiplication=",mul,"\ndivision=",divi,"\n
      power=",power)
```

2. Write a program to take marks as integers and show the total and average.

3. Write a program to input two values and print the modulus values in another variable.

4. Write a python code to find the result of following expression 10 + 5 * 2 - 6 / 3.

2.13.2. Comparision (Relational) Operators:

These operators compare the values on either sides of them and decide the relation among them.They are also called relational operators. Python supports the following relational operators.

Operators	Description
= = (is equal)	If the value of tw operands are equal,then the condition become true.
!= (Not equal to)	If values of two operands are not equal, then condition becomes true.
> (Greater than)	If the value of the left operand is greater than the right operand value ,then condition become true.
<(Lessthan than)	If the value of the left operand is less than the right operand value, then condition become true.
>= (Greater than or equal to)	If the value of left operand is greater than or equal to the value of right operand, then condition becomes true.
<=(Less than or Equal to)	If the value of left operand is less than or equal to the value of right operand, then condition becomes true.

Examples: # 1. Relatinonal operators with integer and string values.

```
>>>x=2;y=2;x==y
True
>>>x='str';y='sTR';x==y
False
>>>x=3;y=5;x!=y
True
>>>x=5;y=4;x>=y
True
>>>x='hello';y='greetings';x<=y
False
```

#2. Relational operators with binary and integer values.

x	y	x==y	x>y	x<y	x>=y	x<=y	x!=y
1	0	0	1	0	1	0	1
0	1	0	0	1	0	1	1
1	1	1	0	0	1	0	0
3	2	0	1	0	1	0	1

In the above table ,the expression produces results in the form of 0's and 1's. 0 means false and 1 means true.

Practice Quiz:

1. Write a Python program to compare x = 6 and y = 4, and print the result.
2. Find the result for the following:
   P=5,q=4
   Print(p,q,(p>q)==(p>=q))
3. If a=5, b=6, c=7,d=7 , then what will be the outcome for the following:
   i) a<=b>=c
   ii) a+b==c>d
   iii) b+c==6+d>=13

2.13.3. Assignment Operator:

Python provides various assingnment operators. The assignment operator = is used to assign values of expressions to a variable. Various shorthand operators for addition, subtraction,mutiplication,division,modulus,exponent and floor division are supported by python. The python assignmet operators are mentioned below:

Operator	Description
=	Assign values from right side operands to left side operand
+=	It adds right operand to the left operand and assign the result to left operand.
-=	It subtracts right operand from the left operand and assign the result to left operand.
*=	It multiplies right operand with the left operand and assign the result to left operand.
/=	It divides left operand with the right operand and assign the result to left operand
%=	It takes modulus using two operands and assign the result to left operand.
**=	It performs exponential(power)calculation on operators and assign value to the left operand.
//=	It performs floor division on operators and assign value to the left operand.

Examples: #1.

```
>>>x=10
>>>x+=5   #x=x+5
15
>>>x/=2    #x=x/2
5
```

#2. Addition

```
a,b=12,5
a+=b
print("the addition is:",a)
```

#3. Multiplication

```
a=7
a*=4
print("The multiplication is:",a)
```

#4. floor division

```
a=15
b=3
a//=b
print("The floor division is:",a)
```

Outputs:

```
the addition is: 17
The multiplication is: 28
The floor division is: 5
```

For practice:

1. Write a program to calculate the area and perimeter of a rectangle.

Solution:

```
length=int(input("Enter length of rectangle:"))
width=int(input("enter width of the rectangle:"))
print("area of rectangle is:",length*width)
print("perimeter of rectangle is :",2*length+2*width)
```

Output:

```
Enter length of rectangle:12
enter width of the rectangle:8
area of rectangle is: 96
perimeter of rectangle is : 40
```

2. Write a python program to convert Fahrenheit to celsius value.

Solution:

```
f=float(input("Fahrenheit to celsius:"))
print("celsius temperature :",(f-32.0)*5.0/9.0)
```

Output:

```
Fahrenheit to celsius:96
celsius temperature : 35.55555555555556
```

2.13.4. Logical Operators:

Logical operators are also called a ***Boolean operators***.With boolean operators we perform logical operations.These are most often used with if and while keywords.Also,these operators are used to combine one or more relational expression. The logical operators are

Operator	Description
and (Logical and)	If both the operands are true then condition becomes true
Or (Logical or)	If any of the operands are true then condition becomes true
Not (Logical not)	Used to reverse the logical state of its operand

Example:

```
a,b,c,d=10,15,14,3
print((a>b)and(c>d))
print((a<c)or(c>d))
print(not(a>d))
```

Output:

False
True
False

For practice:

1.write the python code to print the result for following logical expression:

i) ((a>=b)and(b<=d) ii) ((a==c)or(c>d)) iii) ((a>b)and(a!=b))

Solution:

```
a=int(input("Enter the value of a:"))
b= int(input("Enter the value of b:"))
c= int(input("Enter the value of c:"))
d= int(input("Enter the value of d:"))
print("The values of a b c d is:",a,b,c,d)
print((a>=b)and(b<=d))
print((a==c)or(c>d))
print((a>b)and(not(a!=b)))
```

Output:

```
Enter the value of a:12
Enter the value of b:14
Enter the value of c:45
Enter the value of d:16
The values of a b c d is: 12 14 45 16
False
True
False
```

2.13.5 Bitwise Operators:

Bitwise operator workss on bits and performs bit by bit operation. It works with bits of binary number.the bitwise operators are

Operators	Expression	Description
& (And)	x&y	Bits that are set in both x and y are set
| (Or)	x|y	Bits that are set in either x or y are set.
^ (Xor)	x^y	Bits that are set in x or y but not both are set.
~ (Not)	~x	bits that are set in x are not set,and vice versa.
<< (shift left)	x<<y	Shift the bits of x,y steps to the left.
>> (Shift right)	x>>y	Shift the bits of x,y steps to the right.

a. Bitwise And(&) operator: Returns 1 if both bits are 1,otherwise 0.

Binary notation :

```
    1000   #8
&   1100   #12
=   0100   #4
```

Example program:

```
a=10 #binary value is 1010
b=6  #binary value is 0110
res=a&b #1010&0110 = 0010
print(res)
```

Output:

```
2
```

The variable a is assigned the value 10 (which is 1010 in binary), and b is assigned the value 6 (which is 0110 in binary). The expression a & b performs a bitwise AND operation, comparing each bit of a and b. Only the bits that are 1 in both numbers remain 1 in the result. Thus, 1010 & 0110 equals 0010, which is 2 in decimal.

For pracitce:

1. Write a Python program to find and print the result of bitwise AND for a = 60 and b = 2.

Solution:

```
a=60 #binary value 0011 1100
b=2  #binary value 0000  0010
print(a&b)
```

Output:

```
0
```

b.Bitwise Or (|) Operator: Returns 1 if at least one of the bits is 1, otherwise 0.

Binary notation :

```
    00110   #6
|   00011   #3
=   00111   #7
```

Example:

```
a = 10   # Binary value 1010
b = 6    # Binary value 0110
result = a | b   # 1010 | 0110 = 1110
print(result)
```

Output:

```
14
```

c. Bitwise Xor (^) operator: Returns 1 if the bits are different, otherwise 0.

Binary notation:

```
    001101 #13
^   001111 #15
=   000010 #2
```

For example:

```
>>>6^3
 5
>>>5^18
 23
```

d.Bitwise Shift operators(>>,<<): The shift bitwise operators shift bits to the right or left.These operators are called as arithmetic shift.

Syntax:

number<<n : multiply number 2 to the nth power

number>>n : divide number by 2 to the nth power

To use left shift, for example,

```
    00110   #6
>>  00001   #1
=   00011   #3
```

Here,we shift each of the bits of number six to the right. It is equal to dividing the six by 2. The result is 00011 or decimal 3.

In python the left and right shift is

```
#lefft shift
>>>6>>1
    3
```

```
# right shift
a = 10          # Binary value: 1010
result = a >> 2   # 1010 >> 2 = 0010
print(result)
Output: 2
```

e.Bitwise Not(~) Operator: Inverts all the bits (1 becomes 0, and 0 becomes 1). This operator also known as one's complement.

For example:

```
a = 10      # Binary: 1010
result = ~a  # ~1010 = -(1010 + 1) = -1011
print(result)
```

Output:

```
-1
```

Practice Quiz:

1.Type the following python code and print the result:

```
a,b=30,2
print(a&b)
print(a<<b)
print(a^b)
print(a|b)
```

2.Write a program to find the result of bitwise And for 15,4.

2.13.6 Membership Operator:

Python has membership operators,which test for membership in a sequence,such as strings,lists,or tuples.There are two membership operators

Operator	Description
in	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.

For example:

#1

```
>>>direction=('North','south','East','West')
>>>print("North-West" in direction)
  False
>>>print('North' in direction)
  True
```

#2

```
alp='abcdef'
word="My name is python"
print("f" in alp)
print('a' in alp)
print("my" in word)
print("is" in word)
```

Output:

```
True
True
False
True
```

2.13.7. Identity Operator:

Identity operators compare the memory locations of two objects by using id(object)function. There are two identity operators

Operator	Description
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.

For example:

```
a,b,c=10,10,4
print(a is b)
print(a is c)
print(a is not c)
```

Output:

```
True
False
True
```

The above program assigns the values ***10, 10, and 4 to the variables a, b, and c.*** It then ***uses*** the ***is and is not operators*** to compare the variables. Since both ***a and b*** have the ***same value*** 10, and Python stores small numbers like 10 in the same memory place, ***a is b gives True***. But ***a and c*** have ***different*** values (10 and 4), so a is c gives False. Finally, ***a is not c gives True*** because a and c are not the same.

2.14. Exercise:

1. What is Python?
2. Write the features of Python?
3. How IDLE is supporting to python programming?
4. What is interactive shell?
5. Write the steps to install and run the python tool.
6. What are the features of python IDLE?
7. What is python Identifier?
8. What are python keywords? List some keywords.
9. How indentation are take important role in python coding? Explain.
10. Write a brief note on input( ) and print( ) function.
11. Write the difference between Relational and Logical expression.
12. What do you mean by literals? Define string literals.
13. what is python operators? Explain any five operators.
15. Define identity operator.
16. Write a note about shift operator.
17. Write a python program to compare two variables by using membership operator.
18. Write the difference between string literals and dictionary literals.
19. What are Arithmetic Operators? What are various types of arithmetic operators that we can use in python?
20. Is a=a*2+3 same as a*=2+3? Find the solution .
21. What is an expression?
22. Write the difference between a=10 and a==10.

****

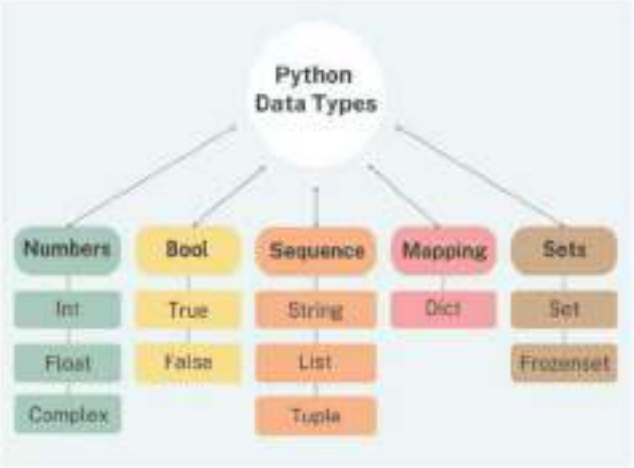
BOOK 3: PYTHON DATA TYPES

3.1. Python Data Types:

In computer programming, data types specify the type of data that can be stored inside a variable. For example,

num = 24

Here, 24 (an integer) is assigned to the num variable. So the data type of num is of the int class.



Python is a dynamically typed language. This means that the user doesn't need to specify the variable type when creating it because Python will figure this out itself.

Python has following standard data types . They are

1. ***Numbers***
2. ***String***
3. ***List***
4. ***Tuple***
5. ***Set***
6. ***Dictionary***

3.2. Data types for Numbers:

Number stores values. Python creates Number objects when a number is assigned to a variable. **For Example;**

a=3,b=5

In the above example, a and b are number objects. Python supports four types of numeric data. They are

i. int (signed integers like 10,3,5,89,etc.):
ii. long(long integers used for a higher range of values like 9080908000L, -0x1983929292L,etc.):
iii. float(float is used to store floating point numbers like1.8,2.45,6.9089,34.21,etc.)
iv. complex(complex numbers like 2.14j,2.0+2.3j,etc.1)

> Note: We can use the type() function to know which class a variable or a value belongs to.

In python, an integer (int) is a whole number. It can be positive or negative or zero. Integers can be of any length, it is only limited by the memory available.

A floating point number is accurate up to 15 decimal places. Integer and floating points are separated by decimal points.

Complex numbers are written in the form , x + yj, where x is the real part and y is the imaginary part.

For practice:

```
a,b,c,d=1,2.6,231456700,4+9j
print("a=",a)
print("b=",b)
print( "c=",c)
print("d=",d)
print(type(a))
print(type(b))
print(type(c))
print(type(d))
```

Output:

```
a= 1
b= 2.6
c= 231456700
d= (4+9j)
<class 'int'>
<class 'float'>
<class 'int'>
<class 'complex'>
```

3.2.1. Mathematical Function:

Python provides various built-in mathematical functions to perform mathematical calculations. For using the below functions, all functions except abs(x),max(x1,x2,x3,…,etc.), min(x1,x2,x3,…..,etc.) and round(x[,n]),pow(x,y) need to import the math module because these functions reside in the math module. The mathematical functions are

Function	Description
abs(x)	Returns the absolute value of x.
sqrt(x)	Finds the square root of x.
max(list)	Returns the largest of its arguments
min(list)	Returns the smallest of its arguments.
round(number,[ndigits])	in case of decimal numbers, x will be rounded to n digits
math. Ceil(x)	Finds the smallest integer not less than x.
math. floor(x)	Finds the largest integer not greater than x.
pow(x,y)	Finds x raised to y
exp(x)	Returns e^x ie exponential of x
log(x)	Finds the natural logarithm of x for x>0
log10(x)	Finds the logarithm to the base 10 for x>0
modf(x)	Returns the integer and decimal part as a tuple. The integer part is returned as a decimal

3.2.2. Trigonometric Functions:

There are several built-in Trigonometric functions in python. the function names and its purpose are

Functions	Description
sin(x)	Returns the sine of x in radians.
cos(x)	Returns the cosine of x in radians.
tan(x)	Returns the tangent of x in radians.
asin(x)	Return the arc sine of x, in radians.
acos(x)	Return the cosine of x, in radians.
atan(x)	Return the tangent of x, in radians.
hypot(x,y)	Returns the Euclidean form ,sqrt(x*x +y*y).
degrees(x)	Converts angle x from radians to degree.
radiand(x)	converts angle x from degree to radians.

3.3. Str(string) Data type:

In python , strings are immutable, it means once you define a string, it cannot be changed during execution. Strings in python are identified as a contiguous set of characters represented in the quotation marks. Python allows either pair of single or double quotes. subsets of strings can be taken using the slice operator([ ] and [:])with index values. These index values are otherwise called as subscript which are used to access to manipulate the strings.

Example : Python code to print the string using quotes

```
a='Hi'                                  # string using single quote
b="Welcome"                             # string using double quote
str1="""This is my favourite place.
My favourite food is Pizza."""          #string using triple quote for multiline
print(a)
print(b)
print(str1)
```

Will print

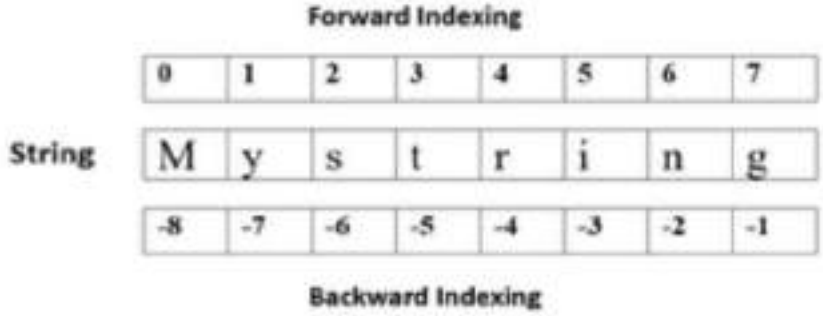
```
Hi
welcome
This is  my favourite place to eat.
My favourite food is pizza.
```

Note: In string data type '+' sign is for concatenation(joining) and the '*' symbol is for repetition

3.3.1. Indexing and String Slicing:

The string data type supports two types of indexing: positive (forward) and negative (backward). The positive subscript 0 is assigned to the first character and n-1 to the last character, where n is the number of character in the string. The negative subscript assigned in reversed order begin with -1.

Example:



3.3.2. String Slicing:

Python also permits us to select a portion of a string using index. This can be done by specifying range of characters and their indexes.

Syntax:

String[start: last: step]

Default start value is 0,stop value is len(string)and step value is 1.

For e.g:

```
str_1='Hello world'
str_2='My name is python'
print(str_1)    #prints complete string
print(str_1[0])    #prints first character of str_1
print(str_2[2:5])  #prints characters from str_2  starting from 3rd to 5th
print(str_1[2:])   #print characters of str_1 from 3rd to end
print(str_1[0:5]) #print characters of str_1 from 1st to 5th
print(str_2[ -1])  # print characters of str_2 by reverse order
print(str_2[:-8])  #print characters of str_2  by reverse order
print(str_1[ : ])   #print characters of str_1 from first to end
```

Output:

```
Hello world
H
 na
llo world
Hello
n
My name i
Hello world
```

For practice 2: Reverse order Slicing

```
name="Foot Ball Player"
print(" The word is: ",name)
print("The reverse order is :",name[ -5:-1])
```

Output:

```
The word is:  Foot Ball Player
The reverse order is : laye
```

3.3.2.1. Stride while Slicing String:

String slicing can take a third parameter. The stride, or the number of characters to be obtained from the string following the first, is specified by the third parameter (step value).

For example: String slicing with third parameter

```
str="python decoder"
print(str[6:14])          # string slicing
str2="hi, welcome to python code"
print(str2[::-1])       # string slicing with third parameter by reserve order
print(str2[4:16:2])       #in slice index numbers will be captured by a stride of
```

Output:

```
decoder
edoc nohtyp ot emoclew ,ih
wloet
```

Practice Quiz:

1. Show the result by applying concatenation and slicing to the provided string.

str1=" String is a datatype in python"

str2='In computer science'

Solution:

```
str1=" String is a datatype in python"
str2='In computer science'
print("the first string :",str1)
print("the sliced string is :",str1[7:17])
print("the second string is :",str2)
print("the concatenated string is :",str2+str1)
```

Output:

```
the first string :  String is a datatype in python
the sliced string is :  is a data
the second string is : In computer science
the concatenated string is : In computer science String is a datatype in python
```

3.3.3. Built-in String functions:

The functions that are predefined in the python are known as built-in functions. we'll cover a number of different python functions in this chapter that we may use to work with strings.

i. len( ) : Returns the length of the string.

Examples:

#1. Python code to display the length by using script mode

```
source="Joe contributes to AI instance"
print("total words are :",len(source))
```

Output:

```
total words are : 30
```

The variable source holds a sentence. When len(source) is used, it returns the number of characters in that sentence — not the number of words. The code then prints that number with the label "total words are:", though the output is technically showing the length of the string in characters, not the number of words.

When working with sentences in Python, you can find out how many words there are by using split(). It breaks the sentence into a list of words, and then you can use len() to count the words.

For e.g:

```
source = "Joe contributes to AI instance"
print("Total words are:", len(source. split()))
```

Output:

```
Total words are: 5
```

#2. Python code to display the length in python shell.

```
>>>a='computer department'
>>>print(len(a))
19
```

ii. upper( ) ,lower ( ) and capitalize( ): The strings that are returned by the functions ***str.upper ( )*** and ***str.lower ( )*** have all of the original string's letters changed to upper or lower-case letters. The ***str.capitalize ( )*** used to capitalize the first character of the string.

Example: Python code to change the case of given word.

```
place="chennai south"
print(place.capitalize())
print(place.lower())
print(place.upper())
```

Output:

```
total words are : 30
Chennai south
chennai south
CHENNAI SOUTH
```

iii. Boolean Return Functions:

In python there are several string functions in python that return a Boolean result. The Boolean results are ***True, False.***

The functions are:

Syntax	Description	Example
isalnum( )	Returns True if the string contains only letters and digits. (no special characers)	>>>a='water' >>>print(a. isalnum()) True >>>a='water bottle' >>>print(a. isalnum()) False
isalpha()	String consists of only alphabetic characters.(no symbols)	>>>'bond007'.isalpha() False >>>'bond'. isalpha() True
islower(), isupper()	string's alphabetic characters are all lower or upper case	>>>print('welcome'. islower()) True >>> 'WELCOME'. isupper() True
istitle( )	string is in title case	>>> 'Good Morning'. istitle() True

iv. join( ),split( )and replace( ): Two strings can be *joined* using the ***str.join( )*** function. We can split *string apart* in the same way that we can link them together by *using* ***str.split( )*** function. An Original string can be passed to the ***str.replace( )*** function, which will return an *altered string* with some replacements.

Example: Python code to display the function of join(),split() and replace().

```
obj="Balu has a balloon"
# 1.By using the join( ) method to add comma and white space to the string
print(",".join(obj))
print(" ".join(obj))
#2. change the original string into opposite
print(",".join(reversed(obj)))
#3.split the strings apart same way that we can link them together.
print(obj. split())
#4. replace the substring "has" into "had" in a new string
print(obj. replace("has", "had"))
```

Output:

```
B,a,l,u, ,h,a,s, ,a, ,b,a,l,l,o,o,n
B a l u h a s a b a l l o o n
n,o,o,l,l,a,b, ,a, ,s,a,h, ,u,l,a,B
['Balu', 'has', 'a', 'balloon']
Balu had a balloon
```

v. find( ) , count ( ) and swapcase( ): The ***str.count( )*** function can be used to count how many times a specific character or a group of characters appear in a string. The ***str.find( )*** function can be used to return the character's position depending on its index number.

The ***str.swapcase( )*** will change of every character to its opposite case vice-versa.

Example:

```
str1="Happiness blooms from within the peace of mind"
# 1. count the no. of times the letter s and p
print(str1.count("s"))
print(str1.count("p"))
#2. find the word "p" and "blooms" from str1
print(str1.find("p"))
print(str1.find("blooms"))
# 3. swapping the letters in opposite case
str1="WonDerful adVEnture"
print(str1.swapcase( ))
```

Output for the above functions:

```
3
3
2
10
wONdERFUL ADveNTURE
```

> **Note:** str.count(str,beg,end) function alsoreturns the number of substrings occurs within the given range.

For practice:

1.Write the python code to print the mirror image of giving word name="Kindness".

Solution:

```
name="kindness"
print("The mirror word is:",name[: :-1])
```

Output:

```
The mirror word is: ssendnik
```

2.Write a program to find the length and count of a string.

Solution:

```
sname=input("Enter the student name :")
a=" The committee met today"
# 1. find the length
print("The length of sname is :",len(sname))
```

```
# 2. count  how many times the word 'committee' is repeats
print("the count of committee is: ",a.count('committee'))
#  3. count the number of times the letter 't'  is in the string
print(a.count('t'))
# 4.count the letter 't' between  indexed  range
print(a.count('t',0,12))
```

Output:

```
Enter the student name :Rajeshwari
The length of sname is : 10
the count of committee is:  1
4
2
```

3. Write a python code to replace a string with another string without using replace( ) and make them to join by '/' by using join( ).

Solution:

```
str1=" Baby saw a big balloon"
#1. replacing by using '+'
str2=str1[ :11] +"  "+ "blue"+str1[15:23]
print("Length is:",len(str1))
print(str2)
# 2. join by '/'
print("/ ".join(str2))
```

Output:

```
Length is: 23
Baby saw a  blue balloon
 / B/ a/ b/ y/  / s/ a/ w/  / a/  /  / b/ l/ u/ e/  / b/ a/ l/ l/ o/ o/ n
```

> **key:** Append ('+=') function is also used to add more string at the end of an existing string.

3.3.4. String Formatting operators:

The formatting operator % is used to construct strings, replacing parts of the strings with the data stored in variables. The format() method formats the specified value(s) and insert them inside the string's placeholder. The placeholder is defined using curly brackets: {}.

Syntax:

```
string.format(value1,value2,….)
("string to be display with %val1 and %val2"%(val1,val2))
```

> Note: Avoid leaving spaces inside the placeholder {}. A space will cause a Key Error.

Example:

```
emp_name=input("Enter your name :")
age=int(input("enter your age :"))
```

```
print("you are {}  and {}years old ".format(emp_name.title(),age))
Cname=input("enter the company name :")
years_exp=int(input("enter the years of experience:"))
print("your working experience in {} is {} years.".format(Cname.title(),years_exp))
```

Output:

```
Enter your name :kathiravan
enter your age :27
you are Kathiravan  and 27years old
enter the company name :rajan consultancy
enter the years of experience:5
your working experience in Rajan Consultancy is 5 years.
```

Some more formatting characters are

- %c - character
- %d or %i - Signed decimal integers
- %s - String
- %x or %X - Hexadecimal integer
- %e or %E - Exponential notation
- %f - Float
- %u - unsigned integer

3.3.5. String operators:

Strings in python supports three operators: '+' for concatenation,'*' for repetition and in, not in for membership. They are shown in table:

Operator	Remarks
+	string concatenation, "hello"+ "world" gives "helloworld"
*	string repetition, "hi"*3 gives "hihihi"
in, not in	checks for membership of element to string, a) 'n' in 'python' returns True b)'I' not in 'python' returns True

Note: String concatenation can only be performed between two strings. You cannot mix any other type with the string. If you try to do you will get Type error. If you want to do with other datatype you need to cast or convert non-string to string data type by str function.

for e.g:

```
>>>s="hi" + str(123)
>>>s
 "hi123"
```

3.3.6. String traversal:

A string is a sequence of characters, it can be easily traversed using a loop. Using a loop each character can be analyzed and processed depending upon the problem at hand.

Example 1: Python program to count number of vowels in a given string using while loop

```
s=input("enter a string :")
i=0
count=0
while i<len(s):
    if  s[i] in 'aeiouAEIOU':
      count +=1
    i=i+1
print("number of vowels=",count)
```

Output:

```
enter a string : mutton briyani
number of vowels= 5
```

This program reads a string from the user and counts how many vowels it contains. It uses a while loop to go through each character of the string one by one. If a character is a vowel (either uppercase or lowercase), it increases the vowel count by one. Finally, it displays the total number of vowels in the string.

Example 2: Python program to count number of vowels in a given string using for loop

```
s=input("enter a string :")
count=0
for char in s:
    if char in 'aeiouAEIOU':
    count +=1
print("number of vowels=",count)
```

Output:

```
enter a string : mini tiffin
number of vowels= 4
```

3.4. List Data Type:

List are the most versatile of python's compound data type. A list is a collection which is ordered and mutable. A list contains items separated by commas and enclosed within square brackets [ ]. Lists are similar to arrays in C,C++ and Java. The one main difference between them is that all the items belonging to list can be of different data type.

Syntax:

```
<name of list>=[ <val1>,<val2>,<val3>]
```

Note: A list contain another list is known as Nested list. The inner list also an element of the variable a.
e.g: a=[1,2,5,[45,67,'hi'],7]

Example: Python program to display the list data types

```
address=['Fero','no:4', 'west coast road','ch-03']
x=[1,34,4.9,'hi']
y=[1,1,2,6,7]
print(x)
print(y)
print(address)
```

Output:

```
[1, 34, 4.9, 'hi']
[1,1,2,6,7]
['Fero', 'no:4', 'west coast road', 'ch-03']
```

The program defines three lists: address, x, and y. The **address** list contains **string elements** that together represent an address. The **list x** includes different types of data: **integers, a float, and a string**. The **list y** contains a sequence of **integers**. The program then uses the print() function to display the contents of each list on the screen, showing all the elements within square brackets as they appear in the list.

3.4.1. Slicing of a List:

The values stored in a list can be accessed using the slice operator([ ] or [:]) with indexes starting at 0 in the beginning of the list and working their way to end-1 same as like string index method. The plus (+) sign is the list concatenation operator, and the asterisk (*) is the repetition operator.

Syntax:

list_name[start : end : step]

- start: is the starting index position of extraction.
- end: is the ending index position of extraction. The end value always continues up to end-1 index position.
- step: represents the incremented or decremented value .

Note: Lists are mutable in python.

Example:

```
lst1=['days in year-2025: ',1,14,8,23,17,17,9]
lst2=['jan', 'feb', 'march', 'april', 'may',' june']
print("The length of list1: ",len(lst1),',',"The length of list2: ",len(lst2))
print(lst1,"\n",lst2)
print(lst1[0])
print(lst2[0:6])
print(lst1+lst2)  #concatenate with '+' operator
print(lst2[2:4]*2)
print("days",lst1,'with',lst2)  #concatenate list
```

Output:

```
The length of list1:  8 , The length of list2:  6
['days in year-2025: ', 1, 14, 8, 23, 17, 17, 9]
['jan', 'feb', 'march', 'april', 'may', 'june']
days in year-2025:
['jan', 'feb', 'march', 'april', 'may', 'june']
['days in year-2025: ', 1, 14, 8, 23, 17, 17, 9, 'jan', 'feb', 'march', 'april', 'may', 'june']
['march', 'april', 'march', 'april']
days ['days in year-2025: ', 1, 14, 8, 23, 17, 17, 9] with ['jan', 'feb', 'march', 'april', 'may',  'june']
```

3.4.1.1. operators + and * with list:

a)The + operator: it can be used for string concatenation as well as for concatenation of two lists.

For e.g:

```
>>>L=[1,2,3] + [4,5]
>>>L
[1, 2, 3, 4, 5]
>>>L=L + list(range(6,11))
>>>L
[1, 2, 3, 4, 4, 5, 6, 7, 8, 9, 10]
```

b)The * operator: The * operator works same as we have seen with string. If used as L*3(L is a list) it simply creates 3 copies of the list L.

For e.g:

```
>>> L=['a','b']
>>> L*3
['a', 'b', 'a', 'b', 'a', 'b']
>>> list('aba')*2
['a', 'b', 'a', 'a', 'b', 'a']
>>> L[0]*3
'aaa'
```

```
>>> L[1]=10
>>> L
['a', 10]
>>> L[1]*30
300
```

3.4.2. Searching Element in a list:

To search an element within a list index( ) is used. This function returns the index position of the first occurrence of an element passed as an argument with this function. But if the element is not found in the list ,it raises an error.

Membership operators are:

- **in : Returns True if the element is found in the list.**
- **not in: Returns True if the element is not in the list.**

Example: Write a program to show the search operation in a list.

```
elements=[10,20,35 ,12,60,5,3,4,2,4,67]
x=elements.index(60)    # will find the index value of 60 from list by index( )
print(x)
print(" The list : ", elements, end=" ")
num=int(input(" \nEnter a number:"))
if num in elements:
   position=elements.index(num)
   print("the given number has found")
   print("the index position  of the number is :",position)
else:
   print("The given number is not found")
```

Output 1:

```
4
 The list :  [10, 20, 35, 12, 60, 5, 3, 4, 2, 4, 67]
Enter a number:3
the given number has found
the index position  of the number is : 6
```

Output 2:

```
4
The list :  [10, 20, 35, 12, 60, 5, 3, 4, 2, 4, 67]
Enter a number89
The given number is not found
```

3.4.3. Adding Elements to a List: append( ),extend( ) and insert ( )

Just like an array in python ,we are able to add one or more elements into a list using append( ), extend( ), and insert( ) methods.

- Using append( ) we can add a single element at the end of a list.
  syntax: list_name. append( )
- Using extend( ) we can add multiple elements at the end of a list.
  Syntax: list_name. extend( )

- Using insert( ) we can insert an element at the beginning, or at any index position in a list.
  Syntax: list_name.insert (position index, element)

Example: Write a program to show use of append ( ), extend ( ), and insert ( )

```
lst1=[12,34,23,21,45]
print("before adding elements :",lst1,end=' ')
 lst1.append(1)        # append  function
lst1.append(10)
print("\n after appending the list is: ",lst1)
lst1.extend(['a','b',7])    #extend function
print("after using the extend the list is :",lst1)
print(lst1)
print("6th index value from  list is :",lst1[6])
lst1.insert(6,'hari')        #insert function
print("after inserting the value at the index value  6 :",lst1)
```

Output:

```
before adding elements : [12, 34, 23, 21, 45]
after appending the list is:  [12, 34, 23, 21, 45, 1, 10]
after using the extend the list is : [12, 34, 23, 21, 45, 1, 10, 'a', 'b', 7]
[12, 34, 23, 21, 45, 1, 10, 'a', 'b', 7]
6th index value from  list is : 10
after inserting the value at the index value  6 : [12, 34, 23, 21, 45, 1, 'hari', 10, 'a', 'b', 7]
```

3.4.4. Deleting elements from list:

- ***List_name.remove(element):-*** Removes mention element from the list. In case duplicate elements ,it removes first appearing value. It gives error if element is not present in the list.
- ***List_name.pop(i):-*** Remove the element present at index 'i'. If no index mentioned ,then it removes last element.
- ***List_name.clear( ):-*** Removes all elements from using indexes.
- ***del*** keyword is used to ***delete elements using indexes***.
- ***del*** keyword is also used ***to delete entire list.***

Example: Python program to display all the list deletion operations

```
lst=['space','venus','mercury','jupitor',45,9.22220,2.3e2]
print("list is:",lst)
print(len(lst))
# 1. remove ()
lst.remove('jupitor')
print("after removing jupitor:",lst)
lst.remove('sun')
print(lst) #it prints ValueError: list.remove(x): x not in list, which means the element is not in the list
# 2. pop(i)
lst.pop(5)
print("after deleting index value[6]:",lst)
lst.pop()
```

```
print("after deleting without index value :",lst) # will delete the last element of the list.
lst.pop(7)
print(lst)  #prints IndexError: pop index out of range, which means total index value is 6
# 3. clear()
lst=['space','venus','mercury','jupitor',45,9.22220,2.3e2]
print(lst)
lst.clear()
print("after  clear()  the list is:",lst)
# 4. del keyword
lst=['space','venus','mercury','jupitor',45,9.22220,2.3e2]
print(lst)
del lst[5]
print("list :",lst)
del lst[0]
print("list :",lst)
del lst[5]  # it prints IndexError: list assignment index out of range. Because index[5] has already deleted.
print("list :",lst)
# delete entire list by using del keyword
lst=['space','venus','mercury','jupitor',45,9.22220,2.3e2]
print("list is:",lst)
del lst
print(lst) #it prints NameError: name 'lst' is not defined. Did you mean: 'list'? because the list has already deleted.
```

Output:

```
list is: ['space', 'venus', 'mercury', 'jupitor', 45, 9.2222, 230.0]
7
after removing jupitor: ['space', 'venus', 'mercury', 45, 9.2222, 230.0]
after deleting index value[6]: ['space', 'venus', 'mercury', 45, 9.2222]
after deleting without index value : ['space', 'venus', 'mercury', 45]
['space', 'venus', 'mercury', 'jupitor', 45, 9.2222, 230.0]
after  clear()  the list is: []
['space', 'venus', 'mercury', 'jupitor', 45, 9.2222, 230.0]
list : ['space', 'venus', 'mercury', 'jupitor', 45, 230.0]
list : ['venus', 'mercury', 'jupitor', 45, 230.0]
list is: ['space', 'venus', 'mercury', 'jupitor', 45, 9.2222, 230.0]
NameError: name 'lst' is not defined. Did you mean: 'list'?
```

3.4.5. Accessing List Data Using for /while loop:

1. for loop in list:

As a list it iterable, we can iterate through a list very easily using the for statement. we can also iterate through a list using the list index .

Syntax:

For <element_ variable> in < list>:
<indented statements>

From the above syntax , the for statement causes the body of the loop to execute once for every element in the <list>. Each time through the loop , the variable you specify as <element_variable> is set to the value of the next element in the list.

Example 1: Generating new list from existing using for loop.

```
l=[1,4,2,5,7,49,23,10]
L1=[ ]
L2=[ ]
for x in l:
   L1.append(x+8)
   L2.append(x*5)
print("The list L1 is =",L1)
print("The list L2 = ",L2)
```

Output:

```
The list L1 is = [9, 12, 10, 13, 15, 57, 31, 18]
The list L2 =  [5, 20, 10, 25, 35, 245, 115, 50]
```

To process a list using **for** loop there is no need to use the range ( ) function. Python can simply set the value of the loop counter to each item in the list. Python allows direct iteration over the elements of the list. This means the loop variable is automatically assigned each item in the list one by one, making the code simpler and more readable.

Example 2 :Program to print all the list elements by using range( ).

```
mylist=[10,20,30,40,50]
for i in range(len(mylist)):
   print(mylist[i],end=' ')
```

Output:

```
10 20 30 40 50
```

This program defines a list of numbers and uses a for loop with range(len(mylist)) to access each index of the list. For each index, it prints the corresponding element on the same line by using the end=' ' argument, which prevents the output from moving to a new line after each print.

2. while loop in list:

In list the while loop continues as long as its predicate is true. When the predicate is false, the loop is broken the code will exit.

Example 3 : Write a program to show how to iterate through a list using while loop.

```
countries=["Antarctica", "Bengal", "Canada", "Denmark"]
i=0
while i<len(countries):
    print(countries[i])
    i+=2
```

Output:

```
Antarctica
Canada
```

This program initializes a list of country names and uses a while loop to print every second element in the list. The loop starts with index i = 0 and continues as long as i is less than the length of the list. Inside the loop, it prints the country at the current index and increases i by 2, effectively skipping every other element.

For Practice:

1: Program to display all the elements in a list by using append function.

```
name=input("Enter your name:")
m1=int(input("mark of python :"))
m2=int(input("mark of maths :"))
m3=int(input("mark of java :"))
x=[ ]
x.append(m1)
x.append(m2)
x.append(m3)
print(x)
print("python:",m1,"maths :",m2,"java :")
```

Output:

```
Enter your name:kathiravan
mark of python :45
mark of maths :67
mark of java :87
[45, 67, 87]
```

Practice Quiz:

1. What will be the output for the following code segment?

```
lst1=[0, -3, 234,321,4354,45,2]
print(lst1)
print(len(lst1))
print("list1 [0] =",lst1[0])
print("list1 [-4:-2]=",lst1[-4:-2])
print("list1 [2] *2 =",lst1[2]*2)
print("list1 [-5]=",lst1[-5])
```

3.4.6. List Traversal:

List traversal means accessing each element of list exactly once and performing some operation on them that may be display, checking for some conditions or any other which the programming problem demands. ***List can be traversed easily using any loop*** either for or while loop. But the easiest one is using for loop.

3.4.6.1. List traversal using while loop:

For the list traversal using while loop we need to have the length of that can be obtained **len** function. Then each element of the list can be accessed using index notation.

Example 1: Displaying all the elements of list one by one using while loop

```
l=[1,4,3,7,8,10,12]
i=0
print("the length of the list is:",len(l))
print(" List elements")
while i<len(l):
    print(l[i], end=' ')
    i=i+1
print("\n out of loop")
```

Output:

```
the length of the list is: 7
List elements
1 4 3 7 8 10 12
out of loop
```

From the above code, we have started from index **0 to len(l)-1** and displayed all elements of list l using **index notation l[i].** The value of I is incremented in every iteration of while loop. When the condition **i<len(l)** becomes false control comes out from loop .

Example 2: Generating new list from existing using while loop

```
lst=[1,3,7,5,7,9,11,13]
l1=[]
l2=[]
i=0
while i<len(lst):
  l1.append(lst[i]+10)
  l2.append(lst[i]*2)
  i=i+1
print("the list is:",lst)
print("The list 1 is:",l1)
print("The list 2 is:",l2)
```

Output:

```
the list is: [1, 3, 7, 5, 7, 9, 11, 13]
The list 1 is: [11, 13, 17, 15, 17, 19, 21, 23]
The list 2 is: [2, 6, 14, 10, 14, 18, 22, 26]
```

3.4.6.2. List traversal using for loop:

List traversal using for loop is the most convenient way and most python programmers often use this method of list traversal. A list is a sequence of items, so the for loop iterates over each item in the list automatically.

Example 1: Displaying all the elements of list one by one using for loop

```
lst=[2,4,6,8,10,12]
print("list elements")
for x in lst:
    print(x, end=' ')
print("\n out of loop")
```

Output:

```
list elements
2 4 6 8 10 12
out of loop
```

The above program defines a list lst containing even numbers. It then prints a header message "List elements:". Using a for loop, it iterates through each element in the list and prints them on the same line separated by spaces, by specifying end=' ' in the print() function. After all elements have been printed, it prints the message "Out of loop" on a new line, indicating that the loop has completed.

Example 2: Count Odd And Even Elements In list.

```
lst=[2,3,6,4,5,7,34,51,32,212,123,10,11]
counteven, countodd=0,0
for x in lst:
    if x%2==0:
    counteven +=1
else:
    countodd +=1
print("Number of even elements=",counteven)
print("number of odd elements=",countodd)
```

Output:

```
Number of even elements= 7
number of odd elements= 6
```

3.4.7. Updating Elements in a list :

We can easily update or modify the list elements just by assigning value at the required index position. With the help of the slicing operation certain positions of a list can be modified .

Example:

```
lst_number=[1,2,2,4,67,12,15,'hari',['w','e','l','c','o','m','e'],10]
print("Before :",lst_number)
lst_number[2]=3          #update the index value at 2 as 3
lst_number.insert(3,5)   #inserting the new element
print (lst_number)
lst_number[7:10]=[18,21]
print(lst_number)        # change the string element into numbers
```

Output:

```
Before : [1, 2, 2, 4, 67, 12, 15, 'hari', ['w', 'e', 'l', 'c', 'o', 'm', 'e'], 10]
[1, 2, 3, 5, 4, 67, 12, 15, 'hari', ['w', 'e', 'l', 'c', 'o', 'm', 'e'], 10]
[1, 2, 3, 5, 4, 67, 12, 18, 21, 10]
```

The program begins by defining a list lst_number that contains a mix of integers, a string, a nested list, and more. Initially, the list is printed to show its original contents. Then, it updates the element at index 2 (which is the second 2) to the value 3.

Next, it uses the insert() function to add a new element 5 at index 3. After that, it modifies a slice of the list from index 7 to 9 (i.e., three elements) and replaces them with the two numbers 18 and 21. This slice includes a string 'hari' and part of the nested list, which are removed and replaced by the new numbers. Finally, the updated list is printed to show the changes.

3.4.8. List functions:

python list functions are

- ***list_name.copy( )*** : Returns a copy of the list.
- ***list_name.count(value)*** : Returns the number of similar elements present in the list.
- ***max(list),min(list)*** : Returns the maximum and minimum value of the list.
- ***sum(list)*** : Returns the sum of values in a list.
- ***list.reverse ( )*** : Reverse the order of the elements in the list.
- ***round(value)*** : The python round() function rounds off the digits of a number and returns the floating point number.
- ***Sort(),sorted()*** : The sort() doesn't return any value. but it changes the original list. Sorted() returns an iterable list.

Example 1: Program to displaying the basic list functions i) max, ii)min, iii)sum, iv)reverse, v)sorted.

```
mylist=[12,1,3,56,43,78,123,2,3,10,100,3]
print("list:" mylist)
print("maximum : ",max(mylist))
print("minimum :",min(mylist))
print("sum of list : ",sum(mylist))
print("reverse of list :",mylist.reverse())
print("sorted list is :",sorted(mylist))
#list( ) function
x=list("warm welcome")
```

```
print(x)
y=list(mylist)
print(y)
```

Output:

```
list: [12, 1, 3, 56, 43, 78, 123, 2, 3, 10, 100, 3]
maximum :  123
minimum : 1
sum of list :  434
reverse of list : None
sorted list is : [1, 2, 3, 3, 3, 10, 12, 43, 56, 78, 100, 123]
['w', 'a', 'r', 'm', ' ', 'w', 'e', 'l', 'c', 'o', 'm', 'e']
[3, 100, 10, 3, 2, 123, 78, 43, 56, 3, 1, 12]
```

Note: list ( ) function is a built-in **constructor** in Python. It construct and returns a list from any iterable

Example 2: Python code for list methods of sort( ),reverse( ),copy( ),round( ) .

1. Program to illustrate the Copy() function.

Solution:

```
a=[23,12,34,56,10,-4,-78]
print("the list is : ",a)
print(len(a))
#copying  from a to b
b=a
print("the second list is:",b)
z=a.copy()
print("after copying the list z is:",z)
```

Output:

```
the list is :  [23, 12, 34, 56, 10, -4, -78]
7
the second list is: [23, 12, 34, 56, 10, -4, -78]
after copying the list z is: [23, 12, 34, 56, 10, -4, -78]
```

From the above program a list named **a** is created with several numbers. The print() function displays the list, and **len(a)** shows how many items are in it. When we write b = a, it means b is not a new list—it just points to the same list as a, so any change in a will also affect b.

To make a real copy, we use a.copy() and store it in z. This creates a separate list, so changes to a will not affect z. Finally, we print all three lists to see the difference.

1.a. Program to print the id of original and copied list.

Solution:

```
a=[23,12,34,56,10,-4,-78]
b=a
z=a.copy()
print("id(a):",id(a))
print("id(b):",id(b))
print("id(z):",id(z))
```

Output:

id(a): 1342699443392

id(b): 1342699443392

id(z): 1342700964800 ***# 'a', 'b' IDs are same but IDs as 'a' and 'z' are different.***

In this code, a list a is created with some numbers. When we assign b = a, both a and b point to the **same list** in memory. So, their id values (which show the memory address) will be the **same**. Then, z = a.copy() creates a **new list** with the same elements, but stored at a **different memory location**.

By printing id(a), id(b), and id(z), we can see that a and b have the same ID, but z has a different one. This shows that b is not a new copy—it's just another name for a, while z is an actual copy.

2. Program to illustrate the Sort( ) method.

```
#sort method
x=[10,100,45,67,34,12,5]
y=["apple", "anchor", "noble", "hexadecimal", "ice- cream", "blossom", "blush"]
x.sort()
print(x)
y.sort(key=len)
print(y)
```

Output:

[5, 10, 12, 34, 45, 67, 100]

['apple', 'noble', 'blush', 'anchor', 'blossom', 'ice -cream', 'hexadecimal']

In this program, the list x contains numbers, and the list y contains words. When we use x.sort(), it arranges the numbers in increasing order. So the smallest number comes first and the largest last.

For the list y, we use y.sort(key=len), which means the words are sorted based on their **length**, not in alphabetical order. The shortest word comes first, and the longest word comes last. This helps in organizing words based on how many characters they have.

3. Program to illustrate the reversed( ) method.

```
# for string
str='JAVA Programming'
print(" The reversed string is:",list(reversed(str)))
# for Tuple
tup=('J','A','V','A')
print("The reversed tuple is:",list(reversed(tup)))
# for range
x=range(7,12)
print("The range of x is:",list(reversed(x)))
#for list
lst1=[3,5,87,5,12,1]
print(" The reversed list is:",list(reversed(lst1)))
```

Output:

The reversed string is: ['g', 'n', 'i', 'm', 'm', 'a', 'r', 'g', 'o', 'r', 'P', ' ', 'A', 'V', 'A', 'J']

The reversed tuple is: ['A', 'V', 'A', 'J']

The range of x is: [11, 10, 9, 8, 7]

The reversed list is: [1, 12, 5, 87, 5, 3]

3.a. Another method for reversed ( ) function for different datatypes by using third parameter.

```
a="Python Tools"
print("The reversed string is:", a[::-1])
lst1=[50,40,30,20,10,0]
print(lst1[::-1])
tup1=('a','b','c','d','e','f')
print("reversed tuple is:",tup1[::-1])
```

Third parameter negative

Output:

```
The reversed string is: slooT nohtyP
[0, 10, 20, 30, 40, 50]
reversed tuple is: ('f', 'e', 'd', 'c', 'b', 'a')
```

Above the Python program (3.a) shows how to reverse a string, a list, and a tuple using the slicing method [::-1]. This method tells Python to go through the elements backwards.

4. Program to illustrate the round( ) method.

```
#for integers
    print(round(10))
#for floating point
    print(round(10.8))
#even choice
    print(round(6.6))
```

Output:

```
10
11
7
```

Key Points: round() is a built-in Python function that **rounds a number to the nearest whole number.**

The above program uses the round() function in Python, which is used to round numbers to the nearest whole number. When you use round(10), it simply returns 10 since it's already an integer. With round(10.8), Python rounds it up to 11 because 10.8 is closer to 11 than 10. For round(6.6), the result is 7, because 6.6 is closer to 7 than to 6. The round() function works well with both integers and floating-point numbers and is helpful when you need to remove decimal places and keep values simple.

Practice Quiz:

1. What will be the output of the following code snippet?

```
even_list=[2,4,6,8,10,12]
even_list[0]=5
print(even_list)
even_list[1:4]=[14,16,18]
print(even_list)
even_list[1:6]=[20,22,,24,26,28]
print(even_list)
even_list. insert(3,12)
    print(even_list)
    even_list. insert(-3,10)
    print(even_list)
```

2. Write a python code to print the maximum and minimum value from a list.

3. By using the for loop write a python code to print all the elements from the following list.
   mylist=['hari',10,20,30]
4. Write a program that prints out the characters of a string in reverse order.
5.write a program which accepts a sequence of comma separated values from user and convert it to a list and tuple

3.4.9.The range ( )function:

Python's range( ) function makes it easy to generate a series of numbers. Sometimes you need to loop with explicit indices (such as the positions at which values occur in a list).

Examples1: Python code to display the range function object.

```
>>>range(5)
range(0, 5)
```

The range function above code has taken an integer and returned a ***range*** object. You can use two variants on the range function to gain more control over the sequence it produce. ***i.e range(start,end),*** the first argument is the starting number ,and the second number is the number resulting sequence goes up to .

3.4.9.1. Using range( ) to Make a list of Numbers:

You can use range command together with the len command on a list to generate a sequence of indices for use by the for loop.

Example 2: Program to display the index value by using range function with using len( ).

```
x=[1,3,-4,7,4,9,-5,4]
for i in range(len(x)):
    if x[i]<0:
      print("Found a negative number at index :",i)
```

Output:

```
Found a negative number at index : 2
Found a negative number at index : 6
```

The above program defines a list x containing both positive and negative integers. It then uses a for loop to iterate over each index of the list using range(len(x)). Inside the loop, it checks whether the current element (x[i]) is a negative number. If the condition is true (i.e., the number is less than 0), it prints a message indicating the index at which the negative number was found.

The range function is also have a third argument that is step value to count backward ,or to count by any amount other than 1.

Syntax: list(range(start,end,step))

Example 3: Program to display the list values by using range( ) with third parameter.

```
square=[]
for i in range(0,18,2):
    s=i*2
    square.append(s)
print("square :",square)
```

Output:

```
square : [0, 4, 8, 12, 16, 20, 24, 28, 32]
```

The program begins by initializing an empty list called square. It then enters a for loop that uses range(0, 18, 2) to generate numbers starting at 0 up to (but not including) 18, incrementing by 2 on each iteration.

For each number i in this sequence (i.e., the even numbers 0, 2, 4, 6, 8, 10, 12, 14, and 16), the program computes s as i * 2 (effectively doubling the value of i). This computed value s is then appended to the square list. Once the loop completes its execution, the program prints the message "square :" along with the content of the list, which will display the doubled even numbers.

3.4.10. List comprehensions:

This method used to create sequence of elements that satisfy a certain conditions. We can directly mention the condition in the list creation .

Syntax:

newlist=[expression for element in list if condition]

The ***for*** loop and ***if*** conditions can be depending upon what you want to achieve with list comprehension.

Example 1: Python program to differentiate a list with and without comprehension

```
# 1. Without list comprehension
L=list(range(1,11))
L1=[]
for i in range(len(L)):
 L1.append(L[i]*L[i])
print(" list without comprehension",L1)
```

```
# 2. With list comprehension
L=list(range(1,11))
L1=[x*x for x in L]
print("With comprehension:"L1)
```

Output:

```
list without comprehension [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
With comprehension: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

- In the first method, the program uses a normal for loop to square the numbers from 1 to 10. It first creates a list L with numbers from 1 to 10. Then it creates an empty list L1 to store the squares. The loop goes through each number in L, squares it (multiplies it by itself), and adds it to L1 using the append() function. After the loop finishes, it prints the list of square numbers.
- In the second method, the program uses **list comprehension** to do the same task in a shorter way. It creates the same list L from 1 to 10. Then it uses one line of code to square each number and store it in the list L1. This is done using [x*x for x in L]. Finally, it prints the list L1. Both methods give the same result [1, 4, 9, 16, 25, 36, 49, 64, 81, 100].

Example 2: Program to find the square of natural numbers

```
num=[x**2 for x in range(1,12)]
print(num)
```

Output:

```
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121]
```

This program uses a list comprehension to create a list of squares of numbers from 1 to 11. The expression x**2 calculates the square of each number x, and range(1, 12) generates numbers from 1 to 11 (excluding 12, because in range ( ) end value is n-1).

3.5. Tuple Data Type:

A **tuple** is an immutable sequence in Python (cannot be changed after creation).Tuples can hold **multiple data types**, like integers, floats, strings, etc . A tuple is a collection of elements that are separated by commas(,) inside parentheses. The tuple of elements can even defined with or without parentheses .

Syntax:

tuple_variable= (val1,val2,val3,...)

tuple_variable=val1,val,2,val3,...

Example 1:

```
x=(10,11,12,13,14,15,3.4,5.333)
y=('a','b','c','d','e')
mytup="madhesh","madhavan", "manikandan", "manish"
print(x)
print(y)
print(mytup)
print(type(x))
```

Output:

```
(10, 11, 12, 13, 14, 15, 3.4, 5.333)
('a', 'b', 'c', 'd', 'e')
('madhesh', 'madhavan', 'manikandan', 'manish')
<class 'tuple'>
<class 'tuple'>
```

This program demonstrates how to create and work with tuples in Python. It defines three different tuples: x contains integers and floats, y contains characters, and mytup contains strings. The print() function displays the contents of each tuple. Finally, the type() function is used to confirm that x is of the tuple type.

→ An empty tuple and single element tuple also we can write.

Example 2:

```
mytup=( )
tup1=(5)
```

In the above example, tup1 will execute but the type of the variable is not a tuple.

To declare a single element tuple we need to write:

```
Mytuple=(5,)
```

The main difference between lists and tuples are:

- List elements of a list is a mutable, whereas the elements of a tuple are immutable.
- list elements are enclosed within square brackets, tuple elements are enclosed in parentheses.

Notes: Tuples are immutable. We can create a tuple without using parenthesis. This is called tuple packing.

e.g. tup=2,3,'CSc',5.3

There is another operation is called unpacking ,'int' his feature tuple elements are stored into different variable.

e.g. tup=(24,"Chowdhury",45.89)

3.5.1. Accessing tuple elements:

Same as arrays and list, tuple elements are also accessed using index values. Tuple index also starts from 0, negative indexes is also allowed in accessing tuple elements. It always starts from-1.

To access the tuple elements we mention the tuple name followed by the tuple index enclosed within [ ]. Slicing operation is also applicable in tuple.

Note: same as list a tuple may contain another tuple or list as an element is called nested tuple.
E.g. x=(1,2,(3,4),(0,2))

Syntax:

Tuple_name[index]

Example 1:

```
x=(10,2,11,12,3,4,15,18,12,1,1)
print("length of x :",len(x))
print("the reverse index of  [-2] :",x[-2])
print("the elements from  [2:5]:",x[2:6])
print(" index value of [7] :",x[7])
```

Output:

```
 length of x : 11
the reverse index of  [-2] : 1
the elements from  [2:5]: (11, 12, 3, 4)
 index value of [7] : 18
```

Practice Quiz:

1. Write a program to show how to access tuple elements for the following tuple: mmy_tupl=(10,20,30,40,50,60,70,80)

3.5.2. Tuple Methods:

Here are some tuple method are mentioned below. Tuple methods are same like list methods.

Method	Description
count( )	Returns the number of occurrence of a specific element in tuple.
index( )	Returns the index of the first occurrence of a specific element in the tuple.
len( )	Returns the number of elements in the tuple.
tuple( )	returns a tuple version of an iterable object.
all( )	Returns True if all elements in the tuple are true, False otherwise.
any( )	Returns True if at least one element
filter( )	Returns the iterator from elements of an iterable for which a function returns true.

Example 1:

```
my_tuple=("techno",'hi',1,2,(2,3,6),6,[2.5,3.8])
a=("techno",(8,4,6),[2.5,3.8])
print(type(my_tuple))
print("my_tuple",my_tuple)
print("index value [2]:",my_tuple[2])
x=my_tuple.index(6)
y=my_tuple.count(2)
print("\t",x)
print("\t",y)
print( "Tuple of [1][2] : ",a[1][2])
```

Output:

```
<class 'tuple'>
my_tuple ('techno', 'hi', 1, 2, (2, 3, 6), 6, [2.5, 3.8])
```

```
index value [2]: 1
  5
  1
Tuple of [1][2] :  6
```

Advantages of Tuple:

There are several advantages to using tuples in python:

1.Performance : Tuples are faster than lists because they are immutable and require less memory.

2. Immutable : Because tuples re immutable, they are safe to use as keys in a dictionary or as elements of a set.

3. Safety : Tuples provide a way to separate and store related data that should not be modified.

4. Readability : Tuples can make code more readable by allowing you to group related data together. This makes it easy to understand the meaning of the data and what it represents.

5. Easy to use : Tuples are easy to create and use. They have a simple syntax and can be created with or without parentheses.

3.5.2.1. Sort a Tuple passing through a list:

Tuples are immutable and therefore cannot be sorted themselves. To sort a tuple ,you need to convert it to a list, sort the list , and convert the list back to a tuple.

Example:

```
num=(10,13,5,7,9,11,13,15,17,19, 0,2,21,23)
print("Tuple", num)
a=list(num)
print("tuple to list is:" ,a)
#1.Sort through list
a.sort()
new_tup=tuple(a)
print("After sort :",new_tup)
# 2. Using the sorted() built-in function
new_tup=tuple(sorted(num))
print("using sorted ( ) :",new_tup)
```

Output:

```
Tuple (10, 13, 5, 7, 9, 11, 13, 15, 17, 19, 0, 2, 21, 23)
tuple to list is: [10, 13, 5, 7, 9, 11, 13, 15, 17, 19, 0, 2, 21, 23]
After sort : (0, 2, 5, 7, 9, 10, 11, 13, 13, 15, 17, 19, 21, 23)
using sorted ( ) : (0, 2, 5, 7, 9, 10, 11, 13, 13, 15, 17, 19, 21, 23)
```

3.5.3. Looping in tuple:

As a tuple is also iterable, we can iterate through a tuple just like a list very easily using the for statement.

Example:

```
number=(1,2,3,4,5)
double=( 'a','b')
for i in number:
    print( number+double)
```

Output:

```
(1, 2, 3, 4, 5, 'a', 'b')
(1, 2, 3, 4, 5, 'a', 'b')
```

(1, 2, 3, 4, 5, 'a', 'b')
(1, 2, 3, 4, 5, 'a', 'b')
(1, 2, 3, 4, 5, 'a', 'b')

3.5.4.Nested Tuple:

A nested tuple is a tuple inside another tuple. Tuple elements can access by using index values. If we have a nested tuple, in each iteration the variable contains tuple.

Example: Program To Iterate Through A Nested Tuple

```
a=( (1,2), (3,4), (5,6))
for i in a:
   print(i, end=' ')
   print(len(i))
print("the length of a is:", len(a))
```

Output:

(1, 2) 2
(3, 4) 2
(5, 6) 2
the length of a is: 3

In this program, the variable a is a tuple that contains three smaller tuples: (1, 2), (3, 4), and (5, 6). Each of these is like a pair of numbers. The for loop goes through each of the small tuples one by one. Inside the loop, it first prints the tuple using print(i, end=' '), which prints it on the same line. Then it prints the length of that tuple using print(len(i)). Since each small tuple has 2 elements, it prints 2 after each tuple.

After the loop ends, the program prints the total length of a using len(a), which tells us how many small tuples are in the main tuple. Since there are three tuples, it prints 3 as the final result.

For practice:

1. create and write a program for nested tuple to access individual members .

Solution:

```
square=((1,2),(2,3),(3,4))
    for i, j in square:
  print("square of {}is {}".format(i, j))
```

Output:

square of 1is 2
square of 2is 3
square of 3is 4

In this program, square is a tuple that holds three small pairs of numbers: (1, 2), (2, 3), and (3, 4). Each pair has two values. The for loop goes through each pair and splits the values into i and j.

So, for the first pair, i = 1 and j = 2, for the second, i = 2 and j = 3, and so on. Inside the loop, the print() function uses format() to display a message like "square of 1 is 2", with the values of i and j filled into the sentence.

Note: In this example, j is not really the square of i, but just a value

2.Python code to creating tuples from strings , lists and sets

Solution:

```
a="Tiny Treasure"
b=[2,3,4,5]
c={12,14,16,18,20}
```

```
print(tuple(a))
print(tuple(b))
print(tuple(c))
```

Output:

```
('T', 'i', 'n', 'y', ' ', 'T', 'r', 'e', 'a', 's', 'u', 'r', 'e')
(2,3,4,5)
(12,14,16,18)
```

This program uses the tuple() constructor to convert different types of iterables into tuples. The string a, list b, and set c are each passed to tuple(), which converts them into tuple format. The print() function then displays the resulting tuples.

3. Python code to display the tuple membership test

Solution:

```
p=(45,67,20,15,10)
print(p)
print(30 in p)
print(15 in p)
print(3 not  in p)
print(10 in p)
```

Output:

```
(45, 67, 20, 15, 10)
False
True
True
True
```

3.6. Set{ } Data Type:

A set is an unordered collection of unique elements. Sets are enclosed within curly braces{ } and separated by the comma(,). We can't change the set elements once set is created, but we can add new elements and remove existing elements. So set is mutable and set elements are immutable.

Sets Vs Lists and Tuples:

Sets are unique and unordered. Lists and Tuple are not unique (duplicates elements allowed)and ordered.

Advantages of sets:

sets are useful to efficiently remove duplicate values from a set or list or tuple and to perform mathematical set operations like union, intersection, set difference and symmetric difference etc.

3.6.1. Creating sets:

In python, set is a collection of unique elements. A set is created by placing all the elements separated by comma(,) within a pair of curly braces{ } or the set( ) function.

Syntax:

```
Set_Variable={ element1,element2,element3,….element n}
Set_Variable=set([element 1, element2,……element n])
```

From the above syntax,

Within the pair of curly braces a set directly using curly brace elements. It automatically removes duplicates and stores only unique elements.

By using set () function , a set by converting another iterable (like a list) into a set. It is especially useful when the original data comes from a list or other sequence.

For e.g.

```
A={10,12,'west','usa',34.678}  # using curly braces
B=set([1,2,3,5])    #using set function
```

The range() function generates a sequence of numbers, and the set() constructor converts that sequence into a set. Since sets store only unique values and do not maintain order, the resulting set may display numbers in any order.

Example 1: Program that print the set from range

```
a=range(5)
print(set(a))
print(set(range(14,25)))
print(set(range (28,10,-3)))
```

Output:

```
{0, 1, 2, 3, 4}
{14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24}
{13, 16, 19, 22, 25, 28}
```

3.6.2. Set operations:

Python supports some set operations such as

i)***Union :*** The union of two sets returns a new set that contains all the elements from both sets. The union operation is performed using the operator | or the union( ) method.

Example 1: Union operation by using | symbol

```
set_1={"apple", "banana", "mango"}
set_2={"banana", "cherry", "watermelon", "elderberry", "mango"}
c=set_1 | set_2
print(c)
```

Output:

```
{'watermelon', 'elderberry', 'mango', 'apple', 'cherry', 'banana'}
```

ii)Intersection : The intersection of two sets returns a new set that contains only the elements that are common to both sets. The intersection operation is performed using the **&** operator or the intersection () method.

Example 2: Intersection operation using & symbol.

```
a={"Madhu", "mala", "mani", "Murase", "malar"}
b={"carrol","cavery","cathrine","madhu","christopher","christy","mani"}
common=a & b
print(common)
```

Output:

```
{'Madhu', 'mani'}
```

iii) Difference : The difference of two sets returns a new set that contains the elements that are in the first set but not in the second set. The difference operation is performed using the - operator or the difference ( ) method.

Example 3: **Difference operation using – symbol.**

```
a={"car", "bike", "pen", "pencil", "notebook", "eraser"}
b={"pencil","notebook","marker","stapler","crayons"}
differ=a - b
print(differ)
```

Output:

```
{'pen', 'bike', 'car', 'eraser'}
```

iv) Symmetric difference: The symmetric difference of two sets returns a new set that contains elements that in either of the sets but not in both. The symmetric difference operation is performed using the ^ operator or the symmetric_difference ( ) method.

Example 4: **Symmetric difference operation using ^ symbol.**

```
group1={"Anna", "Anbu", Brian", "Charlie"," Diana"}
```

```
group2={"Brian", "Charlie", Anbu", "Evena"," Ferona", Fiona"}
friends=group1^group2
print(friends)
```

Output:

{' Ferona', 'Anna', 'Evena', 'Fiona', ' Diana'}

3.6.3. Set Comprehension:

In python, Comprehension is quick way to build sets. Set comprehensions are similar to list comprehensions but produce a set instead of a list. They eliminate duplicate elements automatically.

Example 1: Program to print set comprehension

```
a={x*2 for x in range(0,12)}
print("a is :",a)
b={i for i in range(3,20)}
print("b is :",b)
c={x**2 for x in range(0,20,3)}
print("c is :",c)
import random
d={random.randint(0,5) for i in range(5)}
print("d is:", d)
```

Output:

a is : {0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22}
b is : {3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19}
c is : {0, 225, 36, 324, 9, 144, 81}
d is: {0, 2, 3, 4}

Example 2: Program to display the set comprehension with condition.

```
square={2**x for x in range(10)if x%2==0}
print(square)
```

Output:

{64, 1, 256, 4, 16}

This program uses a set comprehension to generate a set of powers of 2 for even numbers from 0 to 9. The condition if x % 2 == 0 filters only even values of x, and for each such x, 2**x is computed and added to the set. The final set is then printed. Since sets are unordered, the elements may appear in any order.

3.7. Dictionary Data Type (Mapping data type):

Python's dictionaries are kind of hash-table type. It is also a collection but not just collection of values. It stores the data values as pair of *key* and *value*. Each key is separated from its value by a colon ':'. This ***key-value* pair** represents a single element and a comma separated set of these elements forms dictionary. Dictionaries are enclosed by curly braces({ }) and values can be assigned and accessed using square brackets([ ]).

> Note: keys in a dictionary are unique as well as immutable

Basically dictionaries are not sequences ;rather they are mapping. Mappings are collections of objects that stores objects by *key* instead of by relative position.

The *_ dict _* in python represents a dictionary or any mapping object that is used to store the attributes of the object. They are also known as mapping.

Syntax:

Dict_name={key 1:val 1, key 2:val2, key3:val 3....}

Example 1:

```
emp_detail={"e_name":"pradeep","department":"HR","salary":20000}
print(emp_detail)
```

Output:

```
{'e_name': 'Pradeep', 'department': 'HR', 'salary': 20000}
```

An empty dictionary can be created as:

```
Mydict={}
```

→ A dictionary can also be created using constructor as:

Example 2:

```
mydict={ }
mydict=dict([('name','babu'),('age',25),('class','XII')])
print(mydict)
```

Output: {'name': 'babu', 'age': 25, 'class': 'XII'}

3.7.1. Operations on a Dictionary :

Keys in a dictionary are unique as well as immutable, we cannot change them. Elements in a dictionary are mutable. we can add a new *key-value* pair ,delete an existing key-value pair or modify the value of a particular key. Membership operators (in, not in)are used with key values in a dictionary.

3.7.1.1. Adding and Modifying in Dictionary:

In the existing dictionary you can add more values by assigning the values along with key.

Syntax:

```
Dict_name[key]=value
```

Example 1:Adding the new key-value into the existing dictionary.

```
dict={"Rollno":10,"sname":"suji","place":"Karaikudi"}
print("1st Dictionary is:", dict)
#print the specific key
print(dict["Rollno"])
print(dict["place"])
# adding new key-value
dict["phone_no"]=22431246
dict["course"]='BCA'
print("after adding into dictionary:", dict)
```

Key: Dictionary is a mixed collection of elements

Output:

```
1st Dictionary is: {'Rollno': 10, 'sname': 'suji', 'place': 'karaikudi'}
10
Karaikudi
after adding into dictionary: {'Rollno': 10, 'sname': 'suji', 'place': 'karaikudi', 'phone_no': 22431246, 'course': 'BCA'}
```

Example 2: Deleting the specific key from the dictionary.

```
dict={"Rollno":10,"sname":"suji","place":"karaikudi"}
print("1st Dictionary is:", dict)# adding new key-value
dict["phone_no"]=22431246
dict["course"]='BCA'
print("after adding into dictionary:", dict)
```

```
# deleting  a key from dictionary
del dict["phone_no"]
print(dict)
#deleting all the elements from the dictionary
dict. clear()
print(dict)
#delete entire dictionary
del dict
print(dict)
```

Note:
del(dict_name) :- deletes the entire dictionary.
Clear():- delete only the elements in the dictionary.
Del dict_name["key"]:- deletes the specific key-value from the dictionary.

Output:

```
1st Dictionary is: {'Rollno': 10, 'sname': 'suji', 'place': 'karaikudi'}
after adding into dictionary: {'Rollno': 10, 'sname': 'suji', 'place': 'karaikudi', 'phone_no': 22431246, 'course': 'BCA'}
{'Rollno': 10, 'sname': 'suji', 'place': 'karaikudi', 'course': 'BCA'}
{}
<class 'dict'>
```

3.7.2. Dictionary Methods on Nested Dictionary:

The some more dictionary methods are

a. fromkeys(sequence[,value]):- This method creates a new dictionary using the keys from the sequence and set values of all key to value. The default value is None.

b. copy( ) :- This creates a new copy of a dictionary.

c. get(key[,d]) :- Returns the value of the key sent as a argument. If key is not present, returns the default value d.

d. setdefault(key[,d]) :- It is also returns the value of the key sent as argument.

Note: Along with this method dictionary uses some of python's basic functions len( ),str() ,etc.

Example: program to display dictionary methods:

```
dict1=dict.fromkeys([1,3,5])
print("dictionary created without value :",dict1)
#fromkeys()
dict1=dict.fromkeys([1,3,5],0)
print("dictionary created with value :",dict1)
dict2={1:2,2:1,3:2,4:9,7:5}
#get()
print("use of get () metho: ")
```

```
val=dict2.get(4)
print("val of key 3 is :", val)
val=dict2.get(9,'not found')
print("val of key 9 is :", val)
#setdefault()
print("use of setdefault( ) method:")
val=dict1.setdefault(3)
print("val of key 3 is :", val)
val=dict2.setdefault(9,23)
print("val of key 7 is :", val)
#len()
x=dict1
print("updated dictionary is:",x)
x=len(dict1)
print("length of x is :",x)
y=dict2
print("updated dictionar2 is :",y)
y=len(dict2)
print("length of y is :", y)
```

Output:

```
dictionary created without value : {1: None, 3: None, 5: None}
dictionary created with value : {1: 0, 3: 0, 5: 0}
use of get () metho:
val of key 3 is: 9
val of key 9 is: not found
use of setdefault( ) method:
val of key 3 is: 0
val of key 7 is: 23
updated dictionary is: {1: 0, 3: 0, 5: 0}
length of x is : 3
updated dictionar2 is : {1: 2, 2: 1, 3: 2, 4: 9, 7: 5, 9: 23}
length of y is: 6
```

3.8. Type conversion:

In python ,type conversion is the process of converting a value from one data to another.

There are two types of type conversion in python: a) implicit type conversion ,b)explicit type conversion.

3.8.1. Implicit type conversion:

In implicit conversion ,python automatically converts on data type to another when it is safe to do so

e.g. from integer to float

Example:

```
# integer to float
x=12
y=5.8
res=x+y
print(res)
print(type(res)
```

Output:

17.8
<class 'float'>

Key point about implicit:

- Python converts smaller data types (int)to larger data types(float) to avoid data loss.
- Common in arithmetic operations between int and float.

Limitations:- Implicit conversion does not work for incompatible types;

Example 1:

```
x="10"
y=5
z=x+y
print(z)
```

Output:

TypeError: can only concatenate str (not "int") to str

In this program, x = "10" means x is a **string** (because it's in double quotes), and y = 5 is a **number** (an integer). When the code tries to do z = x + y, Python does not allow adding a string and a number together directly. It doesn't know how to mix text with numbers in this way. So, this line causes Type error.

Valid code for conversion:

```
x = "10"
y = 5
z = int(x) + y
print(z)
```

Output:

15

Example 2:

```
x = "hi"
y = 3
z = x * y
print(z)
```

Output:

hihihi

In the example 2, the variable x is a string with the value "hi", and y is a number with the value 3. In Python, when you multiply a string by a number, it repeats the string that many times. So x * y means "hi" will be repeated 3 times. This result is stored in z, and when you print z, it shows "hihihi". There is no error here because Python allows string multiplication with an integer.

3.8.2. Explicit Type Conversion:

In explicit conversion ,also known as type casting , the programmer uses functions to manually convert data types.

Types of Conversion Functions:

Function	Description
int( )	Converts a value to an integer. Truncates decimals for floats.
float( )	Converts a value to floating-point number.
str( )	Converts a value to a string
bool ( )	Converts a value to Boolean (True / False)
list ( )	Converts an iterable (e.g. string ,tuple)into a list.
tuple ( )	Converts an iterable (e.g., string, list) into a tuple.
set ( )	Converts an iterable into a set (removing duplicates

Examples: Explicit conversion

a. Converting Numbers:

```
# float to integer
p=23.45
print(" integer :",int(p))
# integer to float
q=13
print("float:", float(q))
```

Output:

```
Integer : 23
Float: 13.0
```

b. converting string to numbers

```
#str to int
num_str="98"
print("str to int is :",int(num_str))
#  str to float
float_str="34.56"
print("str to float :",float(float_str))
```

Output:

```
str to int is : 98
str to float : 34.56
```

c. Numbers to string

```
#integer to str
x=23
print("int to str :",str(x))
# float to str
pi=3.14159
print("float to str:", str(pi))
```

Output:

```
int to str : 23
float to str: 3.14159
```

d. Converting iterables

```
#str to list
word="hello rose"
print("str to list is :",list(word))
#list to tuple
lst=[1,5,4,7]
print("list to tuple:", tuple(lst))
#list to set
print("list to set is :",set(lst))
```

Output:

```
str to list is : ['h', 'e', 'l', 'l', 'o', ' ', 'r', 'o', 's', 'e']
list to tuple: (1, 5, 4, 7)
list to set is : {1, 4, 5, 7}
```

For practice:

1. write a program to create a list whose elements are divisible by n but nor divisible by n^2 for a given range.

Solution:

```
a=int(input("enter starting number :"))
b=int(input("enter end  number :"))
n=int(input("enter the value of n: "))
n_list=[ ]
for i in range(a,b+1):
    if i%n==0 and i%(n*n)!=0:
        n_list.append(i)
print(n_list)
```

Output:

```
enter starting number :12
enter end  number :24
enter the value of n: 3
[12, 15, 21, 24]
```

2. Write a program to create a dictionary that will contain all ASCII values and their corresponding characters.

Solution:

```
Ascii_dict={}
for i in range (100):
Ascii_dict[i]=chr(i)
print(Ascii_dict)
```

Output:

{0: '\x00', 1: '\x01', 2: '\x02', 3: '\x03', 4: '\x04', 5: '\x05', 6: '\x06', 7: '\x07', 8: '\x08', 9: '\t', 10: '\n', 11: '\x0b', 12: '\x0c', 13: '\r', 14: '\x0e', 15: '\x0f', 16: '\x10', 17: '\x11', 18: '\x12', 19: '\x13', 20: '\x14', 21: '\x15', 22: '\x16', 23: '\x17', 24: '\x18', 25: '\x19', 26: '\x1a', 27: '\x1b', 28: '\x1c', 29: '\x1d', 30: '\x1e', 31: '\x1f', 32: ' ', 33: '!', 34: '"', 35: '#', 36: '$', 37: '%', 38: '&', 39: "'", 40: '(', 41: ')', 42: '*', 43: '+', 44: ',', 45: '-', 46: '.', 47: '/', 48: '0', 49: '1', 50: '2', 51: '3', 52: '4', 53: '5', 54: '6', 55: '7', 56: '8', 57: '9', 58: ':', 59: ';', 60: '<', 61: '=', 62: '>', 63: '?', 64: '@', 65: 'A', 66: 'B', 67: 'C', 68: 'D', 69: 'E', 70: 'F', 71: 'G', 72: 'H', 73: 'I', 74: 'J', 75: 'K', 76: 'L', 77: 'M', 78: 'N', 79: 'O', 80: 'P', 81: 'Q', 82: 'R', 83: 'S', 84: 'T', 85: 'U', 86: 'V', 87: 'W', 88: 'X', 89: 'Y', 90: 'Z', 91: '[', 92: '\\', 93: ']', 94: '^', 95: '_', 96: '`', 97: 'a', 98: 'b', 99: 'c'}

3. Write a program to test whether a variable is a list or a tuple or a set.

solution:

```
def check(a):
  if type(a)==list:
    print(a,"is a list")
  elif type(a)==tuple:
    print(a,"is a list")
  elif type(a)==set:
    print(a,"is a set")
x={1,2,3,4,5}
y=(1,2,4,5,6)
z={1,2,5}
check(x)
check(y)
check(z)
```

Output:

```
[1, 2, 3, 4, 5] is a list
(1, 2, 4, 5, 6) is a list
{1, 2, 5} is a set
```

4. Write a program to create two sets such that one will contain the names of the students who have passed in Java and the other will contain the names of those who have passed in Data Structure.

i. Find the students who have passed in both subjects.
ii. Find the students who have passed in at least one subject.
iii. Find the students who have passed in one subject but not in both.
iv. Find the students who have passed in Java but not in Data Structure.
v. Find the students who have passed in Data Structure but not in Java.

Solution: #Set Operation

```
n=int(input("Enter the number of students who have passed in Java:"))
java=set()
for i in range(n):
    sname=input("Enter name %d:" %(i+1))
    java.add(sname)
n=int(input("enter the number of students who have passed in Data Structure :"))
```

```
datastructure=set()
for i in range(n):
    sname=input("enter name %d :"%(i+1))
    datastructure.add(sname)
print("Students passed in JAVA :",java)
print("Students passed in DATA STRUCTURE :",datastructure)
print("Students passed in Both Subject:", java & datastructure)
print("Students passed in at least one subject:", java | datastructure)
print("Students passed in one subject but not in both :", java ^ datastructure)
print("Students passed in JAVA but not in Data Structure:", java-datastructure)
print("Students passed in DATA STRUCTURE  but not in JAVA:", datastructure-java)
```

Output:

```
Enter the number of students who have passed in Java:3
Enter name 1:anu
Enter name 2:balu
Enter name 3:celsia
enter the number of students who have passed in Data Structure :4
enter name 1 :anu
enter name 2 :celsia
enter name 3 :fero
enter name 4 :manoj
Students passed in JAVA : {'anu', 'raja', 'balu', 'celsia'}
Students passed in DATA STRUCTURE : {'fero', 'anu', 'manoj', 'celsia'}
Students passed in Both Subject: {'anu', 'celsia'}
Students passed in at least one subject: {'anu', 'celsia', 'raja', 'fero', 'manoj', 'balu'}
Students passed in one subject but not in both : {'balu', 'raja', 'fero', 'manoj'}
Students passed in JAVA but not in Data Structure: {'raja', 'balu'}
Students passed in DATA STRUCTURE  but not in JAVA: {'fero', 'manoj'}
```

3.9. Mutable And Immutable Data Types:

Python has data types for collections that can change and the ones that are unable to change. Dictionaries, sets, as well as lists, can be changed i.e. mutable, but strings and tuples can't i.e. immutable.

Difference Between Mutable and Immutable Data Types:

Mutable data types	Immutable data types
An item or object that is capable of being modified after it has been created is called "mutable"	An item or object that is not capable of being modified after it has been created is called "immutable"
List, set, and dictionary are examples.	Frozenset, tuples ,int, float, and bool are examples.
mutable item are not thread-safe by their very nature.	immutable are thread-safe by their fixed nature.
Accessing changeable objects takes more time than accessing fixed objects.	accessing immutable objects is faster than accessing changeable objects.
Whenever we require to modify the size and / or content of an object.	we should use immutable objects when we know we aren't going to modify them at any point.
changing objects can be cheaper in terms of taking less time and space.	changing an item that can't be changed is expensive because each requires making a new copy.

3.10.Excersice:

1. Explain the Data Types available in python.
2. a)What are the data types? How are they important
   b)Write the names of any four data types available in python.
3. How many integer types are supported by python ? name them.
4. What are immutable and mutable types? List immutable and mutable types of python.
5. What are the properties of a list?
6. 'List items are ordered '- what does it mean?
7. What are differences between an array and a list?
8. What are the different ways by which a list can be created?
9. List items are ordered – what does it mean?
10. Write the output for the following code:

```
List1=["BCA","CS",[2,4,6,8,10]]
Print(List1)
Print(List1[0])
Print(List1[0][2])
Print(List1[ ::-1])
```

11. Explain the slicing operation in a list.
12. Is it possible to add multiple items in a list in a single operation? Explain
13. What are the different methods in python by which elements can be added in a list?
14. What will be the output for the following?

```
Num=[2,6,12,18,22]
Num[0]=3
Print(Num)
Num=[1:4]=['a', 'c', 'e']
Print(Num)
```

15. Different between a list and a tuple.
16. How can we delete multiple item from list? Explain with example.
17. Is it possible to sort tuple elements?
18. Create a tuple containing only a single element.
19. Is it possible to modify any tuple element? What happens when the following statements are executed?

```
my_tup=(5,6,9,4,[1,3])
my_tup[3][0]=7
print(my_tup)
```

20. What are the operation that can be done on a set?
21. Explain in detail about list traversal with example.
22. What is frozenset? Explain with example.
23. What is a dictionary in python?
24. Dictionaries are not sequences, rather they are mapping. Explain.
25. Differentiate between get( ) and setdefault ( ).
26. What is the utility of the update ( ) method in a dictionary?
27. Explain nested dictionary with example.

28. What is type conversion? Explain about the types with examples.

29. Write the difference between about mutable and immutable data types.

30. Write a python program to convert the iterables from string to integer and float.

31. 'Tuples are immutable' Explain the statement with example.

32. Write the difference between mutable and immutable data types.

33. Explain the following: a) formkeys() b)get( ) c)len( )

34. What is Type conversion? Explain its types.

35. Write the output for the following

X=[2 ** X for X in range(1,10)]

****

BOOK 4: PYTHON CONTROL STRUCTURES

4.1. Control Structures: The Building Blocks of Logic

Control Structures are the backbone of programming in python, allowing for dynamic execution of code blocks based on certain conditions or the repeated execution of a code block.

Control structures consist of ***conditional statements and loops*** used to manage the execution flow of a program. Control structures guide a program's decisions and repetitions through conditionals and looping constructs.

Note: A program statements that causes a jump of control from one part of the program to another is called control structure or control statements.

4.1.1.Flowchart of Control Structures:

A flowchart is a graphical or symbolic representation of a process. Each step in process is represented by a different symbol and contains a short description of the process step. A flowchart symbol s are linked together with arrows showing the process flow direction.

Flowchart is basically the plan to be followed when the program is written. It acts like a road map for a programmer and guides them how to go from the starting point to the final point while writing in the computer.

4.1.1.1. Flowchart symbols:

A few symbols are needed to indicate the necessary operations in a flowchart. These symbols have been standardized by the American .national Standards Institute(ANSI).

Terminal Symbol:

Terminal symbol is used to indicate the starting (Begin), stopping(End), and pause (Halt) in the program logic flow. It is the first symbol and also the last symbol in the program logic.

Input/Output Symbol:

Input/Output symbol is used to indicate the operation of an input/ output device in the program.

Processing:

Processing symbol is used in a flowchart to represent arithmetic instructions and data movement instructions. Thus all arithmetic processes , such as addition, subtraction, multiplication and division are shown by processing symbols.

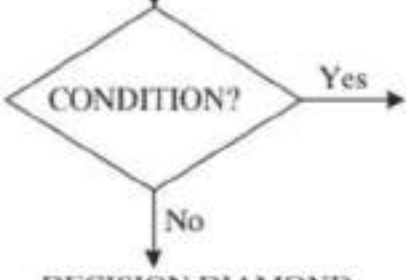
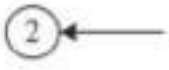
Decision Symbol:

Decision symbol is used in a flowchart to indicate a point at which a decision has to be made and to branch to one of the alternatives.

Connector Symbol:

Whenever a flowchart becomes complex enough and the number and direction of flow lines are hard to understand over more than one page, it is recommended to utilize the connector symbol as substitute for flow lines. This symbol represents an entry from, or an exit to another part of flow chart.

The symbols are mentioned below

SYMBOL	EXAMPLE	DESCRIPTION
 TERMINAL	BEGIN	Start action here
	END	Stop action here
 INPUT/OUTPUT	INPUT X, Y	Take two values from an external source and assign them to X and Y on a terminal
	PRINT X, Y	Write the values contained in X & Y on a terminal / screen
PROCESSING	Z ←(X + Y)	Add the value contained in Y to the value contained in X and place the result in Z
 ENTRY CONNECTOR		An entry in the flowchart is made at the connecting point marked (1)
 DECISION DIAMOND		If condition is satisfied, then *Yes* path is to be followed otherwise *No* path is to be taken
 FLOW LINES		The arrows indicate the paths to systematic solution of the problem.
 CONNECTOR (TRANSFER)		A transfer of problem solution is made at the connector point marked (2) in the flowchart

4.2.What is a Boolean Expression?

The tern Boolean comes from the name of the British mathematician George Boole. Boolean expressions are questions and they should be read as "*Is something equal to/greater than/less than something else?"* and the answer is just a "yes" or "No"(True or False).

A decision control structure can evaluate a Boolean expression or a set of Boolean expression and then decide which block of statements to execute.

For e.g.

In interactive Shell we see

```
>>>True
True
>>>False
False
>>>type(True)
<class 'bool'>
>>>type(False)
<class 'bool'>
```

Note: A Boolean expression is an expression that results in a Boolean value , that is either ***True or False***

4.2.1. Boolean Expressions Using Comparisons:

A simple Boolean expression is written as

Operand 1 comparision_operator operand 2

Where operand 1 and operand 2 can be values , variables or mathematical expressions.

For e.g: x>y . This Boolean expression is a question to the computer and can be read as "is x greater than y?".

The comparison operator can be one of those from below of Python Relational Operator table:

Operator	Expression	Description
==(Equal; not assignment)	x= =y	True if x=y(mathematical equality, not assignment);otherwise false
! =	x!=y	True if x!=y; Otherwise, False
>	x>y	True if x>y; Otherwise, False
<	x<y	True if x<y; Otherwise, False
>=	x>=y	True if x>=y; Otherwise , False
<=	x<=y	True if x<=y; Otherwise, False

Example:

The Boolean expression c > 3 * a - b, using a = 3, b = -5, and c = 7, gives a False outcome.

Reason:

- ✓ in the first line ,a is equal to 3 and b is equal to -5
- ✓ the result of the expression

*3*3 – (-5)=3*3+5=14*

c>14=False

Examples of simple Relational Expression:

Expression	Value
10<20	True
10>20	False
X<12	True if x is less than 12; Otherwise ,False
X!=y	True unless x and y are equal

Example:

```
a=4
b=5
c=a*b<a!=b
print('a=',a, 'b=',b)
print(c)
```

Output:

```
a= 4 b= 5
False
```

For Practice: Analyze and Evaluate:

a	b	c	a= =12	b<=c	c>2*b-a	(b-a)>=3*a-c
3	-4	7				
10	10	23				
-4	-2	-9				

4.2.2. Boolean Expression using Logical and Complex:

A complex Boolean Expression can be built of simpler **Boolean Expression and can be written as:**

BE 1 Logical_Operator BE 2

Where BE 1 and BE 2 can be any Boolean expression. Logical operator can be one of those from the logical operators of *and(&), or(|) and not(!)* operators.

Note: When you combine simple Boolean expressions with logical operators, the whole Boolean expression is called a "complex Boolean Expression",
For e.g. x==4 and y>6

Example of Complex Boolean Expression:

X=10, y=5, z=1

Operator	Expression	Result
and (&)	x>y and x<y	False (True only both Expression is true; otherwise ,False)
or (|)	(x==y) or (y==z)	False(True either 1st or 2nd expression is true; Otherwise false)
not (!)	not (x<2)	True(The whole Boolean expression is True when the Boolean Expression is False and vice versa.)

Example:

```
x=12
y=32
z=8
print(3*x>y and x==y)
print((x!=y) or (y<=z))
print( not x<z)
```

Output:

```
False
True
True
```

4.2.2.1. Operator Precedence: What Comes First in Python

i. Order precedence of Logical operator:

A more complex Boolean expression may use several logical operators like the below expression

name=="Raj" or age >12 and not name=="Priya"

so, which logical operation is performed first?

The order of precedence is: Logical Complements(not) are performed first, Logical conjunctions (and) are performed next, and logical disjunctions (or) are performed at the end.

Example: To display the output for the following Expression

if a=1 , b=-5 , c=7, and the expression is a>b or c>b* and c>2

Solution:

```
a=1
b=5
c=7
print(a>2 or c >b*2 and c > 2)
```

Output:

```
False
```

Note: *The* ***'and'*** *operation has a higher precedence and performed before the* ***'or'*** *operation.*

ii. Order precedence of Arithmetic, Comparison, Membership, and Logical Operators:

In such cases , arithmetic operations are performed at first, comparison and membership operations are performed next, and logical operations are performed at the end.

Example:

```
a=1,b=5,c=7
print(a*b + 2 > 21 or not (c==b/2) and c>13)
```

Output:

```
False
```

- In this program, three variables are given: a = 1, b = 5, and c = 7. The expression inside the print() statement uses a combination of math and logical operators like ***or, not,*** and ***and***. First, a * b + 2 is calculated, which becomes 1 * 5 + 2 = 7. Then it checks if 7 > 21, which is **False**.
- Then, it checks if c == b / 2, or 7 == 2.5, which is also **False**. The not operator turns this into **True**. Then it checks c > 13, which is 7 > 13, again **False**.
- So the expression becomes: False or True and False. In Python, and is solved before or, so True and False becomes **False**, and finally False or False results in **False**. So the program prints the final output is False.

4.2.3. Boolean Expression using Membership Operator:

Membership operators are used in Boolean expressions to test whether a value is part of a sequence, such as a list, string, tuple, or set.

Membership Operator	Description
in	It evaluates to True if it finds a value in the specified sequence; it evaluates to False otherwise.
not in	It evaluates to True if it does not find a value in the specified sequence; it evaluates to False otherwise.

Example:

- X in [1,3,4]. This can be read as "*is x equal to 1, or equal to 3, or equal to 4?*". It can also be written as x==1 or x==3 or x==4.
- 3 in [x, y, a]. this can read *as " is 3 equal to x, or y, or a".*

4.2.4. How to negate Boolean Expression:

Negation is the process of reversing the meaning of Boolean expression. There are two ways used to negate a Boolean expression. i.e.

i) First method: Just use a not operator in front of the original Boolean expression.

For e.g. the expression is x< 3 and y == 5,the negate expression becomes not(x<3 and y ==5).

ii) Second method: To do every operator into negate according to the following table

For e.g.

the expression is

x > 5 and y = =2

the negate expression is x <= 5 or y != 2

Table 4.2.4. Negate operators

Original Operator	Negated Operator
= =	!=
!=	= =
>	<=
>=	>
<	>=
<=	>
in	not in
not in	in
and	or
or	and
not	not

Note: That the not operator remains intact

For practice:

1.Negate the following expression

a) x >= 4 and x <= 10 or y = =3

Solution:

The negate expression is: (x < 5 or x > 10)and y != 3.

b)b != 4

Solution:

The negate expression is: b = =4.

c) b > 5 and not(x > 5)

solution:

The negate expression is: not (b>7 and not (x > 5) or b <= 7 or not (x<=5)

4.3. Sequential Statements:

It is the default mode. Sequence control structure is the way that lines are run one after the other ,in the same order as they appear in the program. This is called line-by-line processing. They might do a number of read or write operations, computations, or assignments to variables.

For e.g: program to print the arithmetic calculations by using input( ) method:

```
a=int(input("enter the first number:"))
b=int(input("enter the second number"))
c=a+b//2
d=c*4
print("The value of c is :",c)
print("The value of d is :",d)
```

Output:

```
enter the first number:12
enter the second number8
The value of c is : 16
The value of d is : 64
```

For practice:

1. Program to calculate the Area and circumference of the circle by using sequential statement.

Solution:

```
r=int(input("enter the radius R="))
pi=3.14
area=pi*r*r
circumference=2*pi*r
print("Area of circle: ",area)
print("circumference of circle= ",circumference)
```

Output:

```
enter the radius R=4
Area of circle:  50.24
circumference of circle=  25.12
```

4.4.Conditional Statements:

Conditional control statements are also called branching or selection statements that execute one group of instructions depending on the outcome of a decision.

Conditional statements in python allow a program to react differently based on whether certain conditions are true or false. The if statement forms the basis of these structures, with optional elif (else if) and else sections providing additional conditions.

Conditional statements are divided into

a. simple if statements

b. if-else statements

c. if-elif-else chains

d. nested if statements – each serving to handle different decision-making scenarios.

4.4.1. Grouping and Indentation:

Every single line of code may be indented by leaving a blank line before it. When compared to other languages, python's indentation requirements are more stringent since they are intended to improve readability.

Key : Python uses four spaces as default indentation spaces.

Note: Python uses indentation to demarcate different parts of a program. Indentation is a tool used by programmers to help explain the logic behind their code to readers.

In python , indentation refers to the technique of inserting whitespace before the first character of a line.

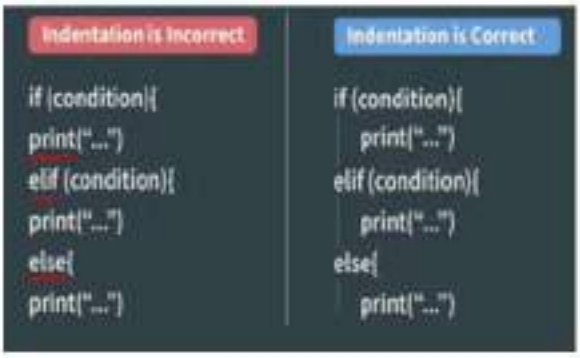
Here' s an example:

1. Single statement on the same line

if 2+2 ==4: print(True)

2. Single statement indented on next line

```
if 2+2 ==4:
    print(True)
```



The test is introduced with the keyword ***if*** and ***terminates with colon***. If the statement block is a single statement , it can occure on the same line or the next. If it occures on the next line, it must be spaced or tabbed in at least one space or tab. If there is no indentation , an error is produced. In below are mentioned the error code:

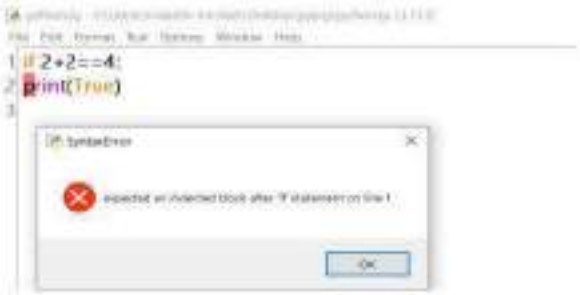


Figure of incorrect indentation

For e.g.

1). Incorrect indentation:

```
if 2+2 ==4:
print(True) #produces a syntax error
```

2).Multiple statements correctly indented

```
if 2*2 >= 4:
    print(True)
    print("code block has ended")
```

Output:

True

code block has ended

In Principle , you can indent with either spaces or tabs, but do not mix them.

For e.g

 # Statement block 1

```
if 3*2<=12:
    print("The Boolean Expression is True")
    print("you can go to the next ")
```

Output:

The Boolean Expression is True

you can go to the next

#Statement block 2

```
if (12-8)!=4:
    print(True)
print("that's not True")
```

Output:

that's not True

From the above two statements, the first statement block both the print statement are executed. In the second case, the second print statement has executed.since it not intended ,it is evaluated as being outside the if structure and thus not contingent on the if test.

4.4.2. The Single -Alternative Selection Structure (if Statement):

An *if* statement consist of a Boolean expression followed by one or more statements.

This is the simplest decision control structure. It includes a statement or block of statements or the " *True* " path only, as presented in the following flowchart fragment , given in general form.

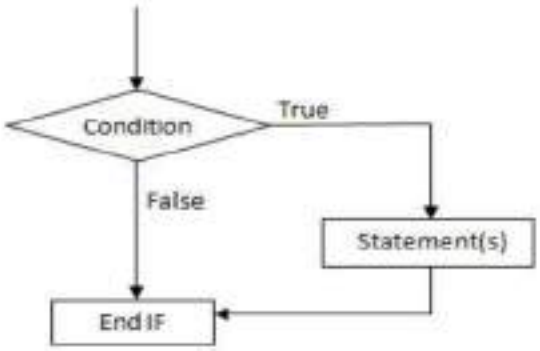


fig: Flowchart for if statement

Control Flow of *if* Statement:

The condition block (Boolean expression)evaluates to *True*, the statement, or block of statements , of the structure is executed; otherwise, the statements are skipped.

Syntax:

if Boolean_Expression :

A statement or block of statements

Note: the statement or block of statements is indented by 4 spaces

Example 1: Program to evaluate and print voting eligibility

```
age=int(input("Enter your age :"))
if age >=18:
  print("You are eligible to vote")
```

Output 1:

```
Enter your age :21
You are eligible to vote
```

Output 2:

```
Enter your age :12
```

The if statement block does not execute or display anything in output 2, because the condition is False for the value 12.

Example 2: Print the highest number by evaluating comparisons between values

```
a=14
b=5
c=6
if a>b:
  print("a is greater")
```

Output:

```
a is greater
```

Example 3: Program to demonstrate how indentation affects the output

#1

```
x=int(input("Enter a number"))
y= int(input("Enter a number"))
if x== y:
  x +=2
  print(x)
```

Output 1:

```
Enter a number12    # When Expression is True
Enter a number12
14
```

Output 2:

```
Enter a number20    # When Expression is False
Enter a number13
```

#2.
```
x=int(input("Enter a number"))
y= int(input("Enter a number"))
if x== y:
   x +=2
print(x)
```

Output1 :

```
Enter a number34
Enter a number23
34
```

Output 2 :

```
Enter a number23
Enter a number23
25
```

This program takes two integer inputs, x and y. It then checks if both values are equal. If they are, it increases x by 2 using the x += 2 statement. Finally, it prints the value of x. If the values are not equal, it prints x as it is.

Example 4: Compare Values and Display Output with If Statements

```
x=int(input("Enter a number"))
y=int(input("Enter a number"))
if x>y:  # condition 1 , Executes this block if true, or checks the next condition.
 x*=2
 print(x)
if x<y:  # condition 2
 x-=12
 print(x)
```

Output 1:

```
Enter a number10
Enter a number4
20
```

Output 2:

```
Enter a number4
Enter a number21
-8
```

Example 5: Indentation in nested independent statement:

 # nested if with two-statement block

```
if (2+2)*2>=7:
 print("Good Morning")
 if 5*4==21:
      print("hi")
      print("Have a great day")
```

Nested independent statement

Output:

```
Good Morning
```

In this program, the if statement checks if (2 + 2) * 2 is greater than or equal to 7. The result is 8, which is true, so it prints "Good Morning". Inside this block, there is another if—this is called a **nested if statement**, meaning one if inside another.

The second condition checks if 5 * 4 equals 21. Since 5 * 4 is 20, the condition is false, and the inner lines ("hi" and "Have a great day") are skipped. So, only "Good Morning" is printed.

#nested if, with independent statement

```
if (3/15)<=2:
  print("Greetings")
  if 7*8>=63:
      print("come again")
print("visit again ")
```

← Independent statement

Output:

```
Greetings
visit again
```

The program first checks if 3 divided by 15 is less than or equal to 2. Since 3 / 15 is 0.2, which is less than 2, the condition is true and it prints "Greetings". Inside this block, there is another if—a **nested if**—that checks if 7 * 8 is greater than or equal to 63. Since 7 * 8 is 56, the condition is false, so "come again" is not printed.

After the if blocks, the program prints "visit again" because that line is **outside** the if statements and runs independently.

For practice:

1. Write a program to take a value from the user and print the square of it , unless it is more than 120. The message" I cannot square numbers that appears if the user types too large a number".

Solution:

```
num=int(input("What number do you want to see the square of ?:"))
if num<120:
   square=num*num
   print("the square of %d is %d"%(num, square))
if num>=120:
   print("\t square is not allowed for numbers over 120\t")
   print("\t Run this program again and try a smaller value.")
print("Thank you for requesting squares.")
```

Output 1:

```
What number do you want to see the square of ?:34
the square of 34 is 1156
Thank you for requesting squares.
```

Output 2:

```
What number do you want to see the square of ?:211
square is not allowed for numbers over 120
Run this program again and try a smaller value.
Thank you for requesting squares.
```

2. Write a program to find the roots of a quadratic equation $root = \frac{-b \pm \sqrt{b^2-4ac}}{2a}$

Solution

```
import math
print("\t Solve the quadratic equation root=(-b±√(b^2-4ac))/2a")
a=int(input("enter a value :"))
b=int(input("enter b value :"))
c=int(input("enter c value :"))
d=b*b-4 *a*c
r1=r2=1
e=math.sqrt(abs(d))
if (d<0):
  print("Roots are imaginary")
if (d==0):
  r1=(-b + e)/(2*a);
  r2=(-b - e)/(2*a);
if(d==0):
  print("Roots are equal")
print("first root is :",r1)
print("second root is:",r2)
```

Output:

```
Solve the quadratic equation root=(-b±√(b^2-4ac))/2a
enter a value :2
enter b value :4
enter c value :7
Roots are imaginary
first root is : 1
second root is: 1
```

4.4.3. Two-Way selection Structure (if -else):

The ***if*** Keyword is followed by a logical expression, after which a colon (:) should be written. Then in a new line, using 4 spaces, as recommended by python, you can write the statements of the branch that are to be executed if the logical expression id *True*.

The colon (:) after is followed by the statements (also indented with 4 spaces) that are to be executed if the logical expression is False.

Syntax:

If logical_expression:

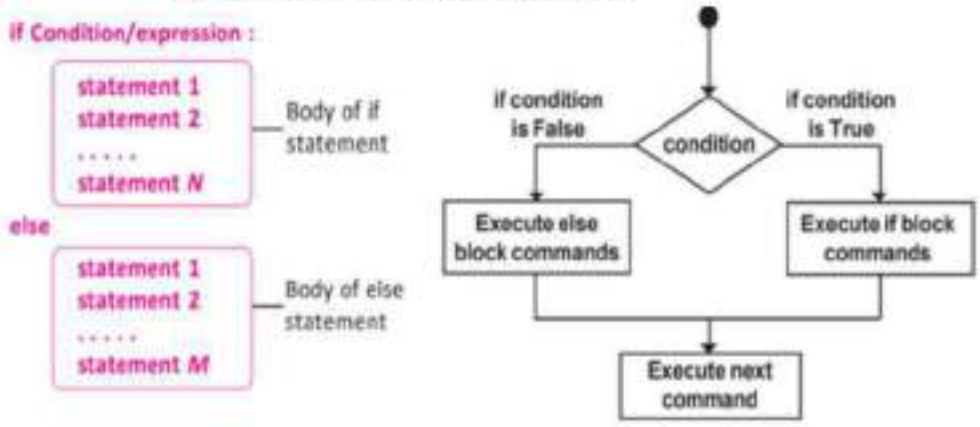
Statements if the logical expression is true

else:

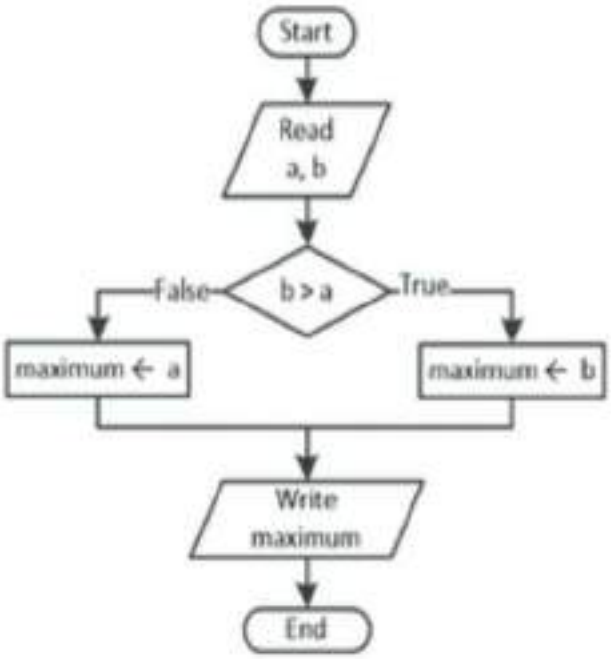
statements if the logical expression is false

Independent statements following the two-way selection structure

Control Flow of if- else structure:



Example: Design a flowchart and write the python program that lets the user enter two numbers A and B and then determines and displays the greater of the two numbers.



Solution:

```
a=float(input("enter the value for a:"))
b=float(input("Enter the value for b:"))
if b >a :
  max=b
else:
  max=a
print("the greatest value is:", max)
```

Output 1:

```
enter the value for a:13
Enter the value for b:34
the greatest value is: 34.0
```

Output 2:

```
enter the value for a:18
Enter the value for b:10
the greatest value is: 18.0
```

Note: From the above example is trying to determine the greatest value and not which variable this value is actually assigned to (to variable A or to Variable B).

For practice:

1. Program to check whether a number is even or odd using an if-else statement.

Solution:

```
x=int(input("enter an integer:"))
if x%2==0:
  x=int(x/2)
  print("The Half of the entered  number is ",x)
else:
  x=int((x+1)/2)
  print("the Half of the entered number increased by one is:",x)
print("-----Statements following the two-way selection structure-----")
```

Output 1:

 # from the true block of if statement ***enter an integer:8***

The entered number is 4

-----Statements following the two-way selection structure-----

Output 2:

 # from the else block of statement

enter an integer:11

the Half of the entered number increased by one is: 6

-----Statements following the two-way selection structure-----

2. Program to assess and display a person's health status using if-else

Solution:

```
s=input("Enter Your sex M or F:")
    if ((s=='M')or(s=='m')):
      print("You are a Male")
      print("you are good at your health:")
    else:
      print("You are a female")
      print("You are beautiful:")
```

Output 1:

Enter Your sex M or F:m

You are a Male

you are good at your health:

Output 2:

Enter Your sex M or F:f

You are a female

You are beautiful

3. Write a program to input and using if-else statement, print whether the number is greater than zero or not.

Solution:

```
num=int(input("What is your number?"))
if(num>0):
  print("more than 0.")
else:
  print("less than 0")
```

Output 1:

What is your number?10

more than 0.

Output 2:

What is your number?-32547

less than 0

4. Write a program to input a number and check whether it is a Buzz number of not. (A number is said to be Buzz number if it ends with 7 or is divisible by 7)

Solution:

```
num=int(input("Enter the number to check Buzz number:"))
if (num%10==7 or num%7==0):
  print(num, "is a Buzz number")
else:
  print(num, "is not a Buzz number")
```

Output 1:

Enter the number to check Buzz number:1457

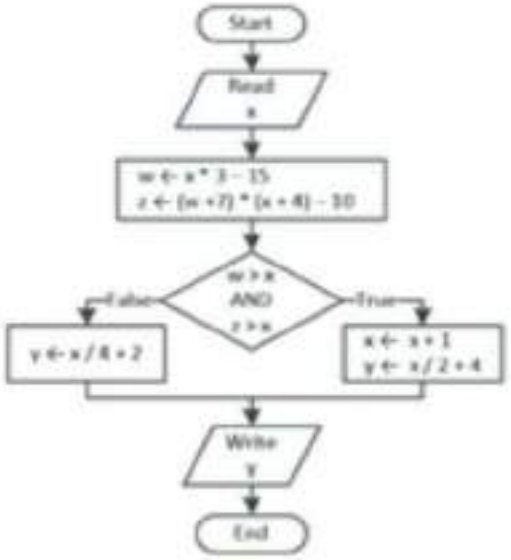
1457 is a Buzz number

Output 2:

Enter the number to check Buzz number:100

100 is not a Buzz number

5. Write the python program for following flowchart:



Solution:

```
x=int(input("enter the value of x :"))
w=x * 3-15
z=(w + 7) * (x + 4) - 10
if ( w > x) and ( z > x ):
    x=x+1
    y=x/2+4
    print("from if  block: ",y)
else:
    y=x/4+2
    print("from else  block :",y)
```

Output 1:

enter the value of x :11

from if block: 10.0

Output 2:

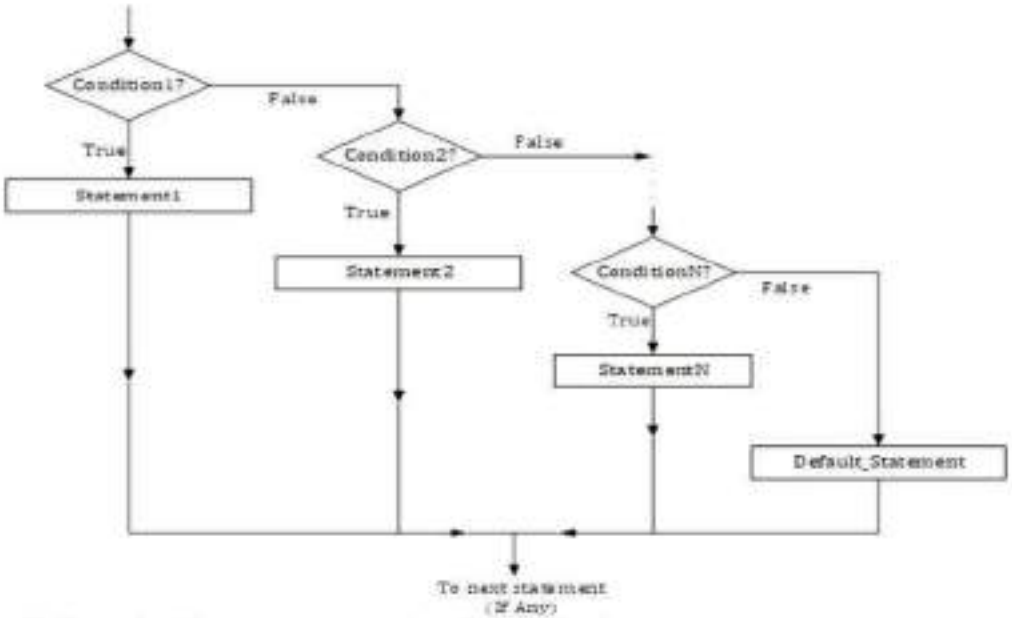
enter the value of x :6

from else block : 3.5

4.4.4. Multi-way Selection:

The ***elif*** statement allows you to check multiple expressions for truth value and execute a block of code as soon as one of the condition evaluates to true. Like the ***else, the elif*** statement is optional. However unlike ***else,*** for which there can be at most on statement, there can be an arbitrary number of elif statement following an ***if.***

Control Flow of if-elif:



Example 1 : print the average marks of a student.

Syntax:

```
if expression1:
   Statement(s)
elif expression2:
  statement(s)
elif expression3:
  statement(s)
else:
   statements(s)
```

Or

```
if expression1:
  Statement(s)
else:
   if expression2:
     statement(s)
  else:
       if expression3:
         statement(s)
       else:
         statement(s)
```

→ First method:

```
avg=int(input("enter the average mark :"))
if avg>80:
  print ("grade A")
  print  ("Excellent")
elif avg>70:
  print ("Grade B ")
  print ("Good")
elif avg>60:
  print ("Grade C")
  print (" Achieved")
elif avg>50:
  print ("pass")
  print ("Good luck")
```

```
else:
  print("grade is F")
  print("fail")
```

Output:

```
enter the average mark :45
grade is F
fail
enter the average mark :72
Grade B
Good
enter the average mark :100
grade A
Excellent
```

→ **Second method:**

```
avg=int(input("enter the average mark :"))
if avg>80:
  print ("grade A")
  print  ("Excellent")
else:
    if avg>70:
       print ("Grade B ")
       print ("Good")
    else:
      if avg>60:
        print ("Grade C")
        print (" Achieved")
     else:
        if avg>50:
          print ("pass")
          print ("Good luck")
        else:
          print("grade is F")
          print("fail")
```

Output:

```
enter the average mark :62
Grade C
 Achieved
enter the average mark :92
grade A
Excellent
enter the average mark :35
grade is F
fail
```

Example 2: Program to print weather remarks.

```
temp=int(input ("enter temperature value: "))
if temp>35:
  print("it is too hot today ")
else:
  if(temp>25)and(temp<=35):
   print("it is warm today")
  else:
   print("it is cooler today")
```

Output 1:

```
enter temperature value: 50
it is too hot today
```

Output 2:

```
enter temperature value: 20
it is cooler today
```

Example 3: program to check the entered character is vowel or consonant.

```
str1=input("enter a character : ")
ch=str1[0]  #extracts first character from str1
if(ch=='a' or ch=='A'):
  print(ch, "is vowel.")
elif(ch=='e' or ch=='E'):
  print(ch, "is vowel")
elif(ch=='i' or ch=='I'):
  print(ch, "is vowel")
elif(ch=='o' or ch=='O'):
  print(ch, "is vowel")
elif(ch=='u' or ch=='U'):
  print(ch, "is vowel")
else:
  print(ch ,"is a consonant.")
```

Outputs:

```
 #1
enter a character : u
u is vowel
#2
enter a character : s
s is a consonant.
#3
enter a character : i am a girl
i is vowel
```

Example 4: Python program that prompts the user to enter an integer between 0 and 999 and then counts its total number of digits and display an error message to the user when they enter a value that is not between 0 and 999.

Solution:

```
x=int(input("Enter an integer (0 - 999):"))
if 0<= x <= 9:
  print( " a 1 -digit integer entered")
elif 10<= x <=99:
  print( " a 2- digit integer entered")
elif 100 <= x <= 999:
  print(" A 3- digit integer entered")
else:
  print("wrong number")
```

Outputs:

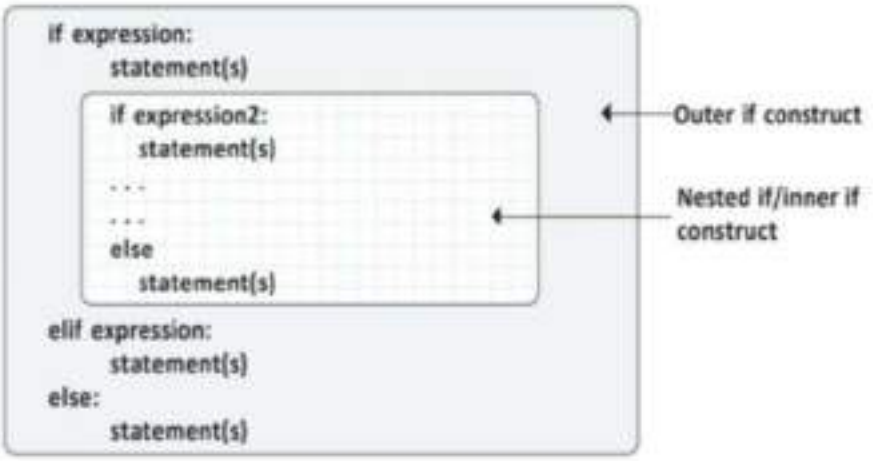
```
Enter an integer (0 - 999):8
a 1 -digit integer entered
```

```
Enter an integer (0 - 999):12
 a 2- digit integer entered
Enter an integer (0 - 999):254
 A 3- digit integer entered
Enter an integer (0 - 999):10231
 wrong number
```

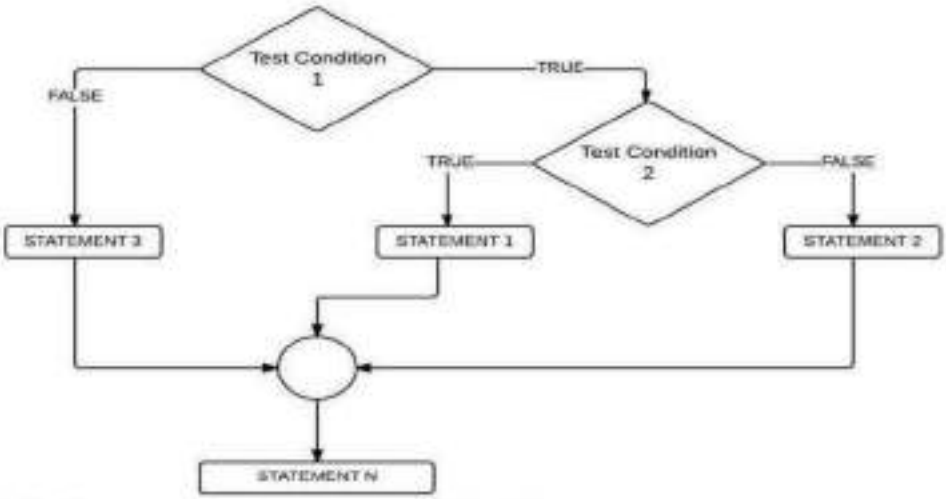
4.4.5. Nested if :

Nested decision control structures are decision control structures that are "nested" (enclosed) within another decision control structure . This means that one decision control structure can nest (enclose) another decision control structure (which then becomes the "nested" decision control structure).

Syntax:



Control Flow of Nested if:



Example 1: Program to print the Nested Decision.

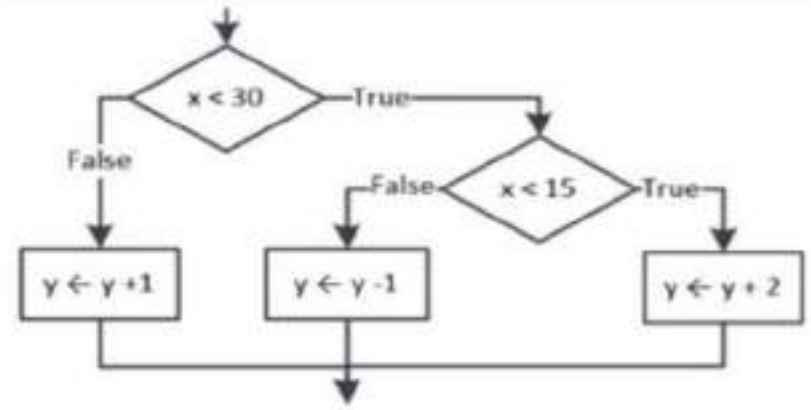
```
x= int(input("enter the x value:"))
y=10
if x<30:
   if x<15:
      y=y+2
      print(y)
   else:
      y-=1
      print(y)
```

```
    else:
       y+=1
       print(y)
```

Output:

```
enter the x value:10
12
enter the x value:31
 11
```

Flowchart to print Nested Decision



Example 2: Program to print the smallest number among three input numbers.

```
num1=int(input("enter the value of num1:"))
num2=int(input("enter the value of num2:"))
num3=int(input("enter the value of num3:"))
if num1<num2:
  if num1<num3:
   smallest=num1
  else:
   smallest=num3
else:
  if num2<num3:
   smallest=num2
  else:
   smallest=num3
print("The smallest number is:", smallest)
```

Output:

```
enter the value of num1:13
enter the value of num2:1
enter the value of num3:41
The smallest number is:1
```

For practice:

1. Write the python program using nested if construct to check a number is positive, negative or zero.

Solution:

```
num=int(input("enter a number :"))
    if num >=0:
        if num==0:
            print("zero")
        else:
            print("positive number")
    else:
        print("negative number")
```

Nested if construct

Output:

```
enter a number :-4
negative number
enter a number :51
positive number
```

In this program, the expression num>=0 is tested and two possibilities happens:

1. If the expression **num>=0** is true , it will follow the inner if condition. This is done with the nested if / else block, which tests the condition **num==0.** If the inner condition is also true, the inner if condition message will display as "**Zero",** otherwise, display as "positive number"
2. If the expression **num>=0** is false, it will follow its own else part without checking the inner if condition.

2. Write a menu driven program to generate a simple calculator to find addition, subtraction, multiplication and division of two numbers. The menu is a given below:

Math calculator Menu

+ =>Addition
- =>subtraction
***=>Multiplication**
/=>Division

Solution:

```
print("Math Calculator")
print("--------------------")
print("+ =>Addition")
print("- =>subtraction")
print("*=>Multiplication")
print("/=>Division")
opt=input("enter an operator[either +, - ,* or /]:")
if(opt=='+' or opt=='-' or opt =='*' or opt =='/'):
        num1=int(input("enter number 1 :"))
        num2=int(input("enter number 2:"))
 if(opt=='+'):
        sum=num1+num2
        print("The sum of",num1,"and",num2,"is=",sum)
 elif(opt=='-'):
   if(num1>num2):
        diff=num1-num2
   else:
        diff=num2-num1
        print("The difference between ,",num1,"and",num2, "is=",diff)
 elif(opt=='*'):
        mul=num1*num2
```

```
            print("The multiplication of",num1,"and",num2,"is=",mul)
    else:
        print("invalid option selected")
```

Output 1:

```
 Math Calculator
 --------------------
 + =>Addition
 - =>subtraction
 *=>Multiplication
 /=>Division
 enter an operator[either +, - ,* or /]:+
 enter number 1 :14
 enter number 2:3
 The sum of 14 and 3 is= 17
```

Output 2:

```
 Math Calculator
 --------------------
 + =>Addition
 - =>subtraction
 *=>Multiplication
 /=>Division
 enter an operator[either +, - ,* or /]:0
 invalid option selected
```

3. Star complex is a shopping complex in chennai city. The store offers Diwali discount for its happy customers as given below:

Purchase amount(₹)	Discount(₹)
<=5000	400
>5000 and <=10000	800
>10000 and <=20000	1000
>20000	2000

If the purchase amount is more than ₹20000 , given an additional discount of 3% on the amount after discount. Write a program to calculate the total discount and net amount after total discount.

That is: totdiscount = discount + additionaldiscount

Netamount=purchaseamount – discount

and print the purchase amount, discount additional discount and net amount.

Solution:

```
print("\t star complex\t")
additional_discount=0
purchase_amount=float(input("Enter purchase amount:"))
if(purchase_amount<=5000):
  discount=400
elif((purchase_amount>5000) and (purchase_amount<=10000)):
  discount=800
elif((purchase_amount>5000) and (purchase_amount<=20000)):
  discount=1000
elif(purchase_amount>20000):
  discount=2000
  additional_discount=purchase_amount *3/100
totdiscount=discount+ additional_discount
```

```
netamount=purchase_amount - totdiscount
print("==***===****====****====****====****===****==**")
print("purchase amount is :%8.2f"%purchase_amount)
print("discount amount is : %6.2f"%discount)
print("Total discount is :%9.2f"%totdiscount)
print("net amount is :%9.2f"%netamount)
```

Output:

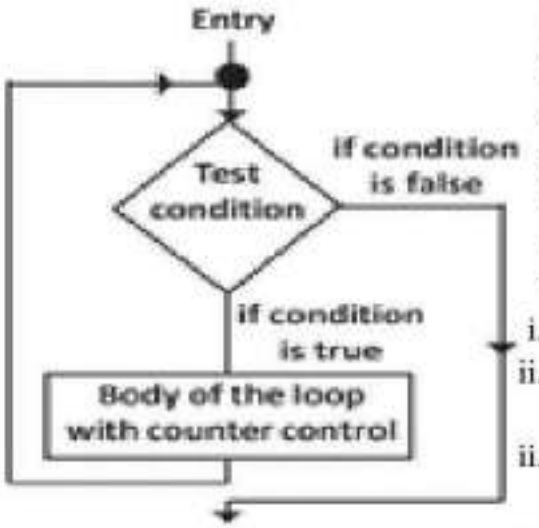
```
star complex
Enter purchase amount:12000
==***===****====****====****====****===****==**
purchase amount is :12000.00
discount amount is : 1000.00
Total discount is :  1000.00
net amount is : 11000.00
```

4.5. Iterative Control (Loops and Iteration):

When you want to execute a block of code more than once, iteration statements help you do this. And" ***loop is defined as a segment of code that executes multiple times. That is why iteration statements are also known as looping statements.*** " The statement or block of statement executed in a loop as the body of the loop or simply as the loop body. Loops can generally be categorized as either ***count-controlled or event controlled.***

- In a **count-controlled** loop, the body of the loop is executed a given number of times based on counter value.
- With **event-controlled** loops, the body is executed until some event occurs. The number of iterations cannot be determined until the program is executed.

The python programming language supports the following types of loops.



Structure of iterative control

- **While loop:** Repeats a statement or group of statements while a given condition is True. It tests the condition before executing the loop body.
- **For loop:** This type of loop executes a code block multiple times and abbreviates the code that manages the loop variable.
- **Nested loop:** We can iterate a loop inside another loop.

There is a common structure of all types of loops, such:

i. There is a control variable, called the loop counter.
ii. The control variable must be initialized; in other words, it must have an initial value.
ii. The increment/decrement of the control variable , which is modified each time the iteration of the loop occurs.
iv. The loop condition that determines if the looping should continue or the program should break from it.

Note: The basic principle is that a loop is a structure that allows repeated execution of a block of statements . Within a loop structure, a condition called Boolean Expression.

If the expression result is True, a block of statements called the loop body executes and the Boolean expression is evaluated again and again as long as the

expression is True, the statements in the loop continue to execute. When the Boolean evaluation is False, the loop ends or terminates.

4.5.1. Loop Control Statements:

Loop control statements in python provide programmers with the ability to control the flow of execution within loops. These statements include ***break, continue, and pass.*** Loop control statements are essential for enhancing the flexibility and efficiency of loops.

These control statements are discussed in deeply in 4.9

What is state in iteration?
Transition from one process to another process under specified condition within a time is called state.

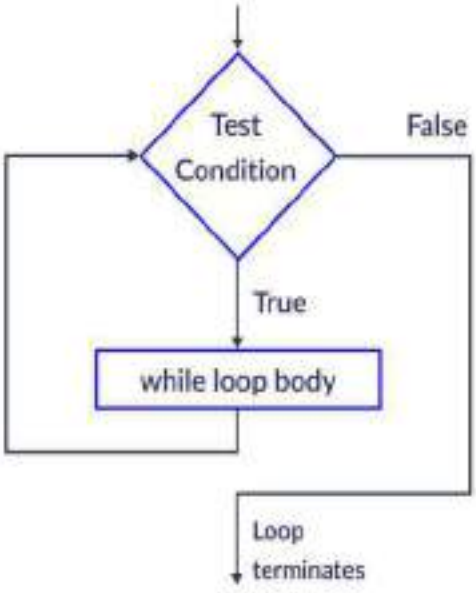
Sl.no	Name of the control statement	Description
1.	Break statement	This command terminates the loop's execution and transfer the program's control to the statement next to the loop.
2.	Continue statement	This command skips the current iteration of the loop. The statements following the continue statement are not executed once the python interpreter reaches the continue statement.
3.	Pass statement	The pass statement is used when a statement is syntactically necessary, but no code is to be executed.

4.6.The While loop :A while loop statement in python programming language repeatedly executes statement(s) as long as a given condition is true.

Syntax:

While expression / test condition:
Statement block

- While: This keyword is used to mark the beginning of a while loop.
- Expression / test condition: After **while keyword** a condition expression is written that determines whether to continue the loop (evaluates to either True or False).
- (:) : a colon is written to indicate start of loop code block under the while loop.
- Statement block: a new line is entered and one line or multi-line loop block is written with uniform indention as per programmer's choice.



Flowchart of while loop

Note: Remember to update the variables or conditions within the loop to ensure the condition eventually becomes False and the loop exists, preventing an infinite loop.

Example 1: program for while loop as count-controlled loop

```
count=0
while count<5:
  print("The count is :",count)
  count +=1
print("Loop finished")
```

Output:

```
The count is : 0
The count is : 1
The count is : 2
The count is : 3
The count is : 4
Loop finished
```

In the above program the while loop will run for value of variable count as 0,1,2,3,4. When the value of variable count will reach 5, the condition will return False and the loop will break.

Example 2: Program for while loop as event – controlled loop

```
total=count=0
value=int(input("enter the first grade:"))
while value >=0:
  total += value
  count += 1
  value=int(input("enter the next grade (or<0 to quit):"))
  avg=total / count
print("The average grade is %4.2f"%avg)
```

Output:

```
enter the first grade:45
enter the next grade (or<0 to quit):56
enter the next grade (or<0 to quit):48
enter the next grade (or<0 to quit):78
enter the next grade (or<0 to quit):63
enter the next grade (or<0 to quit):-12
The average grade is 58.00
```

Consider the above problem of entering multiple exam grades from the user and then computing the average grade. We need to enter multiple grades but we loop, thus leading to the name event-controlled loop.

Example 3: Some programs using while loop

1.
```
num=4
while num != 0:
    print(num)
    num -=1
```

Output:

```
4
3
2
1
```

This loop prints num starting from 4 and going down to 1,until the num becomes 0 which is the terminating condition of the loop.

2. program to print the value of n by using while loop.

```
n=int(input ("Enter the value of n:"))
i=1
while(i<=n):
  print(i)
  i=i+1
```

Output:

```
Enter the value of n: 4
1
2
3
4
```

Here, the while loop is controlled by a loop counter variable i. The loop will only terminate when i equal to n or greater than n

3. Program to print the first name and the given name is equal or not.

```
first_name=input("what is you first name ?")
name=input("enter a name:")
while name!= first_name:
    print("the name  you entered is not your first name")
     name=input("enter a name:")
print("welcome", name)
```

Output:

```
what is you first name ?manojan
enter a name: michael
the name  you entered is not your first name
enter a name: manojan
welcome manojan
```

Example 4: Program to find the factorial of an input number

```
fact= i=1
N=int(input("Enter any number to find factorial:"))
i=1
while (i<=N):
   fact=fact*i
    i+=1
print("the factorial of %d is %2f"%(N, fact))
```

Output:

```
Enter any number to find factorial:5
the factorial of 5 is 120.000000
```

The program computes the factorial of a user-provided number using a while loop. It initializes a variable fact to 1, then iteratively multiplies it by each integer from 1 up to the input number. This loop-based approach demonstrates the use of control structures in Python and is a common way to calculate factorials without using built-in functions or recursion.

Step	i	fact = fact × i
1	1	$1 \times 1 = 1$
2	2	$1 \times 2 = 2$
3	3	$2 \times 3 = 6$
4	4	$6 \times 4 = 24$
5	5	$24 \times 5 = 120$

Example 5: program to print mathematical table by using while loop

Solution:

```
i=t=1
n=int(input("Enter a number:"))
while i<10:
 t=n*i
 print(n, " * ", i, " = ",t)
 i=i+1
```

Output:

```
Enter a number:4
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
```

Example 6: Write a Python Program to print the following pattern:

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
```

Solution:

```
n=int(input("Enter the number of rows:"))
i=1
while i<= n:
 print( " * " * i)
 i += 1
```

4.6.1. The Infinity Loop:

An infinity loop is a never-ending loop. It does not have any stopping condition. In other words, in a while loop, the conditions will always be True. In order to break an infinite while loop , you have to manually stop it by interrupting the code execution. For instance , the condition True is always true and when in a while loop, this creates an infinite loop.

Syntax:

While True:

Statement(s)

Instead of specifying a condition, we have written True, Which creates an infinite loop.

Note: Whether or not you meant to create an infinite loop, pressing ctrl + c in the python terminal or in IDLE you can press ctrl+F6 to stop the loop as well.

Example 1: Code for infinite loop in python using while loop.

```
while True:
  print("hello")
print("this line is over")
```

Output:

This infinite loop has stopped by pressing ctrl + F6

Example 2: Infinite loop with single line statement.

Solution:

```
counter=0
while (counter<3): print("hi, everyone")
```

Output:

Note: To avoid infinite loops, make sure that the condition of the loop can eventually become False, or use a break statement to exit the loop when a certain condition is met.

Example 3:

```
i=0
while True:
   print(i)
   i=i+1
   if i==6:
      break
```

Output:

```
0
1
2
3
4
5
6
```

In this example , the break statement causes the loop to exit when i becomes 6, preventing the loop from running indefinitely. The output will be the numbers 0 through 6.

4.6.2. Using else with while Loop:

In python, the else clause of a while loop can be used to specify a block of code that should be executed after the loop has finished executing, but only if the loop completed normally(i.e. , if the loop was not existed prematurely by a break statement).

Example 1: program to demonstrate the use of else clause with a while loop.

Solution:

```
i=0
while i<10:
  print(i)
  i=i+1
else:
  print("All done")
```

Output:

```
0
1
2
3
4
5
6
7
8
9
All done
```

Example 2: Program to demonstrate if the while loop is exited prematurely by a break statement, the else clause will not be executed.

Solution:

```
i=0
while True:
    print(i)
    i=i+1
    if i==5:
        break
else:
    print("done")
```

In this example , the while loop will print the numbers 0 through 4, and then exit when I becomes 5. The else clause will not be executed.

Output:

```
0
1
2
3
4
```

4.7. The For loop:

In python, the for loop has the ability to iterate over the items of any sequence , such as list or a string and perform operations on them sequentially. This statement must end with a colon (:) character and statements to be executed on each iteration of the loop must be indented below.

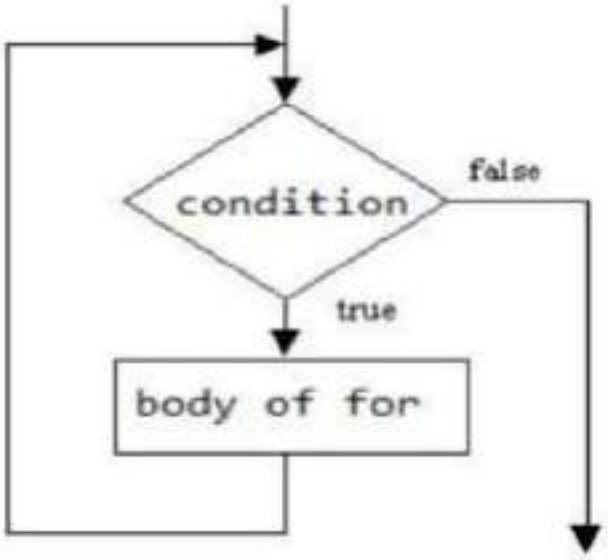
Syntax:

```
for loop_variable in sequence:
    Program statement 1
    Program statement 2
    ……..
    Program statement N
else:
    Program statement 1
    Program statement 2
    ……..
    Program statement M
```

From the syntax:

- **for:** The keyword ***for*** is used to mark the beginning of a for loop.
- **Loop_variable:** after for, a loop variable is written that represents each element in the sequence. It can be any valid variable name chosen by the programmer.
- **In :** The keyword in is written after loop variable. It indicates the sequence being iterated over.
- **Sequence:** After keyword in , the sequence variable is written , which refers to any iterable object containing multiple elements.
- (:) : After the sequence variable, a colon (:) is written to indicates start of loop code block under the for loop.
- **Block statements:** After writing else: statement , a new line is entered and one line or multi-line loop block is written with uniform indention as per programmer's choice.

Flow chart of else statement with for loop:



Example 1: Program to demonstrate to print the sequence values:

```
for name in "Arjun Dhas":
print("Current character is:",name)
```

Output:

```
Current character is: A
Current character is: r
Current character is: j
Current character is: u
Current character is: n
Current character is:
Current character is: D
Current character is: h
Current character is: a
Current character is: s
```

In the above example, the "Arjun Dhas" is a string and holds a sequence of characters including the space.

Example 2: Program to print variable iteration:

```
f=["apple", "banana", "carrot", "dragon fruit", "Egg plant"]
for fruit in f:
  print("I love to eat ",fruit)
```

Output:

```
I love to eat  apple
I love to eat  banana
I love to eat  carrot
I love to eat  dragon fruit
I love to eat  Egg plant
```

One element of sequence ***variable f*** ,is stored in variable ***fruit*** in each iteration of for loop. The loop is iterated for each element of the sequence ***variable f.***

❖ for loop with else statement:

Python supports to have an else statement associated with a loop statement. If the else statement is used with for loop, the else statement is executed when the loop has finished iterating the list. A break

statement can be used to stop a for loop. In this case , the else part is ignored. Finally, a for loop's else part runs if no break occurs.

Example: Python code to find whether an item is present in the list ornot.

```
lst=[10,20,30,40,12,13,14,15,50,100]
num=int(input("Enter the number to be searched in the list:"))
for i in range(len(lst)):
   if lst[i]==num:
      print("number found at:" ,(i+1))
      break
else:
   print("number not found ")
```

Output 1:

```
Enter the number to be searched in the list:12
number found at: 5
```

Output 2:

```
Enter the number to be searched in the list:25
number not found
```

The program searches for a user-input number in a predefined list using a for loop. It iterates through each element of the list by index and compares it with the input number. If a match is found, it prints the position (index + 1) where the number is located and exits the loop using break.

If the loop completes without finding the number, the else block associated with the loop executes, indicating that the number was not found in the list.

4.7.1. Using range( ) Function:

A for in loop iterates over the items of any list or string in the order that they appear in the sequence , but you cannot directly specify the number of iterations to make, a halting conditions or the size of iteration step. It is used to generate a range of numbers.

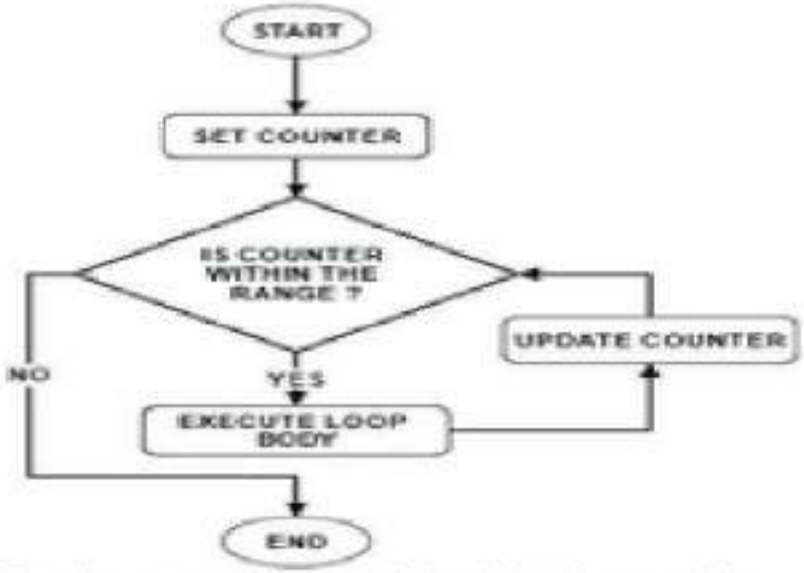
Syntax:

range (start, stop, step)

Here,

- The range function accepts three arguments, two are optional**. i.e.** Start and step are optional.
- The first parameter Start is the starting number of range. If the start argument is omitted , it defaults to 0.
- The second parameter Stop is the last number of range where the range will end. The exact end position is: stop – 1. The end value cannot be omitted.
- The Step is the increment/ Decrement option. If this third parameter is missing, the increment value is det to 1.

Flowchart of for loop using range function:



The range ( ) function generates lists containing arithmetic progression

Range function	Range object(output)
range(10)	0,1,2,3,4,5,6,7,8,9
range(1,10)	1,2,3,4,5,6,7,8,9
range(1,10,2)	1,3,5,7,9
range(10,0,-1)	10,9,8,7,6,5,4,3,2,1
range(10,0,-2)	10,8,6,4,2
range(1,1)	empty
range(1,-1,-1)	1,0
range(1,-1)	empty
range(-5,5)	-5,-4,-3,-2,-1,0,1,2,3,4
range(0)	empty

For example 1: Python program to show the working of range ( ) function.

Solution:

```
print(range(15))
print(list(range(10)))
print(tuple(range(15)))
print(list(range(1,15,3)))
```

Output:

```
range(0, 15)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14)
[1, 4, 7, 10, 13]
```

Example 2: program to demonstrate for loop in range ( ) and print the results in single line.

```
for i range(10):
    print(i, end=" ")
```

Output:

```
0 1 2 3 4 5 6 7 8 9
```

This loop starts from 0 and goes up to 9 (one less than 10). The end=" " keeps all the numbers on the same line, separated by spaces.

Example 3: python program to display the values generated by for loop.

```
for i in range (1,8):
    print(i)
```

Output:

```
1
2
3
4
5
6
7
```

This loop begins at 1 and continues until it reaches 7. Because it doesn't use the end argument, each number is printed on a separate line by default.

Example 4: Program to generate n terms of Fibonacci sequence.

```
n=int(input("enter the number  of terms :"))
f=0
s=1
if n<=0:
  print("The Fibonacci series is :",f)
else:
  print(f ,s, end=" ")
  for x in range (2,n):
    next_term=f + s
    print(next_term, end=" ")
    f=s
    s=next_term
```

Output:

```
enter the number  of terms : 9
0 1 1 2 3 5 8 13 21
```

Example 5: program to generate the negative values by using for loop.

```
for a in range(21,-1,-2):
    print(a, end=" ")
```

Output:

```
21 19 17 15 13 11 9 7 5 3 1
```

In the above for loop, the range(21,-1,-2) prints number from 21 to 1 in reverse order. It is possible to let the range start with different numbers, or to specify a different increment (even negative).

Example 6: Python program using else with a for Loop in Python

```
for i in range(5):
   print(I, "I\'m a Bsc Student")
else:
   print("inside My college")
   print(i, "I\'m enjoying it.")
```

Output:

```
0 I'm a Bsc Student
1 I'm a Bsc Student
```

```
2 I'm a Bsc Student
3 I'm a Bsc Student
4 I'm a Bsc Student
inside My college
4 I'm enjoying it.
```

In this program, a for loop runs 5 times, starting from 0 to 4. For each value, it prints the current number along with the message "I'm a Bsc Student". Once the loop finishes without interruption (i.e., no break statement), the else block gets executed. In the else block, it prints "inside My college" and again prints the last value of i with the message "I'm enjoying it.". This example shows that the else block after a for loop runs only after the loop completes normally.

4.7.1.1. Using for in List and Tuple:

As a string is simply a list of character, the for in statement can loop over each character. Similarly, a for in statement can loop over each element in a list, each item in a tuple , each number of a set or each key in a dictionary.

Here are two examples which prints a tuple value and a list values using for loop.

a) Using Tuple:

Example :program to display the directions :

```
>>>direction=("North","south","east","west","notrh_west","south_east")
>>>for dr in direction:
    print("current direction is :",dr)
```

Output:

```
current direction is : North
current direction is : south
current direction is : east
current direction is : west
current direction is : notrh_west
current direction is : south_east
```

b) Using List:

```
days=["Sunday","Monday","Tuesday","Wednesday","Thursday","Friday"]
for i in days:
  print("choose a colour for ",i)
```

Output:

```
choose a colour for  Sunday
choose a colour for  Monday
choose a colour for  Tuesday
choose a colour for  Wednesday
choose a colour for  Thursday
choose a colour for  Friday
```

4.7.2. Definite Loop:

For loop in python is a definite iterative statement. It is called a definite loop because it iterates for a range of values or a sequence. The iteration statement is executed for each value of the range.

The range value can be numeric , or it may be some successive elements of string, list, or tuple, we may how many times the loop will be executed.

Syntax:

```
for <variable> in <sequence / item in range>:
  Statement in body of loop
else:
  statements
```

Here variable will take new value from the range or sequence in every iteration . else statement is not mandatory however, if mentioned, if will be executed after all the iterations of the for loop.

For e.g.

```
for i in range(4):
  print(i)
```

Output:

```
0
1
2
3
```

Note: The value of i in this example will be start from 0 and will go to 5-1.

4.7.3. Using else statement with for Loop:

Similar to the while -loop, the for loop also allows the use of optional else, break, continue statements within the loop. The else phrase is only used when the execution is finished. If we break out of the loop or an error is raised, it won't be carried out. This code can help you comprehend if-else statements better.

Syntax:

```
for variable in iterable object:
    Loop body statements
else:
    Statements to be executed before exit
Code lines after the loop
```

Example 1: program to demonstrate the for loop using else .

Solution :

```
word=input("Enter sentences:")
for s in word:
   if s=='c' or s=='a':
      print("  omitted")
   else:
      print(s)
else:
   print("finished")
```

Output 1:

```
Enter sentences: lovely baby
l
o
v
e
l
y

b
 omitted
b
y
finished
```

Output 2:

```
Enter sentences: cat runs
 omitted
 omitted
 t
 r
 u
 n
 s
 finished
```

The above program asks the user to enter a sentence. It then goes through each letter in that sentence one by one using a for loop. If the letter is 'c' or 'a', it prints " omitted" instead of showing the letter. For all other letters, it prints them as they are. After the loop finishes checking all the letters, it prints "finished" using the else part of the for loop.

4.7.4. Iterating in sorted and reversed order:

- Whenever we print the elements of a list using a ***for*** loop, the elements will be printed in the order they appear because the list elements have an inherent order, and the iteration is defined in that order only.

Example 1:

```
num=[2,4,5,2,5]
for numbers in num:
    print(numbers, end=' ')
```

Output:

```
2 4 5 2 5
```

- If we want the iteration done in a sorted order of elements, we can use the sorted function.

Example 2:

```
num=[2,4,5,2,1,0,7,5]
print("the unsorted list is: ",list(num))
print("The sorted list is")
for numbers in  sorted(num):
   print(numbers, end=' ')
```

Output:

```
the unsorted list is:  [2, 4, 5, 2, 1, 0, 7, 5]
The sorted list is
0 1 2 2 4 5 5 7
```

- Similarly we can loop over a sequence in reverse order using the ***reversed*** function.

For e.g.

```
num=[2,4,5,2,1,0,7,5]
print("the unsorted list is: ",list(num))
print("The reversed and sorted list is")
for numbers in  reversed(sorted(num)):
   print(numbers, end=' ')
```

Output:

```
the unsorted list is:  [2, 4, 5, 2, 1, 0, 7, 5]
The reversed and sorted list is
7 5 5 4 2 2 1 0
```

Note: Dictionaries can also be iterated over in sorted order using the sorted function
For e.g. sorted by key

```
prices={'apple':200,'banana':100,'grapes':50,'mango':85,'cherry':275,'guava':45}
for fruit in sorted(prices. keys()):
  print(fruit, prices[fruit],end=' | ')
print()
for fruit, price in sorted(prices. items( )):
  print(fruit, price, end='|')
```

Output:

```
apple 200 | banana 100 | cherry 275 | grapes 50 | guava 45 | mango 85 |
apple 200|banana 100|cherry 275|grapes 50|guava 45|mango 85|
```

Here, the sorting is done based on keys; if we want to sort according to values , we can invert the keys and values by using *zip* function.

For e.g.

```
prices={'apple':200,'banana':100,'grapes':50,'mango':85,'cherry':275,'guava':45}
print()
for fruit, price in sorted( zip(prices. values( ),prices. keys())):
  print(fruit, price, end='/')
```

Output:

```
45 guava/50 grapes/85 mango/100 banana/200 apple/275 cherry/
```

4.7.5. Definite vs Infinite Loops:

- For loop is a definite loop. A definite loop is a program loop in which the number of times the loop will iterate can be determined before the loop is executed.
- While loop is an indefinite loop, where the result of the condition determines the iteration. Indefinite loop is a program loop in with the number of times the loop will iterate is not known before the loop is executed. It is also known as a conditional loop.
- The difference between ***for*** **and** ***while*** loop are mentioned below:

Property	For loop	While loop
Definite/ indefinite	For loop is a definite loop, where the number of iterations is defined at the beginning of the loop.	While loop is an indefinite loop, where the result of the condition determines the iteration.it is also known as a conditional loop.
Finite / infinite loops	For loops are used with collections only. Hence , they iterate a finite number of times.	While loops need not necessarily be finite. But the condition should be specified.
Counter variables	For loops define the counter variable during declaration.	While a loop has no built-in loop control variable , the programmer can create any variable and condition the loop.
Number of iterations	In the for loop , the number of loop iterations is defined at the beginning of the loop.	In the while loop, there is no specific count of iterations. Here ,the loop iterates until the condition is met.
Application	Loops are convenient when the number of iterations is known.	While loops are convenient when the number of iterations is unknown.
Supporting methods	Range( ) and xrange( ) methods can be used with for the loop	No specific methods are available in python to help in iteration or defining the counter in while loops.
Speed	For loop is faster than while loop	While the loop is comparatively slower as it has to test conditions on every iterations.

4.8. Nested loop:

When we use one loop inside another loop then it is called nested loops. When the condition for the outer loop is True control is transferred to the inner loop. The outer loop cannot terminate until the inner loop terminates.

Syntax of nested while:

```
while <test_condition>:
    while <test_condition>:
        Statements(inner loop)
    Statements(outer loop)
Statements (outside the outer loop)
        ( or )
While loop_ condition:
Sequence of statements,s1
for loop_ condition:
    sequence of statements,s2
sequence of statements,s3
```

Syntax of nested for:

```
for var in sequence:
    for var1 in sequence:
        Statements
    Statements
Statements
    ( or )
for loop_condition:
    sequence of statements,s1
    while loop_condition:
        sequence of statements,s2
    sequence of statements,s3
```

Example 1: program to print the nested wl

```
i=1
while i<=5:
    j=1
    while j<=i:
        print(j, end=" ")
        j=j+1
    print()
    i=i+1
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

This program uses nested while loops to print a right-angled number triangle. The outer loop controls the number of rows from 1 to 5. For each row i, the inner loop prints numbers from 1 to i on the same line using end=" ". After the inner loop finishes, a newline is printed using print(). This pattern continues until all 5 rows are printed.

Example 2: program for nested for loop

```
for i in range(1,6):
  for j in range(1,i+1):
   print(j, end=" ")
  print( )
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Example 3: program to print table of 2 and 3 using nested while loops.

```
f_num=2
while f_num<=3:
 s_num=1
 print("The multiplication table ",f_num)
 while s_num<=10:
  multiple=f_num*s_num
  print(f_num, "X" , s_num, "= ",multiple)
  s_num=s_num+1
 print( )  #prints new empty line
 f_num=f_num+1
```

Output:

The multiplication table 2	The multiplication table 3
2 X 1 = 2	3 X 1 = 3
2 X 2 = 4	3 X 2 = 6
2 X 3 = 6	3 X 3 = 9
2 X 4 = 8	3 X 4 = 12
2 X 5 = 10	3 X 5 = 15
2 X 6 = 12	3 X 6 = 18
2 X 7 = 14	3 X 7 = 21
2 X 8 = 16	3 X 8 = 24
2 X 9 = 18	3 X 9 = 27
2 X 10 = 20	3 X 10 = 30

Example 4: program to evaluate the power series. The series looks like, $s = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots + \frac{x^n}{n!}$ ***where*** $0 < x < 1$

Solution:

```
import math
x=int(input("enter the value of x :"))
n=int(input("enter the value of n :"))
s=1.0 + x
term, ctr=0 , 2
while ctr<=n:
   pw=math. pow(x, ctr)
   fact=1
   for i in range(1, ctr+1):
    fact=fact*i
   s=s+(pw/fact)
   ctr +=1
print("sum of the series is %2f"%s)
```

Output:

```
enter the value of x :0
enter the value of n :3
sum of the series is 1.000000
```

- This Python program calculates the sum of a mathematical series of the form: $1 + x + x^2/2! + x^3/3! + ... + x^n/n!$, which resembles the expansion of the exponential function e^x. The program begins by importing the math module to use the pow() function.

- It then takes two inputs from the user: x (the number to be raised to powers) and n (the number of terms in the series). It initializes the sum s with the first two terms, 1.0 + x, and sets a counter ctr starting from 2.
- A while loop runs from 2 to n, calculating each term using x^ctr / ctr!. The factorial is computed using a for loop, and each term is added to the total sum s.
- Finally, the program prints the total sum formatted to six decimal places. This is a classic way to approximate exponential values using Taylor series in Python.

Example 5: Program to print a list in reverse order.

Solution:

```
num=[10,12,14,20,22,24,30,32,34,40,42,44,50]
print("The given list is: ",end=" ")
for i in num:
  print(i, end=" ")
ln=len(num)
i=0
ctr=-1
print(" \n The reverse order list elements are :",end= " ")
while i<=ln - 1:
  print(num[ctr],end=" ")
  ctr-=1
  i+=1
```

Output:

```
The given list is:  10 12 14 20 22 24 30 32 34 40 42 44 50
The reverse order list elements are : 50 44 42 40 34 32 30 24 22 20 14 12 10
```

Example 6:Program to print the squares and cubes of numbers from 1 to 10 using a table format.

Solution:

```
print("\n The table format is:")
print("\n Number\t square \t cube")
for i in range(25):
  print("~",end=' ')
print( )
for x in range(1,11):
  print("{0:2d} {1:10d} {2:10d}".format(x,x*x,x*x*x))
for i in range(25):
  print("~",end=' ')
```

Output:

```
The table format is:
 Number  square    cube
~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
 1   1   1
 2   4   8
 3   9  27
 4  16  64
 5  25  125
 6  36  216
 7  49  343
 8  64  512
 9  81  729
10  100 1000
~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~ ~
```

4.9. The depth of Loop control statements:

Now we will discuss the loop control statements in detail.

sl.no	Control statement	Description
1.	Break statement	This command terminates the loop's execution and transfer the program's control to the statement next to the loop.
2.	Continue statement	This command skips the current iteration of the loop. The statements following the continue statement are not executed once the python interpreter reaches the continue statement.
3.	Pass statement	The pass statement is used when a statement is syntactically necessary, but no code is to be executed.

4.9.1. Continue Statement:

It returns the control to the beginning of the loop.

Example 1: program to show how continue statements works.

Solution:

```
for str in " warm weather":
    if str=='e' or str=='a' or str=='t':
        continue
    print("current letter: ",str)
```

Output:

```
current letter:
current letter:  w
current letter:  r
current letter:  m
current letter:
current letter:  w
current letter:  h
current letter:  r
```

Example 2: Code to print sequence of values.

Solution:

```
for num in range(1,11):
 if num==6:
  continue
 if num%2==0:
  print(num)
```

Output:

```
2
4
8
10
```

The continue statement escape the code below the continue statement within the loop.

4.9.2. Break statement:

It stops the execution of the loop when the break statement is reached.

Example 1: python code by using break statement.

Solution:

```
str=input("enter sentences:")
    for i in str:
     if str=='e' or str=='E':
      break
     print("The letter is :",i)
```

Output:

```
enter sentences: coffee addict
The letter is : c
The letter is : o
The letter is : f
The letter is : f
The letter is : e
The letter is : e
The letter is :
The letter is : a
The letter is : d
The letter is : d
The letter is : i
The letter is : c
The letter is : t
```

Example 2: program to find a country from list by using break in for loop.

Solution:

```
names=["India","US","Australia","Melbourn","Nepal","Srilanka","Pakisthan","Melbourn"]
search="Melbourn"
lcount=0
for name in names:
   lcount=lcount+1
   if name==search:
       print("Name Found : ",name)
       break
print(f "Loop runs {lcount} times")
```

Output:

```
Name Found :  Melbourn
Loop runs 4 times
```

As soon as the word Melbourn found, break statement is executed and exited from the loop.

4.9.3. Pass statement:

Loops that are empty are made with pass statement. Classes, functions, and empty control statements are also used with the pass statement.

- The pass statement is a null statement
- The difference between a comment and a pass statement in python is that while the interpreter ignores a comment entirely; pass is not ignored.
- The pass statement does nothing.

Example 1: Python code for pass statement.

Solution:

```
for a in "python loops":
```

```
    pass
print("last letter: ", a)
```

Output:

```
last letter: s
```

Example 2: Program to print the greater or equal to numbers.

Solution:

```
x=10
if x<5:
  pass # no action is taken if x is less than 5
else:
  print("x is greater than or equal to 5 :")
```

Output:

```
x is greater than or equal to 5 :
```

4.9.4. Difference between *break* and *continue*:

Feature	break statement	continue statement
purpose	It terminates the current loop prematurely and executes the remaining statement outside the loop. **Syntax:** **break**	It terminates the current iteration and transfer the control to the next iteration in the loop. **Syntax:** **continue**
Effects	Exits the loop immediately when encountered	Jumps to the next iteration of the loop.
Usage	Typically used when a specific condition is met and further iterations are unnecessary.	Typically used to skip certain iterations based on specific conditions.
Applications	Used to exit the loop when a certain condition is met, regardless of the loop's remaining iterations.	Used to skip the current iteration of the loop and proceed to the next iteration.
Control Flow	Breaks out of the loop entirely.	Skips the remaining code in the current iteration and continues with the next iteration.

Example
 	
Output:
	for i in range(10):
	if i== 5:
	break
	print(i)
	0 1 2 3 4
		for i in range(10):
		if i== 5:
		continue
		print(i ,end=" ")
		
		0 1 2 3 4 6 7 8 9

Practice Quiz:

1. Find the result of each the following complex Boolean expression when variables a, b, c and d contain the values 6,-3,4 and -7 respectively.

   i.) (3* a+b / 5 – c *b /a > 4)and (b!= -3)

   ii.)(a*b-c / 2 + 21 * c / 3 != 8)or not (a>=5)

2. Find the output for the following code:

```
a=float(input("enter a number for a:"))
b=float(input("enter a number for b:")
if b!=3:
  x=a/(b – 4)
print(x)
```

3. Find the error and get the output for the following code:

```
if =5
x=if+5
print(x)
```

4. Find the absolute value of two variables from the following code; The input value for the two executions are (i) -11 , and (ii)11.

```
x=int(input())
y=6
if abs(x) <10:
  y+=x
  x-=1
if abs(x) >10:
  y*=3+y
print(x, "," , y)
```

5. Write a program to enter the marks in 3 subjects and calculate total and assign grade if total >= 200 grade is 'A' otherwise grade is 'B'.

6. Write a python program to print the following pattern:

```
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

7. Write a python program to enter your name and print it five times using while loop

8. Write a program to print the tables from 1 to 5 using nested for loop

9. Write the code to find the sum of first n natural numbers using a:

   i) while loop ii)for loop iii)without any loop

10. Will these two loops give the same output?

i)
```
L= [2,3,4,1,5,6,9,0]
for n in L:
    print(n, end=" ")
```

ii)
```
i=0
while i< len(L):
    print(L[i],end=' ')
    i + =1
```

11. write the output for the following codes:

a)
```
x=5
while x:
print(x, end=' , ')
```

b)
```
for item in [1,2,4,,3]:
    print(item * 4, end=' ')
    print(item)
```

c)
```
a=[3,8,-3.4,-2,-5,6]
s=0
for i in a:
  if i > 0 :
```

```
    s+=i
print(s)
```

4.10. Boolean Flag:

- Boolean flags labelling technique involves the use of binary indicators to assign labels to data instances. These indicators often represented as Boolean variables (***true/ false or 1/ 0***), are associated with specific characteristics or properties that help identify the desired label.
- By examining the presence or absence of these flags , data instances can be automatically labelled. Often the condition of a given while loop is denoted by a single Boolean variable, called a Boolean flag.
- Boolean flags are also useful in controlling loops. For instance, in a while-loop, a flag can determine whether the loop should continue running or stop.

For e.g.

```
flag=True
while flag:
  print(' Inside loop')
  flag=False
print('outside loop')
```

Output:

```
Inside loop
outside loop
```

Note: In python, a Boolean flag is a variable used to signal whether a condition is True, False, often used to control the flow of program.

- In real-world applications, Boolean flags are used everywhere—from tracking user authentication status (is_logged_in) to enabling or disabling features (debug_mode) or monitoring the completion of tasks (task_completed).

Example 1 : Accessing login simulation

```
authenticated = False
username = input("Username: ")
password = input("Password: ")
if username == "admin" and password == "1234":
  authenticated = True
if authenticated:
  print("Welcome!")
else:
  print("Access denied.")
```

Output:

```
Username: admin
Password: 23fedv
Access denied.

Username: admin
Password: 1234
Welcome!
```

In the above program the Boolean flag is determines the login progress.

Example 2: Python program to check for value change in list

```
a = [1, 2, 2, 2, 3]
changed = False
for i in range(1, len(a)):
   if a[i] != a[i-1]:
       changed = True
       break
  if changed:
      print("List contains a change in value.")
```

Output:

List contains a change in value.

This Python program checks whether there is a **change in value** between consecutive elements in a list. The list a = [1, 2, 2, 2, 3] is defined first. A flag variable changed is initially set to False. The for loop starts from the second element (index 1) and compares each element with the one before it.

If any pair of consecutive elements are **not the same**, it sets changed = True and exits the loop using break. After the loop, the if changed: condition is checked. If it is True, it prints "List contains a change in value."

In this example, since a[0] = 1 and a[1] = 2 are different, the program detects a change immediately and prints the output.

Example 3: Finding the first match in a list:

```
numbers = [11, 22, 35, 46, 57]
found = False
for num in numbers:
   if num % 7 == 0:
    print("Found a multiple of 7:", num)
    found = True
    break
if not found:
   print("No multiples of 7 found.")
```

Output:

Found a multiple of 7: 35

4.11. String, List and Dictionary Manipulation:

In Python, the ability to manipulate data efficiently is a crucial part of writing effective programs. Among the most commonly used data structures in Python are **strings**, **lists**, and **dictionaries**. Each of these structures serves a unique purpose and offers powerful built-in tools for handling and transforming data.

4.11.1. String manipulation:

String is vital in various computing domains, including web development , data science, and automation. In python strings are immutable sequences of Unicode characters.

This immutability means that once a string is created , it cannot be modified in place, and any operations that seem to modify a string actually produce a new one. This characteristics ,while might seem limiting at first, contributes to the efficiency and safety of string manipulation in python. The more string methods are briefly explained in book 3(3.3 onwards).

Basic string operations:

1) Concatenation: Joining two or more strings into one perhaps the most straightforward operation. Python uses the '+' operator for string concatenation.

Example:

```
a="hello,"
b="John."
Msg =a+ b+"!"
print(Msg)
```

Output:

hello,John.!

2) Slicing: python allows for extracting part of a string using slicing. The syntax for slicing is [**start : end :step**],where start is beginning of the index , end is the ending index but not included in the result, and step is the stride between each character.

Example:

```
alphabet="abcdefghijklmnopqrstuvwxyz"
slice_ltr=alphabet[ : : 3 ]
print(slice_ltr,end=" ")
```

Output:

Adgjmpsvy

In this program, the string alphabet contains all the letters from a to z. The slicing alphabet[::3] means we are picking every third letter from the string, starting from the beginning. So it selects letters like 'a', 'd', 'g', and so on. The result is stored in slice_ltr, and then printed. The output will be the letters that are at every third position in the alphabet.

3) Replace and Find:

- The find() method is used to search for a substring within a larger string and return the index of its first occurrence. If the substring is not found, find() returns -1 rather than raising an error, which makes it safe to use in conditions. *The format of find function is* ***str. find(substring, start,end)***
- The replace() method is used to substitute all or a specified number of occurrences of a substring with another string. Since strings are immutable, this method returns a new string with the changes applied. The format for replace function is string. replace(old, new, count)

Example:

```
sentence = "I love ice cream"
print(sentence.replace("love", "like"))
print(sentence.find("cream"))
```

Output:

```
I like ice cream
11
```

4) Case Conversion: Python, known for its clarity and simplicity, offers a robust set of string methods that make case conversion both intuitive and powerful. These methods are widely used in formatting outputs, standardizing user input, comparing text, and handling case-insensitive operations. The below methods are deeply explained in

Python provides several built-in string methods for case conversion:

- **upper()**
- **lower()**
- **capitalize()**
- **title()**
- **casefold()**
- **swapcase()**

Example:

```
text = "hello world"
print(text.upper())
text = "HELLO WORLD"
print(text.lower())
text = "python is fun"
print(text.capitalize())
text = "hello from the other side"
print(text.title())
text = "PyThOn"
print(text.swapcase())
```

Output:

```
HELLO WORLD
hello world
Python is fun
Hello From The Other Side
pYtHoN
```

Case insensitive string comparisons:

Comparing string in case insensitive way seems like something that's trivial , it is essentially converting all text to lowercase, with some additional transformations. It is supported by the str. casefold( ) method (new in python 3)

For any string containing inly Latin characters casefold( ) function produces the same result as lower( ), with only two exceptions- the micro sign 'µ' is changed to the Greek lowercase mu (most looks the same in most fonts)and the German Eszett or "sharp s" (β) becomes "ss":

Example 1: Python program to display the lower case letters by using casefold function.

```
text = "pYtHon"
text1="HELLO EVERYONE"
# convert all characters to lowercase
l_str = text. casefold()
l_str2=text1.casefold()
print(l_str)
print(l_str2)
```

Output:

```
python
hello everyone
```

Example2: program to display the casefold( ) as an aggressive lower( ) method:

```
text1 = "Straße"
text2 = "strasse"
```

```
#comparing the lower case letters in text1 and text2
print(text1.casefold() == text2.casefold())
text = 'groß'
# convert text to lowercase using casefold()
print('Using casefold():', text. casefold())
# convert text to lowercase using lower()
print('Using lower():', text.lower())
```

Output:

```
True
Using casefold(): gross
Using lower(): groß
```

From the above example the casefold( ) method also converts 'ß' to lowercase whereas lower ( ) doesn't.

5) String Formatting method:

To address the limitations of the % operator , python introduced the str.format method, which uses { } braces as place holder.

Example:

```
print("My name is {} and I am {} years old.".format("John", 30))
```

Output:

```
My name is John and I am 30 years old.
```

4.11.2. List manipulation:

List provide a flexible and powerful way to manage sequential data in python. They are easy to create and allow efficient access and modification of elements by index. Adding and removing elements are supported by several intuitive methods, and slicing techniques enable extracting sublists for focused operations.

Built-in methods like sort ( ), reverse ( ),and count( ) further simplify list manipulation tasks. Iteration with for loops and elegant list comprehensions facilitate processing elements and generating new list. The other list methods are deeply explained in book 3(3.4 onwards)

a) Searching in Lists: Python lists support searching using in, index(), and count()

Example:

```
fruits=["apple", "banana", "cherry"]
print("apple" in fruits)
print(fruits.index("cherry")) # display the index value of "cherry"
print(fruits.count("banana"))  #display the count of "banana"
```

Output:

```
True
2
1
```

b) List Comprehensions:

- Python also provides list comprehensions, a concise and expressive way to create new lists by applying expressions and conditions to existing iterables.

- A list comprehension is a compact way of creating a new list by performing an operation on each item in an existing list(or other iterable), optionally filtering items.

Examples 1:

```
squares = [x**2 for x in range(5)]
print(squares)
```

Output:

[0, 1, 4, 9, 16]

Note: List comprehensions are for creating new lists, not for executing but the .append() method is used to modify an existing list by adding an element to the end of it.

You shouldn't use append() inside list comprehensions, because append() returns None, which will fill your list with None.

Example 2:

```
a= []
y= [a. append(x**2) for x in range(5)]
print(a)
print(y)
```

Output:

```
[0, 1, 4, 9, 16]
[None, None, None, None, None]
```

Example 3: List comprehension without using append function to create a list

```
evens_squared = [i ** 2 for i in range(10) if i % 2 == 0]
print(evens_squared)
```

Output:

```
[0, 4, 16, 36, 64]
```

List creation using append function in loop

```
evens_squared = []
for i in range(10):
    if i % 2 == 0:
        evens_squared. append(i ** 2)
    print(evens_squared)
```

Output:

```
[0, 4, 16, 36, 64]
```

c)reverse() , reversed( ), sorted( ):

- ***Reverse ( ) method*** is a mutator which returns an iterator that access the list's elements in reverse order.
- ***Reversed ( ) method*** is a non-mutator which returns a new reversed list.
- Sort( ) method sorts the items in place, which is to mutate the list into a sorted order.
- Python also has a built-in function ***Sorted( )*** returns a new list that is ordered according to the specified key or natural ordering . Function sorted( ) is non-mutating, whereas the sort( ) method is a mutator.

Example:

```
num=[3,3,4,6,78,32,67,1,6]
```

```
sorted_numbers=sorted(num)
rev_num=list(reversed (num))
print("Sorted list:", sorted_numbers)
print("reversed numbers:", rev_num)
```

Output:

```
Sorted list: [1, 3, 3, 4, 6, 6, 32, 67, 78]
reversed numbers: [6, 1, 67, 32, 78, 6, 4, 3, 3]
```

d) count ( ):

- The count( ) method returns the number of times a specified element appears in the list.

Example:

```
fruits = ['apple', 'banana', 'orange', 'apple', 'banana', 'apple']
apple_count = fruits.count('apple')
print(f " 'apple' appears {apple_count} times in the list.")
```

Output:

```
'apple' appears 3 times in the list.
```

4.11.3. Dictionary Manipulation:

Dictionaries in python are mutable. Python dictionaries are dynamic data structures that allow for the seamless modification of key-value pairs. The operations for adding , updating , and deleting entries are fundamental for efficiently managing and manipulating data.

In dictionary we can put duplicate values, but key should be different.

For e.g:

This dictionary is valid as keys are unique. Duplicate values are allowed	this dictionary is not valid as key are duplicate.
x={1:"apple",2:"apple",3:"banana"}	x={1:"apple",2:"hi banana",3:"apple"}

Python provides a variety of built-in methods for dictionary manipulation, including ***adding, removing, and iterating over key-value pairs.*** The dictionary methods are briefly explained in book 3(3.7 onwards).

1) Creating an empty dictionary:

Example:

```
n_dict={ }
n_dict["apple"]=20
n_dict["orange"]=45
n_dict["banana"]=20
print(n_dict)
```

Output:

```
{'apple': 20, 'orange': 45, 'banana': 20}
```

2) Update ( ) Method:

Example:

```
n_dict={ }
n_dict["apple"]=20
n_dict["orange"]=45
n_dict["banana"]=20
print(n_dict)
add_item={"mango":112,"grape":50}
n_dict.update(add_item)
```

```
print(n_dict)
```

Output:

```
{'apple': 20, 'orange': 45, 'banana': 20}
{'apple': 20, 'orange': 45, 'banana': 20, 'mango': 112, 'grape': 50}
```

3) Counting with Dictionaries

Example 1:

```
text = "banana"
count = {}
for char in text:
  count[char] = count.get(char, 0) + 1
  print(char, end=" ")
```

Output:

```
b a n a n a
```

Example 2: Program to count frequency and store in a new dictionary.

```
colors = ['red', 'blue', 'red', 'green', 'blue', 'red']
color_count = {}
for color in colors:
    color_count[color] = color_count.get(color, 0) + 1
print(color_count)
```

Output:

```
{'red': 3, 'blue': 2, 'green': 1}
```

4) Dictionary comprehension: It follows the syntax { key: value for vars in iterable}

Example:

```
s_dict={x: x**2 for x in range(6)}
print(s_dict)
```

Output:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

5) get ( ) method: Dictionary provides get() which allows for a default return if the key does not exist, averting potential key errors.

Example:

```
info={'name':'alice','age':30}
print(info['name'])
#modifying age value
info['age']=36
#adding a new key-value pair in dictionary
info['city']='NYC'
print(info. get('country', 'USA'))
```

Output:

```
alice
USA
```

6)Dictionary operations: Dictionary supports operations like ***membership testing(in),length checking(len()), and copying.***

Example 1:

```
info={'name':'alice','age':30}
print(info['name'])
```

```
#modifying age value
info['age']=36
#adding a new key-value pair in dictionary
info['city']='NYC'
print(info. get('country', 'USA'))
#membership testing
print('name' in info)
#length checking
print(len(info))
#creates  a shallow copy
info1=info. copy()
print(info1)
```

Output:

```
alice
USA
True
3
{'name': 'alice', 'age': 36, 'city': 'NYC'}
```

Example 2: Python program for dictionary to counting word frequencies.

```
text='apple banana rose apple jasmine rose apple banana jasmine rose jasmine apple'
words=text. split()
w_count={ }
for a in words:
  w_count[a]=w_count. get(a,0)+1
print(w_count)
```

Output:

```
{'apple': 4, 'banana': 2, 'rose': 3, 'jasmine': 3}
```

4.12.Building blocks of python programs:

Below constructs are not just for python programs, they are part of every programming language from machine language up to the high-level languages.

- **Input:** Get data from the "outside world". This might be reading data from a file, or even some kind of sensor like a microphone or GPS. In our initial programs, our input will come from the user typing data on the keyboard.
- **Output:** Display the results of the program on a screen or store them in a file or perhaps write them to a device like a speaker to play music or speak text.
- **sequential execution:** Perform statements one after another in the order they are encountered in the script.
- **conditional execution:** Check for certain conditions and then execute or skip a sequence of statements.
- **repeated execution :** Perform some set of statements repeatedly, usually with some variation.
- **Reuse:** Write a set of instructions once and give them a name and then reuse those instructions as needed throughout your program.

It sounds almost too simple to be true, and of course it is never so simple. It is like saying that walking is simply "putting one foot in front of the other". The "art" of writing a program is composing and weaving these basic elements together many times over to produce something that is useful to its users.

4.13. Understanding and using ranges:

- The range type is python's built – in sequence generator. It is used to create a sequence of numbers.
- Like strings and tuple , ranges are immutable. The range function return an object of type range. The range function takes three integer arguments :i.e. start, stop, step.

Example:

```
ex_val=range(1,10,2)
for num in ex_val:
  print(num, end=" ")
```

Output:

```
1 3 5 7 9
```

- As you can see, the variable num is assigned a new value for each iteration of the loop. This is called enumerating a loop. The keyword ***in*** is used to denote that you only care about the values which are in the range( ) function returned values.
- We use the len() function again to get the size of the list and that numbers gets passed into range( ) to give a list numbers.

Example:

```
view=['abc', 'def', 'ghi', 'jkl']
print(len(view))
print(list(range(len(view))))
```

Output:

```
4
[0, 1, 2, 3]
```

About range( ) function has explained briefly in book 3(3.4.9).

4.13.1. Descending Loops:

A for loop can operate in either direction , incrementing upward or decrementing downward. The range( ) function , however , only works downward unless you tell it otherwise.

Example:

```
for i in range(10,1,-1):
    print(i)
```

Output:

```
10
9
8
7
6
5
4
3
2
```

4.13.2. Looping Helper functions: enumerate( ), zip( )

- In addition to looping control statements, python has several iterator helper functions. A couple of the most common ones are the *enumerate and zip* function.
- The ***enumerate( )*** function takes an iterable as input and adds a counter to an iterable and returns it as an enumerate object and also enables you to iterate over items while also returning the index of the iteration. This prevents you from having to keep track of the iteration index yourself.
- The ***zip()*** function pairs elements from multiple iterable objects (like lists or tuples) and return together into tuples. It stops when the shortest iterable is exhausted.

Example: Program to illustrate the zip( )and enumerate( ) function.

```
# enumerate( ) function:
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
        print(index, fruit)          # iteration over index based
for i, fruit in enumerate(fruits, start=1):
        print(i, fruit)              #iteration by custom index
#Zip( ) function:
names = ['Alice', 'Bob', 'Charlie']
scores = [85, 90, 78]
for name, score in zip(names, scores):
  print(f"{name} scored {score}")
for x, y in zip(range(3), range(10, 13)):
  print(x, y)
```

Output:

```
0 apple
1 banana
2 cherry
```
Indexed based output

```
1 apple
2 banana
3 cherry
```
Custom Index output

```
Alice scored 85
Bob scored 90
Charlie scored 78
```
Combined elements from two list names, scores

```
0 10
1 11
2 12
```
Combined elements by using range function

4.14. Exercise:

1. Define control structure.
2. What is flow chart? Explain its properties.
3. What is complex Boolean Expression?
4. Define order precedence of logical operator.
5. what are the membership operators can use in Boolean Expression?
6. What are the negate Boolean Expression for the following operator?

a) <= b) != c) and d) not

7.Write the output for the following python code.

```
weather="sunny"
if weather =="sunny":
  print("let's go for a picnic!")
elif weather =="rainy":
  print("it's a day for reading and cozy blankets!")
else:
print("what a beautiful day!")
```

8.Write the output for the following.

a)
```
for fruit in ["jackfruit", "banana", "cherry", "strawberry"]:
  print(f" I Love eating {fruit}!")
```

b)
```
count=5
while count>0:
    print(f "countdown:{count}")
  count -=1
```

9. What are conditional statements ? Explain any two statements.

10. Write a note about how the indentation are important in conditional statements.

11.Write note on controlled statements are used in iteration.

12. what is called infinite loop?

13. Write a python program to print mathematical table using while loop.

14. write a brief note about Nested if statement.

15. Draw the flow chart of Nested if statement.

16. write a note on following statements: i)if-else ii)elif iii)enumerate( ) iv)zip()

17. Identify the syntax errors and find the output in the following python program

```
x=float(input())
y← -5
if x*y/2>20
  y=*1
  x+=4*x²
print(x y)
```

18. Write the python program that prompts the user to enter a number , and then displays the message "positive" when the user provided number is positive.

19. Find the output for the following code:

```
a=20
z=a*10
w=(z-4)*(a-3)/7+36
if a< z >=w:
  y=2*a
else:
  y=4*a
print(y)
```

20. Rewrite the following python program using correct indentation.

```
a=float(input())
```

```
if a<1 :
y=5+a
print(y)
elif a<5:
y=23 / a
print(y)
elif a<10 :
y=5*a
print(y)
else:
print("Error")
```

****

BOOK 5: PYTHON FUNCTION

5.1. Python Functions:

The most essential component of an application is its functions. A function is an organized block of reusable code that may be called anytime it is defined. ***Python enables us to break down a big program into fundamental building pieces known as functions***.

- The functions is made up of the collection of programming statements denoted by a function can be called multiple times to offer reusability and modularity to the python program.
- The Function assist the programmer in breaking down the program into smaller parts. It effectively organizes the code and prevents cod repetition. As the program expands , function helps to arrange it.

Python has a number of built-in functions, such as ***range( ) and print( ).*** However, the user can create its own functions, which are called as user-defined functions.

Key: A function in python is a block of code designed to perform a particular task. Defined using the 'def' keyword, function can accept input in the form of parameters and produce output through return values.

There are mainly three types of functions.

a) ***User-defined function:-*** The user-defined functions are those define by the user to perform the specific task.
b) ***Built-in functions:-*** The built-in functions are those functions that are pre-defined in python.
c) ***Module:-*** A python module is a file containing python code(functions, classes, or variables)that can be imported and reuse in other programs. More about module and its methodology is deeply explained in book 6.

5.1.1. Advantages of Functions in python:

There are the following advantages of python functions.

- Using functions, we can avoid rewriting the same logic/code again and again in a program.
- We can call python functions multiple times in a program and anywhere in a program.
- We can track a large python program easily when it is divided into multiple functions.
- Reusability is the main achievement of python functions.
- However, function calling is always overhead in a python program.

5.2. What is Program Routine ?

A program routine is also called as a function. A function or routine can be defined as a bundle of instructions to perform a specific task. A function or routine can be called as many times as required in the program.

From the image, when the routine B execution gets terminated, the control automatically returns to the calling main program. The main objective of writing a routine/ function is to increase the level of abstraction and to promote software reuse.

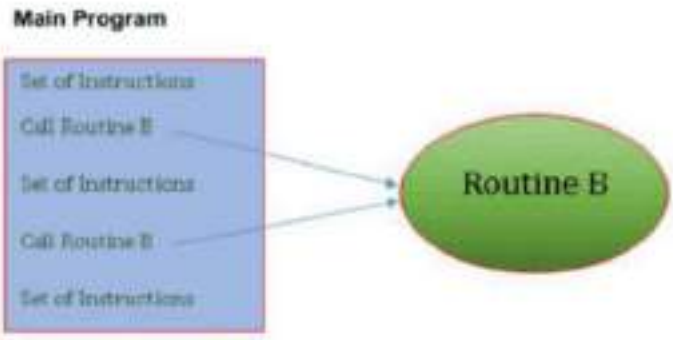


Figure 5.2 Program routine/

5.3. Function Definition:

In order to define a user defined python function , we need to use the keyword **'def'** before the function name. the function name should be followed by parenthesis and should be ended with a **: (colon symbol).**

Syntax of user-defined function:

```
def <Function Name> (<parameters>):
        <Function Statement1>
        <Function Statement2>
            .
            .
```

Note: User-defined functions are also called subprograms or subroutines. The collection of such functions builds a library of functions.

From the above syntax , there can be N number of statements in any given function. But they should be indented from the keyword **' def '**. A function body that has one or more python statements indented at the same level.

For e.g:

```
>>>def demo ( ):
        Print("let's learn about functions in python")
```

From the above example, the first line is called as a function header and it contains the keyword **' def '**. demo is the name of the function.

Example for user-defined Function: python program to define the user-defined function definition by using three parameters.

```
def sum(a,b,c):                          <- Function Header
    print("value a:",a)
    print("value b:",b)
    print("value c:",c)                  } Function body with parameters
    print("sum is:", a+ b+ c)
a=10
b=24
c=34                                     } Arguments
sum(a,b,c)
```

Output:

```
value a: 10
value b: 24
value c: 34
sum is: 68
```

From the above example, the total number of parameters inside the parenthesis (a,b,c) indicates the number of values to be passed to the function. These parameters are called as **"arguments ".**

5.3.1.Nested Function:

In python, nested functions are functions defined within other function, the ***inner*** function is ***nested*** inside the outer. This allows for modular and organized code, as well as the ability to create functions that can be customized and reused within the scope of the enclosing function.

For e.g:

```
def f(x,y):
    w=10
    def g(z):
        return z**3+ w
    return g(x+y)
print(f(2,3))
print(g(3))
```

Nested function

Output:

```
135
Traceback (most recent call last):
File "C:/Users/nishanthi michael/Desktop/pyprg/dict.py", line 7, in <module>
print(g(3))
NameError: name 'g' is not defined
```

From the above example, the nested function ***g*** on line 3 can see and use w outside its definition but within f. we cannot call a nested function by name outside its enclosing function. If we call the nested function of ***g(3)*** , It will produce result as ***NameError.***

Note: Nested functions can be useful for creating closures, which are functions that "remember" the values of variables from the enclosing scop, even after the enclosing function has finished executing.

Example :

```
def outer_fn(x):
    print(f"Executing outer function with {x}")
    def inner_fn(y):
        print(f"Executing inner function with {y}")
        return y*y
    res=inner_fn(x)
    return res
output=outer_fn(7)
print(output)
```

Nested function

Nested function

Outer function call

Output:

```
Executing outer function with 7
Executing inner function with 7
49
```

In this example, the inner_fn( ) has access to the x parameter from the outer_fn, even though it is defined within the outer_fn( ).

5.3.2. Function as Parameter to Another:

Functions can be passed as arguments to other functions, which is powerful technique in functional programming. This allows to create higher-order functions that can operate on other functions, making the code as more flexible and reusable.

For e.g: Python program demonstrates how functions can be passed into other functions, showcasing the flexibility and expressive power of Python.

```
def apply(func, x):
        return func(func(x))
        def square(x):
            return x * x
        res=apply(square,4)
        print(res)
```

Output:

256

The code defines two functions: apply and square. The ***square function is straightforward***. It takes a single number x and returns its square (x * x).

For example, square(4) would return 16.

The function apply is where the concept of higher-order functions is introduced. It takes two parameters: func, which is expected to be a function, and x, which is the value to operate on. Inside the body of apply, the function fun is applied twice—once to the input x, and again to the result of that first application. In effect, it returns func(func(x)).

When apply(square, 4) is called, the execution proceeds as follows:

1. ***First, square(4) is evaluated, which gives 16.***
2. ***Then, square(16) is evaluated, which gives 256.***

Thus, the result stored in res is 256, and the final print(res) statement outputs this value.

Example :

```
def square(n):
    return n * n
def apply_to_all(func, data):
    return [func(x) for x in data]
nums = [1, 2, 3, 4]
print(apply_to_all(square, nums))
```

Output:

[1, 4, 9, 16]

The above code defines a function square to compute the square of a number, and another function apply_to_all that applies a given function (func) to each item in a list (data). It then uses these to square each number in the list [1, 2, 3, 4], resulting in [1, 4, 9, 16].

5.4. More on Function:

Function are one of the most fundamental building blocks of programming in python. They allow developers to break programs into smaller, reuseable, and more manageable pieces of code. ***Functions*** are python's answer to this problem.

5.4.1. Call by reference in python:

In python, calling a function via reference implies giving the real value as an argument. All functions are called by reference, which means that any modifications made to the reference within the function return to the original value referred by the reference.

Syntax:

```
Variable=function_name(argument_list)
                    Or
  Variable=function_name( )
```

Example 1:

```
def cal(x,y):
    z=x*y/x
    return z
x=int(input("Enter the value of x :"))
y=int(input("Enter the value of Y:"))
c=cal(x,y)    #function call
print("the value of", x,' and', y ,'=',c)
```

Output:

```
Enter the value of x :14
Enter the value of Y:12
the value of 14  and 12 = 12.0
```

This Python program demonstrates the use of a user-defined function to perform a basic arithmetic operation. The function cal(x, y) takes two numeric inputs, multiplies them, and then divides the result by the first input. Inside the function, the expression x * y / x is calculated and stored in the variable z.

Since x is used for both multiplication and division, this simplifies to just y (as long as x is not zero).The program prompts the user to input two integer values, x and y, using the input() function and converts them to integers.

Finally, the result using a print() statement, showing both input values and the computed output in a readable format. This python code illustrates function definition, user input handling, and output display.

Key: A function can be called by specifying its name, followed by a list of arguments enclosed within the parenthesis.

If the function does not pass any arguments then an empty pair of parentheses should follow the name of the function.

Example 2: Python program to print the first 10 natural numbers.

```
def natural_no():
    for i in range(1,11):
        print(i)
print(natural_no( ))
```

Output:

```
1
2
3
4
5
6
7
8
9
10
None
```

Example 3: Python code to find the biggest of three numbers using function.

```
n1=int(input("First Number:"))
n2=int(input("Second number :"))
n3=int(input("Third number:"))
def big_num():  #function definition
    if(n1>=n2)and(n1>=n3):
        big=n1
    elif(n2>=n1)and(n2>=n3):
        big=n2
    else:
        big=n3
    print("The biggest given number is:", big)
    big_num()  #function call
```

Output:

```
First Number:23
Second number :12
Third number:46
The biggest given number is: 46
```

For practice:

1. Write a program to find the cube of a number.

```
def cube(a):
    c=a*a*a
    return c
a=int(input("enter the number:"))
print("cube of a number", a,'=',cube(a))
```

Output:

```
enter the number:8
cube of a number 8 = 512
```

Note: A variable in a function header is called an arguments. The arguments are used to pass information from the calling function to the called function.

2. write a program to passing immutable object(list) by using function call.

```
def change_lst(list1):
        list1.append(20)
        list1.append(30)
        list1.append(25)
        print("list inside function=",list1)
list1=[10,15,35,40]
change_lst(list1)       #function call
print("list outside function=",list1)
```

Output:

```
list inside function= [10, 15, 35, 40, 20, 30, 25]
list outside function= [10, 15, 35, 40, 20, 30, 25]
```

3. Python program with arguments to check whether the number passed as an argument to the function is even or odd.

```
def even(x):
    if(x%2==0):
        print(x," "," is even number")
    else:
        print(x," ","is odd number")
even(2)    #call the function with argument
even(13)
```

Output:

```
2   is even number
13  is odd number
```

5.4.2. Returning Function:

In python , when a function wants to give a result to the caller, it uses a ***return*** statement and specifies the value to hand back.

Syntax:

return<expression to be returned as output>

The return statement can be an argument ,a statement, or a value, and it is provided as output when a particular job or function is finished. A declared function will return an empty string if no return statement is written.

Example:

```
#with return statement
def square(num):
   return num**2
print("with return statement")
print(square(34))
```

Output:

```
with return statement
1156
```

Note: When a defined function is called, a return statement is written to exit the function and return the calculated value.
That a 'return' statement cannot be used outside the function.

5.4.2.1. Returning More Than One Value :

In most other programming languages, the return statement can only return either no values or single value. In python, just as you can pass as many values as you want into a function, you can also return any number of values:

Syntax:

return<value1>,<value2>,<value3>,....

Example1: First method to print the value of circle.

```
def area(r1,r2,r3):
    return  3.142*r1*r1,3.142*r2*r2,3.142*r3*r3
a1,a2,a3=area(3,1.5,2)
print("Area of circle with different radius are:",a1,a2,a3)
```

Output:

Area of circle with different radius are: 28.278 7.0695 12.568

In this method, the python program defines a function called area() that calculates the area of three circles using their radius values. It uses the formula area = 3.142 × radius × radius for each circle. The function returns all three area values at once. These values are then stored in the variables a1, a2, and a3.

Finally, the program prints the areas of the three circles together in one line. The program shows how to use a function to return multiple values and print them clearly.

Example 2: Second method to print the radius of circle.

```
def area(r1,r2,r3):
    return  3.142*r1*r1,3.142*r2*r2,3.142*r3*r3
t=area(3,1.5,2)
print("Area of circle with different radius are:", t)
```

Output:

Area of circle with different radius are: (28.278, 7.0695, 12.568)

In this second method, program defines a function called area that calculates the areas of three circles using the formula Area = 3.142 × radius × radius. The function takes three radius values as input (r1, r2, and r3) and returns their areas. The values 3, 1.5, and 2 are passed to the function, and the result (three area values) is stored in the variable t.

5.4.3. Calling Functions That Return None (Void function):

If a function is not going to return any value instead it's going to do some task like displaying some message to the user, then it's called as a ***Non-value returning function.*** It doesn't have a ***'return'*** statement.

Example 1 :

```
#without return statement
def square(num):
    num**2
print("without return statement")
print(square(12))
```

Output:

without return statement
None

This program defines a function square(num) that calculates the square of a number using num**2, but it does not return the result. Since there is no return statement, the function gives no output back to the caller. When print(square(12)) is executed, it displays None because the function doesn't return any value.

Note: Every function in python is a value returning function only. If there is no explicit return from the function, it returns a special value ***none.***

Example 2:

```
def nonvalue():
    print("This is calling Non-value return statement")
nonvalue()
```

Output:

This is calling Non-value return statement

In the above code , This Python program defines a function called nonvalue() that does not return any value. Instead, it simply prints a message from print function.

This is an example of a function with **no return statement**, often used when you just want to display something or perform an action without needing a result back.

Example 3: Python program to illustrate the direct print addition.

```
def add():
    a=10
    b=23                    Output: 33
    c=a+b
    print(c)
add( )
```

The above example is a **void function** that performs an addition of two numbers (10 and 23) and prints the result (33). It does not return any value. Because the function **does not accept any input** — meaning, it has **no parameters inside the parentheses**. Instead, it uses two fixed values (a = 10 and b = 23) **inside the function**.

When the function is called using add(),these two numbers are added together, and the result is stored in the variable c. The function then prints the value of c.

Key: A function that does not include a return statement, or uses return with no value, is considered a void function.

5.4.4.Returning Function:

Functions can also return other functions, which can be useful for creating higher-order functions or for implementing design patterns.

For e.g:

```
def get_function(multiplier):
    def multiply_by(number):
        return number **multiplier
    return multiply_by
result=get_function(4)
print(result(10))
```

Output:

```
10000
```

From the above example, get_function( ) returns another function multiply_by. When get_function(5) is called, it returns the multiply_by function with a specific multiplier. This returned function can then be used to perform multiplication.

Example:

```
def first(factor):
    def second(x):
        return x*factor
    return second
double=first(4)
triple=first(3)
print("From double: ",double(5))
print("from triple :",triple(5))
```

Output:

From double: 20

from triple : 15

5.5.Function Arguments and Parameters:

When we are define a function in a program ,we need to know about the arguments and parameters.

- ***Arguments*** are the actual values that are passed to the function when it is being called. The arguments are included in parenthesis. We can pass any number of parameters, but a comma must separate them.
- ***Parameters*** are the variables that are specified at the time of function declaration in the function header.

Example 1:

```
def func(name):
    print("Hi", name)
func("Mano")     # Calling the function
```

Output:

Hi Mano

Example 2:

```
def add(a,b):
    return a+b
#taking values from the user
a=int(input("enter value for a:"))
b=int(input("enter value for b:"))

#printing the sum of a and b
print("sum=",add(a,b))
```

Output:

enter value for a:41

enter value for b:25

sum= 66

5.6.Passing Arguments to Functions in Python:

- When a function is called, variable are created for receiving the function's arguments. These variables are called parameter variables.(Another commonly used term is formal parameters.).
- The values that are supplied to the function when it is called are the arguments of the call.(These values are also commonly called the actual parameters.). Each parameter variable is initialized with the corresponding argument.
- When we pass a variable to a function python, a new reference to object is created. Parameter passing in python is the same as reference passing in Java.
- Actual parameter values are passed to the formal parameters in two ways. They are

  i)pass- by-value and

  ii)pass- by-reference.

i. **Pass-by Value:**

When parameters are passed by value, the value of the actual parameter is copied into the formal parameter variable. Hence changes made to them inside the function body are not reflected back in the caller. ***Immutable objects are like strings , tuples are passed by value.***

Example:

```
def pass_value(x):
    x="Hello June!, "+x      #x is modified here
    print("Inside the function definition:", x)
s=input("Enter a string:")    #before function call
pass_value(s)                 #function call
print("After Function call:", s)
```

Output:

```
Enter a string: how is your holiday
Inside the function definition: Hello June!, how is your holiday
After Function call: how is your holiday
```

- This program demonstrates how parameter values are passed into a function in Python. The function pass_value(x) takes a single argument and modifies it by concatenating the string "Hello June!, " to it. The modified value is printed within the function.
- However, since strings are immutable in Python and passed by value (for immutable types), the original string provided as input remains unchanged outside the function.
- This is confirmed by printing the value after the function call, which displays the original, unmodified input..

ii. **Pass-by reference:**

When parameters are passed by reference, formal and actual parameters become aliases and refer to the same memory location. Hence changes made to the formal parameters inside the function are reflected back in the caller. ***Mutable objects like lists, sets and dictionaries are passed by reference.***

If you pass an ***Immutable objects (like an int, float, str, or tuple),*** you can't change the original object inside the function.

Example: python code to display the pass-by reference by using mutable set content.

```
def add_element(s):
    s.add(100)
    s.add(212)
    print("Inside function:", s)
my_set = {1, 2, 3}
add_element(my_set)
print("Outside function:", my_set)
```

Output:

```
Inside function: {1, 2, 3, 100, 212}
Outside function: {1, 2, 3, 100, 212}
```

In above illustration , the set was **modified in place** inside the function (because it's mutable), the change is reflected outside the function too.

- This program demonstrates the behavior of mutable data types, specifically a set, when passed to a function. The function add_element(s) receives a set as an argument and modifies it by adding two elements: 100 and 212.

- Because sets in Python are mutable, these changes directly affect the original set my_set even outside the function scope. As a result, the output will reflect the updated set both within and after the function call, illustrating how functions can modify mutable objects in place.

5.6.1. Functions are Objects:

In python, functions are objects (like instances of a class). Like any object

- A function can be assigned to a variable
- A function can be passed into a function as an argument, and
- A function can be the return value of a function

i. Assign function to variable:

Assigning function to a variable allows you to call the function using the variable name, which can be useful for creating more generic or reusable code. When you assign a function to a variable you don't use the ( ) but simply the name of the function name.

For e.g:

```
def add(x,y):
   return x+y
def sub(x,y):
   return x-y
def apply(func, x, y): # this function takes a function object as its first argument
   return func(x,y)        #invoke the function received
print(apply(add,4,5))
print(apply(sub,56,12))
```

Output:

```
9
44
```

ii. Returning an Inner function object from an outer function:

In Python, a function can return another function defined inside it. This is called returning an inner function object. The inner function becomes a first-class object that can be assigned to a variable and called later.

For e.g:

```
def choose_operation(op):
    def add(a, b):
        return a + b
    def subtract(a, b):
        return a - b
    if op == "add":
        return add
    else:
        return subtract
# Get a function based on input
operation = choose_operation("add")
print(operation(5, 3))
```

```
operation = choose_operation("subtract")
print(operation(5, 3))
```

Output:

```
8
2
```

5.7.Types of Arguments:

There may be several types of arguments that can be passed at the time of the function call. They are

i. Required arguments / Positional arguments
ii. Keyword arguments
iii. Default arguments
iv. Variable-length arguments

5.7.1. Required arguments:

The number of arguments in the function call should match exactly with the function definition. The python interpreter will display an error if one of the parameters is not given in the function call or if the position of the arguments is changed.

Example 1:

```
def fun(name):
    mesg="Hi"+" "+name+" "+"How are You"
    return mesg
name=input("Enter the name:")
print(fun(name))
```

Output:

```
Enter the name: Anton Ryan
Hi Anton Ryan How are You
```

The above example has one parameter (name) in the function definition, and only one argument has passed during the function call from the user.

Example 2:

```
def details(name,age,place):
    print("Name:", name)
    print("Age:", age)
    print("Place:", place)
    return
details("Michael",42,"chennai")
```

Output:

```
Name: Michael
Age: 42
Place: chennai
```

Example 3:

```
def prtme(str):
  print (str)
  return
prtme()
```

When the above code executed , it produces the following result as an error of missing argument. Because Python requires all positional arguments to be passed when calling a function unless default values are defined.

Output:

```
Traceback (most recent call last):
    File "C:/Users/nishanthi michael/Desktop/pyprg/dict.py", line 5, in <module>
    prtme()
TypeError: prtme() missing 1 required positional argument: 'str'
```

5.7.2. Keyword Arguments:

Python interpreter is able to use the keywords provided to match the values with parameters even though if they are arranged in out of order. This function call allows us to pass the parameters in arbitrary order.

The parameters' names are keyword and matched in function calling and definition. These types of arguments are also called named arguments because we use the name of the arguments and assignment operator(=) followed by value.

Example 1 : program to print mark detail by using keyword arguments.

```
def info(name,dept,sub1,sub2):
    print("Name:", name)
    print("Department:", dept)
    print("Subject 1:",sub1)
    print("Subject 2:",sub2)
    return
info(name="shanu")
dept="CSC",sub1=76,sub2=56)
```

Output:

```
Name: shanu
Department: CSC
Subject 1: 76
 Subject 2: 56
```

Example 2: program to print the passing keyword arguments from a dictionary

```
def welcome_message(name, language="English"):
    print(f "Welcome, {name}! Language selected: {language}")
user_info = {"name": "Lena", "language": "Spanish"}
# Unpacking dictionary as keyword arguments
welcome_message(**user_info)
```

Output:

```
Welcome, Lena! Language selected: Spanish
```

Note: python allows you to pass multiple keyword arguments represented as **kwargs. It works the same way as *args, except it saves the argument in dictionary format.

This type of argument is important when we don't know how many arguments we'll need ahead of the number.

Example:

```
def food(**kwargs):
    print(kwargs)
    return
food(a='Apple', b='Briyani', c='lemon soda')
food(fruit='orange', vegetable='beans')
food(snack='chips')
```

Output:

```
{'a': 'Apple', 'b': 'Briyani', 'c': 'lemon soda'}
{'fruit': 'orange', 'vegetable': 'beans'}
{'snack': 'chips'}
```

5.7.3. Default Arguments:

A default argument is a parameter that assumes a default value is not provided in the function call for that argument. It allows us to assign default values to the formal arguments.

Default arguments can be assigned to the arguments which do not have matching arguments in the function call. The default values are assigned to the arguments as we initialize variables.

For e.g:

```
def interest(principal, time, rate=0.10):
```

Here, three parameters are defined(principal , time, rate), the parameter rate is a default value(0.10). If in a function call, the value for rate is not provided. Python will fill the missing value with rate=0.10. The function call statement can be,

```
Si=interest(1000,5)
```

This statement passes the value 1000 to principal, 5 to time and uses default value 0.10 to rate, because the third argument is missing in the function call statement.

An argument in a function header can have ***default value*** if and only if all ***should be added from right to the left.*** Below mentioned some examples of valid and invalid function headers with default values.

- def interest(principal, time, rate=0.10): #legal function header
- def interest(principal, time=3,rate=0.10): #legal function header
- Def interest(principal=1000,time=3,rate=0.10): #legal function header
- Def interest(principal=1000,time,rate): #illegal function header

Example 1: program to print the simple interest with default argument.

```
def interest(principal,time,rate=0.10):
```

```
        return principal*time*rate
principal=float(input("Enter the principal amount:"))
time=int(input("enter the time:"))
si=interest(principal,time)
print("simple interest:", si)
```

Output:

```
Enter the principal amount:1500
enter the time:3
simple interest: 450.0
```

Example 2: program to display a person details with default argument.

```
def detail(name,age=23):
    print("My name is", name, "and age is", age)
detail(name=" Ferona")
```

Output:

```
My name is  Ferona and age is 23
```

Example 3: program to display the error by defining the improper default argument

```
def func(x=10,y):
    print("x :",x)
    print("y:",y)
func(20)
```

Here, the above program will produce the result of

Output:

5.7.4. Variable-Length Arguments:

if we want to specify more arguments than specified while defining the function, variable length arguments are used. It is denoted by * symbol before parameter. We may pass any number of arguments by utilizing variable-length parameters.

Syntax:

```
def function_name(*variable_name):
```

Example 1:

```
def msg(*names):
    print("Type of passed arguments is", type(names))
    print("printing the passed arguments")
    for n in names:
        print(n)
msg("Micheal", "Manojan", "Kamal", "Raja", "Sundar")
```

Output:

```
Type of passed arguments is <class 'tuple'>
printing the passed arguments
Micheal
Manojan
Kamal
Raja
Sundar
```

In the above illustration, we passed *names as a variable-length argument in the preceding code. We called the function and passed values that are internally handled as tuples.

The tuple, like the list, is an iterable sequence. We used a for loop to traverse the *arg names to print the supplied values.

Example 2:

```
def aa1(*str):
    ans=[]
    for l in str:
        ans.append(l.upper())
    return ans
obj=aa1('python', 'java', 'data science')
print(obj)
def function(**kargs_list):
    ans=[]
    for key,value in kargs_list. items():
        ans.append([key,value])
    return ans
obj=function(first='Python', second='java', third='data science')
print(obj)
```

Output:

```
['PYTHON', 'JAVA', 'DATA SCIENCE']
[['first', 'Python'], ['second', 'java'], ['third', 'data science']]
```

5.8. Grouping Python Functions by Arguments:

Functions in Python can be grouped by their parameter structure, determining how they receive and process input values. The categories are

- Function with no arguments and no return values
- Function with arguments and with no return values
- Function with arguments and with return values
- Recursive function

i. Function with no arguments and no return values:

In this method, the function performs an independent task. The function does not receive or send any arguments.

Example:

```
def f_name():
  for i in range(1,7):
    print(i)
print(f_name())
```

Output:

```
1
2
3
4
5
6
None
```

ii. Function with arguments and with no return values:

In this method, the function receives some arguments and does not return any value.

Example:

```
def avg(x,y,z):
    sum=x+y+z
    average=sum/3
    print("Average : ",average)
x=int(input("enter the x value:"))
y=int(input("enter the y value:"))
z=int(input("enter the z value:"))
print("Average:", avg(x,y,z))
```

Output:

```
enter the x value:8
enter the y value:7
enter the z value:6
Average :  7.0
Average: None
```

iii. Functions with no arguments and with return values:

Example:

```
def greatest():
    if a > b:
        return a
    else:
        return b
a=int(input("Enter the a value: "))
b=int(input("Enter the b value :"))
print("Greatest :" ,greatest())
```

Output:

```
Enter the a value: 12
Enter the b value :15
Greatest : 15
```

5.8.1. Recursive function:

- ***Recursive function is a function that calls by itself. The process of calling a function by itself is called recursion.***
- The recursive function calls by itself till the stopping condition is encountered. If the stopping condition is not present then the recursive function enters into an infinite loop.
- Each recursive call reduces the problem into smaller subproblems until a base case is reached, which terminates the recursion.

What is Recursion?

Recursion is a programming technique where a function calls itself to solve a problem. Recursive functions are often used to solve problems that can be broken down into smaller, similar subproblems.

5.8.1.1. Key components of recursion:

1. **Base case:**
   - The condition under which the recursion stops.
   - Prevents infinite recursion and a stack overflow.
2. **Recursive case:**
   - The part of the function where it calls itself.
   - Break the problem into smaller subproblem.

Example 1:

```
def countdown(n):
    if n<=0:        #base case
        print("Done!!")
    else:           #recursive case
        print(n)
        countdown(n-1)    # recursive call
#call the recursive function
countdown(5)
```

Output:

```
5
4
3
2
1
Done!!
```

The code shows how a recursive function can repeat an action (printing) with updated input until a condition stops it.

- If n <= 0, it prints "Done!!" — this is the **base case** (stopping condition).
- Otherwise, it prints n and **calls itself** with n-1 — this is the **recursive case**.

Example 2: program to display the sum of digits by using recursion

```
def sum_of(n):
    if n==0:
        return 0
    else:
        return n%10 +sum_of(n//10)
print(sum_of(9541207))
```

Output:

```
28
```

This recursive function calculates the sum of the digits of a number by adding the last digit (n % 10) to the result of the same function called on the number without its last digit (n // 10), until n becomes 0.

5.8.1.2. Types of Recursion:

1. Direct Recursion: A function directly calls itself.

Example:

```
def factorial(n):
    if n==0:  #base case
        return 1
    else:     #recursive case
        return n*factorial(n-1)
print(factorial(6))
```

Output:

720

From the code, The function ***factorial is directly calling itself — factorial(n - 1)*** — which is the definition of ***direct recursion***. The function factorial(n) checks if n is 0. If it is, it returns 1 (this is the base case). Otherwise, it multiplies n by the factorial of n - 1.This repeats until n becomes 0.

2.Indirect Recursion: A function calls another , which in turn calls the original function.

Example:

```
def even(n):
    if n==0:
        return True
    else:
        return odd(n-1)
def odd(n):
    if n==0:
        return False
    else:
        return even(n-1)
# test indirect recursion
print(even(4))
print(odd(3))
```

Output:

True

True

In the above cod, the function even(n) calls odd(n-1) and vice versa instead of calling itself. They keep calling each other , forming an indirect recursive loop.

5.8.1.3. Advantages and Disadvantages of recursion function:

Advantages:

→ ***Elegance:*** Recursive solution are often concise and easier to read.

→ ***Suitable for Divide-and-conquer:*** Ideal for problems like merge sort, quick sort, and tree traversal.

Disadvantages:

→ ***Performance Overhead:*** Each recursive call adds a frame to the call stack , leading to higher memory usage.

→ ***Risk of stack Overflow:*** if the recursion depth is too large , it can result in a Recursion Error

5.8.2. Lambda Function (Anonymous function):

Lambda functions are tiny functions that can take any number of arguments but can have only one statement or expression as its body. The function has no name and hence also called an anonymous function.

Syntax: Variable=**lambda formal_parameters: expression**

Lambda is a keyword, expression may include *formal_parameters*.
The value of the expression is assigned to the variable.

Example 1: By using Lambda function to calculate the cube of a given number:

```
cube=lambda x:x**3
print(cube(6))
```

Output:

216

In this example , the lambda function lambda x:x**3 takes single argument x and returns its cube.

Example 2: Python program to check the number is odd or even by using lambda function.

```
even=lambda x:x%2==0
print(even(7))
print(even(8))
```

Output:

```
False
True
```

Example 3: Python program to sort a list of tuples by the second element by using lambda function.

```
pairs = [(1, 3), (2, 1), (4, 2)]
pairs.sort(key=lambda x: x[1])
print(pairs)
```

Output:

```
[(2, 1), (4, 2), (1, 3)]
```

5.8.2.1. Using Lambda function in filter( ),map( ),reduce( ) and Comprehension:

Instead of using a for loop to iterate through all the items in iterable (sequence), you can use the following functions to apply an operating to all the items. This is known as ***functional programming*** or ***expression-oriented programming.***

i)filter(func, iterable):- It return an iterator yielding those items of iterable for which func(item) is True.

For e.g: program to use with filter( ) to get even numbers :

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens)
```

Output:

```
[2, 4, 6]
```

This code uses a lambda function with filter() to select only the even numbers from the list nums, and converts the result into a new list called evens.

ii) map(func, iterable):- Apply (or map) the function on each item of the iterable.

For e.g:

```
lst=[11,12,22,33,44,21,55]
a=list(map(lambda x: x*2,lst))
print(a)
```

Output:

```
[22, 24, 44, 66, 88, 42, 110]
```

The lambda x: x*2 doubles each number, and map() applies it to all elements in lst.

iv) reduce(func, iterable) (in module functools):-Apply the function of two arguments cumulatively to the items os a sequence, from left to right, so as to reduce the sequence to a single value.

For e.g:

```
lst=[1,23,41,14,13,15]
from functools import reduce
a=reduce(lambda x,y: x+y, l st)
print(a)
```

Output:

107

The code adds all numbers in the list using reduce() and prints the total sum((1+23)+(41+14)+(13+15)).

v) List Comprehension:- A one-liner to generate a list as discussed in the earlier section.

For e.g:

```
lst=[x**x for x in range(1,8) if x%2==0]
print(lst)
```

Output:

[4, 256, 46656]

For practice:

1. Write a python function with default value to find value of x^2+2x+5 where x is known.

Solution:

```
def expression(x=4):
    return x**2 + 2*x + 5
print("Default x=4:", expression())
print("x=3:", expression(3))
```

Output:

Default x=4: 29
x=3: 20

2. Write a python function to compare two words.

Solution:

```
def words():
    word1 = input("Enter first word: ").lower()
    word2 = input("Enter second word: ").lower()
    if word1 == word2:
        print("Words match (case-insensitive).")
    else:
        print("Words do not match.")
words()
```

Output 1:

Enter first word: Quantum
Enter second word: harmony
Words do not match.

Output 2:

Enter first word: nature
Enter second word: Nature
Words match (case-insensitive).

3. Write a python function to provide a sports grade or an arts grade using default arguments.

Solution:

```
def res(total, sports=None, arts=None):
    print('Total marks= ',total)
    if sports is not None:
        print('sports grade ',sports)
        if arts is not None:
            print('arts grade', arts)
res(98,'A')
res(79,'B','C')
res(89)
```

Output:

```
Total marks= 98
sports grade A
Total marks= 79
sports grade B
arts grade C
Total marks= 89
```

5. Write a function to print Fibonacci series.

Solution:

```
def fib(n):
    a,b=0,1
    while(a<n):
        print(a, end=' ')
    a,b=b, a+b
fib(150)
```

Output:

```
0 1 1 2 3 5 8 13 21 34 55 89 144
```

6. Write a python function to calculate factorial of entered number.

Solution:

```
def fact(n):
    res=n
    for i in range(1,n):
        res *=i
    return res
number=int(input("Enter a number:"))
factorial=fact(number)
print("Factorial of number ",number, "is", factorial)
```

Output:

```
Enter a number:7
Factorial of number 7 is 5040
```

7. Write a python recursion function to check the given word are palindrome or not.

Solution:

```
def palindrome(s):
  if len(s)<=1:   #Base case
    return True
  if s[0]!=s[-1]:
    return False
  return palindrome(s[1:-1])   #recursive case
print(palindrome("radar"))
print(palindrome("hello"))
```

Output:

```
True
False
```

8. Write the program to find the exponent.

Solution:

```
def power(a,b):
    if a==0:
```

```
        return 0.0
    elif b==0:
        return 1.0
    elif b>0:
        return a*power(a,b-1)
    else:
        return 1/a *power(a,b+1)
x=float(input("Enter the base: "))
y=int(input("Enter the Exponent :"))
print(x, "Raised to the power of", y,'=',pow(x,y))
```

Output:

```
Enter the base: 12
Enter the Exponent :2
12.0 Raised to the power of 2 = 144.0
```

5.8.3. Decorators and Generators in python:

i) Decorators:

In python , a decorator is a callable (function) that takes a function as an argument and return a replacement function. Recall that functions are objects in python. i.e. , you can pass a function as argument, and return a function. A decorator is a transformation of a function. Python uses the @ symbol to denote the replacement .

For e.g :

```
def double(func):
    def wrapper(*args,**kwargs):
        result=func(*args,**kwargs)
        return result * 2
    return wrapper
@double
def add(a,b):
    return a+b
print("The value is:", add(5,8))
```

Output:

```
The value is: 26
```

This program uses a decorator to change the result of a function. The double function is a decorator that takes another function and multiplies its result by 2. The @double line means the add function will now return double the result. When we call add(5, 8), it first adds the numbers (5 + 8 = 13), then the decorator makes it 13 × 2 = 26.

So the final output is: "The value is: 26".

Example 2: Decorators can be used to modify the behaviour of functions that work with strings.

```
def to_upper(func):
    def wrapper(string):
        return func(string.upper())
    return wrapper
@to_upper
```

```
def greet(name):
    return f"Hello,{name}!"
print(greet("raja raja chozhan"))
```

Output:

Hello,RAJA RAJA CHOZHAN!

ii)Generator:

Generator are a special type of function in python that can be paused and resumed , allowing them to generate a sequence of values on-the-fly, rather than creating and returning a full list at once.

Example:

```
def count(n):
    i=0
    while i<n:
        yield i
    i += 1
counter=count(4)
print(next(counter))
print(next(counter))
print(next(counter))
print(next(counter))
```

Output:

```
0
1
2
3
```

This program uses a generator to give numbers one by one. The count(n) function starts from 0 and uses yield to return numbers up to n - 1. The next() function is used to get each value from the generator.

5.9. Python Built-in functions:

Python built-in functions are defined as functions with pre-defined functionality in python. Built-in functions are the functions that are built into python and can be accessed by programmer any time without importing any module(file).

Built-in functions are the ***names***, followed by an optional list of arguments that represents information being passed to the information.

Note: The built-in functions available in python are provided by default and programmer need not import any module for them.

Python has numerous built-in functions, which are mentioned below:

Name	Description
abs()	The abs() function is used to get the absolute (positive) value of a given number.
all()	The all() function is used to test whether all elements of an iterable are true or not.
any()	The any() function returns True if any element of the iterable is True. The function returns False if the iterable is empty.
ascii()	This function returns a string containing a printable representation (escape the non-ACSII characters)of an object.

bin()	The bin() function is used to convert a n integer number to binary string. The result is a valid python expression.
bool()	the bool() function is used to convert a value to a Boolean.
chr()	The chr() function returns the string representing a character whose Unicode point is the integer.
classmethod()	This function is used to convert a method into a class method.
dict()	This dict() function is used to create a new dictionary.

And some other built-in functions are also there like ***dir(), filter( ), float( ), id( ),input( ),int( ),len( ), list( ), next( ),pow( ), type( ),etc.***

5.10. variable Scope: The Life Area of a Variable

Variable scopes in python dictate visibility and lifespan. The scope of variables depends upon the location where the variable is being declared. The variable declared in one part of the program may not be accessible to the other parts.

In python , the variable are defined with two types of scopes. a)Global scope
b)Local scope

Note: The scope determines where a variable can be accessed or referenced within a program, and the lifetime refers to the duration for which the variable exists in memory.

5.10.1.Local Scope:

- A local variable in python is a variable defined within a specific block of code ,such as a function or a loop.
- It is accessible only within that block and has a limited lifespan, existing only for the duration of the block's execution.

Note: In python , if a variable is assigned within a function , it is automatically considered local unless explicitly declared as global using the 'global' keyword, which allows it to be accessed outside the function.

For e.g:

```
def function():
    x=15    #local scope
    print(f"Inside function: x= {x*2}")
function()
```

Output:

Inside function: x= 30

In this example, x is a local variable within the function() and cannot be access from outside the function.

5.10.1.1. Lifetime of Local scope:

Local variables are created when the functions is called and destroyed when the function exits.

Example:

```
def outer():
    msg = "I am local to outer"
    def inner():
        print(msg)
    inner()
outer()
```

Output:

I am local to outer

From the example, msg is a local variable to outer(), it is **accessible to inner()** because it was defined inside the same enclosing scope. But once outer() finishes, msg no longer exists.

5.10.2. Global Scope:

- Variables declared outside any function or block reside in the global scope, accessible throughout the module or entire program.
- They can be used by any part of the program.
- To define a variable as global scope, or to across / manipulate a global variable inside a function, ***'global'*** keyword is used.

For e.g 1:

```
y=25   # Global variable
def my_func():
    print(f"Inside function: y={y}")
my_func()
print(f"Outside function: y={y}")
```

output:

```
Inside function: y=25
Outside function: y=25
```

Here , y is a global variable defined outside the function. It can be accessed both inside and outside the my_func().

5.10.2.1. Lifetime and modifying of Global Scope within Function:

Global variables exist throughout the program's execution and are destroyed when the program terminates. This means it can be used and modified across different functions if declared properly.

For e.g:

```
a = 2  # Global variable
def f(x):
    def g(x):
        global a
        return x * (y + a)
    y = 1    # Local variable
    a = 0  # Local 'a', different from global 'a'
    z = g(x + 1) + y
    return z
f(1)
```

Explanation:

- a = 2 is a global variable.
- Inside f, a new local a = 0 is defined, but it **does not affect** the global a.
- Inside g, global a is declared, so it **uses the global a = 2**.
- g(2) returns 2 * (1 + 2) = 6.
- Final result is 6 + 1 = 7.

Output:

7

In the above code, the nested function g uses global a, so it refers to the original global variable (a = 2), not the local one. The lifetime of global variable lasts throughout the whole program because the local a inside f() does not change the global one.

5.11.Exercise:

1. Which keyword is used for function?
2. Define function.

3. What are the parts of the function?
4. What are the advantages of function?
5. What are the arguments supported by python?
6. List and define the types of function in python.
7. List and write description about any five Built-in function.
8. What are the arguments supported by python?
9. Differentiate between argument and parameter.
10. Write a note on default argument with proper example.
11. How do you return a value from a function?
12. Write the purpose of return statement.
13. Write short note on Built-in function.
14. Write a function to check entered number is even or odd.
15. What is function header?
16. Write the output of following function:

a) def add(a,b):
   i. c=a+b
   ii. return(c)
   iii. p=add(5,72)
   iv. print(p)
   v. q=add(3,6)
   vi. print(q)

b) b)def f(x):
   i. y=x**x
   ii. return(y)

c) print("testing ...")
d) print("passing the value 12")
e) z=f(5)
f) print("the function returns:", z)

17. What are *args and **kwargs, and how can they be used in a function?
18. Find the output and find what type of function is the following code?

```
def factorial(n):
    if n==0:
        return 1
    else:
        return n * factorial(n-1)
result=factorial(5)
print(result)
```

19. What is the output of the following code?

```
def test(a,b):
    return a - b
print(test(b=10,a= -5))
```

20. What is the difference between the local and global variable?
21. What is recursion, and how can it be used in python?
22. What is called lambda function ?

BOOK 6: PYTHON CLASSES And OBJECTS

6.1. Objects In Python:

Python is an Object Oriented Programming Language. Classes and Objects are the key features of OOP. Object-Oriented Programming(OOP) is a programming paradigm that focuses on creating objects, Which are instances of classes.

The two main players in OOP are ***objects*** and ***classes.***

→ ***Classes*** are used to create objects(objects are instances of the classes with which they were created).***Classes*** define the structure and behaviour of objects, and objects are the building blocks of OOP.

→ When ***objects*** are created by a class, they inherit the class attributes and methods. They represent concrete items in the program's domain. The object is also called an ***Instance*** of a class of objects. Python strings, lists, and dictionaries are all examples of python objects.

Note: "Objects are python's abstraction for data. All data in a python program is represented by objects or by relations between objects."

6.2. Software Objects:

In Python, a **software object** is an instance of a class that combines both data and the functions that operate on that data. Objects provide a structured way to organize code, allowing programmers to model real-world entities.

For example, an object representing a student might store the student's name and marks, and contain methods to calculate their average or display their details. To define this the python code is

Simple Program: Student Class to calculate the average

```
class Student:
    def set_details(self, name, mark1, mark2):
        self.name = name
        self.mark1 = mark1
        self.mark2 = mark2

    def calculate_average(self):          #method 1
        return (self.mark1 + self.mark2) / 2

    def display(self):                    #method 2
        print(f"Name: {self.name}")
        print(f"Average: {self.calculate_average()}")

s = Student()    # Create object
s.set_details("Ravi", 80, 90)      # Set values
s.display()      # Display result
```

Output:

```
Name: Ravi
Average: 85.0
```

The software objects are…

- ***Class:*** A blueprint or template defining the structure and behavior of objects.
- ***Instance:*** A specific object created from a class.
- ***Attributes:*** Variables holding the object's data or state.
- ***Methods:*** Functions defining the object's actions or behavior.
- ***Encapsulation:*** Bundling data and methods within a class, hiding internal details.

- ***Inheritance:*** Creating new classes based on existing ones, reusing and extending functionality.
- ***Polymorphism:*** Allowing objects of different classes to respond to the same method call in their own way.

6.3. Defining Classes in python:

A class in python serves as blueprint for creating objects, encapsulating data for the object and methods to manipulate the data. To define a class, the ***class*** keyword is employed followed by the class name and a colon (:).

Syntax:

```
Class class_name:
    Statements
```

For e.g: program to display a simple creation of class.

```
class fruit:
    apple=5
print(fruit)
```

Output:

```
<class '__main__.fruit'>
```

In this program, a class named fruit is defined with a class variable apple set to 5. When you print fruit, you're printing the class itself, not an object or variable inside it. So, Python shows the class type:<class '__main__.fruit'>

This means that fruit is a class defined in the __main__ module (the current script).

6.3.1. Create an Inner class:

Python allows to define a class within another class, known as an inner class. Inner classes can access the attributes and methods of the outer class.

For e.g:

```
class outerclass:
    def __init__(self,outer_value):
        self.outer_value=outer_value
        self.inner_obj=self.Innerclass(outer_value *2)
    def outer_method(self):
        print(f"outer value: {self.outer_value}")
        self.inner_obj.inner_method()

    class Innerclass:
        def __init__(self,inner_value):
            self.inner_value=inner_value
        def inner_method(self):
            print(f"Inner value: {self.inner_value}")
outer_obj=outerclass(15)
outer_obj.outer_method()
```

Output:

```
outer value: 15
Inner value: 30
```

The above program shows how one class can be defined inside another (called an inner class). The outer class stores a value and also creates an inner class object. The inner class has its own method and uses a value based on the outer class. When the outer method is called, it prints both the outer and inner values.

6.3.2. Class Namespace:

When a class is defined, a namespace is created specifically for that class. This namespace contains the class's attributes (variables) and methods (functions). These names are only accessible within the scope of the class or through an instance of the class.

The structure of a class may include attributes, methods, and a constructor.

6.4. Attributes:

Attributes in python classes are ***variables*** that hold data related to a particular object. These are essentially variables defined within a class and can be used to store and retrieve data. They may be instance variables or class variables.

- ***Instance variables:*** these are specific to an object(i.e., an instance of a class).each object instance can have a different value for these variables.
- ***Class variables:*** Shared across all instances of the class. Changes to class variables reflect across all instances.

6.5.Methods:

Methods in classes are functions that define behaviours or actions associated with a class. They operate on class instances and their data. Methods in python classes are recognized by their indents and rely on their first arguments, typically named ***self,*** which refers to the instance.

Example 1: A simple example to define and creating a class.

```
class person:
    def __init__(self,name,age):
        self.name=name      #class attribute
        self.age=age
    def greet(self):
        print(f"hello,my name is {self.name} and i am {self.age} years old.")
a=person("Semmozhi Selvi",34)
a.greet()
```

Output:

```
hello,my name is Semmozhi Selvi and i am 34 years old.
```

In this example, we define a *person class* with an *__init__* method (a constructor) that initializes the name and age attributes, and a *greet* method that prints a greeting.

What is self? The Link Between Data and Functions in a Class

In python's object-oriented programming , the keyword '***self***' *is always the first attribute of an instance method, it* occupies role in defining and referencing instance-specific data. When a ***method*** is defined within a class, it is designed to operate on an object instance, and the '***self*** ' parameter is used to access both attributes and other methods.

Example 2: program to display how we can share the attributes and methods with all instances.

```
class square( ):
    side=5
    def area(self): #self is a reference to an instance
        return self.side**2
sq=square()
```

```
print(sq.area())   #side value will find from the class method area
print(square.area(sq))  #equivalant to sq.area()
sq.side=10
print(sq.area())
```

Output:

```
25
25
100
```

Example 3: Python program to display how class variables are shared between objects.

```
class inventory:
    items=[ ]
    def __init__(self,name):
        self.name=name
    def add_item(self,item):
        inventory.items.append(item)
inventory1=inventory("Tool shed")
inventory1.add_item("Hammer")
print(inventory.items)     #both  inventories share the same items list
inventory2=inventory("Garage")
inventory2.add_item("saw")
print(inventory1.items)
print(inventory2.items)
```

Output:

```
['Hammer']
['Hammer', 'saw']
['Hammer', 'saw']
```

This program shows that the items list is a class variable, meaning it's shared by all objects of the inventory class. When inventory1 adds "Hammer" and inventory2 adds "saw", both objects see the same list because they use the same shared items. That's why the final output shows both items in all objects.

6.5.1. Class Methods and Their Types:

Python offers different types of methods within classes, each serving a distinct purpose in the design and behavior of objects. The three methods are

- Instance method
- Class method and
- Static method

Each of this methods plays a unique role in interacting with the class or its instances.

i)Instance Method:

→ This is the most common form of methods found within classes. They are defined with the first parameter named ***'self'***, which acts as a reference to the specific instance of the class.

→ Instance methods operate on data that belong to an object, accessing and modifying instance attributes stored in the object's internal namespace.

→ When a method is called on an instance, python automatically passes the instance as the first argument to the method.

→ This design allows instance methods to maintain or update the state of the object.

Example 1:

```
class bankaccount:
    def __init__(self,owner,balance):
        self.owner=owner
        self.balance=balance
    def deposite(self,amount):
        self.balance +=amount
        print("deposite complete.new balance:",self.balance)
    def withdraw(self,amount):
        if amount > self.balance:
            print("withdrawal  failed: insufficient funds")
        else:
            self.balance -= amount
            print("withdrawal complete.new balance :",self.balance)
ac=bankaccount("Manoj",2000)
ac.deposite(150)
ac.withdraw(600)
```

Output:

```
deposite complete.new balance: 2150
withdrawal complete.new balance : 1550
```

This example shows how instance method *deposite* and *withdraw* modify the instance attribute *balance* and print messages related to transactions. The use of ***'self'*** ensures that the correct account instance is updated.

Example 2:

```
class pet:
    def __init__(self,name, age):
        self.name=name
        self.age=age
    #instance methods
    def speak(self):
        return f"{self.name} makes  a sound."
    def birthday(self):
        self.age+=1
#create an instance
mypet=pet("Bublu",2)
#calling instance methods
print(mypet.speak())
mypet.birthday()
print(mypet.age)
```

Output:

```
Bublu makes  a sound.
3
```

ii)Class Method:

- → In contrast to instance methods, class methods operate on the class itself rather than on individual instances.
- → They are defined using the ***@classmethod*** decorator and take a first parameter named ***cls,*** which represents the class rather than an instance.
- → Class methods are useful when a method needs to access class attributes or for defining alternative constructors that create instances in a specific manner.

Example:

```
class emp:
        raise_percentage=1.05      #class attribute shared across instances
        def __init__(self,name,salary):
           self.name=name
           self.salary=salary
        def apply_raise(self):
           self.salary *= emp.raise_percentage
           print(f"New salary for {self.name}: {self.salary}")
        @classmethod
        def update_raise_percentage(cls,percentage):
           cls.raise_percentage= percentage
        print("updated raise percentage to :",cls.raise_percentage)
        @classmethod
        def from_string(cls,emp_str):
           name,salary=emp_str.split("-")
           return cls(name.strip(),float(salary.strip()))
emp1=emp("balu",50000)
emp1.apply_raise()
emp.update_raise_percentage(1.10)
emp2=emp.from_string("michael-70000")
emp2.apply_raise()
```

Output:

```
New salary for balu: 52500.0
updated raise percentage to : 1.1
New salary for michael: 77000.0
```

The above example illustrates two primary uses of class methods. The first *,update_raise_percentage,* changes a class-level attribute that affects all employee objects. The second, *from_string* ,serves as an alternative constructor, facilitating the creation of an employee object from a string representation.

iii)Static Method:

- → static method, defines with the @staticmethod decorator, are the third type of methods within python classes.
- → Static methods do not automatically receive either ***self*** or ***cls*** as parameter, meaning that they cannot access or modify the instance or class attributes directly.
- → Static methods are useful when the method's functionality is related to the class.

Example:

```
class mathoperations:
    @staticmethod
    def add_numbers(a,b):
        return a+b
    @staticmethod
    def multiply_numbers(a,b):
        return a*b
result_add=mathoperations.add_numbers(12,14)
result_mul=mathoperations.multiply_numbers(6,8)
print("Addition result:",result_add)
print("multiplication result:",result_mul)
```

Output:

```
Addition result: 26
multiplication result: 48
```

The above example, static methods *add_numbers* and *multiply_numbers* perform arithmetic operations without accessing or modifying any class or instance-specific data. They are self-contained processes that belong to the class but operate independently of any object state.

Example : program to display the working status of a library by using instance, class, and static method.

```
class library:
    library_name="central library"
    def __init__(self, book_title, author):
        self.book_title=book_title
        self.author=author
    #instance method
    def book_info(self):
        return f"Title: {self.book_title},Author:{self.author}"
    #class method
    @classmethod
    def set_library_name(cls,name):
        cls.library_name=name
    #static method
    @staticmethod
    def is_open(day):
        # assuming the library is closed on Sundays
        return day.lower()!="Sunday"
#create an instance and call methods
book=library("1982","George Orwell")
print(book.book_info())    #using instance method
library.set_library_name("Westside Library")    #using class method
print(library.is_open("monday"))    # using static method
```

Output:

```
Title: 1982,Author:George Orwell
True
```

6.5.2. The Constructor: The __init__( ) method

In python , the __init__( ) method serves as a constructor, used to initialize objects, variables, and methods within a class. When an instance of a class is created, the __init__()method is automatically executed.

Example 1: program to display constructor with default value.

```
class product:
   def __init__(self, name, price, discount=0):
       self.name = name
       self.price = price
       self.discount = discount
 def get_final_price(self):
       return self.price - (self.price * self.discount / 100)
  def display(self):
       print(f"Product: {self.name}")
       print(f"Price: ${self.price}")
       print(f"Discount: {self.discount}%")
       print(f"Final Price: ${self.get_final_price():.2f}")
       print("------------------------")
# Creating instances
product1 = product("Laptop", 13000)
product2 = product("Smartphone", 8000, discount=10)
# Display output
product1.display()
product2.display()
```

Output:

```
Product: Laptop
Price: $13000
Discount: 0%
Final Price: $13000.00
------------------------
Product: Smartphone
Price: $8000
Discount: 10%
Final Price: $7200.00
------------------------
```

This approach is similar to other programming languages like C,C++,Dart, and Java which use a main() function to initiate execution. However, Unlike main(),__init() is specific to object instantiation.

6.6.Object Creation :

An objects is an instance of a class, they are created through the instantiation of classes. The ***instantiation*** process ***allocates memory*** for the new object and initializes its state ***through the __init__ constructor.*** To create an object, you call the class name as if it were a function, passing any required arguments to the constructor.

Syntax:

```
Object_name=class_name( )
```

Note: *The process of creating an object from a class is called as instantiation.*

Concept Snapshot:

In python, classes are instantiated by creating objects, but in some cases, they can be executed without explicitly creating an object.

1. Instantiating a class: Normally , to use a class, you create an instance(object) of it.

For e.g:

```
class greet:
    print("Hello,\nGood day Buddy")
#creating an object and calling method
obj=greet
```

Output:

```
Hello,
Good day Buddy
```

2. Running a class without Instantiation:

In python, a class can contain class-level methods that can be called without creating an object.

For e.g:

```
class greet:
    print("Hello,\nGood day Buddy")
#calling  the method without an instance
greet
```

Output:

```
Hello,
Good day Buddy
```

Example:

```
class car:
    def __init__(self,model,year):
        self.model=model
        self.year=year
#object creation
mycar=car("Toyota  Model 5 ",2022)
print(mycar.model)  # accessing model attributes
mycar.year+=1    #Modifying year attribute
print(mycar.year)
```

Output:

```
Toyota  Model 5
2023
```

Each instantiation of a car object results in a distinct memory allocation corresponding to the object's attributes and methods, providing a personalized piece of data.

Accessing Attributes:

In python, we can access the attributes of an object using the ***dot (.) operator.***

For e.g:

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
# Create a Rectangle object
rect = Rectangle(10, 5)
# Access attributes and method
print("Width:", rect.width)
print("Height:", rect.height)
print("Area:", rect.area())
```

Output:

```
Width: 10
Height: 5
Area: 50
```

The above program defines a class Rectangle that represents a rectangle shape with **width** and **height**, and includes a method to calculate its **area**. Then, ***rect.width*** and ***rect.height*** access the object's attributes using the **dot operator**.

6.6.1. Object Namespace:

Python classes are constructed as blueprints for creating objects that encapsulate both data and behavior. When a new object is initiated, python allocates a unique namespace for that object.

This namespace is a mapping of names(or identifiers) to their corresponding attributes and methods.

These attributes are defined within the __init__ method or can be added dynamically.

The *'self'* parameter acts as the reference to the object on which the current method is being called, enabling the correct resolution of names within the object's namespace.

Example 1:

```
class sample:
    def __init__(self,number):
        self.number=number
    def display(self):
        print("The number is: ",self.number)
s=sample(45)   # object creation 's'
s.display()  #method called to print the output
```

Output:

```
The number is:  45
```

📌 **Remember This:**

In python, instantiating a class with or without the __init__() constructor behaves differently.

1. Without __init__():

When a class does not have an __init__ method, an instance can be created simply by referencing the class name:

For e.g:

```
Class Myclass:
    Pass
#instantiating
Object=Myclass
```

2. with __init__():

When a class has an __init__() constructor, it must be instantiated with parentheses:

For e.g:

```
Class Myclass:
  def __init__(self):
      print("instance")
#instantiating
Object=Myclass
```

Example 2: program to display the instance variables using the self-parameter.

```
class car:
    def __init__(self,brand,model,year):
        self.brand=brand
        self.model=model
        self.year=year
    def display(self):
        print("the brand is:",self.brand)
        print("the model is:",self.model)
        print("the year is:",self.year)
  #creating instances
car1=car("Toyota", "Camry", 2022)
car2=car("Honda","Accord",2021)
car1.display()
car2.display()
```

Output:

```
the brand is: Toyota
the model is: Camry
the year is: 2022
the brand is: Honda
the model is: Accord
the year is: 2021
```

6.6.2.Object Destruction:

Destructors are special methods invoked at the end of the lifecycle of objects, when they must be deleted. In python the ***__del__( )*** method is invoked when an object is about to be destroyed.
This method automatically deletes an object that is no longer in use. This automatic destroying of an object is known as garbage collection.

Note: *Define by using the __del__ method is used to delete an instance/ object when it is not needed anymore. The method takes no arguments, and returns no values.*

Example: program to display the object creation and destruction

```
class employee:   #defining the class
  #default constructor that only prints a message
  def __init__(self):
    print("Employee created")
  #destructor deletes the object and print a message
  def __del__(self):
    print("employee deleted")
emp1=employee()
emp2=employee()
#destroy objects emp1 and emp2. destructor method is called
del emp1
del emp2
```

Output:

```
Employee created
Employee created
employee deleted
employee deleted
```

In this program, a class called employee is defined with two special methods: a constructor and a destructor. The constructor method __init__() is automatically called when an object is created, and it prints "Employee created".
The destructor method __del__() is called when an object is deleted using del, and it prints "employee deleted". When two objects emp1 and emp2 are created, the constructor runs for both and prints the message twice. Later, when emp1 and emp2 are deleted using del, the destructor runs for each and shows the deletion message.

Example 2:

```
class point():
  def __init__(self,x=0,y=0):
    self.x=x
    self.y=y
  def __del__(self):
    class_name=self.__class__.__name__
    print(class_name,"destroyed")
pt1=point()
pt2=pt1
pt3=pt1
```

```
print(id(pt1),id(pt2),id(pt3))
del pt1
del pt2
del pt3
```

Output 1:

```
2149036615600 2149036615600 2149036615600
point destroyed
```

Output 2:

```
1948540561328 1948540561328 1948540561328
point destroyed
```

Garbage Collection:

Python's automatic memory management , known as garbage collection, is responsible for reclaiming the memory occupied by objects that are no longer in use. Python's garbage collector periodically checks for unreachable objects and frees the memory occupied by them.

GC Methods:

Python provides several methods to interact with the garbage collector

- → *gc.collect(): Manually triggers a garbage collection.*
- → *gc.get_count():Returns the number of garbage collection generations.*
- → *gc.get_threshold(): Returns the current collection thresholds.*
- → *gc.set_threshold(): Set the collection thresholds.*

6.7. Encapsulation:

Encapsulation is one of the pillar of object-oriented programming. It is based on the idea of wrapping up the attributes and methods in a class and controlling access when instantiating a new objects/instances. ***It emphasizes how encapsulation promotes information hiding by controlling access is levels using public, private and protected modifiers.***

Instead, access modifiers are used to dictate and control how the instance attributes can be accessed.

For e.g:

```
class rectangle:
    def __init__(self,width,height):
        self.width=width
        self.height=height
    def area(self):
        return self.width *self.height
    def display(self):
        print(f"width:{self.width}.heigth:{self.height}")
```

The ***rectangle*** class uses encapsulation to group related data and behavior. It provides methods to access and operate on the data safely and clearly, making it easy to create and manage multiple rectangle objects.

Advantages of Encapsulation:

Encapsulation offers several benefits related to data management and object-oriented design:

- ***Controlled Access:*** Encapsulation restricts direct access to internal data, allowing only controlled interaction through defined methods to protect and manage the data safely.
- ***Modularity:*** Grouping related data and methods fosters code modularity, making systems more manageable by separating different part of a program into discrete unit.
- ***Ease of Maintenance:*** Modular and well-encapsulated code is inherently easier to traverse and adapt.

> Don't Miss This:
>
> - Encapsulation is the process of binding data (attributes) and methods into single unit(class) and hiding the internal implementation details from the outside world.
> - It helps in achieving data abstraction, which is the process of providing only the essential features of an object to the outside world and hiding the internal implementation details.

6.7.1. Encapsulation in Python:

In python, encapsulation can be achieved using different levels of attribute visibility: ***public, private, and protected*** facilitated through naming conventions rather than specialized keywords as utilized in other languages such as Java or C++.

1. ***Public and private Attributes:***
   - *Public attributes:* Declared using **self.attribute_name,** making them accessible within and outside the class.
   - *Private attributes:* Defined with double underscores(**__attribute _name**), restricting access from outside the class.
2. ***Public and private methods:***
   - ***Public methos:*** Defined with self as the first parameter(e.g: def my_method(self):)
   - ***Private methods:*** prefixed with double underscores (e.g: def __my_method(self):) , making them inaccessible outside the class.
3. ***Protected attributes / methods:***
   - Indicated by a single underscore prefix, _, these are intended for internal use within classes and sub-classes, not accessible from client code.

Example:

```
class employee:
    def __init__(self, name, salary):
        self.__name = name      # private attribute
        self.__salary = salary  # private attribute

    def get_name(self):         # public method
        return self.__name
```

```
        def get_salary(self):       # public method
            return self.__salary
    # Creating an instance of the class
    emp = employee("Raja", 70000)
    # Accessing private attributes using public methods
    print("Employee Name:", emp.get_name())
    print("Employee Salary:", emp.get_salary())
    print(emp.__name)
```

Output:

```
Employee Name: Raja
Employee Salary: 70000
Traceback (most recent call last):
  File "C:/Users/nishanthi michael/Desktop/pyprg/sysprg.py", line 19, in <module>
    print(emp.__name)
AttributeError: 'employee' object has no attribute '__name'
```

The program defines a class named employee that stores information about an employee's name and salary. It also demonstrates how to protect data using private attributes and provide controlled access using methods.

The __init__ method is called automatically when a new object is created. self.__name and self.__salary are private attributes, indicated by the double underscores (__). This means they cannot be accessed directly from outside the class.

Since __name is private, trying to access it directly will cause an AttributeError.

For practice:

1. Write a program to print the employee details of a company by using inner class method.

Solution:

```
class Company:
    def __init__(self, name):
        self.name = name
        self.employees = []
    class Employee:
        def __init__(self, name, role):
            self.name = name
            self.role = role
        def display(self):
            return f"{self.name} is a {self.role}"
def add_employee(self, name, role):
    employee = self.Employee(name, role)
    self.employees.append(employee)
def display_employees(self):
    for employee in self.employees:
        print(employee.display())
# Usage
company = Company("TechCorp")
```

```
company.add_employee("Alice", "Manager")
company.add_employee("Bob", "Developer")
company.display_employees()
```

Output:

```
Alice is a Manager
Bob is a Developer
```

2. write a python program to display the result of addition and multiplication of two attributes using static method.

Solution:

```
class mathcal:
    @staticmethod
    def add(x,y):
      return x+y
    @staticmethod
    def multiple(x,y):
      return x*y
a,b= 4,71
s=mathcal.add(a,b)
print(f"addition of {a} and {b} is : {s}")
c,d=4,14
mul=mathcal.multiple(c,d)
print(f"the multiplication of {c} and {d} is: {mul}")
```

Output:

```
addition of 4 and 71 is : 75
the multiplication of 4 and 14 is: 56
```

3. write a python program to display the nested data or relationships with other objects by using __init__() method.

Solution:

```
class engine:
  def __init__(self, horsepower):
    self.horsepower = horsepower
class car:
  def __init__(self, make, model, horsepower):
    self.make = make
    self.model = model
    self.engine = engine(horsepower)  # fixed typo here
  def display(self):
    print(f"Car Make: {self.make}")
    print(f"Car Model: {self.model}")
    print(f"Horsepower: {self.engine.horsepower}")
# Creating a car with an engine
mycar = car("Tesla", "Model W", 670)
# Display car details
mycar.display()
```

Output:

```
Car Make: Tesla
 Car Model: Model W
Horsepower: 670
```

4.Write the python code to calculate the geometry values of circle, square, rectangle.

Solution:

```
class geometry:
    def __init__(self):
        self.pi=3.14
    def circle(self,radius):
        area=pow(radius,2)*self.pi
        return area
    def square(self,size):
        area=size*size
        return area
    def rectangle(self,width,height):
        area=width*height
        return area
geo=geometry()
pi=geo.pi
circle=geo.circle(6)
square=geo.square(14)
rec=geo.rectangle(4,5)
print(f"pi= {pi}")
print(f"Areaof a circle:{circle}")
print(f"Area of a square:{square}")
print(f"Area of a rectangle:{rec}")
```

Output:

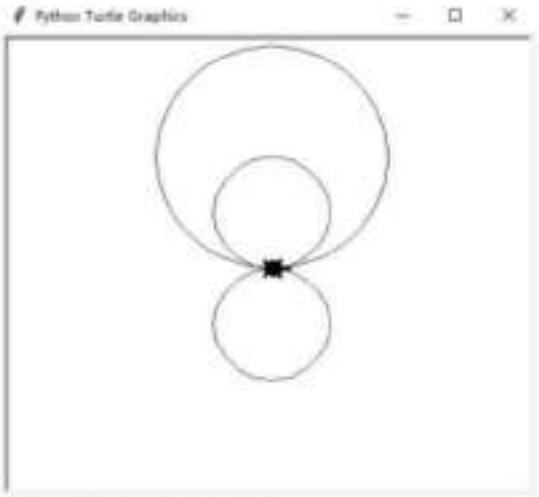
```
pi= 3.14
 Areaof a circle:113.04
 Area of a square:196
 Area of a rectangle:20
```

6.8. Turtle Graphics:

Python also include a basic graphics package that can be used to create simple drawings . The python Turtle module is a powerful tool for drawing geometric shapes, animations, and patterns moving the turtle (cursor) in different directions on the screen. The turtle follows a set of programmed instructions, allowing users to create intricate designs. This module is per-packaged in python and can be imported using the import turtle statement.

Turtle graphics are widely used in educational environments to introduce programming concepts in a fun and interactive way. It helps beginners understand loops, functions, and event-driven programming by visually representing code execution.

Fig of python turtle graphics cursor



6.8.1. Features of Python Turtle:

→ ***Cursor as a Drawing tool:*** The *turtle* act as a pen or cursor controlled by user-defined commands. It draws shapes or patterns as it moves(Forward, backward, left, and right with a specific length) on the screen.

→ ***Customizable Appearance:*** User can change the turtle's color, shape(predefined shapes arrow, classic, square, triangle, and circle), and size.

→ ***Drawing Geometric Shapes:*** By combining movement commands and loops, users can create polygons, fractals, spirals, and stars, among other complex patterns.

→ ***Animation Capabilities:*** Animations can be created by adjusting the speed, timing, and using loops to control the turtle's motion.

Turtle Origins:

The Turtle Graphics package , which dates back to 1996, was originally designed to help teach programming to children. The package creates a turtle, which is mechanical object or cursor, that you control by issuing commands.

6.9. Basic Turtle Methods:

i. ***import turtle or from turtle import *:*** To create a drawing , first import the turtle graphics package into your program.

ii. ***Heading:*** The direction the turtle is facing, measured as an angle in degrees (0 degrees is East, 90 degrees is North, 180 degrees is West, and 270 degrees is South).

iii. ***turtle.pendown:*** The turtle carries a pen that can be either up or down. When the pen is down, the turtle draws on the screen as it moves. Initially, the pen is in the up position.

iv. ***turtle.forward(100):*** The turtle starts in the center of a graphics window and faces towards the east. The turtle can be move forward or backward relative to its current position. We move the turtle forward 100 screen pixels.

v. ***turtle.right(90):*** The turtle moves in the direction it currently faces. To change its direction , you turn the turtle either left or right and specify the angle by which it should turn. The angle is given in degrees.

vi. ***input( ):*** After drawing the scene, the program has to pause and wait for the user to terminate or end the program by using *input ( ) function.*
For e.g: response=input("press ENTER to quit")

vii. ***win.wait( ):*** If you fail for user input, the program terminates and you will not see the graphics window or the image drawn by the turtle.

viii. ***penup() :*** raises the turtle's pen so that it can't draw while it moves.

ix. ***setposition(x, y) and setpos(x,y):***sets the position of the turtle to the specified x and y coordinates.

x. ***setheading(angle) :***sets the heading angle of the turtle to the specified degree.

xi. ***dot(size, color)*** :draws a dot of the specified size and color at the turtle's current position.

xii. ***circle(radius, extent):***draws a circle with the radius and extent you specify (the percentage of the circle to be drawn).

xiii. ***begin_fill():***begins to fill the space that the turtle's movements have enclosed.

xiv. ***end_fill():***stops occupying the space that the turtle's movements have enclosed.

Example 1: ***# turtle forward***

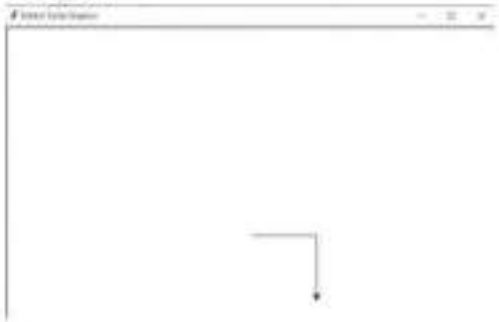
```
import turtle
turtle.pendown()
turtle.forward(100)
```

Output:

Example 2: ***# turtle right side forward***

```
import turtle
turtle.pendown()
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
```

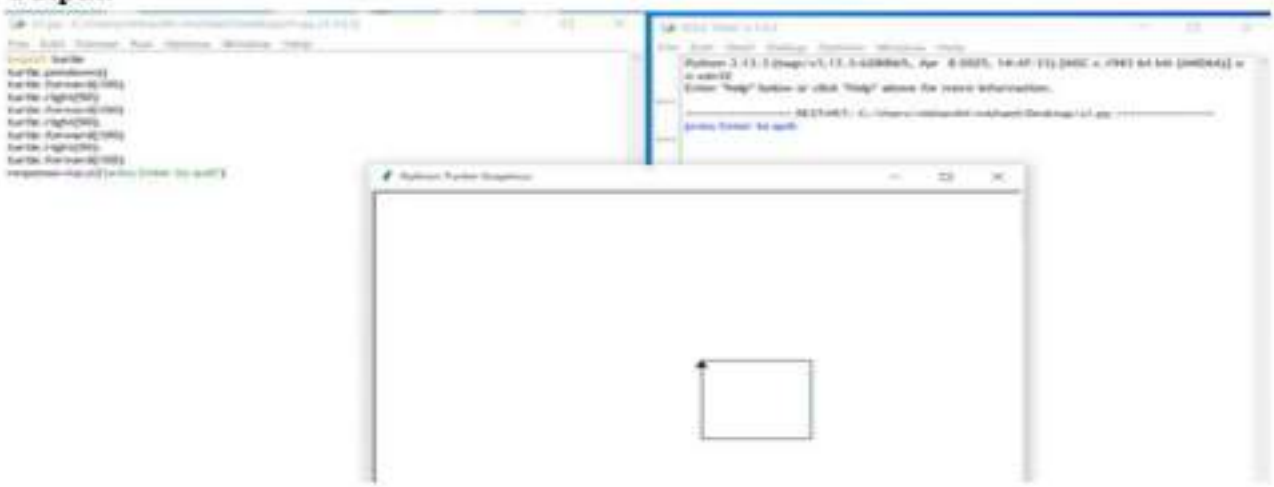
Output:



Example 3: ***#draw a square by using right and forward command***

```
import turtle
turtle.pendown()
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
turtle.right(90)
turtle.forward(100)
response=input("press Enter to quit")
```

Output:



Example 4 : Program to draw a circle using Turtle setpos(angle) method:

```
import math
import turtle
#draw the circle using turtle
def drawcircleTurtle(x,y,z):
    #move to the start of circle
    turtle.up()
    turtle.setpos(x + r, y)
    turtle.down()
    for i in range(0,365,5):
        a=math.radians(i)
        turtle.setpos(x + r*math.cos(a),y + r*math.sin(a))
drawcircleTurtle(100,100,50)
turtle.mainloop()
```

Output:

This program uses the Turtle graphics module along with the Math library to draw a circle by calculating points along its circumference using trigonometric functions, and plotting them step-by-step with the turtle pen.

- Lifts the pen by using up( ) function and moves the turtle to the starting point of the circle: **rightmost point** on the circle's edge.

- Then it puts the pen down by down( ) function to start drawing.
- The for loop runs from 0 to 360 degrees in steps of 5.
- Converts angle i to radians (because math.cos() and math.sin() need radians).
- turtle.mainloop():Keeps the Turtle graphics window open until the user closes it.

Note:

→ clone( ): Creates a duplicate instance of the turtle at the current position.

→ home(): Moves the turtle to the origin(0,0) and resets the heading to 0 degrees.

→ undo(): Removes the last action performed by the turtle, such as drawn line.

Example: Python turtle program to draw different shapes by using turtle method.

```
import turtle
window1=turtle.Screen()
window1.bgcolor("black")
#create a star shape
star_t=turtle.Turtle()
star_t.color("red")
star_t.shape("turtle")
#executing loop 5 times for a star
for i in range(5):
    #moving turtle 100 units forward
    star_t.forward(100)
    star_t.right(144)
# create a hexagon
hex_t=turtle.Turtle()
hex_t.shape("square")
hex_t.color("purple")
hex_t.penup()
hex_t.goto(150,200)
hex_t.pendown()
#draw 6 sides of hexagon
for i in range(6):
    hex_t.forward(100)
    hex_t.right(60)
#create an octagon
oct_t=turtle.Turtle()
oct_t.shape("arrow")
oct_t.color("orange")
oct_t.penup()
oct_t.goto(-250,100)
oct_t.pendown()
#draw 8 sides of octagon
for i in range(8):
```

```
    oct_t.forward(100)
    oct_t.right(45)
window1.exitonclick()
```

Output:

- The given Python program uses the turtle module to create an artistic drawing consisting of a star, a hexagon, and an octagon, each drawn using separate turtle objects with different colors and shapes. The drawing window is initialized with a black background using turtle.Screen().
- The first turtle, named star_t, is shaped like a turtle and colored red. It draws a five-pointed star by repeating a forward movement of 100 units and turning right by 144 degrees five times.
- Next, a second turtle called hex_t, with a square shape and purple color, is used to draw a hexagon. The pen is lifted, the turtle is moved to the position (150, 200), and then the hexagon is drawn by repeating a forward movement of 100 units and a right turn of 60 degrees for six sides.
- Finally, an orange-colored turtle named oct_t, shaped like an arrow, is used to draw an octagon. It is moved to the position (-250, 100) and then draws eight equal sides by turning 45 degrees after each 100-unit forward movement.
- The program ends with window1.exitonclick(), which keeps the window open until the user clicks to close it.

Example 5: python turtle program to display the pencolor(color) and color(color):

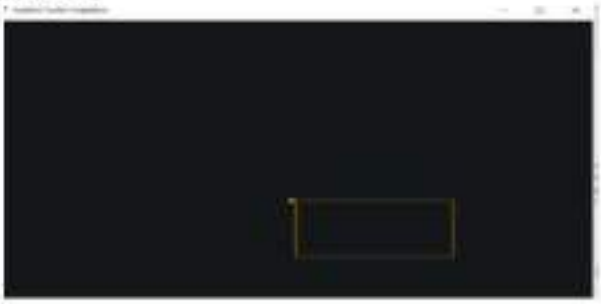
These set the line color for drawing . it accepts:

- Named colors: "red","green","blue".etc.
- Hexadecimal format: #rrggbb", where r,g,b, are values from 0-9 or A-F.

Solution:

```
import turtle
turtle.bgcolor("#14171A")
turtle.color("orange")
turtle.pensize(1)
for width in [200,100,200,100]:
    turtle.forward(width)
    turtle.right(90)
turtle.done()
```

Output:



- This Python program uses the turtle module to draw a rectangle. It sets the background color to dark grey (#14171A) and the drawing color to orange.

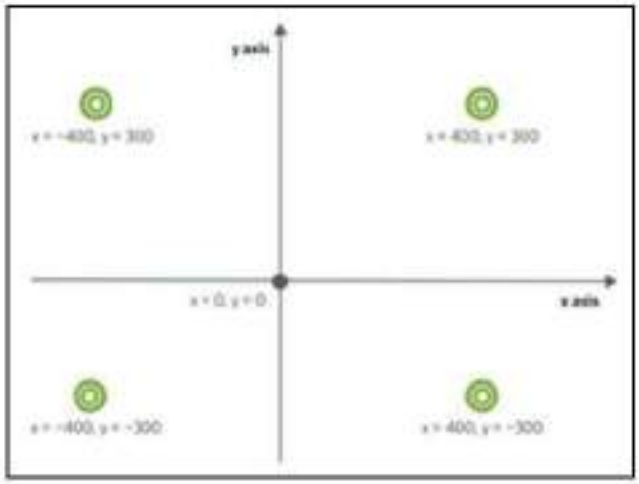
- The pen size is set to 1, making the lines thin. A loop is used to move the turtle forward with side lengths of 200 and 100 alternately, turning right 90 degrees after each side.
- This creates a rectangle shape. Finally, turtle.done() is called to complete and display the drawing.

6.10. Turtle Attributes:

Python's turtle graphics give you a lot of options for controlling the turtle cursor and drawing various shapes and patterns on the screen.

Turtle Attributes

- ***position()*** -provides a tuple of coordinates as the current position of the turtle, represented by its x and y coordinates (in pixels).



- ***heading()*** -returns the current heading angle of the turtle in degrees(0 degrees is East, 90 degrees is North, 180 degrees is West, and 270 degrees is South)..
- ***color()*** -returns a tuple of RGB values representing the turtle's current color.
- ***pensize()*** -returns the current pen size of the turtle, a Boolean attribute indicating whether the turtle's pen is currently "down" (drawing) or "up" (not drawing)..
- ***speed()*** -returns the current speed of the turtle.

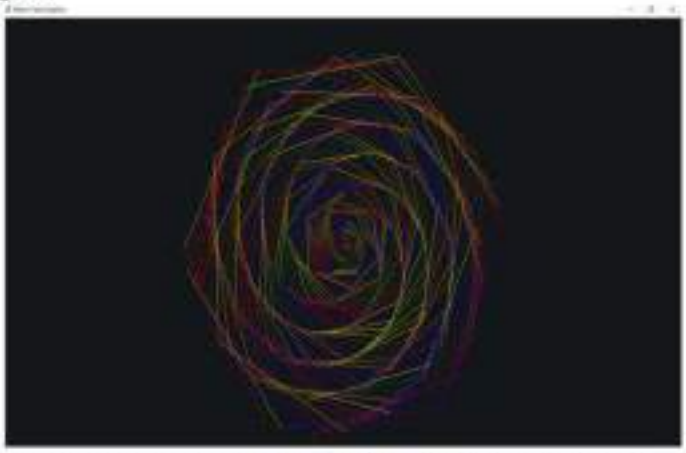
Examples of shapes created using Turtle: i)Colourful spiral flower with random module.

Solution:

```
import turtle
import random
t=turtle.Turtle()
t.speed(0)
turtle.bgcolor("#14171A")
t.pensize(1)
colors=["red", "yellow", "green", "blue","orange","purple"]
def flower(petals,angle_step):
    for i in range(petals):
        t.pencolor(random.choice(colors))
        t.forward(i)
        t.left(angle_step)
```

```
flower(360,59)
turtle.done()
```

Output:



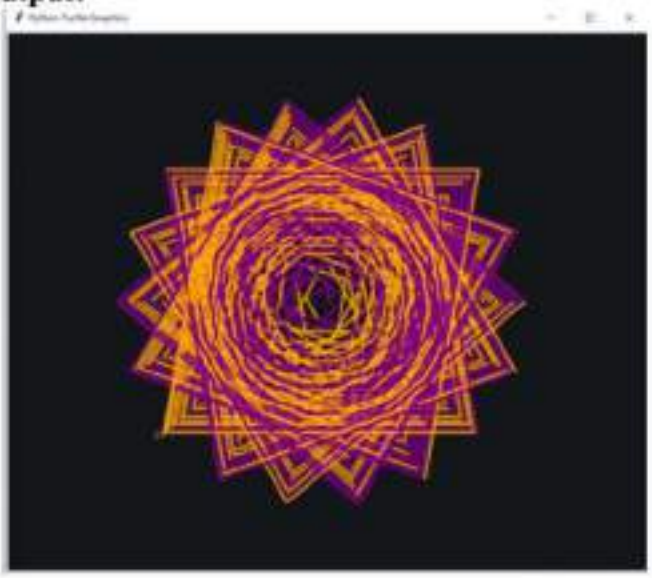
This Python Turtle program creates a colourful spiral by drawing 360 lines, each time moving forward by a growing distance (t.forward(i)) and turning left 59 degrees (t.left(59)). The color of each line is randomly chosen (random.choice(colors)), and the background is set to dark (turtle.bgcolor("#14171A")) for contrast.

ii) Flower with spiral petals by with pencolor method:

Solution:

```
import turtle
import math
import random
t=turtle.Turtle()
t.speed(0)
turtle.bgcolor("#14171A")
t.pensize(1)
colors=["orange","purple"]
def flower(petals,angle_step,size_factor):
    for i in range(petals):
        t.pencolor(random.choice(colors))
        t.forward(math.sqrt(i)*size_factor)
        t.left(angle_step)
flower(360,100,20)
turtle.done()
```

Output:



This Python program uses the turtle module to draw a beautiful spiral flower pattern with a mathematical twist. It begins by importing the turtle, math, and random modules. A turtle object t is created and set to maximum speed for smooth drawing. The background color is set to dark grey (#14171A), and the pen size is set to 1. A list of two colors, orange and purple, is defined for random color selection during drawing.

The function flower(petals, angle_step, size_factor) controls the pattern. It draws the flower by repeating a loop for the number of petals (360 in this case). In each step, the turtle moves forward by a distance that grows with the square root of the current step (math.sqrt(i) * size_factor), making the spiral gradually expand.

The turtle also turns left by a fixed angle (100 degrees), creating a rotating effect. The pen color changes randomly between orange and purple in each step. Finally, turtle.done() displays the completed spiral flower drawing and keeps the window open.

iii) Draw a square with fillcolor(color),begin_fill(color) and end_fill(color) method:

The fillcolor defines the fill color for closed shapes.

begin_fill():Marks the start of filling.

End_fill(): Completes and fills the drawn shape.

Color can be specified in two formats:

- Named colors: "red","green","blue".etc.
- Hexadecimal format: #rrggbb", where r,g,b, are values from 0-9 or A-F.

Solution:

```
import turtle
tut=turtle.Turtle()
turtle.bgcolor("#14171A")
tut.fillcolor("#209010")
tut.begin_fill()
for i in range(4):
    tut.forward(200)
    tut.right(90)
tut.end_fill()
turtle.done()
```

Output:

This Python program uses the turtle module to draw a filled green square on a dark background. It begins by creating a turtle named tut and sets the background color to dark gray (#14171A). The fill color for the shape is set to a shade of green (#209010).

The begin_fill() function marks the start of the shape to be filled. Then, using a loop that runs four times, the turtle moves forward by 200 units and turns right by 90 degrees each time, creating the four sides of a square.

After completing the square, end_fill() is called to fill it with the chosen green color. Finally, turtle.done() displays the completed drawing.

iv) Shapes(shape), shapesize(stretch_wid, stretch_len, outline) and getshapes() methods:

- The shape() sets the shape of the turtle cursor.
- The shapesize() modifies the shape's width, length, and outline thickness.

The getshapes() returns a list of all available shapes. The following are available shapes.

Shape	Icon	Shape	Icon
arrow		triangle	
square		turtle	
circle		blank	
classic			

Example : program to draw a "Turtle" using all Turtle Shapes.

Solution:

```
import turtle
shapes=turtle.getshapes()
print(shapes)
turtle.shape("turtle")
turtle.shapesize(stretch_wid=10, stretch_len=20, outline=2)
```

Output:

Above the Python program demonstrates how to list and customize turtle shapes using the turtle module. It starts by calling turtle.getshapes(), which returns a list of all available built-in turtle shapes like "arrow", "turtle", "circle", "square", etc. This list is printed to the console.

Then, the shape of the turtle cursor is set to "turtle" using turtle.shape("turtle"). After that, the turtle's size is customized using turtle.shapesize(), where:

- stretch_wid=10 makes the turtle taller,
- stretch_len=20 makes it longer,
- outline=2 sets the thickness of the shape's border.

This code modifies the appearance of the turtle on the screen, but does not draw anything. To see the turtle, you can add turtle.done() at the end, or move the turtle with turtle.forward() commands.

Note:

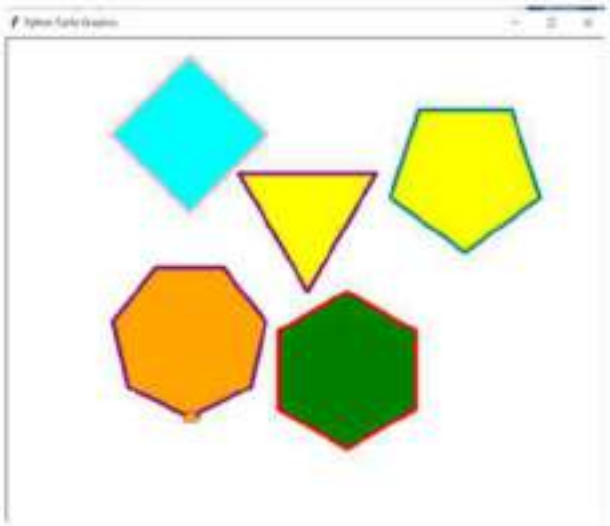
- **turtle.teleport(x,y):** Moves the turtle to a new position without drawing, similar to **penup( ) and pendown( )** but with specified coordinates.
- What will the appearance be without using penup() and pendown()

Example : program to display different shapes by using teleport() method.

Solution:

```
import turtle
turtle.pensize(5)
turtle.shape("turtle")
turtle.begin_fill()
turtle.color("purple", "yellow")
turtle.begin_fill()
turtle.circle(radius=100,steps=3)
turtle.teleport(x=-150, y=100)
turtle.color("pink", "cyan")
turtle.circle(radius=100,steps=4)
turtle.teleport(x=200, y=50)
turtle.color("teal","yellow")
turtle.circle(radius=100, steps=5)
turtle.teleport(x=50, y= -200)
turtle.color("red","green")
turtle.circle(radius=100,steps=6)
turtle.teleport(x=-150,y=-160)
turtle.color("purple", "orange")
turtle.circle(radius=100, steps=7)
turtle.end_fill()
turtle.mainloop()
```

Output:



- This Python turtle program creates multiple colorful polygon shapes on the screen using different colors and positions. It starts by setting the turtle's pen size to 5 and changing its shape to a turtle. The program uses the circle() function with the steps parameter to draw various polygons: a triangle (3 sides), square (4 sides), pentagon (5 sides), hexagon (6 sides), and heptagon (7 sides).
- The turtle is moved to new positions using the teleport() function so the shapes don't overlap. Each polygon is drawn in a different color combination using turtle.color(), where the first color is the outline and the second is the fill.
- However, only the last shape is filled because begin_fill() is not used before each shape. Finally, turtle.mainloop() keeps the window open to display the drawing.

For practice:

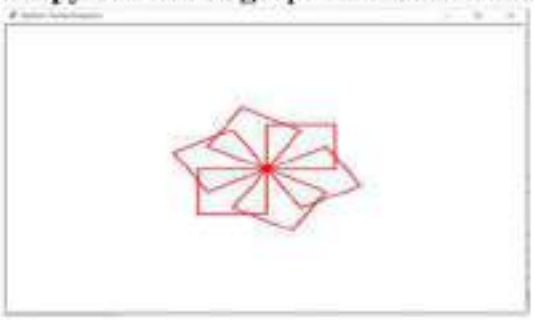
1. Write a colourful square flower by using turtle methods and attributes.

Solution:

```
import turtle
t=turtle.Turtle()
t.speed(0)
turtle.bgcolor("#14171A")
t.pensize(3)
colors=["red","green", "blue", "orange", "purple"]
for i in range(100):
    t.pencolor(colors[i% len(colors)])
    t.forward(i*6)
    t.right(91)
turtle.done()
```

Output:

2. Write a python turtle graphics code for the following output by using nested loop function.



Solution:

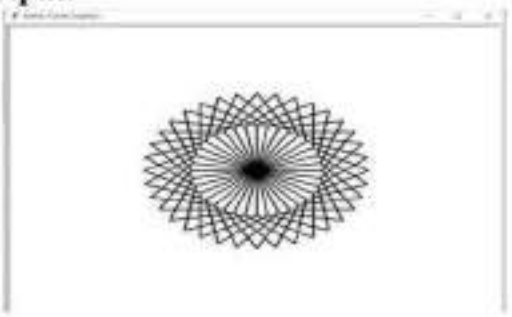
```
import turtle
tut=turtle.Turtle()
tut.pensize(3)
tut.color("red")
for n in range(6):
    for i in range(4):
        tut.forward(100)
        tut.left(90)
    tut.right(60)
```

3. Write a python code to create a simple pattern by using turtle forward command.

Solution:

```
import turtle
a=turtle.Turtle()
my_screen=turtle.Screen()
a.pensize(3)
for _ in range(36):
    for _ in range(2):
        a.forward(100)
        a.right(60)
        a.forward(100)
        a.right(120)
    a.right(10)
turtle.done()
```

Output:



4. Python cod to draw a star as a turtle and filled with yellow color.

Solution:

```
from turtle import *
fillcolor("yellow")
begin_fill()
for t in range(5):
    fd(80)
    rt(120)
    fd(80)
    rt(-48)
end_fill()
```

Output:

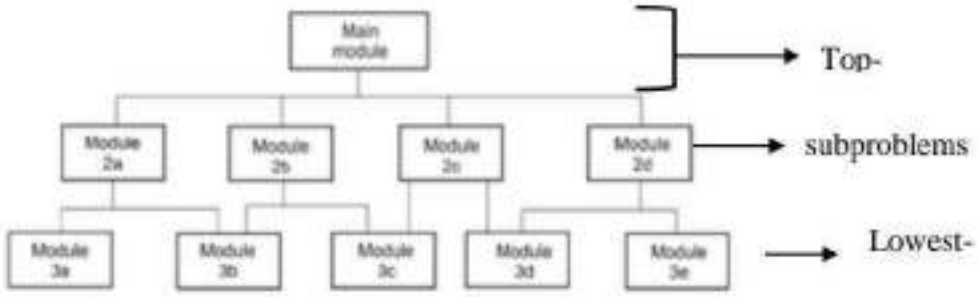
6.11. Modular Design:

To design and implement large and complex problems, two principles are essential: ***decomposition*** and ***abstraction.*** A problem may be too large and complex to solve as single unit.

An important strategy in problem solving is *divide and conquer.* It consist of partitioning a large problem into ***subproblems*** , easier to solve, and easier to manage.

Each subproblem can be solved individually. These ***subproblems*** or ***modules*** need to be well organized in order to achieve the overall solution to the problem.

The ***abstraction*** principle is applied in this technique. The **top-level** module includes the design at the highest level of abstraction, and is the easiest to understand and describe because it includes no details of the design. The **lowest-level** modules contain the all the necessary detail, at the lowest level of abstraction.



Modular design.

6.11.1. Top-down Design:

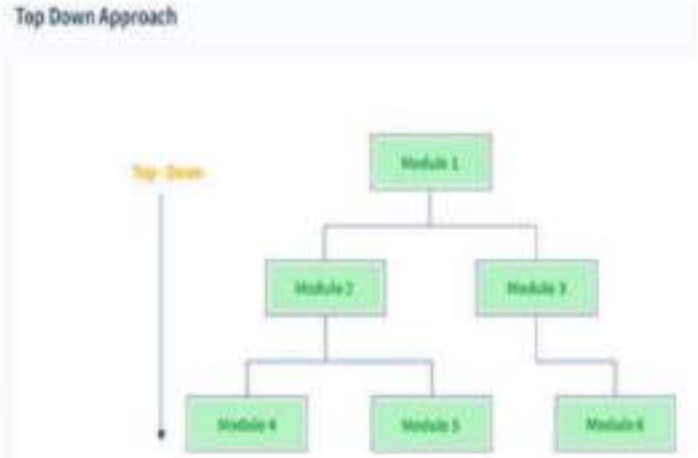
Top-down design is the design methodology whereby high-level functions are defined first, and the lower level implementation details are filled in later. A design can be viewed as a hierarchical tree.

The top level block represents the entire chip. ***The next lower level block*** also represent the entire chip but divided into the major function blocks of the chip. ***Intermediate level blocks*** divided into the functionality into more manageable pieces. ***The bottom level*** contains only gates and macro functions, which are vendor-supplied high level functions.

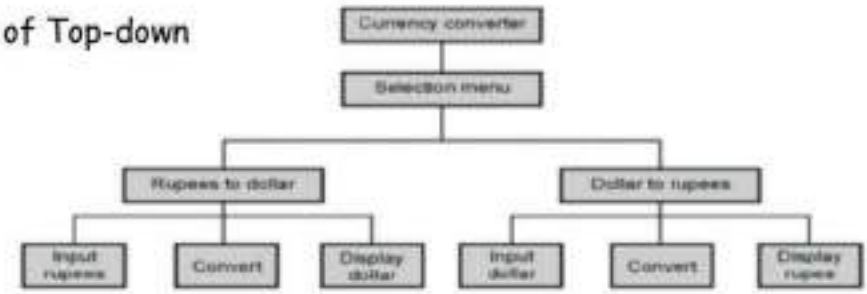
For example:

Consider a currency converter system. We can design this system using top down design approach as follows:

- At the top level we will design the selection menu. *The selection menu is refined into two choices-* ***Rupees to dollar*** *conversion and* ***Dollar to rupees*** conversion subsystem
- Each subsystem is further refined for input module, conversion module and output module.



Example of Top-down



Note: *Top-down design means to repeatedly divide a problem into (smaller) subproblems, until they are directly solvable. A top-down design can be represented by a structure chart.*

6.12. Python Modules:

A module is a file containing python definitions and statements. These files have a ***.py*** extensions and can be imported into other python scripts or modules to make use of their functionality.

A Python module is essentially a way to organize python code into a separate file, allowing for better code organization and reusability. Each module can contain functions, classes, or variables that can be imported int other modules or scripts to be utilized.

6.12.1. structure of a Module:

A python *Module* is typically a ***.py*** file containing python code. It may include function definitions, variable assignments, class definitions, and other executable code. The structure of a module is flexible, allowing developers to organize their code in a way that make sense for the specific application.

For e.g: Create a module :(greet.py)

code:

```
def welcome(name):
    print("Hello, " + name +"!" +"Glad to have you for the party.")
pi=3.14159
class circle:
    def __init__(self,radius):
        self.radius=radius
    def area(self):
        return pi*self.radius **2
```

Importing the entire module : (imp.py)

Code:

```
# importing the entire module
import greet
greet.welcome("Stephen")
print(greet.pi)
circle=greet.circle(5)
print(circle.area())
```

Output: by executing the imp.py we will get this

```
Hello, Stephen! Glad to have you for the party.
3.14159
78.53975
```

In the above program demonstrates how to create and use a custom Python module by defining functions, variables, and a class in greet.py, and then importing and accessing them in imp.py using the import statement.

6.12.2. How to Import a Module?

We can import modules with the ***import*** command. Usually import statements occur at the beginning of the python module.

Python provides different ways to import modules:

- **import module_name:** It imports the whole module.

- **from module_name import function_name:** It imports specific functions or classes.
- **import module_name as alias:** It import a module and assign it a shorter or custom name(alias) for easier access within code. It for frequently used libraries with long names.
- **from module_name import * :** It can import all names(definitions) from a module using *.

6.13.Types of Modules:

Python supports two main types of modules:

i. Built-in modules and
ii. User-defined modules

6.13.1. Built-in modules:

Python's ***built-in modules*** are pre-installed libraries that provide ***various functionalities***, such ***as mathematical operations, file handling, date and time manipulation, system interactions, and web-related tasks.*** These modules are part of python's standard library , so you don't need to install them. The commonly used built-in modules are ***math, os, sys, datetime, and random.***

6.13.1.1. The math module:

This module contains a set of commonly-used mathematical functions, including number-theoretic functions (such as factorial); logarithmic and power functions; trigonometric (and hyperbolic)functions; angular conversion function from the math module are discussed below:

Function	Description
fabs( )	This function returns the absolute value of the x.
ceil( )	This method returns the smallest whole number greater than or equal to x.
floor( )	This method returns the largest whole number less than or equal to x.
log10( )	this log10() function returns the base-10 logarithm of num. If num is negative, a domain error occurs. If num is zero, a range error may occur.
exp( )	This function returns the exponential number E raised to the power of a.
sqrt( )	This function returns the non-negative square root of x. If x is negative , a domain error occurs.
sin( )	It calculates the sin of the angle whose value is entered in radians.
cos( )	It calculates the cosine of the angle whose value is entered in radians.
tan( )	It calculates the tangent of the angle whose value is entered in radians.
degrees( )	It converts angle x from radians to degree.
radians( )	It converts angle x from degrees to radians.

Illustration to import a module in different ways:

import: it is simplest and most common way to use module in our code.

Example:

```
import math
x=math.pi
print("The value of pi is :",x)
```

Output:

```
The value of pi is : 3.141592653589793
```

from import: it is used to get a specific function in the code instead of complete file.

Example:

```
from math import pi
x=pi
print("The value of pi is :",x)
```

Output:

```
The value of pi is : 3.141592653589793
```

import with renaming: it can import a module by remaining the module as our wish. **Example:** import math as m x=m.pi print("The value of pi is :",x) **Output:** The value of pi is : 3.141592653589793	**import all:** it can import all names from a module using *. **Example:** from math import* x=pi print("The value of pi is :",x) **Output:** The value of pi is : 3.141592653589793

Example 1: python program to find distance between two points.

```
import math
x1=int(input("Enter x1:"))
y1=int(input("Enter y1:"))
x2=int(input("Enter x2:"))
y2=int(input("Enter y2:"))
distance=math.sqrt((x2-x1)**2)+((y2-y1)**2)
print (distance)
```

Output:

```
Enter x1:5
Enter y1:4
Enter x2:8
Enter y2:12
67.0
```

Example 2:program to find the mathematical calculations using math module.

```
import math
print(math.sqrt(25))
print("2 raised to the power 3:", math.pow(2, 3))
# Factorial
print("Factorial of 5:", math.factorial(5))
# GCD (Greatest Common Divisor)
print("GCD of 24 and 36:", math.gcd(24, 36))
print("\n Trigonometric functions:")
# Radians and Degrees
angle_deg = 90
angle_rad = math.radians(angle_deg)
print("90 degrees in radians:", angle_rad)
print("sin(90 degrees):", math.sin(angle_rad))
# Inverse trigonometric function
print("arcsin(1):", math.degrees(math.asin(1))) # Converts result back to degrees
print("\n Rounding functions:")
# Floor and Ceiling
print("Floor of 3.7:", math.floor(3.7))
print("Ceiling of 3.7:", math.ceil(3.7))
# Logarithms
```

```
print("\n Logarithmic functions:")
print("Natural log of 10:", math.log(10))      # log base e
print("Base-10 log of 1000:", math.log10(1000))
```

Output:

```
5.0
2 raised to the power 3: 8.0
Factorial of 5: 120
GCD of 24 and 36: 12
Trigonometric functions:
90 degrees in radians: 1.5707963267948966
sin(90 degrees): 1.0
arcsin(1): 90.0
Rounding functions:
Floor of 3.7: 3
Ceiling of 3.7: 4
Logarithmic functions:
Natural log of 10: 2.302585092994046
Base-10 log of 1000: 3.0
```

Example 3: Program to print the mathematical function values.

```
import math
print("absolute value  of -123 :",abs(-123))
print("square root of 45 is :",math.sqrt(45))
print("ceiling of  12.2 :",math.ceil(12.2))
print("floor value of 23.5 is :",math.floor(23.5))
print("Maximum value is :",max(23,5,1,67,9,3),"\t","minimum value is    :",(23,5,1,67,9,3))
print("12.5467 rounded to 2 decimal places :",round(12.5467))
print("natural logarithm of 3 is :",math.log(3))
```

Output:

```
absolute value  of -123 : 123
square root of 45 is : 6.708203932499369
ceiling of  12.2 : 13
floor value of 23.5 is : 23
Maximum value is : 67      minimum value is : (23, 5, 1, 67, 9, 3)
12.5467 rounded to 2 decimal places : 13
natural logarithm of 3 is : 1.0986122886681098
```

Example 4:program to display the angle values by using math module.

```
import math
print("sin (90) :",math.sin(90))
print("cos (90) :",math.cos(90))
print("tan (90) :",math.tan(90))
print("asin(1) :",math.asin(1))
print("acos (1) :",math.acos(1))
print("hypotenuse of 3 and 4 :",math.hypot(3,4))
print("degree of 90 :", math.degrees(90))
```

Output:

```
sin (90) : 0.8939966636005579
cos (90) : -0.4480736161291701
tan (90) : -1.995200412208242
asin(1) : 1.5707963267948966
acos (1) : 0.0
hypotenuse of 3 and 4 : 5.0
degree of 90 : 5156.6201
```

6.13.1.2. The Standard library modules (sys and os):

i) os module:

The os module provides a portable way of using operating system-dependent functionality, allowing python scripts to interact with the operating system. It includes functions for file and directory operations, environment variable management, and built-in support for executing shell commands. Some OS methods for File and Directory Management are mentioned below

Function	Description
mkdir	Creates a directory
makedirs	creates a directory along with any necessary parent directories.
rmdir	Removes an empty directory.
removedirs	Removes multiple nested directories.
rename	renames a file or directory.
renames	Rename a file, creating any needed intermediate directories.
replace	replace a file or directory.
remove	Deletes a file.
unlink	Alias for remove(),used to delete a file.
listdir	List files and directories in a specified path.
scandir	Returns an iterator for directory entries.
walk	Generates file names in a directory tree.
getcwd	Returns the current working directory
getppid	Returns the process ID of the current process
getlogin	Retrieves the username of the logged-in user.
uname-result	Retrieves system-related information like OS name and version.
chdir	Changes the current working directory
curdir	Represents the current directory (".")
pardir	Represents the parent directory("..")
pathsep	Separator used for file paths in PATH environment variable.
extsep	Extension Separator.

For e.g:

```
import os
current=os.getcwd()
print("current Directory:", current)
files=os.listdir(current)
print("Files:", files)
```

```
os.mkdir("pyprogram")
```

Output:

```
current Directory: C:\Users\nishanthi michael\Desktop\pyprg
Files: ['bitwise.py', 'ctrl example.py', 'ctrl.py', 'dict.py', 'indent 1.py', 'indent.py', 'lst.py',
'new_floder', 'python.py', 'python_prg', 'str.py', 'str1.py', 'v1.py', 'variable 1.py', 'xxxx.py']
```

This code uses the os module to work with the file system: it gets and prints the current directory, lists its contents, creates a new folder named pyprogram, and then deletes that folder.

This code imports the os module to perform basic file system operations:

a. **os.getcwd() – Gets the current working directory and prints it.**
b. **os.listdir(current) – Lists all files and folders in the current directory.**
c. **os.mkdir("python_prg") – Creates a new directory named "python_prg".**
d. **os.rmdir("python_prg") – Removes (deletes) the "python_prg" directory.**

Example 2:

```
import os
mk=os.makedirs("nisha\pyprogram")
print(mk)
lstdir=os.listdir()
print(lstdir)
#Rename
os.rename("movie ","Tv")
name=os.listdir()
print(name)
#remove
os.remove("pp1.py")
rmve1=os.listdir()
print(rmve1)
#replace
a=os.replace("pycoding","py")
print(a)
#scandir
files=os.scandir()
for items in files:
  print(items)
#walk
for dirs in os.walk("media"):
        print(dirs)
```

Output:

```
  None
['check.py', 'media', 'melody', 'movie', 'nisha', 'pp1.py', 'program', 'pycoding', 'songs', 'testing']
['check.py', 'media', 'melody', 'nisha', 'pp1.py', 'program', 'pycoding', 'songs', 'testing', 'Tv']
 ['check.py', 'media', 'melody', 'nisha', 'program', 'py', 'songs', 'testing', 'Tv']
DirEntry 'check.py'>
<DirEntry 'media'>
<DirEntry 'melody'>
```

```
<DirEntry 'nisha'>
<DirEntry 'program'>
<DirEntry 'py'>
<DirEntry 'songs'>
<DirEntry 'testing'>
<DirEntry 'Tv'>
('media', ['music'], [])
('media\\music', ['melody'], [])
('media\\music\\melody', [], [])
```

ii)SYS Module:

The sys module provides access to some variables used or maintained by the python interpreter, as well as functions that interact with the execution environment.

Some of sys methods system and environment information are given below:

Function	Description
argv	Retrieves command-line arguments passed to the script.
executable	returns the path of the python interpreter binary.
platform	Returns the name of the operating system.
getwindowsversion	Returns version information about windows OS (windows only).
version	Returns python version
version_info	Provides version details as a tuple
orig_argv	Retrieves the original command-line arguments.
meta_path	list meta path finder used for importing modules.
path	list directories where python looks for modules.
implementation	provides detail about python implementation (e.g: CPython, PyPy).
modules	Displays all currently loaded modules.
getsizeof()	Returns the memory size of an object in bytes.
getrefcount()	Returns the reference count of an object.
gettrace()	Returns the trace function set for debugging.
maxsize	Retrieves the maximum size of a python object.
stdin/ stdout	Standard input stream / Standard output stream.
displayhook	controls how output is displayed in interactive mode.
exit()	Terminates the program with an optional exit status.
getfilesystemencoding()	Returns the encoding used for file system operations.
byteorder()	indicates the byte order of the system.

For e.g:

```
import sys
print("Arguments passed:", sys.argv)
print("Python Version :",sys.version)
sys. exit(0)
```

Output:

```
Arguments passed: ['C:/Users/nishanthi michael/Desktop/pyprg/dict.py']
Python Version : 3.13.2 (tags/v3.13.2:4f8bb39, Feb  4 2025, 15:23:48) [MSC v.1942 64 bit
(AMD64)]
```

This code prints the list of command-line arguments, displays the current Python version, and then exits the program with a success status.

From the program

- **import sys:** Imports the sys module, which provides access to system-specific parameters and functions.
- **sys.argv:** Returns a list of command-line arguments passed to the script. sys.argv[0] is always the script name.
- **sys.version:** Prints the version of Python currently being used.
- **sys. exit(0):** Exits the script with exit status 0, which usually means "no error".

Example 2:

```
import sys
w=sys.path
print(w)
x=sys.executable
print(x)
```

Output:

['C:/pyprg numpy,etc', 'C:\\python folder\\Lib\\idlelib', 'C:\\python folder\\python313.zip', 'C:\\python floder\\DLLs', 'C:\\python folder\\Lib', 'C:\\python folder', 'C:\\Users\\nishanthi michael\\AppData\\Roaming\\Python\\Python313\\site-packages', 'C:\\python folder\\Lib\\site-packages']

C:\python floder\pythonw.exe

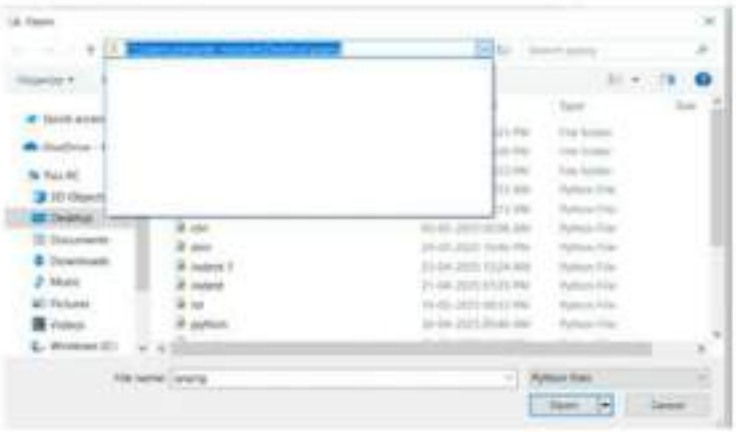
Example 3:

```
import sys
def add():
    x=sys.argv[1]      #first argument
    y=sys.argv[2]      #second argument
  #converting argument to float
    sum=float(x)+float(y)
    return sum
#calling the function
res=add( )
print(res)
```

To run the above code follow the steps given below:

Step 1: Save your code in a file named, ***for example: sysprg.py***

Step 2: Copy the floder path from the saved location.



Step 3: open the command prompt and navigate the folder where your file is saved. Use the cd command.

For e.g: cd C:\Users\nishanthi michael\Desktop\pyprg

Step 4: Run the script and provide **two numbers** as arguments then press enter.

For e.g: python sysprg.py 12 23

Step 5: Tou will see the expected output like 35.0

6.13.1.2.1. Popular Third-Party Libraries:

In addition to the standard library, python's ecosystem includes a vast array of third-party libraries, often accessible via the python packages index(PyPI).These libraries are pivotal in expanding python's functionalities. The third-party libraries are

- NumPy
- Pandas
- Matplotlib and seaborn
- Flask
- Scikit-learn

*Note: To test whether the **pip packages** was installed correctly in Python, you simply try to **import it** in your Python environment (terminal, IDE, or Jupyter Notebook).*

a)Use the import Statement

step 1: *After installing a package (e.g., pandas, numpy), open a Python environment and type the following code into script mode.*

step 2: *Open the command prompt*

python code:

```
import pandas
import numpy
import matplotlib
import seaborn
import flask
print("All packages imported successfully!")
```

step 3: Save the file with .py extension.

for e.g: lib.py

*If you see **no errors**, the packages are installed correctly, then you'll see the message of print statement ("All packages imported successfully") in python shell.*

***Tip:** If you get an error like "ModuleNotFoundError", it means the package is not installed or you're using the wrong Python interpreter.*

To fix this problem use pip install library_name (e.g: pip install numpy) in the terminal or command prompt to install it. Always check that you're using the correct Python interpreter in your IDE.

i) NumPy:

NumPy is a foundational library for numerical computing, conferring support for large multi-dimensional arrays and matrices, along with a collection of high-level mathematical function to operate on these arrays. NumPy's efficiency in computations make it an essential tool for scientific and analytics based project.

Example:

```
import numpy as np
array=np.array([1,2,3,4,5])
print("Mean :",np.mean(array))
print("sum :",np.sum(array))
#reshaping array
matrix=array.reshape((1,5))
print("matrix reshape:", matrix)
```

Output:

```
Mean : 3.0
sum : 15
matrix reshape: [[1 2 3 4 5]]
```

This program demonstrates basic operations using the NumPy library in Python. The numpy module is imported as np, and an array containing the numbers 1 to 5 is created. Using NumPy functions:

a) np.mean(array) calculates the mean (average) of the array.
b) np.sum(array) calculates the sum of all elements.
c) The reshape((1, 5)) function changes the one-dimensional array into a 2D matrix with 1 row and 5 columns.

This example shows how NumPy is useful for mathematical operations and reshaping data structures.

ii) pandas:

Pandas is an indispensable library for data manipulation and analysis, providing data structures like Data Frame and series , which are built on top of numpy.

It enables data cleaning , transformation , and visualization , indispensable in data science workflows.

Example:

```
import pandas as pd
data={'Name':['Anbu','bala', 'chandran'],'Age':[23,21,25]}
df=pd.DataFrame(data)
print("DataFrame Head :\n",df.head())
print("descriptive statistics :\n",df.describe())
ages=df['Age']
print("Ages :",ages)
```

Output:

```
DataFeame Head :
      Name  Age
0     Anbu   23
1     bala   21
2  chandran   25
descriptive statistics :
       Age
count   3.0
mean   23.0
std     2.0
min    21.0
25%    22.0
50%    23.0
75%    24.0
max    25.0
Ages : 0   23
1   21
2   25
Name: Age, dtype: int64
```

This code demonstrates how to use the pandas library to organize and analyze data. A dictionary is created with names and corresponding ages. This data is converted into a DataFrame, which looks like a table.

The head() function is used to show the first few rows, and describe() gives summary statistics such as average age and count. The Age column is then selected and printed separately.

iii) Matplotlib and Seaborn:

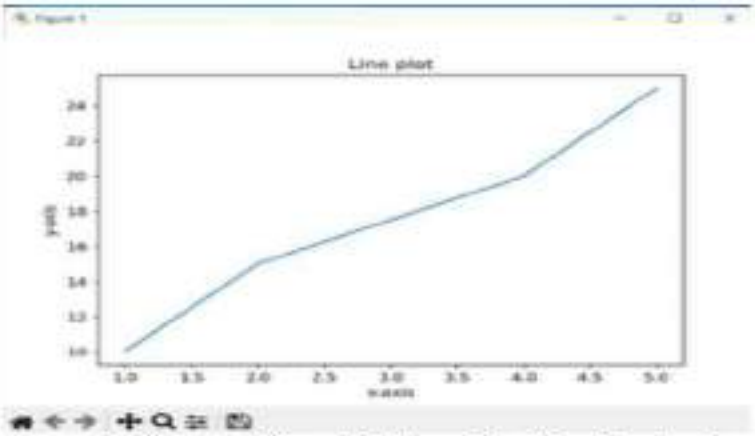
Matplotlib is a multi-platform data visualization library built on numpy arrays, Extremely useful for creating static, animated, and interactive visualization . It supports generating plots like ***line, bar, histogram, and scatter plots.***

Example 1:

```
import matplotlib.pyplot as plt
x=[1,2,4,5]
y=[10,15,20,25]
plt.plot(x,y)
plt.title("Line plot")
```

```
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.show()
```

Output:

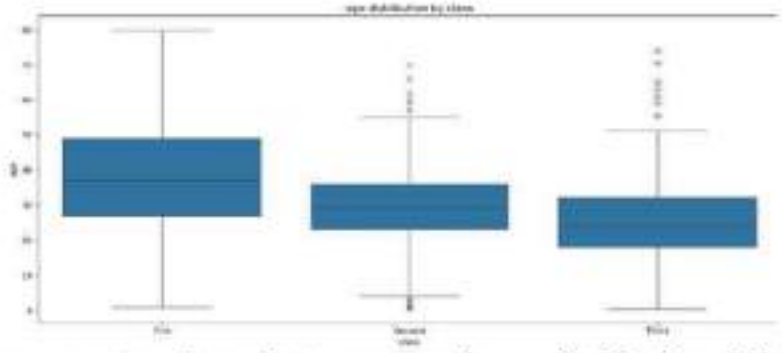


Seaborn builds on matplotlib, providing a high level interface for drawing attractive and informative statistical graphics.

Example 2:

```
import seaborn as sns
import matplotlib.pyplot as plt
titanic=sns.load_dataset("titanic")
sns.boxplot(x="class", y="age", data=titanic)
plt.title("age distribution by class")
 plt.show()
```

Output:



This program creates a box plot to compare the age distribution of Titanic passengers by class. It uses the seaborn library for plotting and loads the built-in Titanic dataset. The boxplot() function shows how the ages are spread across each class (First, Second, Third). The chart title is set using plt.title(), and plt.show() is used to display the plot.

iv)Flask:

Flask is a light weight web framework written in python that assists developers in prototyping web applications quickly and effectively. Due to its simplicity and flexibility, Flask is well-suited for small to medium-scale applications.

Example:

```
from flask import Flask
app=Flask(__name__)
```

```
@app.route("/")
def home():
    return "hello , babe"
if __name__=="__main__":
    app.run(debug=True)
```

Output:

```
 * Serving Flask app 'pp1'
 * Debug mode: on
[31m[1mWARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.[0m
 * Running on http://127.0.0.1:5000
[33mPress CTRL+C to quit[0m
 * Restarting with stat
```

When you run a Flask app, this message shows that the program is working. It says the app (pp1) is running in debug mode, which helps developers by showing errors and automatically restarting the server when code changes.

The warning reminds you that this server is only for testing, not for real websites. The app is available at http://127.0.0.1:5000, and you can stop it by pressing Ctrl + C.

6.13.1.3. The Random module:

The random module is one of python's most widely used libraries, particularly in areas such as game development, simulation, cryptography, and statistical modelling. Python's built-in *random* module provides functions for generating random numbers in a non-linear or unsequential manner.

Syntax:

```
Import random
```

Some most common random number generator functions in the random modules are listed below in the following table:

Name	Description
random( )	This function returns a floating point random number ranging from 0.0 to 1.0 (only the lower range limit is inclusive).
random.choice(iterable)	Randomly selects a single element from a sequence (list, tuple, string, etc).
random.choices(iterable)	Returns a list of k randomly selected elements from a sequence, with optional weights for probability control.
randint(a,b)	This function returns an integer random number ranging from a to b (both range limits are inclusive).
random.uniform(a,b)	this function returns a floating point random number ranging from a to b.
random.randrange(start,end)	this function is used to return a randomly selected element from range(start,stop,step)
random.shuffle(iterable)	Randomly rearranges the elements of a mutable sequence.

Example 1: Python program to demonstrate the random module operations.

```
import random
print(random.randint(1,10))
```

```
print(random.random())
fruits=["apple","banana","cherry","orange"]
print(random.choice(fruits))
numbers=[1,2,3,4,5]
random.shuffle(numbers)
print(numbers)
```

Output:

```
3
0.641907522608282
banana
[4, 2, 1, 5, 3]
```

This program uses Python's built-in random module to perform different random operations:

- ✓ random.randint(1, 10) gives a random integer between 1 and 10 (including both).
- ✓ random.random() returns a random decimal number between 0 and 1.
- ✓ random.choice(fruits) picks a random item from the fruits list.
- ✓ random.shuffle(numbers) randomly rearranges the items in the numbers list.

Each time you run the program, the output may change because the results are randomly generated.

Example 2: Program for 'The Guess Game'

The Guss Game is an interactive game where the user inputs within a specified range. If the entered number matches the randomly generated number by the program, the user wins. Otherwise, they must continue guessing until they find the correct number. However, the game can be terminated at any time by entering a non-digit character.

Solution:

```
import random
def getrandomnumber():
  value=random.randint(0,10)
  return value
while True:
  getinput=input("Guess a number 0-10:")
  if getinput .isalpha():
    print("Input is not a number \n Game Over")
    break
  else:
    value=getrandomnumber()
    if value==int(getinput):
      print("You've won!\nCongratulations")
      break
    else:
      print("Wrong\nGuess again")
```

Output 1:

```
Guess a number 0-10:3
Wrong
Guess again
Guess a number 0-10:4
```

You've won!
Congratulations

Output 2:

Guess a number 0-10:a
Input is not a number
Game Over

6.13.1.4. Calendar Module:

The calendar module is essential for working with dates and time in a system. It allows you to display the calendar for any given year, past or future. Additionally, it provides various methods for checking leap years, retrieving the number of days in a month, and working with months and weekdays.

Some most common calendar module methods are mentioned below:

Function	Description
calendar.calendar()	The main module used for handling date-related functionalities.
calendar.datetime()	A separate module in python that works with date and time objects.
calendar.day_abbr()	A list of abbreviated weekday names(e.g:['mon','tue','wed'...]).
calendar.day_name()	A list of full weekday names(e.g:{'Monday','Tuesday',..]).
calendar.firstweekday()	sets the first day of the week(e.g: setfirstweekday(calendar.Sunday)).
calendar.format(formatstring)	used for formatting calendar output.
calendar.formatstring	A string pattern used for formatting calendar output.
calendar.isleap(year)	checks if a given year is leap year(returns True or False).
calendar.leapdays(y1,y2)	Runs the number of leap years between two years (exclusive).
calendar.main()	Runs a demo of the calendar module when executed as a script.
calendar.month(year,month)	Returns a string representation of a month's calendar.
calendar.monthcalendar(yr,m)	Returns a list of weeks for the specified month. Each week is a list of seven integers, where 0 represents days outside the month.
calendar.prcal(yr,w,l,c)	Prints a formatted calendar for full year.
calendar.timegm(tuple)	converts a time tuple (like one returned by time.gmtime()) int a unix timestamp(seconds since epoch).

Example 1: Python program to display the calendar of 2025

```
import calendar
cal=calendar.calendar(theyear=2025)
print(cal,end='|')
```

Output:

```
                                  2025
      January                     February                    March
Mo Tu We Th Fr Sa Su        Mo Tu We Th Fr Sa Su        Mo Tu We Th Fr Sa Su
          1  2  3  4  5                        1  2                        1  2
 6  7  8  9 10 11 12         3  4  5  6  7  8  9         3  4  5  6  7  8  9
13 14 15 16 17 18 19        10 11 12 13 14 15 16        10 11 12 13 14 15 16
20 21 22 23 24 25 26        17 18 19 20 21 22 23        17 18 19 20 21 22 23
27 28 29 30 31              24 25 26 27 28              24 25 26 27 28 29 30
                                                        31
```

Example 2: Python program to check the given era is a leap year or not.

```
import calendar
a=calendar.isleap(2024)
b=calendar.isleap(2025)
c=calendar.month(2026,1)
d=calendar.leapdays(2014,2026)
print(a)
print(b)
print("==========================")
print(c)
print("==========================")
print("The total number of  leap days from 2014 to 2026 is: ",d)
```

Output:

```
True
False
==========================
   January 2026
Mo Tu We Th Fr Sa Su
       1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
==========================
The total number of leap days from 2014 to 2026 is:  3
```

6.13.1.5. Datetime Module:

The datetime module in python is used to work with dates and times based on the system's current settings. It provides several methods for handling months, years, time zones , and formatting date-time values.

Some datetime modules are mention below:

Function	Description
Astimezone	Converts the given datetime object to the specified time zone(tz). It is useful for handling time zones when working with global applications.
combine(date, time)	Combines a date object and a time object into a single datetime object.
Ctime	Returns a string representation of the date-time in a human-readable format.
Date	Extracts and returns the date portion from datetime object.
Day	Returns the day of the month as an integer(1-31).
Dst	Returns the daylight saving time (DST) offset for time zones that observe DST transitions.
Now	Returns the current local date-time. If a time zone (tz) is provided, it returns the current date-time in that time-zone.
Fold	Used when dealing with ambiguous times in time zone that observe DST transitions.
fromisocalendar(yr,wk,d)	Converts an ISO calendar (year, week number, weekday)into a datetime object.
fromtimestamp	Converts a timestamp (seconds since epoch) into a datetime object.
Isoweekday	Returns the weekday as an integer(1=Monday, 7=Sunday).
today()	Returns the current local date.
tzinfo()	Returns information about the time zone of the datetime object.
tzname()	Returns information about the time zone of the datetime object.

Example: Python program to print the system date and time by using datetime module.

```
from datetime import datetime
x=datetime.now()
print(x)
y=datetime.now().day
print(y)
z=datetime.now().microsecond
print(z)
day=datetime.now().today()
print(day)
```

Output:

```
2025-05-27 23:15:45.271147
27
327755
2025-05-27 23:15:45.339083
```

6.13.1.5.1. Datetime String Format in Python

The datetime module in Python allows you to format date and time into **custom strings** using the .strftime() method. This is useful for displaying dates in readable formats or for saving data consistently.

Common Formatting Codes

Here are some of the most used strftime format codes:

Code	Meaning	Example
%Y	Year (4 digits)	2025
%y	Year (2 digits)	25(for 2025)
%m	Month (01–12)	05
%B	Full month name	March
%b	Abbreviated month name	Mar
%d	Day of the month (01–31)	25
%A	Full weekday name	Sunday
%a	Abbreviated weekday name	Sun
%H	Hour (24-hour clock, 00–23)	13
%I	Hour (12-hour clock, 01–12)	01
%p	AM or PM	PM
%M	Minute	45
%S	Second	30
%Z	Time zone	UTC
%c	Full date and time representation	Sat Mar 3014:30"45 2025

Example:

```
from datetime import datetime
today=datetime.now()
print(today)
a=today.strftime("%d-%m-%y")   #date
print(a)
b=today.strftime("%d %B, %Y")    #date
print(b)
c=today.strftime("%d %B,%Y %H:%M:%S")  #datetime
print(c)
d=today.strftime("%c")  #datetime
print(d)
```

Output:

```
2025-05-27 23:40:15.137805
27-05-25
27 May, 2025
27 May,2025 23:40:15
Tue May 27 23:40:15 2025
```

For Practice:

1. **Write a program to calculate area of the parallelogram , area of the square, area of the circle and the volume of the cone. Import pi constant from math module.**

Solution:

```
from math import pi
h=5.0
b=2.0
r=1.5
area_parallelogram=h*b
print("The area of the parallelogram is: ",area_parallelogram)
area_square=b**2
print("The area of the square is:",area_square)
area_circle=pi*r**2
print("The area of the circle is:",area_circle)
volume_cone=1.0/3*pi*r**2*h
print("The volume of the cone is :",volume_cone)
```

Output:

```
The area of the parallelogram is:  10.0
The area of the square is: 4.0
The area of the circle is: 7.0685834705770345
The volume of the cone is : 11.780972450961723
```

2. Write a python object-oriented program to handle fancy math concepts power, square root by using default value.

Solution:

```
class power(object):
    default_exponent=2
    def __init__(self,exponent=default_exponent):
        self.exponent=exponent
    def of(self,x):
        return x**self.exponent
class realpower(power):     #subclass of power for real numbers
    def of(self,x):
        if isinstance(self.exponent,int) or x>=0:
            return x**self.exponent
        raise valueerror('Fractional powers of negative numbers are imaginary')
print('power:',power)
print('power.default_exponent:',power.default_exponent)
square=power()
root=power(0.5)
print('square:',square)
print('square.of(3)=',square.of(3))
print('root.of(3)=',root.of(3))
print('root.of(-3)',root.of(-3))
real_root=realpower(0.5)
print('real_root.of(3)=',real_root.of(3))
print('real_root.of(-3)=',real_root.of(-3))
print('done')
```

Output:

```
power: <class '__main__.power'>
power.default_exponent: 2
square: <__main__.power object at 0x000001CEE8816900>
square.of(3)= 9
root.of(3)= 1.7320508075688772
root.of(-3) (1.0605752387249068e-16+1.7320508075688772j)
real_root.of(3)= 1.7320508075688772
Traceback (most recent call last):
  File "C:/pyprg numpy,etc/pp1.py", line 22, in <module>
    print('real_root.of(-3)=',real_root.of(-3))
  File "C:/pyprg numpy,etc/pp1.py", line 11, in of
    raise valueerror('Fractional powers of negative numbers are imaginary')
NameError: name 'valueerror' is not defined. Did you mean: 'ValueError'?
```

3. Python code to implement encapsulation in python class.

Solution:

```
class bankaccount:
    def __init__(self,owner,balance):
        #private variables
        self.__owner=owner
        self.__balance=balance
    def deposite(self,amount):
        self.__balance +=amount
        print(f"Deposited {amount} into the account.")
    def withdraw(self,amount):
        if self.__balance>=amount:
            self.__balance -=amount
            print(f"Withdraw {amount} from the account.")
        else:
            print("Insufficient funds.")
    def get_balance(self):
        return self.__balance
    def get_owner(self):
        return self.__balance
    def get_owner(self):
        return self.__owner
account=bankaccount("Gopal",1500)
print(f"Accout Owner:{account.get_owner()}")
print(f"Initial balance:{account.get_balance()}")
account.deposite(500)
account.withdraw(450)
print(f"Final balance: {account.get_balance()}")
```

Output:

```
Account Owner:Gopal
Initial balance:1500
Deposited 500 into the account.
Withdraw 450 from the account.
```

Final balance: 1550

6.14. File Handling:

The file is a basic storage for data or information. It acts as a container to hold the data or information. The files store on secondary storage devices permanently for future reference.

Examples for files are.. document file, database file , source code file, executable file, object code file etc.

Python is used to perform certain operations on files. Every programming language offers provision to use and create files through programs.

Types of data files:

Depending on how data are stored and retrieved, the files are classified into two types.

- ***Text Files:*** It is a file that stores information in ASCII characters. In text files, each line of text is terminated with a special character known as EOL(End of line) character or delimiter character. When this EOL character is read or written, certain internal translation take place.
- ***Binary Files:*** It is a file that contains information in the same format as it is held in memory. In binary files, no delimiters are used for a line and no translations occure here.

6.14.1. Text Files:

A file is a collection of characters or bytes represented using a code format (ASCII or UTF or Unicode).These files are called as text files such as notepad files, word files, programming language source programs, etc. Some files are stored as sequence of bits. These types of files are called as binary files such as image, video, audio, object, executable files.

Methods of Files:

OPEN - open()
CLOSE - close()
READ - read()
WRITE - write()

6.14.2. File Modes:

In python , the basic file modes are 'r'- reading a file, 'w'- writing a file', 'x' – opens a file for exclusive creation. Some of the model in which the file open for various operations such as read , write, append etc. the following table describes each of the file types.

Mode	Description
r	The file is opened in read-only mode. The file pointer is present at the start. If no access mode is defined, the file is opened in this mode by default.
r+	It opens the file for both reading and writing. The file pointer is present at the start of the file.
rb	It converts the binary file to read-only mode. The file pointer is present at the start of the file.
rb+	It opens the file in binary format for reading and writing. The file pointer is present at the start of the file.
w	It only allows you to write into the file. If a file with the same name already exists, It is overwritten; otherwise it is created. The file pointer is present at the start of the file.

w+	It opens the file for both writing and reading. It differs from r+ in that it overwrites the previous file if one exists, while r+ leaves the previously written file alone. If no such file exists, it creates one. The file pointer is present at the start of the file.
wb+	It opens the file in binary format for both writing and reading. The file pointer is present at the start of the file.
a	The file is opened in append mode. If there is one, the file pointer is at the end of the previously written file. If no file with the same name already exists, it creates a new one.
a+	It opens a file for both appending and reading. If a file exists, the file pointer stays at the end of it. If no file with the same name already exists, it creates a new one.
ab	It opens the file in binary format in append mode. The pointer is t the end of the file that was previously written. If no file of the same name remains, it produces a new binary file.
ab+	It opens a binary file for appending and reading. The file pointer is already at the file's end.
x	Open a file for exclusive creation. If the file already exists, the operation fails.

6.15. opening and closing files:

In order to perform various operations on file we need to ***open a file*** in a specific mode and perform the operations. once the operations are done, the opened file must be closed. To perform various tasks such as , modifying, appending data, deleting data, reading, writing etc. we use different built-in functions of python.

To opening a file: use open( ) member function.

Syntax:

<file_objectname>=open(<filename>)

<file_objectname>=open(<filename>,<module>)

For e.g:

Openfile=open("sample.txt")

Here, Openfile is a file object and open( ) is a member function and sample.txt is a name of the file to be opened. By default the file will be opened in read only mode.

In order to open a file in write only mode, use open () member function as ,

openfile=open("sample.txt", "w")

Here, openfile is a file object and open( ) is a member function and sample.txt is a name of the file to be opened and the "w" indicates mode which is write only mode.

Example: Code to demonstrate opening a file "r" mode and reading its content.

Note: The file is expected to contain plain text data.

Code:

```
f=open("sample .txt", "r")
if f:
  print("File opened")
```

Output:

File opened

We passed text file (smple.txt) as the first argument and opened the file in read mode with ***r*** as the second argument in the above example. The f holds the file object, and if the file is successfully opened, the print statement is executed.

Note: We have to store all the text files(for e.g: sample.txt from the text editor) and python program files(for e.g: abc.py) in the same directory.

6.15.1. Closing a File:

We can perform any operations on the file using the file system that is currently open in python; thus, it is best to practice closing the file once all processes have been completed.

After we've completed all of the file's operations, we'll use the ***close( )*** method to close it. When the close( ) method on a file object is called, any unwritten data is deleted.

Syntax:

File_object. close()

Text file (sample.txt)

Note: closing a file will free up the resources that were tied with the file and is done using python close () method.

Example:

```
f=open("sample.txt","r")
content=f.read()
print(content)
f .close()
```

Output:

The above method is not entirely safe. If an exception occurs when you are performing some operations with the file, the code exits without closing the file. The best way to do this is using the ***"with"*** statement. This ensures that the file is closed when the block inside ***with*** is exited.

6.16. Reading a File:

- We can create and write into the text file by from Notepad ⟶ file ⟶ New
- Then click File ⟶ save ⟶ enter the name and save.

Or

To follow the following command to create and open a text file in notepad from command prompt

- Type the command of start notepad file_name.txt, then press enter button .
- When you hit on enter it shows a pop up box whether to open or not?
- Click on "yes" to open else "no" to cancel.

6.16.1. Reading the Entire File: read()

Python provides several methods for reading data from a file. ***The most common methods are read(), readline(), and readlines()***

i) The following code demonstrates opening a file in "r" mode and reading its content of a text file by using "with" statement.

For e.g:

```
with open("sample.txt","r") as file:
    content=file.read()
        print(content)
```

The file sample.txt is opened in read mode, its content is read into a variable, printed, and the file is closed automatically.

Output:

From another perspective:

Using a certain number of characters is another way to read a text.

For e.g: program to opening text file in read mode.

```
f=open("view.txt", "r")
data=f.read(25)
print(data)
f.close()
```

Output:

Text file

The above code will read first twenty five characters(data=f.read(25)) of stored data and return it as a sting.

Points to Remember:

strip( ) method: .strip() removes spaces, tabs, and newline characters from both ends of each line.

→ *when processing text data, it is common to post-process the content for further use. This might include operations such as stripping new line characters, converting data to lowercase, or performing pattern matching to extract specific information.*

→ *For instance , the strip( ) method is often used in combination with line-reading methods to eliminating extraneous whitespace or newline characters.*

→ *Also we can use lstrip( )method to remove whitespaces from the left of the string and rstrip( )method to remove whitespaces from the right of the string.*

6.16.2. Read Line by Line: readline()

The readline() function in python makes it easy to read a file by line by line. The readline() method reads the file's lines starting at the beginning, so if we call it three times, we'll get the first three, we'll get the first line of data from the file each time it is called:

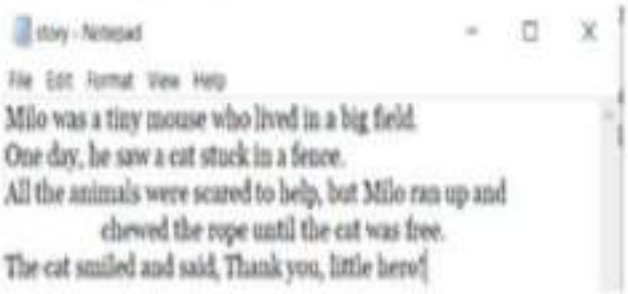
Example:

```
with open("story.txt","r") as f:
   print(f.readline())
```

Output:

Milo was a tiny mouse who lived in a big field.

Text file



6.16.2.1.Read Specific Number of Characters:

If we wanted to get back every line in the file, separated properly. The same function will be used but in different format. The Fileobject.readlines() method is used to accomplish this. It returns a list of lines up to the End of the File(EOF)

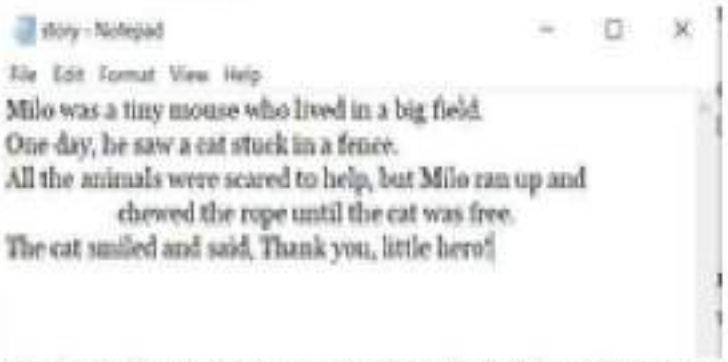
Example: Return the few bytes from the first line

```
f = open("story.txt", "r")
print(f.readline(10))
```

Output:

Milo was a

Text file



Example 1: Python program to print all the lines from a text file.

```
obj1=open("story.txt","r")
# first line
L1=obj1.readline()
#second line
L2=obj1.readline()
# third line
L3=obj1.readline()
#printing all the lines
print(L1)
print(L2)
print(3)
obj1.close()
```

Output:

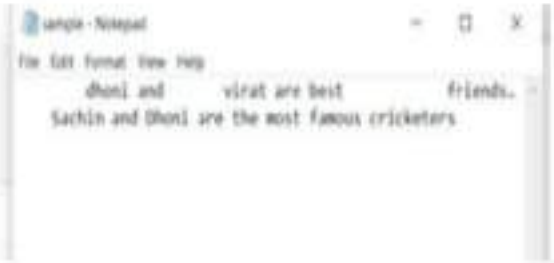
```
Milo was a tiny mouse who lived in a big field.
One day, he saw a cat stuck in a fence.
All the animals were scared to help, but Milo ran up and
```

Example 2: program to display stripping newline characters.

Solution:

```
with open("sample.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Text File



Output:

The above program reads each line from sample.txt, uses .strip() to remove newline and extra spaces, and prints clean output.

6.16.3. Reading file through a loop:

The loop-over method can be used to read all the lines from a file in a more memory accessible and fast manner. The corresponding code is simple and easy to read, which is a benefit of using this method.

Example: Python program to print all the lines from a text document by using loop.

```
with open("story.txt","r") as f:
    for x in f:
        print(x)
```

Output:

```
Milo was a tiny mouse who lived in a big field.
One day, he saw a cat stuck in a fence.
All the animals were scared to help, but Milo ran up and
        chewed the rope until the cat was free.
The cat smiled and said, Thank you, little hero!
```

This Python program reads and prints the contents of a file named story.txt line by line. It uses the with open("story.txt", "r") as f statement to open the file in read mode, ensuring that the file is automatically closed after reading.

Inside the with block, a for loop is used to go through each line of the file. The line is stored in the variable x and printed using print(x). This will display each line from the file on the screen.

However, since each line in a text file already ends with a newline character, using print(x) may result in extra blank lines between outputs. To avoid that, x.strip() can be used to remove unwanted newline characters.

Another method to get back every line from the text file is readlines( ).

Method 1:

```
with open("story.txt","r") as f:
    print(f.readlines())
```

Output:

['Milo was a tiny mouse who lived in a big field. \n', 'One day, he saw a cat stuck in a fence.\n', 'All the animals were scared to help, but Milo ran up and \n', '\t chewed the rope until the cat was free.\n', 'The cat smiled and said, Thank you, little hero!\n', '\n']

The method 1 Python code reads the entire contents of a file named story.txt and prints all the lines at once as a list. It uses the with open("story.txt", "r") as f statement to open the file in read mode, which also ensures the file is properly closed after reading.

Inside the block, f.readlines() reads all the lines from the file and returns them as a list, where each element is a line from the file (including newline characters '\n').

The print() function then displays this list on the screen. This method is useful when you want to work with or view all lines at once rather than reading them one by one.

OR

Method 2:

```
f=open("one.txt","r")
data=f.readlines()
print(data)
f.close()
```

Output:

```
['Hello Python']
```

- The above method 2 Python program reads the contents of a file named one.txt and prints all the lines as a list. It begins by opening the file in **read mode** using f=open("one.txt","r").
- The readlines() function is then used to read all lines from the file and store them in the variable data as a list, where each item in the list is a line from the file, including newline characters.
- The print(data) statement displays the entire list of lines.
- Finally, the file is closed using f.close() to free system resources. This method is useful when you want to work with all lines of a file at once.

Example 2: Python program to read and display the contents of a text file entered by the user.

```
filename=input("Enter file name:")
f=open(filename, "r")
done=0
while not done:
        aline=f.readline()
        if aline != "" :
           print (aline)
        else:
           done=1
f.close( )
```

Output:

```
Enter file name:exe.txt
This is  rainy season.
Take an umbrella and go.
```

6.17. Writing to a File:

When it is necessary to write into a file, the file can be opened in the **"w"** mode. When a file is opened in this mode, any existing in the file is erased. This mode is suitable when the entire content of the file needs to be replaced.

For e.g:

```
with open("sample.txt","w") as file:
   file.write("dhoni and Virat is best friends in cricket.\n")
   file.write("Sachin and Dhoni are the most famous cricketers")
```

Output:

Open the sample.txt file in a text editor to view the updated content.

Existing Text file (sample.txt)

New Text File(sample.txt)

6.17.1. Using Append Mode to create and write:

Using the append mode 'a' as the second argument in the open() function allows us to create a new file if it doesn't exist, or add new content to the end of an existing file without overwriting its current contents.

Example: Write a python Program to open and write "Hello Python" into a file.

Solution:

```
a=open("myfile.txt","a")
a.write("Hello World")
a.close()
```

Output:

If we keep on executing the same program for more than one time then it append the data that many times.

Example: Writing a python program to write the content " Welcome to the new AI World" for the existing file.

Solution:

```
with open("myfile.txt","w") as f:
    f.write("Welcome to the new AI World")
```

Output:

6.17.2. The r+ Method:

This method is enables both reading and writing operations on a file. When using this mode , the file is positional at the beginning of the file, allowing the program to read existing data and modify content as needed. However , caution is necessary because any modifications any overwrites existing content.

For e.g:

```
with open("story.txt","r+") as f:
        current_content=f.read()
        f.seek(0)      #moves the pointer back to the beginning of the file
        f.write("Modified content: Short story\n")
        f.write(current_content)
```

Output:

```
Modified content: Short story
Milo was a tiny mouse who lived in a big field.
One day, he saw a cat stuck in a fence.
All the animals were scared to help, but Milo ran up and
 chewed the rope until the cat was free.
 The cat smiled and said, Thank you, little hero!
```

Note: ***writelines()*** method also used to write multiple lines into a file. It accepts an iterable (list, tuple, etc) of strings.

for e.g:

```
line=['apple', 'orange', 'banana', 'dragon fruit']
with open('test1.txt','w') as f:
        f.writelines('\n'.join(line))
```

Output:

6.18. Significance of file pointer in file handling:

A file pointer is a maker that indicates the current position in a file where the next read or write operation will take place. In other words , the file pointer is a cursor that keeps track of where the next byte or character will be read or written in a file.

The significance of file pointer in file handling are:

i. ***Positioning:*** The file helps to position the cursor in the file so that the next read or write operation takes place at the desire location in the life.

ii. ***Navigation:*** The file pointer allows navigation to different parts of the file. It can be moved to a specific location in the file, or it can be moved a certain number of bytes or characters from its current position.

iii. ***Overwriting and inserting:*** File pointer allows overwriting or inserting data at any position in the file. By moving the file pointer to a specific location, data can be written to that location , overwriting or inserting the existing data.

iv. ***File size:*** The file pointer also helps in determining the size of the file. By subtracting the position of the file pointer from the end of life, we can determine the size of the file.

v. ***Efficient reading and writing:*** The file pointer enables efficient reading and writing of data by allowing the operating system to only read or write the data that is needed.

6.19. tell( ), seek( ) and truncate( ) functions:

1. tell( ): The tell( )method returns the current position of the file pointer, which is the number of bytes from the beginning of the file. It's a simple method to check where the pointer currently points.

2.seek(): This method is used to change the position of the file pointer to a specific location within the file. It takes two arguments:

- ***offset:*** The number of bytes to move the pointer.
- ***whence:*** (Optional) Specifies the reference point for the offset. It can be:
  - 0: Beginning of the file (default).
  - 1: Current position of the file pointer.
  - 2: End of the file.

3.truncate(): This method resizes the file to a specified size. It takes one optional argument:

- size: The desired size of the file in bytes. If not provided, it truncates the file at the current position of the file pointer. If the specified size is less than the current file size, it truncates the file, removing data. If it's larger, it may pad the file with null bytes, depending on the operating system.

Example:

```
file = open("story.txt", "r+")
print("Initial position:", file.tell())
file.seek(3)
print("Position after seek:", file.tell())
file.truncate(10) # Resizes the file to 10 bytes
print("Position after truncate:", file.tell())

file.close()
```

Output:

```
Initial position: 0
Position after seek: 3
Position after truncate: 3
```

6.20. Exception

An Exception is an error that occurs when a program is being executed. Non-programmers understand exceptions as instances that do not follow a general rule.

Even if a sentence or expression's syntax is correct, it can still produce an error when executed. Exception I python are errors that are observed during execution but are not always serious.

When a python code throws an error , an exception object is formed. The program will be forced to end suddenly if the code does not explicitly handle the exception . exceptions are usually ignored by programs , resulting in error message.

For e.g :

```
name='Kanupriya'
age=39
age+name
```

Output:

```
Traceback (most recent call last):
File "C:/Users/nishanthi michael/Desktop/file/getting ip.py", line 3, in <module>
 age+name
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Python exception come in various forms, and the form is written alongside the message: in two cases , they are ***TypeError*** and ***ZeroDivisionError.*** Based on the type of exception , the remaining portion of the error line contains information about what caused the error.

6.20.1. Exception handling in python file:

- Exception handling allow us to handle those problems and either recover or present nicer error message than the stack traces that are shown something breaks and we don't expect it. The two core elements of exception handling are ***try*** and ***except keywords.***
- The ***try*** block of code is what we are going to try to execute. If an error occurs in our ***try*** block, we have an ***except*** statement to handle it. Two other elements that may appear in exception handling blocks are ***else*** and ***finally.***
- The ***else*** keyword is used for code that should run if no ***exception*** is raised, and the ***finally*** keyword is used for code that should be run regardless of errors.

Example 1 :

```
try:
   f=open("aaa.txt")
   print(f.read())
   f.close()
except IOError:
   print("Error occurred opening file")
except:
   print("unknown error occurred")
else:
   print("file contents successfully read")
finally:
   print("Thanks for Playing!")
```

Output from finally

```
Small Steps Every
Day Lead To Big Results.
file contents successfully read
Thanks for Playing!
```

Example 2:

```
try:
   f=open("aa1.txt")
   print(f.read())
   f.close()
except IOError:
   print("Error occured opening file")
except:
   print("unknown error occured")
else:
   print("file contents successfully read")
finally:
   print("Thanks for Playing!")
```

Output from try block:

```
Error occured opening file
Thanks for Playing!
```

From the above programs the first example has defined the file aaa.txt **exists** and is read successfully , so try block runs without error.

And from the second example When the file aa1.txt is not found, the IOError is caught and the message 'Error occurred opening file' is displayed, while the finally block runs regardless of the error, printing 'Thanks for Playing!

For Practice:

1. Write a python program to cleaning up and processing each line in a file.

Solution:

```
with open("thought.txt","r") as f:
    for line in f:
        # Remove leading/ trailing whitespace and convert text to lowercase
        cleaned_lined=line.strip().lower()
        # Further processing could include spliting the line into tokens
        tokens=cleaned_lined.split()
        print(tokens)
```

Output:

2. Program to display the file methods of open(), close(), read() and write.

```
obj=open("test.txt","w")
obj.write("Python Programming\n")
obj.write("It is a programming language \n")
obj.write("It supports file handling\n")
obj.write("Lets dive into python .")
obj.close()
#opening a file in read mode
fobj=open("test.txt","r")
#initially the file pointer is at 0
print("pointer position : ", fobj.tell())
#reading the content of the file
l1=fobj.readline()
print(l1)
l2=fobj.readline()
print(l2)
#changing the file pointer location to 10
fobj.seek(10)
print("pointer position :",fobj.tell())
l3=fobj.readline()
print(l3)
print("pointer position:",fobj.tell())
fobj.seek(20)
print("pointer position:", fobj.tell())
```

Output:

```
pointer position :  0
Python Programming
It is a programming language
pointer position : 10
gramming
pointer position: 20
pointer position: 20
```

3. Write a python program to create and add the following text into a file.
"This is rainy season.
Take an umbrella and go."

Solution:

```
with open ("exe.txt","a") as f:
    f.write("This is  rainy season.\n")
    f.write("Take an umbrella and go.")
```

Output:

4.Write a python program that reads a text file and counts and a is play the counts of words 'and', 'or' ,and 'not' and display them on the screen.

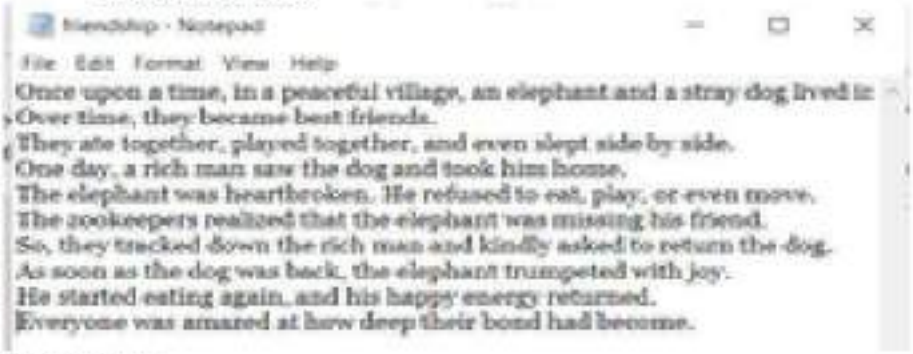
Solution:

```
andcount, orcount,notcount=0,0,0
with open("friendship.txt","r") as f:
    content=f.read().lower()
    words=(content.split())
    for i in words:
        if i=="and":
            andcount +=1
        if i=="or":
            orcount +=1
        if i=="not":
            notcount +=1
    print(f"Number of words : and={andcount}, or={orcount},not={notcount}")
```

Output:

```
Number of words : and=5, or=1,not=0
```

Friendship. Text



6.21. Exercise:

1. What is instantiation?
2. How can you tell if a def statement in python defines a function or class method?
3. What are constructors used for? What special name does python use for them?
4. Hoe do you create module?
5. Define classes and object.
6. What is garbage collection?
7. Explain in detail about destructor and their use in python.
8. How do you create private fields and methods in a python class?
9. Explain the difference between public, private, protected content in python.
10. What is the purpose of creating a turtle object in turtle graphics?
11. How can you lift the pen off the canvas so that the turtle can move without drawing?
12. What is Top-down design? Explain with suitable block diagram and example.
13. How the append method is work in file operation? Explain with proper example.
14. What are the different ways to open and close a file?
15. What are the different read and write methods that are used most frequently in python?
16. Write a program to read a text file and count all the vowels present in the file.
17. What are some common modes used when opening a file in python?
18. What is an exception handling in python?

****

BOOK 7: DATABASE PROGRAMMING

7.1. What is Databases?

An organized collection of data enables users to efficiently store and manage data along with their ability to retrieve it efficiently. A database exists in multiple forms which includes:

7.2. Interface Python with SQL Database:

The Python programming language has powerful features for database programming. Python supports various database like MySQL, Oracle, SyBase, PostgreSQL , etc. Python also supports Data Definition Language(DDL), Data Manipulation Language(DML) and Data Query Statements.

For database programming , the python DB API(Application Programming Interface) is a Widely used module that provides a database application programming interface. Using python structure, DB-API provides standard and support for working with database. The API consists of:

- Bring in the API module
- Obtain database connection
- Issue SQL statements and then store procedures
- Close the connection

7.2.1. Installing Python PyMySQL:

Python provides different ways to connect and access the MySQL database and tables to process data. In this chapter we use PyMySQL to access MySQL server.

PyMySQL and MySQLdb provide the same functionality and they are both database connectors for python. Basically the libraries inside PyMySQL and MySQLdb are enabling python programs to talk to a MySQL server.

To install PyMySQL to access MySQL database, the following requirements are needed:

1. Python:
    - CPython or Python 3.x
    - PyPy - 4.0
2. MySQL Server:
    - MySQL server 4.1 or 8.0
    - MariaDB 5.1 or higher

Before installing first check whether PyMySQL is already installed on your system. To test open command prompt or terminal, then start python shell and type the following code:

```
>>> import PyMySQL
```

if it is not showing any error like **ModuleNotFoundError: No module named 'PyMySQL'**

we can install PyMySQL by using pip. Pip is a package management system used to install and manage software packages written in python.

To install PyMySQL to access MySQL database ,follow these steps:

i. Open the command prompt.
ii. Change the directory into Python installation scripts, i.e., C:\python floder\Scripts>
iii. Type the following:

C:\python3\Scripts> pip install PyMySql

You will see a successfully installed window as shown below

After installing PyMySQL package in the virtual environment, you are ready to work with actual database. Once we have PyMySQL installed , we can create a connection object.

7.3. Connecting to a DATABASE:

Before connecting to a database, following things should be done to assure the right connectivity.

i. Create a database called mydb in MySQL by using following SQL command in SQL Query Tab:
   CREATE DATABASE mydb;
ii. The user id and password used to access mydb database is "user" and "Type your MySQL password" respectively.
iii. Create a table "student" in mydb in MySQL Workbench. The command is .
iv. The table STUDENT has fields ROLLNO, NAME, AGE, COURSE, GRADE.

The following code shows how to connect MySQL database with python.

For e.g 1: program to check the connection between the MySQL and Python.

```
import pymysql
#open database connection
mydb = pymysql.connect(
  host="localhost",
  user="root",
  password="Fero_17Frano_23")
print("✅ Connection successful!")
print(mydb)
```

Output:

```
✅ Connection successful!
<pymysql.connections.Connection object at 0x00000177591FBE00>
```

The ***import pymysql*** line is used to include the module that allows Python to connect with a MySQL database. The ***pymysql.connect()*** function connects Python to the MySQL server using the given host, username, and password. If the connection is successful, it shows a success message and prints the connection object. However, no specific database is selected in this connection.

Example 2: program to check the connection between the Python and MySQL and prints the Database version.

```
import pymysql
mydb = pymysql.connect(
```

```
    host="localhost",
    user="root",
    password="Fero_17Frano_23",
    database="sample")
cursor=mydb.cursor()
cursor.execute("SELECT VERSION()")
data=cursor.fetchone()
print("Database version:",data)
print("✅ Connection successful!")
print(mydb)
mydb.close()
```

Output:

```
Database version: ('8.0.42',)
✅ Connection successful!
<pymysql.connections.Connection object at 0x0000027305D1BE00>
```

Using pymysql, the above program logs into MySQL, runs a version check, shows the version and connection info, and ends the session.

7.3.1. Use of Connection Objects:

Connection objects should respond to the following methods:

a) .close( ) : This method sends the quit message and closes the database connections to give up resources.

b) .commit( ) : The execution of **commit** statement makes all the modification made by the current transaction become permanent.

c) .rollback( ): Transaction **rollback** becomes necessary if an error occurs the execution of a transaction.

d) .cursor( ) : Return a new cursor object using the connection. The primary purpose of this object is to provide SQL queries using execute() method.

7.3.2. Use of Cursor Object:

When you have a connection object associated with a database, you can create cursor objects. These objects represents a database cursor, which is used to manage the context of a fetch operation. The cursor object enables to run SQL queries using the cursor's execute() method.

a) .close() : This method , closing a cursor. Once you close cursor object you cannot execute any SQL statements.

b).execute(operation [, parameters]): prepare and execute a database operation(query or command). The parameters are option but they can be tuple, list or dictionary.

c) .fetchone() : Fetch the next row of a query result set, returning a single sequence, or None when no more data is available.

d) .fetchmany([size=cursor.arraysize]): Fetch the next set of rows of a query result, returning a sequence of sequences (e.g: a list of tuples). An empty sequence is returned when no more rows are available.

e) fetchall() : Fetch all (remaining)rows of a query result, returning them as a sequence of sequences.

7.4. Creating TABLES:

Once a connection is successfully established, we can create the tables. Creation of tables, insertion, updation and deletion operations are performed in python with the help of execute() statement. The following code shows how to create a table in MySQL from Python.

For e.g: program to create a table inside the database sample to enter the department details.

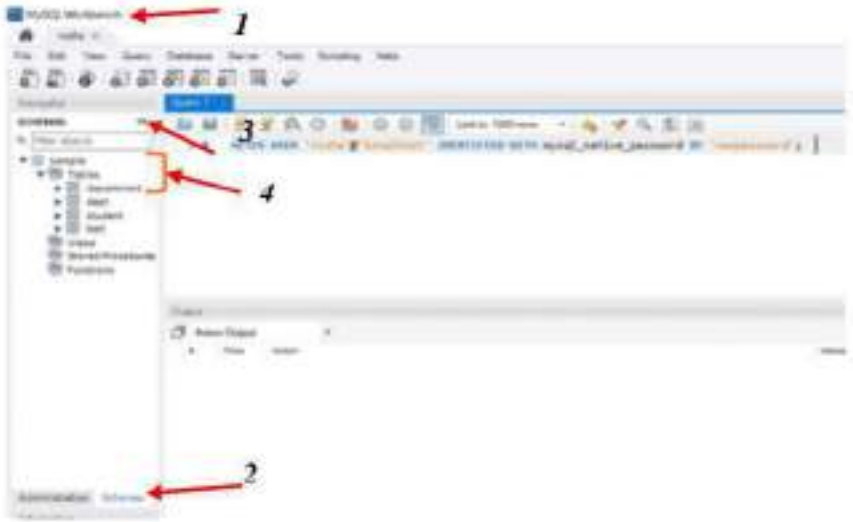
```
import pymysql
#open database connection
db=pymysql.connect(host="localhost",user="root",password="Fero_17Frano_23",
  database="sample")

#prepare a cursor object using cursor() method
cursor=db.cursor()

#create a TABLE as per requirement
sql="""create TABLE Department( DeptNo INTEGER, Dept_Name CHAR(20), Location
CHAR(25))"""
cursor.execute(sql)
db.commit()
#disconnect from server
db.close()
```

The program creates a table called Department with fields DeptNo, Dept_Name and Location . the SQL statement for creating the table is stored in sql and the sql is passed to execute () method for creating the table.

If you don't have any error message in python shell window , then you can check if your table was created successfully in MySQL Workbench, follow these instructions.

Way to view the table:

open **MySQL Workbench** → go to **Schemas** → Refresh the **"Schemas"** →click your database **(sample)** → check under **Tables(your created table Department)** .



Example 2: Program to create a table with unique key for each record.

```
import pymysql
#open database connection
db=pymysql.connect(host="localhost",    user="root",    password="Fero_17Frano_23",
database="sample")
#prepare a cursor object using cursor() method
cursor=db.cursor()
cursor.execute("CREATE TABLE customers (id INT AUTO_INCREMENT
        PRIMARY KEY, name VARCHAR(255),
            address VARCHAR(255))")
```

The above program demonstrates how to create a new table with specific fields, which can be viewed in the MySQL Workbench terminal.

Output:

Customers table with fields

Example 3: program to display the Database tables in python shell.

```
import pymysql
mydb = pymysql.connect(
  host="localhost",
  user="root",
  password="Fero_17Frano_23",
  database="sample")
mycursor = mydb.cursor()
mycursor.execute("Show tables;") #database command to show tables in python shell
myresult = mycursor.fetchall()
for x in myresult:
  print(x)
```

Output:

```
('department',)
('dept',)
('student',)
('test',)
```

The above Python program connects to a MySQL database and uses the ***SHOW TABLES*** command to list all tables. A ***for loop*** is used to print each table name one by one.

Additional points:

1. ***Constraints in Database Programming:*** *Database integrity refers to the accuracy or correctness of data in data base. The types of database constrains are*

- *Type constraints: It specify what is legal to a given type.*
  For e.g:
  want to define Age as a Type. Then , we can specify the following type constraints in plain English.

  TYPE Age Representation (Rational) CONSTRAINT Age > 0

- *Attribute Constraints: It specify that a specified column is of a specified type.*
  For e.g: CREATE TABLE employee(emp_no INTEGER, emp_name char(32),.....)
  Here emp_no is an integer; the emp_name is a string ,these declarations are automatically add attribute constraints to column definition.
- *Instance constraints: It apply to a specific instance or conditions.*
  For e.g: CREATE TABLE student(R_num INTEGER NOT NULL, Total_Marks INTEGER CHECK(Total_Marks <= 100))

The keyword Check for specifying constraints.

2. ***Keys:*** Keys are fundamental to relational database. The power of relational database comes from the fact that we can split related data into different tables and logically link them together by using keys. The different types of Keys database is
   - ***Primary Key:*** *It will identifies every record in a table uniquely.*
   - ***Alternate Key or Secondary Key:*** *These types of keys are may or may not identify a record uniquely but help in faster searching.*
   - ***Composite Key:*** *A key consisting of two or more columns is called as a composite key.*
   - ***Super Key and Key:*** *It is a set of columns that uniquely identifies every row in a table, which a key is a minimal set of such columns.*
   - ***Foreign Key:*** *It is a set of column in one table, which is a key in another table.*
   - ***Candidate Key:*** *A table can have more than one set of columns that could be chosen as the key.*

7.4.1.Database Operations:

There are various operations to perform within python program. Some of the functions are defined below:

Database Operations	Description
INSERT	It is used to create a record into a table.
UPDATE	It is used to update those available or already existing records.
SELECT	It fetches useful information from the database.

DELETE	It is used to delete records from the database.
COMMIT	It is used to make the changes permanently into database table.
ROLLBACK	It works like "undo" , which reverts all the changes that have made

What is Database Transaction?

Database transaction represents a single unit of work. Any operation which modifies the state of MySQL database is a transaction. The database transactions are commit() and rollback() methods.

- The commit() sends a commit statements to the MySQL server, committing the current transaction. This is useful when have multiple INSERT,UPDATE, and DELETE statements within the same connection.
  syntax: connection.commit()
- MySQL connection .**rollback** revert the changes made by the current transaction.
  syntax: connection.rollback()

7.4.2. INSERT OPERATION:

Once a table is created , we need to insert values to the table. The values can be inserted using the INSERT statement of SQL.

Inserting Rows/Records into Table:

- MySQL/Python provides to insert one or many rows into a table at a time. To insert a row, first you have to write the queries to insert different data, then execute with the help of cursor. To insert new rows into a MySQL table, follow the steps below:
  - ✓ Connect to the MySQL database server by creating new connection object.
  - ✓ Initiate a cursor object from connection object.
  - ✓ After execution commit the changes to the database.
  - ✓ Close the database.

For Example 1: program to insert the multiple values into the existing MySQL database sample.

```
import pymysql
mydb = pymysql.connect( host="localhost", user="root", password="Type your MySQL password", database="sample")
cursor = mydb.cursor()
sql="""INSERT INTO Department(DeptNo,Dept_Name,Location) VALUES(13,'Computer Science', 'Bangalore')"""
cursor.execute(sql)

sql="""INSERT INTO Department(DeptNo,Dept_Name,Location) VALUES(14,'BCA', 'Bangalore')"""
cursor.execute(sql)

sql="""INSERT INTO Department(DeptNo,Dept_Name,Location) VALUES(12,'Hotel Management', 'Bangalore')"""
cursor.execute(sql)
```

```
mydb.commit()
cursor.execute("SELECT * FROM Department")
myresult=cursor.fetchall()
for x in myresult:
  print(x)
mydb.close()
```

Output 1: from python shell

```
(13, 'Computer Science', 'Bangalore')
(14, 'BCA', 'Bangalore')
(12, 'Hotel Management', 'Bangalore')
```

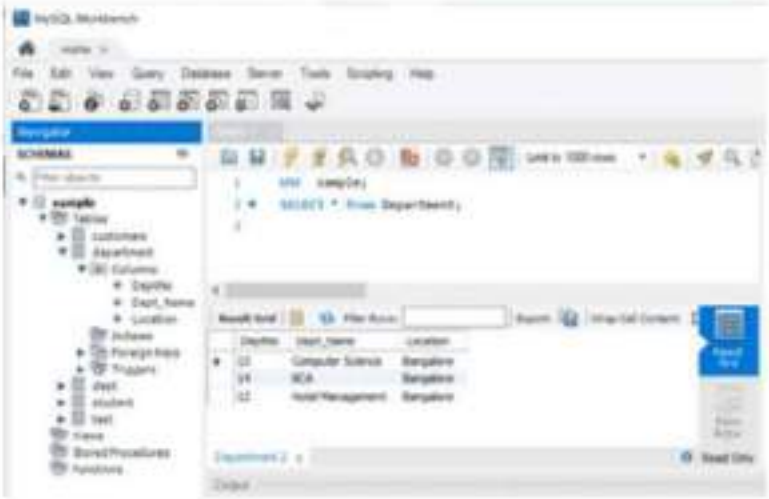
Output 2:

To view from MySQL Workbench follow the follow the instructions.

open **MySQL Workbench** → go to **Schemas** → Refresh the **"Schemas"** →click your database **(sample)** → check under **Tables**→ click the table Department → then type the following SQL command in SQL Query Tab.

- USE sample;
- SELECT * from Department;

Image of workbench output:



Example 2: Simple Python Code to Create a Table and Insert Values.

```
import pymysql
mydb = pymysql.connect(host="localhost", user="root", password="Fero_17Frano_23")
cursor = mydb.cursor()
 cursor.execute("CREATE DATABASE Stud_details")
cursor.execute("USE Stud_details")
cursor.execute("""CREATE TABLE studentdetails(S_Name varchar(40), Address varchar(255),
Course char(20))""")
sql="""INSERT INTO studentdetails(S_Name,Address,Course) VALUES ('abi','Chennai-24','Bsc
Maths')"""
try:
   cursor.execute(sql)
    mydb.commit()
```

```
except:
    mydb.rollback()
cursor.execute("SELECT * FROM studentdetails")
res= cursor.fetchall()
for row in res:
    print(row)
cursor.close()
mydb.close()
```

Output:

```
('abi', 'Chennai-24', 'Bsc Maths')
```

Example 2.1: To do programming to insert as many as many as records, the following program uses function called **records( )** with parameters for a data entry program as per users choice to insert records into **studentdetails** table.

Coding:

```
import pymysql
def Records(roll, name, gender, address, course, department, T_marks):
    # Connect to MySQL database
    mydb = pymysql.connect(
        host="localhost",
        user="root",
        password="Fero_17Frano_23",
        db="Stud_details" )
    cursor = mydb.cursor()
    # Create table only if it does not already exist
    cursor.execute("""CREATE TABLE IF NOT EXISTS studentdetails (RollNo
        INTEGER(20), Name VARCHAR(40), Gender CHAR(2), Address VARCHAR(255),
Cours CHAR(30), Department CHAR(20),
        T_Marks INTEGER(10) ) """)
    # Insert data safely using parameters
    sql = """INSERT INTO studentdetails (RollNo, Name, Gender, Address, Course,
        Department, T_Marks) VALUES (%s, %s, %s, %s, %s, %s, %s)"""values = (roll, name,
gender, address, course, department, T_marks)
    try:
        cursor.execute(sql, values)
        mydb.commit()
        print(" Record inserted successfully.")
    except Exception as e:
        print(" Error:", e)
        mydb.rollback()
```

```
    finally:
        cursor.close()
        mydb.close()
def display_records():
    # Function to display all records in the table
    mydb = pymysql.connect(
        host="localhost",
        user="root",
        password="Fero_17Frano_23",
        db="Stud_details"    )
    cursor = mydb.cursor()
    cursor.execute("SELECT * FROM studentdetails")
    print("\nStudent Records:")
    for row in cursor.fetchall():
        print(row)
    cursor.close()
    mydb.close()
def main():
    while True:
        roll = int(input("Enter the Roll No: "))
        name = input("Enter Name: ")
        gender = input("Enter Gender (M/F): ")
        address = input("Enter Address: ")
        course = input("Enter Course: ")
        department = input("Enter Department: ")
        T_marks = int(input("Enter Total Marks: "))

        Records(roll, name, gender, address, course, department, T_marks)
        more = input("\nDo you want to enter another record? (Y/N): ")
        if more.lower() != 'y':
            break
    # Show all inserted records
    display_records()
if __name__ == '__main__':
    main()
```

Output:

```
Enter the Roll No: 12
Enter Name: ani
Enter Gender (M/F): f
Enter Address: chennai-23
Enter Course: Bsc maths
Enter Department: Maths
Enter Total Marks: 336
 Record inserted successfully.
```

Do you want to enter another record? (Y/N): Y
Enter the Roll No: 21
Enter Name: Raju
Enter Gender (M/F): m
Enter Address: chennai-4
Enter Course: BCA
Enter Department: CS
Enter Total Marks: 450
Record inserted successfully.

Do you want to enter another record? (Y/N): Y
Enter the Roll No: 10
Enter Name: Michael
Enter Gender (M/F): M
Enter Address: chennai-27
Enter Course: BCom
Enter Department: Commerce
Enter Total Marks: 434
Record inserted successfully.

Do you want to enter another record? (Y/N): N
Student Records:
(12, 'ani', 'f', 'chennai-23', 'Bsc maths', 'Maths', 336)
(21, 'Raju', 'm', 'chennai-4', 'BCA', 'CS', 450)
(10, 'Michael', 'M', 'chennai-27', 'BCom', 'Commerce', 434)
We can see the inserted table value into MySQL Workbench also

7.5. UPDATE OPERATION:

UPDATE operation is done to modify existing values available in the table. We can update one or more records at the same time. In other words , the UPDATE statement replaces old values of one or more data belonging to several rows with new values.

The rows updated are based on the WHERE clause specified in the UPDATE statement.

For e.g: Program to update studentdetail table data where total marks by adding 10 to each record:

```
import pymysql
def Update_Records():
   # Connect to MySQL database
   mydb = pymysql.connect(
      host="localhost", user="root", password="Fero_17Frano_23", db="Stud_details"    )
   cursor = mydb.cursor()
   sql="UPDATE studentdetails SET T_Marks=T_Marks+10 WHERE GENDER='F' "
   mydb.commit()
   cursor.execute(sql)
   cursor.execute("SELECT * FROM studentdetails")
   for row in cursor.fetchall():
      print(row)
   print("Records updated")
   cursor.close()
   mydb.close()
def main():
   Update_Records()
if __name__=='__main__':
   main()
```

Output:

```
(12, 'ani', 'f', 'chennai-23', 'Bsc maths', 'Maths', 346) # T_Marks has updated
(21, 'Raju', 'm', 'chennai-4', 'BCA', 'CS', 450)
(10, 'Michael', 'M', 'chennai-27', 'Bcom', 'Commerce', 434)
Records updated
```

7.6. DELETE OPERATION:

We can delete records from a database using Python. The MySQL DELETE query from python application is used to delete from the MySQL database in Python. By using the DELETE operation ,we are able to perform the following:

- → Delete single row, multiple rows, single column, and multiple columns.
- → Delete table and an entire database.

Example 1: program to perform deletion operation to delete all rows of data using the SQL query statement with python .

```
import pymysql
mydb = pymysql.connect(
host="localhost",
user="root",
password="Fero_17Frano_23",
database="sample")
cursor = mydb.cursor()
cursor.execute("DELETE FROM department")
```

```
mydb.commit()
print("Records Deleted")
cursor.execute("SELECT * FROM department")
mydb.close()
```

Output:

Records Deleted

Output from MySQL Workbench

From the above program , the command **"DELETE FROM department"** removes all rows from the department table, leaving the table structure intact but empty.

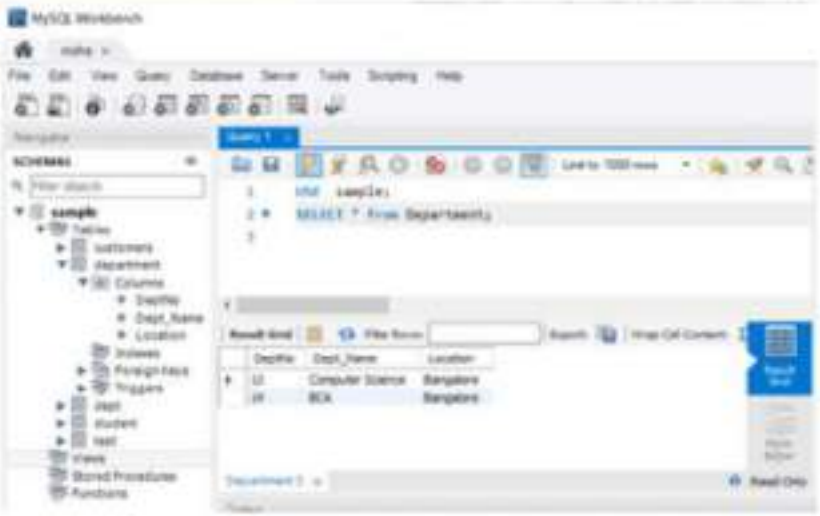
Example 2: Program to perform a single row delete operation using for loop.

```
import pymysql
mydb = pymysql.connect(
   host="localhost",
   user="root",
   password="Fero_17Frano_23",
   database="sample")
cursor = mydb.cursor()
cursor.execute("DELETE from Department WHERE DeptNo='12'")
mydb.commit()
cursor.execute("SELECT * FROM Department")
myresult=cursor.fetchall()
for x in myresult:
   print(x)
mydb.close()
```

Output 1:

(13, 'Computer Science', 'Bangalore')
(14, 'BCA', 'Bangalore')

Output 2: from MySQL Workbench.



7.7. READ OPERATION:

The READ operation on any database means to fetch some useful information from the database. The **SELECT** statement is used to read data from the database . It allows users to retrieve and display stored information for processing, reporting, or analysis. Retrieving the results using methods like ***fetchone() or fetchall()*** to display records from the database.

For e.g: program to select fields from database table.

```
import pymysql
# Connect to the MySQL database
mydb = pymysql.connect(host="localhost",user="root",password="Fero_17Frano_23",
   database="Emp_2025"  )
cursor = mydb.cursor()
# Query to select rows with salary between 40000 and 60000
query = "SELECT EmpNo, Name, Salary FROM empsalary "
cursor.execute(query)
rows = cursor.fetchall()
# Print header
print(f"{'EmpNo':<10}{'Name':<20}{'Salary':<10}")
print("-" * 40)
# Print each row formatted
for row in rows:
   empno, name, salary = row
   print(f"{empno:<10}{name:<20}{salary:<10}")
# Close the connection
cursor.close()
mydb.close()
```

Output:

```
EmpNo     Name              Salary
----------------------------------------
3210      suresh            46350
213       Kanija Rose       380200
1110      Anand             64000
30324     Maduri            71400
54361     Androlin Maxmilan 64780
```

7.7.1. fetchone() Method:

The fetchone() method returns the next row of a query result set or None in case there is no row left.

Example 1:

```
import pymysql
# Connect to the MySQL database
mydb = pymysql.connect(
   host="localhost",
   user="root",
   password="Fero_17Frano_23",
   database="stud_details" )
```

```
cursor = mydb.cursor()
cursor.execute("SELECT * FROM stud_marks")
row=cursor.fetchone()
if row:
   print("Rollno :", row[0])
mydb.close()
```

Output:

Rollno : 12

The above Python program uses the pymysql library to connect to a MySQL database named **stud_details**. It retrieves the **first record** from the table **stud_marks** using the SQL SELECT * command. The fetchone() method fetches a single row, and if a row is found, it prints the value of the **first column**, which is the student's **Roll Number**.

Example 2: program to display all records from the table by using fetch one() method.

```
import pymysql
# Connect to the MySQL database
mydb = pymysql.connect(
   host="localhost",
   user="root",
   password="Fero_17Frano_23",
   database="stud_details"  )
cursor = mydb.cursor()
cursor.execute("SELECT * FROM stud_marks")
row=cursor.fetchone()
while row is not None:
   print(row)
   row=cursor.fetchone()
mydb.close()
```

Output:

```
(12, 'Niaha', 'F', 432)
(13, 'Nilan', 'M', 486)
(26, 'Raju chandran', 'M', 535)
(37, 'Ferniminix', 'M', 562)
(10, 'Kaviyazhini', 'F', 592)
```

The above program executes a SQL SELECT query to retrieve all records from the stud_marks table. Using a while loop and the fetchone() method, it iteratively fetches and prints each row until no more rows are left.

7.7.2. fetchall() Method:

This method fetches all the rows in a result set. If some rows have already been extracted from the result set, then it retrieves the remaining rows from the result set.

Example:

```
import pymysql
# Connect to the MySQL database
mydb = pymysql.connect(
```

```
    host="localhost",
    user="root",
    password="Fero_17Frano_23",
    database="stud_details"  )
cursor = mydb.cursor()
cursor.execute("SELECT  name FROM stud_marks")
row=cursor.fetchall()
for i in row :
    print("Name:",i[0])
    mydb.close()
```

Output:

```
Name: Niaha
Name: Nilan
Name: Raju chandran
Name: Ferniminix
Name: Kaviyazhini
```

The program executes a SQL query to select only the name column from the stud_marks table using cursor.execute(). It retrieves all resulting rows using the fetchall() method and iterates over them using a for loop. For each row, it prints the name in a formatted way using print("Name:", i[0]).

7.7.3. rowcount Method:

This is a read-only attribute and returns the number of rows that were affected by an execute() method.

Example:

```
import pymysql
# Connect to the MySQL database
mydb = pymysql.connect(
    host="localhost",
    user="root",
    password="Fero_17Frano_23",
    database="stud_details"  )
cursor = mydb.cursor()
cursor.execute("SELECT  name FROM stud_marks")
row=cursor.fetchall()
num_rows=cursor.rowcount
print("Number of Records: ", num_rows)
mydb.close()
```

Output:

```
Number of Records:  5
```

For practice:

Example 1: program to display all the table names present inside a server database:

Solution:

```
import pymysql
mydb = pymysql.connect(
    host="localhost",
```

```
    user="root",
    password="Fero_17Frano_23",
    database="sample")
mycursor = mydb.cursor()
mycursor.execute("SELECT table_name FROM information_schema.tables WHERE
table_schema = 'sample';")
myresult = mycursor.fetchall()
for x in myresult:
    print(x)
```

Output:

```
('customers',)
('department',)
('dept',)
('student',)
('test',)
```

Example 2: Write the python program for the following table of data.

EmpNO	Name	DOJ	Gender	Salary	Dept_code
3210	Suresh	2021-4-23	M	46350	12
213	Kanija Rose	2025-1-3	F	380200	10012
1110	Anand	2020-3-15	M	64000	12
30324	Maduri	2025-4-2	F	71400	2
54361	Androlin Maxmilan	202-3-15	M	64780	3001

Solution:

```
import pymysql
mydb = pymysql.connect(
host="localhost",
user="root",
password="Fero_17Frano_23")
cursor = mydb.cursor()
cursor.execute("CREATE DATABASE Emp_2025")
cursor.execute("USE Emp_2025")
cursor.execute("CREATE TABLE empsalary(EmpNo INT(20), Name CHAR(40), DOJ
DATE, Gender CHAR(2), Salary INT(10),Dept_Code INT(20))")
sql="""INSERT INTO empsalary(EmpNo,Name, DOJ, Gender, Salary, Dept_code)
VALUES(3210,'suresh','2021-04-23', 'M',46350, 00012)"""
cursor.execute(sql)
sql="""INSERT INTO empsalary(EmpNo,Name, DOJ, Gender, Salary, Dept_code)
VALUES(213,'Kanija Rose','2025-01-03', 'F',380200, 10012)"""
cursor.execute(sql)
sql="""INSERT INTO empsalary(EmpNo,Name, DOJ, Gender, Salary, Dept_code)
VALUES(1110,'Anand','2020-03-15', 'M',64000, 012)"""
cursor.execute(sql)
```

```
sql="""INSERT INTO empsalary(EmpNo, Name, DOJ, Gender, Salary, Dept_code)
VALUES(30324,'Maduri','2025-04-02', 'F',71400, 002)"""
cursor.execute(sql)
sql="""INSERT INTO empsalary(EmpNo,Name, DOJ, Gender, Salary, Dept_code)
VALUES(54361,'Androlin Maxmilan','2021-03-15', 'M',64780, 30012)"""
cursor.execute(sql)
mydb.commit()
cursor.execute("SELECT * FROM empsalary")
result=cursor.fetchall()
for i in result:
    print(i)
cursor.close()
mydb.close()
```

Output:

```
(3210, 'suresh', datetime.date(2021, 4, 23), 'M', 46350, 12)
(213, 'Kanija Rose', datetime.date(2025, 1, 3), 'F', 380200, 10012)
(1110, 'Anand', datetime.date(2020, 3, 15), 'M', 64000, 12)
(30324, 'Maduri', datetime.date(2025, 4, 2), 'F', 71400, 2)
(54361, 'Androlin Maxmilan', datetime.date(2021, 3, 15), 'M', 64780, 30012)
```

Example 3: Write the python interface program to read all the rows from Employee table whose salary in between 40000 and 60000. Also display the rows in a formatted manner.

Solution:

```
import pymysql
# Connect to the MySQL database
mydb = pymysql.connect(
    host="localhost",
    user="root",
    password="Fero_17Frano_23",
    database="Emp_2025" )
cursor = mydb.cursor()
# Query to select rows with salary between 40000 and 60000
query = "SELECT EmpNo, Name, Salary FROM empsalary WHERE Salary BETWEEN 40000
AND 60000"
cursor.execute(query)
rows = cursor.fetchall()
# Print header
print(f"{'EmpNo':<10}{'Name':<20}{'Salary':<10}")
print("-" * 40)
# Print each row formatted
for row in rows:
    empno, name, salary = row
    print(f"{empno:<10}{name:<20}{salary:<10}")
# Close the connection
cursor.close()
mydb.close()
```

Output:

```
EmpNo    Name          Salary
-----------------------------------------
3210     suresh        46350
```

7.8. Transaction Control:

A transaction is a logical unit of work that contains one or more SQL statements. A transaction is an atomic unit. The effects of all the SQL statements in a transaction can be either all committed (applied to the database) or all rolled back. Transaction is a mechanism to ensure data consistency. It ensures 4 properties generally referred to as ACID properties.

- ***Atomicity:*** it ensures that all operations within the work unit are completed successfully; Otherwise, the transaction is aborted at the point of failure, and previous operations are rolled back to their former state.
- ***Consistency:*** it ensures that the database properly change states upon a successfully committed transaction.
- ***Isolation:*** it enable transactions to operate independently and transparent to each other.
- ***Durability:*** it ensures that the result or effect of a committed transaction persists in case of a system failure.

7.9. COMMIT and ROLLBACK Operation:

- The COMMIT command is the transactional command used to save changes invoked by a transaction to the database. The COMMIT command saves all transactions to the database since the last COMMIT or ROLLBACK command.

  Syntax:

  ```
  db.commit()
  ```

- The ROLLBACK command is the transactional command used to undo transactions that have not already been saved to the database. Th ROLLBACK command can only be used to undo transactions since the last COMMIT or ROLLBACK command was issued.

  Syntax:

  ```
  db.rollback()
  ```

  Example:

```
import pymysql
# Connect to the MySQL database
mydb = pymysql.connect(
  host="localhost",
  user="root",
  password="Fero_17Frano_23",
  database="stud_details")
cursor = mydb.cursor()
try :
  sql="""INSERT INTO studentdetails(RollNo, Name, Gender, Address, Course,
Department, T_Marks) Values(27,'George','M','Kanchipuram','BBA','
Logistics',450)"""
  cursor.execute(sql)
  mydb.commit()
```

```
        print("Record inserted successfully.")
        cursor.execute("SELECT * FROM studentdetails")
        rows=cursor.fetchall()
        for i in rows:
            print(i)
    except Exception as e:
        print("Error occured :",e)
        mydb.rollback()
    finally:
        mydb.close()
```

Output:

```
    Record inserted successfully.
(12, 'ani', 'f', 'chennai-23', 'Bsc maths', 'Maths', 336)
(21, 'Raju', 'm', 'chennai-4', 'BCA', 'CS', 450)
(10, 'Michael', 'M', 'chennai-27', 'Bcom', 'Commerce', 434)
(27, 'George', 'M', 'Kanchipuram', 'BBA', ' Logistics', 450)
```

7.10. Disconnecting From a Database:

A close( ) method is called to disconnect from the database.

Syntax:

```
db.close()
```

If the connection to a database is closed by the user with the ***close( )*** method, any outstanding transactions are rolled back by the database. However, instead of depending on any of database lower level implementation details, the application would be better to call commit or rollback explicitly.

7.11. Exception Handling in Database:

There are many sources of errors in databases. Database modules define a top-level exception Error that is a base class for all other errors. The following exceptions are for more specific kinds of database errors:

Exception	Description
InterfaceError	Errors related to the database interface, but not the database itself.
DatabaseError	Errors related to the database itself.
DataError	Errors related to the processed data. For e.g: bad type conversion, division by zero, etc.
OperationalError	Error related to the operation of the database itself. For e.g: a lost connection.
IntegrityError	Error When relational integrity of the database is broken.
InternalError	Internal error in the database. For e.g: if a stale cursor.
ProgrammingError	Error in SQL queries.
NotSupportedError	Error for methods in the database API that aren't supported by the underlying database.

Python offers robust exception-handling mechanisms that can be integrated seamlessly with database operations. Python's ***try-except-finally*** structure allows developers to detect and manage such errors gracefully.

- The try block contains the database operations like SELECT, INSERT, or UPDATE.
- If an error occurs, the program control moves to the except block, which is used to display appropriate error messages or handle the issue accordingly.
- Finally, the finally block ensures that resources like database connections are released properly, avoiding memory leaks or locked tables.

Syntax:

```
try:
    # Execute query
     mydb.commit()
except <SpecificError>:
    # Handle it
finally:
    # Close connection
```

Example:

```
import pymysql
try:
   mydb = pymysql.connect(
   host="localhost",
   user="root",
   password="Fero_17Frano_23",
   database="Emp_2025" )
   cursor = mydb.cursor()
   cursor.execute("SELECT * FROM empsalary")
   results = cursor.fetchall()
   for row in results:
      print(row)
except pymysql. ProgrammingError as e:
   print(f"SQL error: {e}")
except pymysql.OperationalError as e:
   print(f"Database connection error: {e}")
except pymysql. Error as e:
   print(f"An error occurred: {e}")
finally:
   if mydb:
      mydb.close()
```

Output:

```
(3210, 'suresh', datetime.date(2021, 4, 23), 'M', 46350, 12)
(213, 'Kanija Rose', datetime.date(2025, 1, 3), 'F', 380200, 10012)
(1110, 'Anand', datetime.date(2020, 3, 15), 'M', 64000, 12)
(30324, 'Maduri', datetime.date(2025, 4, 2), 'F', 71400, 2)
(54361, 'Androlin Maxmilan', datetime.date(2021, 3, 15), 'M', 64780, 30012)
```

From the above program The entire operation is enclosed within a try-except-finally block to handle errors effectively:

- If a **SQL syntax or command error** occurs, a ProgrammingError is caught.
- If there is an issue with **connecting to the database**, such as wrong credentials or server issues, an OperationalError is handled.
- Any **other general MySQL errors** are caught by the generic pymysql. Error exception.

Regardless of whether the operation succeeds or fails, the finally block ensures that the database connection is properly closed using mydb.close() .

7.12. Exercise:

1. How will you connect to MySQL from Python?
2. How will you create tables in MySQL from python?
3. Write python programs using MySQL module for (a) to (d) using following specifications of HOSPITAL table. Assume that you are using a MySQL database .

Use appropriate connection and cursor object to perform SQL query operations.

Field Name	Data Type	Constraints
PNo	Number(4)	Primary Key
Name	Char(20)	UNIQUE
Age	INI(2)	
Department	Char(15)	Default="OPD"
Dateofadm	Date	DefaultSYSDATE
Charges	Decimal(7,2)	Check>100
Sex	Char(1)	

a)Write a program to insert the following data into the above table:

1	Arpit	63	Surgery	2018-01-21	1300	M
2	Zarina	25	ENT	2018-12-14	1240	F
3	Kareem	34	Orthopaedic	2018-02-16	1500	M
4	Arun	18	Surgery	2018-04-01	16200	M
5	Zubiya	30	ENT	2018-01-15	1250	F
6	Kalki	25	ENT	2018-04-28	1250	F
7	Ankita	51	Cardiology	2018-02-14	1850	F
8	Arun Kushal	59	Cardiology	2018-03-14	1800	M

b) Write a program to fetch read records of patients that are admitted in the month of February.

c)Write a program to update the female patient's records by giving a discount of 300.

d) Write a program to delete the ENT patient records.

4.What is Cursor object?

5. What happens when you close a connection object before commit() in python program?

6. Explain two methods that fetches rows from a database table.

7. Write the difference between commit() and rollback().

8. what is the purpose of close() method in related to cursor?

****

BOOK 8: PYTHON DICTIONARIES And SETS

8.1. About Dictionaries and Sets:

The dictionary and sets are used to store data in which there is no explicit or implicit ordering. Both are quite similar. The difference is that a dictionary has a key-value pair. A set can be thought of a collection of unique keys.

8.2. Dictionaries:

A dictionary in python is an unordered collection of items that store data in key-value pairs, offering a practical mechanism for mapping unique keys to values. The Key-Value pairs need to be enclosed in { }. Dictionaries are mutable, enabling dynamic data manipulation.

Creating Dictionaries:

Dictionaries can be crafted using curly braces with key-value pairs or through the dict() constructor.

Example 1:

```
bin_colors={"primary_color":"yellow","prioption_color":"green","final_color":"Red"}
print(bin_colors)
```

Output:

```
{'primary_color': 'yellow', 'prioption_color': 'green', 'final_color': 'Red'}
```

Python's dictionary comprehensions also allow dictionaries to be constructed in a succinct manner:

Example 2:

```
squares={x: x**2 for x in range(6)}
print(squares)
```

Output:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

- To retrieve a value associated with a key, either the get function can be used or the key can be used as the index:

for e.g 3:

```
bin_colors={"primary_color":"yellow","prioption_color":"green","final_color":"Red"}
a=bin_colors.get('prioption_color')
print(a)
```

Output:

```
Green
```

- To update a value associated with a key.

For e.g 4:

```
bin_colors={"primary_color":"yellow","prioption_color":"green","final_color":"Red"}
bin_colors['primary_color']="purple"
print(bin_colors)
```

Output:

```
{'primary_color': 'purple', 'prioption_color': 'green', 'final_color': 'Red'}
```

When constructing dictionaries, values can be of any type, while keys are restricted to any being immutable types(meaning no lists, sets, or dictionaries can be used as keys). For example: the following code is a valid dictionary:

```
>>>E={"country":["Berlin","Thailand","US"],"Id":23,4:5,6:9.1}
>>>E["Id"]
23
>>>E["city"]
['Berlin', 'Thailand', 'US']
>>>E[6]
9.1
```

8.3. dict ( ) Type in Python:

The ***dict*** type is not only widely used in our programs but also fundamental part of the python implementation. Modules namespaces, class and instance attributes, and function keyword arguments are some of the fundamental constructs where dictionaries are deployed. The built-in functions live in __builtins__._dict_

Because of their crucial role, python ***dicts*** are highly optimized. *Hash tables* are the engines behind python's high-performance dicts. Sets are also implemented with hash tables as well.

All mapping types in the standard library use the basic dict in their implementation, so they share the limitation that the keys must be hashable (the values need not be hashable, only the keys.

What is Hashable?

An object is hashable if it has a hash value which never changes during its lifetime (it needs a *_hash_( )* method), and can be compared to other objects(it needs an *_eq_()* method).Hashable objects which compare equal must have the same hash value[....]

the atomic immutable types (str, bytes, numeric types) are all hashable. A frozen set is always hashable, because its elements must be hashable by definition.

for e.g:

```
>>>tr=(1,3,4,(5,6,3,5))
>>>hash(tr)
-5611131492770032 81
>>>tl=(1,5,6,3,[89,19])
>>>hash(tl)
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    hash(tl)
TypeError: unhashable type: 'list'
>>>td=(2,3,frozenset([5,67]))
>>>hash(td)
        5768629268929338612
```

From the coding the tl is a tuple, it contains a **list**, which is a mutable type. Since hashable objects must be immutable, Python prevents hashing this tuple.

8.3.1. Dictionary Methods:

Python provides dictionary method for more advanced dictionary manipulations. Here are mentioned few of the most common ones .

- ***dict.clear():*** Deletes all key-value pairs in the dictionary, resulting in an empty dictionary.
- ***dict.copy():*** this copies all key-value pairs from a dictionary, creating an identical dictionary in a new memory location, modifying key-value pairs of the original dictionary will not affect the copy, and vice versa.
- ***dict.items():*** Returns a dynamic view of the dictionary key-value items, provided as (key,value) tuples. It is 'dynamic' because if the dictionary changes the dynamic view assigned to a variable earlier will also change. The dynamic view can be converted to other data structures: list(dict.items()) or tuple(dict.items())
- ***dict.keys()*** :and dict.values(): Similar to dict.items(), however provides only the keys or values respectively.
- ***dict.update():*** Updates a dictionary with a new value for a given key; if the key does not exist, the key-value pair is added to the dictionary.

For Practice:

1. Write python program to count Number of times each word appears in a sentence.

Solution:

```
def main():
    count_words=dict()
    sentence=input("Enter a sentence:")
    words=sentence.split()
    for each_word in words:
        count_words[each_word]=count_words.get(each_word,0)+1
    print("The number of times each word appear in a sentence is:")
    print(count_words)
if __name__=="__main__":
    main()
```

Output:

```
Enter a sentence :Don't wait for the perfect moment. Take the moment and make it perfect.
The number of times each word appear in a sentence is:
{"Don't": 1, 'wait': 1, 'for': 1, 'the': 2, 'perfect': 1, 'moment.': 1, 'Take': 1, 'moment': 1, 'and': 1, 'make': 1, 'it': 1, 'perfect.': 1}
```

2. Write python program to count the Number of Characters in a string using dictionaries. Display the Keys and their values in alphabetical order.

Solution:

```
def character_dict(word):
    character_count=dict()
    for each_character in word:
        character_count[each_character]=character_count.get(each_character,0)+1
    sorted_keys=sorted(character_count.keys())
    for each_key in sorted_keys:
        print(each_key,character_count.get(each_key))
```

```
def main():
    word=input("Enter a string: ")
    character_dict(word)
if __name__=="__main__":
    main()
```

Output:

```
Enter a string: With the new day comes new strength and new thoughts.
 9
. 1
W 1
a 2
c 1
d 2
e 6
g 2
h 5
i 1
m 1
n 5
o 2
r 1
s 3
t 6
u 1
w 3
y 1
```

3. Program to create a dictionary using a list of tuples of key and value pair.

Solution:

```
Dict = dict([(1, 'Home Sweet home'), (2, 'Hot Coffee')])
print("\n Dictionary by using list of tuples of key and value as a pair: ")
print(Dict)
```

Output:

```
Dictionary by using list of tuples of key and value as a pair:
 {1: 'Home Sweet home', 2: 'Hot Coffee'}
```

4. Python program to input information for n numbers of students as given below:

a. Name

b. Registration Number

c. Total Marks

The user has to specify a value for n number of students. The program should output the registration number and marks of a specified student given his name.

Solution:

```
def student_details(number_of_students):
    student_name = { }  # Corrected variable name for consistency
    for i in range(number_of_students):
```

```
        name = input("Enter the Name of the student: ")
        registration_number = input("Enter student's Registration Number: ")
        total_marks = input("Enter Student's Total Marks: ")
        student_name[name] = [registration_number, total_marks]  # Fixed
    student_search = input("Enter name of the student you want to search: ")
    if student_search not in student_name:
        print("Student you are searching is not present in the class.")
    else:
        print("Student you are searching is present in the class.")
        print(f"Student's Registration Number is {student_name[student_search][0]}")
        print(f"Student's Total Marks is {student_name[student_search][1]}")
def main():
    number_of_students = int(input("Enter the number of students: "))
    student_details(number_of_students)
if __name__ == "__main__":
    main()
```

Output:

```
Enter the number of students: 2
Enter the Name of the student: Arul Sughana
Enter student's Registration Number: 1AEITI8CS03
Enter Student's Total Marks: 667
Enter the Name of the student: Bala Manikandan
Enter student's Registration Number: 3AFITI4CS05
Enter Student's Total Marks: 786
Enter name of the student you want to search: Arul Sughana
Student you are searching is present in the class.
Student's Registration Number is 1AEITI8CS03
Student's Total Marks is 667
```

8.3.2. dict Comprehensions:

A ***dictcomp*** builds a dict instance by producing Key:value pair from any iterable. The following examples shows how to use a dictionary comprehension to select a subset of key-value pairs from a dictionary that meet a particular condition.

Example 1: Program shows the use of dict comprehensions to build two dictionaries from the same list of tuples.

```
codes=[(86,'China'),(81,'India'),(1,'United
States'),(63,'Indonesia'),(92,'Pakistan'),(446,'Nigeria'),(7,'Russia'),(81,'Japan')]
country_code={country: code for code, country in codes}
print(country_code)
dict_compre={code:country.upper() for country, code in country_code. items() if code<50}
print(dict_compre)
```

Output :

```
{'China': 86, 'India': 81, 'United States': 1, 'Indonesia': 63, 'Pakistan': 92, 'Nigeria': 446, 'Russia':
7, 'Japan': 81}
```

{1: 'UNITED STATES', 7: 'RUSSIA'}

This Python program uses dictionary comprehensions to create and filter dictionaries. First, it takes a list of country codes and names, and creates a dictionary where each country name is the key and its code is the value. If a country code appears more than once, the last one will replace the previous. Then, it creates another dictionary(dict_compre) from the first one, but only keeps the countries with codes less than 50. In this second dictionary, the keys are the codes and the values are the country names in uppercase. This helps to filter and format data easily using simple Python code.

Example 2:

```
For_squares={x: x**2 for x in range(6)}
for_even_squares={x: x**2 for x in range(7) if x%2==0}
print(For_squares)
print(for_even_squares)
```

Output:

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
{0: 0, 2: 4, 4: 16, 6: 36}
```

From the line 1, It iterates through numbers from 0 to 5 (since range(6) excludes 6).For each number x, it creates a key–value pair where:

- **Key** = x
- **Value** = x squared (x**2)

From the line 2, It loops through numbers from 0 to 6 (range(7)). The condition if x % 2 == 0 ensures that only even numbers are included. Each even number becomes a key, and its square becomes the corresponding value.

Key Point:

Dictionary comprehensions allow you to build dictionaries efficiently, and they can include **conditional logic** to control which elements are added.

8.4. Sets

A set is a mutable and unordered collection of unique elements. Set is written with curly brackets { }, being the elements separated by commas.

It can also be defined with the built-in function set([iterable]). This function takes as argument an iterable (i.e., any type of sequence, collection, or iterator), returning a set containing unique items from the input (duplicated elements are removed).

Example:

```
# Create a Set using various data types
# A string
print(set('Mrs.Deviyani'))
# a list
a=set(['Mayank', 'Vardhman', 'Mukesh', 'Mukesh'])
print("set from a list :",a)
# A tuple
t=set(('Lucknow', 'Kanpur', 'India'))
print("set from tuple:",t)
```

```
# a dictionary
d=set({'Sulphur': 16, 'Helium': 2, 'Carbon': 6, 'Oxygen': 8})
print("set from dictionary:",d)
```

Output:

```
{'e', '.', 'n', 's', 'r', 'D', 'M', 'i', 'a', 'v', 'y'}
# The set() function takes the string 'Mrs.Deviyani' and converts it into a set of its unique characters.Note: Sets are unordered, so the character order in the output may vary each time.
set from a list : {'Mayank', 'Vardhman', 'Mukesh'}
#The list contains duplicate values: 'Mukesh' appears twice.When converted to a set, duplicates are removed.
set from tuple: {'Kanpur', 'India', 'Lucknow'}
# As all elements are already unique, no change other than type conversion.
set from dictionary: {'Oxygen', 'Helium', 'Sulphur', 'Carbon'}
#When a dictionary is converted to a set, only the keys are included in the result.
The  values are ignored.
```

📌 **Note:** As both sets and dictionaries use curly brackets. Python uses the existence of key-value pairs to distinguish between the two . however, an empty dictionary and an empty set looks the same! By default { } initialise an empty dictionary. To initialise an empty set, use set( )

for e.g:

```
>>>a={}
>>>print(type(a))
<class 'dict'>
>>>b=set() # not { }
>>>print(type (b))
<class 'set'>
```

8.4. 1. Frozen Sets:

A frozen set is a special set in python whose elements are frozen, which means they become immutable. No further elements can be added or removed from that set. The built-in function frozen set is frozenset( ) or by using curly braces with a 'f' prefix.

A frozen set can be useful in situations where you want to use a set as a key in a dictionary or as an element in another set, but you don't want the set to be modified.

Example 1 : Program to print set using frozenset( ).

```
a=frozenset({12,13,14,15})
print(set(a))
```

Output:

```
{12, 13, 14, 15}
```

This program first creates a frozenset named a, which is an immutable collection of unique elements. Then it converts the frozenset back into a regular set using the set() function. The result is a mutable set containing the same elements, which is then printed.

Example 2 : Program to print set using set literal.

```
phone=frozenset(['apple','oppo','vivo','Samsung'])
print(f"{phone}")
```

Output:

```
frozenset({'apple', 'Samsung', 'oppo', 'vivo'})
```

This program creates a frozenset named phone from a list of smartphone brand names. It then prints the frozenset using an f-string (f"{phone}"), which formats and displays the contents of the frozenset. The elements may appear in any order, as sets and frozenset are unordered collections.

8.4.2. Set Datatype and Operations:

A **set** is an unordered collection of unique, hashable elements. Although implemented using a similar structure to dictionaries, sets serve a distinct purpose. They are mainly used for **checking membership**, i.e., determining whether an item exists in the set. This operation is highly optimized for both presence (hits) and absence (misses). Unlike dictionaries, which associate keys with values, sets store only keys — making them ideal for fast existence checks and removing duplicates.

Example 1:

```
s = {10, 50, 20}
print(s)
print(type(s))
```

Output:

```
{10, 20, 50}
<class 'set'>
```

Example 2: Program To Print Set From Dictionary By Using Key( ), Values( ), Items( ) Function.

```
a={2.5:3,"name":3,5:'c',"place":"london"}
b={'x':12,'y':14,'z':34}
print("set a keys are:",set (a))  #only keys are considered
print("set b keys are :",set(b))
print(set (a.keys())) # also keys are considered
print(set (a.values())) #only values are considered
print(set (a.items())) #key & values are considered
```

 # 1. Set Union Operation: join the two sets by using the keyword union

```
c=(set (a.keys()).union(set (a.values())))
print("set union of a is:",c)
d=(set(a)).union(set(b))
print ("set union of a ,b  is :",d)
```

 # 2. Set Intersection Operation: it prints the common elements in two sets.

```
e=d.intersection(c)
print("set intersection of d and c is:",e)
```

#3. Set Symmetric_Difference Operations: it includes all the elements that are in two sets but not the one that are common to two sets.

```
x=e.symmetric_difference(c)
print("the symmetric_difference of e and c is:",x)
```

Output:

```
set a keys are: {2.5, 5, 'place', 'name'}
set b keys are : {'x', 'y', 'z'}
{2.5, 5, 'place', 'name'}
{3, 'c', 'london'}
{(5, 'c'), (2.5, 3), ('name', 3), ('place', 'london')}
set union of a is: {2.5, 3, 'london', 5, 'place', 'c', 'name'}
set union of a ,b  is : {2.5, 5, 'place', 'y', 'z', 'x', 'name'}
set intersection of d and c is: {2.5, 5, 'place', 'name'}
the symmetric_difference of e and c is: {3, 'london', 'c'}
```

8.4.2. Accessing and Modifying sets:

Elements in a set can be accessed using a loop, but not by index or key since sets are unordered. Sets have a variety of methods for adding and removing elements .

For e.g:

```
m_set={1,2,3,5,7}
print(type(m_set))
print(m_set)
m_set.add(7)
print("new set:",m_set)
m_set.update([7,8,9])
print("updated set:",m_set)
m_set.discard(7)
print("set after removing 7 is:",m_set)
```

Output:

```
<class 'set'>
set()
new set: {7}
updated set: {8, 9, 7}
set after removing 7 is: {8, 9}
```

For further set operations and method refer in book 3(3.6)

8.5. Dictionary vs Set :

Aspect	Dictionary (dict)	Set (set)
Definition	A dictionary is a collection of key-value pairs.	A set is a collection of unordered, unique elements.
Syntax	Written as {key: value} for e.g:{"name": "Alice", "age": 22}	Written as {value1, value2} Example: {10, 20, 30}
Purpose	Used to store and retrieve data based on unique keys.	Used to store unique values and perform mathematical set operations.
Structure	Each element consists of a key and its associated value.	Each element is a single value; no associated key.

Duplicates	Keys must be unique; values can be duplicated.	All elements must be unique; duplicates are automatically removed.
Element Type	Keys must be immutable (e.g., int, str, tuple); values can be of any type.	Elements must be immutable and hashable.
Access	Accessed via keys: my_dict['name']	Membership tested using in: if 20 in my_set:
Ordering	Maintains insertion order (Python 3.7+).	Maintains insertion order (Python 3.7+), but ordering is not significant.
Mutability	Mutable: can add, update, or delete key-value pairs.	Mutable: can add or remove elements.
Use Cases	Useful for mapping, associative arrays, configuration data, databases.	Useful for filtering duplicates, testing membership, and mathematical operations.
Common Methods	.keys(), .values(), .items(), .get(), .update()	.add(), .remove(), .union(), .intersection(), .difference()

For e.g:

```
# Dictionary
my_dict = {"name": "Alice", "age": 25, "city": "New York"}
print(my_dict["name"])  Output: Alice

# Set
my_set = {1, 2, 3, 3} # Duplicate 3 will be removed
print(my_set)
```

Output:

```
Alice
{1, 2, 3}
```

8.6. Exercise:

1.Define a dictionary.

2. Write short notes on the following methods.

   a. keys( )
   b. values( )
   c. get(key)
   d. clear( )

3. Briefly explain how a dictionary is created with an example.

4. What is set datatype?

5. Write a note about dictionary vs sets.

6.Write the purpose of dict() method.

****

Book References:

1. Limitations of problem solving: https://bscbcanotes.wordpress.com/
2. Computer algorithms: https://www.techtarget.com/whatis/definition/algorithm
   https://bscbcanotes.wordpress.com/
3. Computer hard ware:
https://namesentrancement565.weebly.com/blog/list-three-hardware-components-commonly-installed-on-a-desktop-computer
https://en.wikipedia.org/wiki/Computer_hardware
4. Binary number system: https://arith-matic.com/notebook/binary-numbers
5. Python features: https://www.naukri.com/code360/library/features-of-python
6. Python variables and data types:
   Provided by Python: Variables and Data Types The Academic Center for Excellence 1 January 2022
   7. keywords
   Tpoint Tech https://www.tpointtech.com/python-keywords
8. Introduction to Computing & Problem Solving With PYTHON:
   By Jeeva Jose, P.Sojan Lal
9. Comments: https://gargsacademy.com/comments-in-python/
10. Python float() ,int():
   Python float() Function: Key Concepts [With Examples]
   By Ravikiran A S
11.**PYTHON ESSENTIALS:**
   By Mr. Ajay Gupta, Dr. Prabhat Kumar Srivastava, Ms. Mamta Srivastava, Mrs. Priya Gupta
12.Literals: https://www.wscubetech.com/resources/python/literals
https://www.scientecheasy.com/2022/09/literals-in-python.html/
13.Operators:
   Computer Science with Python
   By Reeta Sahoo, Gagan Sahoo
14.Data Types, DB PROGRAMMING:
   Introduction to Computing & Problem Solving With PYTHON
   By Jeeva Jose, P.Sojan Lal
15. List,tuple,set,dict:
   1.Computer Science with Python
   By Reeta Sahoo, Gagan
   2. Data Structures and Algorithms using Python
   By Subrata Saha
   3. PYTHON PROGRAMMING FOR COMPUTER SCIENCE
   By Prof. Chetan N. Ratho
   4. Learn Python Programming Systematically and Step by Step: Immensely popular and Highly-demanded programming language in the world
   By Chaitanya Patil
16. Type Conversion:
   The Python Code: A Practical Guide for Beginners
   By Nitin, Ankit Kumar, A
   Programming in Python
   By Davis Miller

17. Boolean Expression:

1.Python and Algorithmic Thinking for the Complete Beginner: Learn to think like a programmer by mastering Python programming and algorithmic foundations
By Aristides Bouras
2.Advanced Algorithm Mastery: Elevating Python Techniques for Professionals
By Adam Jones

18. if statement, nested if:

Comp-Computer Science-TB-12
By Reeta Sahoo, Gagan
Python for Linguists
By Michael Hammond

19.if-else:

Python knowledge building step by step from the basics to the first desktop application
By Dr. Csaba Dobreff

20. Loop Control Statements:

PYTHON PROGRAMMING: BASICS TO MACHINE LEARNING
By KUMARI, LALITA, SHYAM, RADHEY

21. String,List,Dictionary Manipulation:

Advanced Data Structures in Python: Mastering Complex Computational Patterns
By Adam Jones
Python Data Structures Explained: A Practical Guide with Examples
By William E. Clark

22.FUNCTIONS:

Fundamentals of Problem Solving and Python Programming
By Dr. R. K. Patjoshi, S. Patro, S. R. Jena
PYTHON PROGRAMMING
By Dr.M Sirish Kumar, G.R
Python for Data Science
By A. Lakshmi Muddana,
Learning Python: Introduction and Basic Object-Oriented Programming: Your Step By Step Guide To Easily Learn Python in 7 Days (Python for Beginners, Python ... for Beginners, Learn Python
By Neos Thanh

23.Python objects:

1. Python 3 Object-oriented Programming
By Dusty Phillips
2.Python OOP Step by Step: A Practical Guide with Examples
By William E. Clark

24.Turtle Graphics:

Python Graphics: Unleashing Creativity with Code
By Williams Asiedu

****